



TASKING VX-toolset for TriCore User Guide

Copyright © 2019 TASKING BV.

All rights reserved. You are permitted to print this document provided that (1) the use of such is for personal use only and will not be copied or posted on any network computer or broadcast in any media, and (2) no modifications of the document is made. Unauthorized duplication, in whole or part, of this document by any means, mechanical or electronic, including translation into another language, except for brief excerpts in published reviews, is prohibited without the express written permission of TASKING BV. Unauthorized duplication of this work may also be prohibited by local statute. Violators may be subject to both criminal and civil penalties, including fines and/or imprisonment. Altium[®], TASKING[®], and their respective logos are registered trademarks of Altium Limited or its subsidiaries. All other registered or unregistered trademarks referenced herein are the property of their respective owners and no trademark rights to the same are claimed.

Table of Contents

1. C Language	1
1.1. Data Types	2
1.1.1. Half Precision Floating-Point	3
1.1.2. Fractional Types	4
1.1.3. Packed Data Types	5
1.1.4. Changing the Alignment: <code>__unaligned</code> , <code>__packed</code> and <code>__align()</code>	7
1.2. Accessing Memory	8
1.2.1. Memory Qualifiers	8
1.2.2. Placing an Object at an Absolute Address: <code>__at()</code>	11
1.2.3. Accessing Bits	12
1.3. Data Type Qualifiers	13
1.3.1. Circular Buffers: <code>__circ</code>	13
1.3.2. Accessing Hardware from C	14
1.3.3. Saturation: <code>__sat</code>	16
1.3.4. External MCS RAM Data References: <code>__mcsram</code>	16
1.3.5. External PCP PRAM Data References: <code>__pram</code>	17
1.3.6. Shared Data between TriCore and PCP: <code>__share_pcp</code>	17
1.4. Multi-Core Support	19
1.4.1. Scratchpad RAM	19
1.4.2. Program Flash Memory (PFLASH)	20
1.4.3. Distributed Local Memory Unit SRAM (DLMU)	21
1.4.4. Compile Time Core Association	21
1.5. Shift JIS Kanji Support	29
1.6. Using Assembly in the C Source: <code>__asm()</code>	30
1.7. Attributes	36
1.8. Pragmas to Control the Compiler	41
1.9. Predefined Preprocessor Macros	50
1.10. Switch Statement	51
1.11. Functions	53
1.11.1. Calling Convention	53
1.11.2. Register Usage	55
1.11.3. Inlining Functions: <code>inline</code>	55
1.11.4. Interrupt and Trap Functions	57
1.11.5. Intrinsic Functions	64
1.12. Compiler Generated Sections	79
1.12.1. Rename Sections	80
1.12.2. Influence Section Definition	82
2. C++ Language	83
2.1. C++ Language Extension Keywords	83
2.2. C++ Dialect Accepted	83
2.2.1. C++03 Mode	84
2.2.2. GNU C++ Mode	85
2.2.3. Anachronisms Accepted	86
2.2.4. Atomic Operations	87
2.3. Namespace Support	89
2.4. Template Instantiation	90
2.4.1. Instantiation Modes	91
2.4.2. Instantiation <code>#pragma</code> Directives	92

2.4.3. Implicit Inclusion	93
2.5. Inlining Functions	94
2.6. Extern Inline Functions	95
2.7. Pragmas to Control the C++ Compiler	95
2.7.1. C pragmas Supported by the C++ compiler	96
2.8. Predefined Macros	97
2.9. Precompiled Headers	101
2.9.1. Automatic Precompiled Header Processing	101
2.9.2. Manual Precompiled Header Processing	104
2.9.3. Other Ways to Control Precompiled Headers	104
2.9.4. Performance Issues	105
3. Assembly Language	107
3.1. Assembly Syntax	107
3.1.1. Deviations from the Instruction Set Manual	108
3.2. Assembler Significant Characters	109
3.3. Operands of an Assembly Instruction	109
3.4. Symbol Names	110
3.4.1. Predefined Preprocessor Symbols	110
3.5. Registers	111
3.5.1. Special Function Registers	112
3.6. Assembly Expressions	112
3.6.1. Numeric Constants	113
3.6.2. Strings	113
3.6.3. Expression Operators	114
3.7. Working with Sections	115
3.8. Built-in Assembly Functions	117
3.9. Assembler Directives and Controls	130
3.9.1. Assembler Directives	131
3.9.2. Assembler Controls	176
3.10. Macro Operations	192
3.10.1. Defining a Macro	192
3.10.2. Calling a Macro	192
3.10.3. Using Operators for Macro Arguments	193
4. Using the C Compiler	199
4.1. Compilation Process	199
4.2. Calling the C Compiler	200
4.3. The C Startup Code	202
4.4. How the Compiler Searches Include Files	206
4.5. Compiling for Debugging	206
4.6. Compiler Optimizations	207
4.6.1. Generic Optimizations (frontend)	208
4.6.2. Core Specific Optimizations (backend)	211
4.6.3. Optimize for Code Size or Execution Speed	214
4.7. Static Code Analysis	217
4.7.1. C Code Checking: CERT C	218
4.7.2. C Code Checking: MISRA C	220
4.8. C Compiler Error Messages	222
5. Using the C++ Compiler	225
5.1. Calling the C++ Compiler	225
5.2. How the C++ Compiler Searches Include Files	227

5.3. C++ Compiler Error Messages	228
6. Using the Assembler	231
6.1. Assembly Process	231
6.2. Calling the Assembler	232
6.3. How the Assembler Searches Include Files	233
6.4. Assembler Optimizations	234
6.5. Generating a List File	235
6.6. Assembler Error Messages	235
7. Using the Linker	237
7.1. Linking Process	238
7.1.1. Phase 1: Linking	239
7.1.2. Phase 2: Locating	240
7.2. Calling the Linker	242
7.3. Linking with Libraries	243
7.3.1. How the Linker Searches Libraries	245
7.3.2. How the Linker Extracts Objects from Libraries	246
7.4. Incremental Linking	247
7.5. Cross-Linking	248
7.6. Linking Core-Specific Projects into a Multi-Core Application	249
7.7. Importing Binary Files	251
7.8. Linker Optimizations	252
7.9. Controlling the Linker with a Script	254
7.9.1. Purpose of the Linker Script Language	254
7.9.2. Eclipse and LSL	254
7.9.3. Preprocessor Macros in the Linker Script Files	256
7.9.4. Structure of a Linker Script File	258
7.9.5. The Architecture Definition	262
7.9.6. The Derivative Definition	265
7.9.7. The Processor Definition	268
7.9.8. The Memory Definition	268
7.9.9. The Section Layout Definition: Locating Sections	270
7.9.10. Locating in a Multi-core Processor Environment	272
7.9.11. Locating Private Sections in ROM	274
7.9.12. Stack Size Estimation	275
7.9.13. MPU Configuration Data	280
7.9.14. Boot Mode Headers	283
7.9.15. Booting AURIX SCR	289
7.10. Linker Labels	289
7.11. Generating a Map File	292
7.12. Linker Error Messages	292
8. Using the Utilities	295
8.1. Control Program	295
8.2. Make Utility amk	297
8.2.1. Makefile Rules	297
8.2.2. Makefile Directives	299
8.2.3. Macro Definitions	299
8.2.4. Makefile Functions	301
8.2.5. Conditional Processing	302
8.2.6. Makefile Parsing	302
8.2.7. Makefile Command Processing	303

8.2.8. Calling the amk Make Utility	304
8.3. Make Utility mktc	305
8.3.1. Calling the Make Utility	306
8.3.2. Writing a Makefile	307
8.4. Eclipse Console Utility	316
8.4.1. Headless Build	316
8.4.2. Generating Makefiles from the Command Line	317
8.5. Archiver	319
8.5.1. Calling the Archiver	319
8.5.2. Archiver Examples	321
8.6. HLL Object Dumper	323
8.6.1. Invocation	323
8.6.2. HLL Dump Output Format	323
8.7. ELF Patch Utility	331
8.7.1. ELF Patch Command File	331
8.7.2. Data Reference Modification File	332
8.7.3. ELF Symbol Renaming Command File	335
8.8. Expire Cache Utility	337
8.9. Proftool Utility	338
9. Using the Debugger	339
9.1. Reading the Eclipse Documentation	339
9.2. Creating a Customized Debug Configuration	339
9.3. Pipeline and Cache During Debugging	347
9.4. Troubleshooting	347
9.5. TASKING Debug Perspective	348
9.5.1. Debug View	349
9.5.2. Breakpoints View	351
9.5.3. File System Simulation (FSS) View	357
9.5.4. Disassembly View	358
9.5.5. Expressions View	358
9.5.6. Memory View	359
9.5.7. Compare Application View	360
9.5.8. Heap View	360
9.5.9. Logging View	361
9.5.10. RTOS View	361
9.5.11. Registers View	361
9.5.12. Trace View	363
9.5.13. Devices View	363
9.6. Multi-core Hardware Debugging	365
9.7. Programming a Flash Device	366
9.7.1. Boot Mode Headers	368
10. Tool Options	371
10.1. Configuring the Command Line Environment	377
10.2. C Compiler Options	379
10.3. C++ Compiler Options	480
10.4. Assembler Options	624
10.5. Linker Options	670
10.6. Control Program Options	724
10.7. Make Utility Options	806
10.8. Parallel Make Utility Options	834

10.9. Archiver Options	848
10.10. HLL Object Dumper Options	863
10.11. ELF Patch Utility Options	897
10.12. Expire Cache Utility Options	913
11. Influencing the Build Time	923
11.1. SFR File	923
11.2. MIL Linking	923
11.3. Optimization Options	924
11.4. Automatic Inlining	924
11.5. Code Compaction	924
11.6. Compiler Cache	924
11.7. Header Files	925
11.8. Parallel Build	925
11.9. Section Concatenation	926
12. Profiling	927
12.1. What is Profiling?	927
12.1.1. Methods of Profiling	927
12.2. Profiling using Code Instrumentation (Dynamic Profiling)	928
12.2.1. Step 1: Build your Application for Profiling	929
12.2.2. Step 2: Execute the Application	931
12.2.3. Step 3: Displaying Profiling Results	933
12.3. Profiling at Compile Time (Static Profiling)	936
12.3.1. Step 1: Build your Application with Static Profiling	936
12.3.2. Step 2: Displaying Static Profiling Results	937
13. Position Independent Code and Data	939
13.1. Using the Example PIC/PID Projects	940
13.2. Create Your Own PIC/PID Projects	941
14. Libraries	945
14.1. Library Functions	949
14.1.1. assert.h	949
14.1.2. cinit.h	949
14.1.3. clock.h	949
14.1.4. complex.h	950
14.1.5. cstart.h and cstart_tcx.h	951
14.1.6. ctri.h	951
14.1.7. ctype.h and wctype.h	951
14.1.8. dbg.h	952
14.1.9. errno.h	952
14.1.10. etsimath.h	953
14.1.11. except.h	953
14.1.12. fcntl.h	954
14.1.13. fenv.h	954
14.1.14. float.h	955
14.1.15. float_config.h	956
14.1.16. inttypes.h and stdint.h	956
14.1.17. io.h	956
14.1.18. iso646.h	957
14.1.19. libfloat.h	957
14.1.20. limits.h	957
14.1.21. locale.h	957

14.1.22. malloc.h	958
14.1.23. math.h and tgmath.h	958
14.1.24. setjmp.h	962
14.1.25. sevt.h	962
14.1.26. signal.h	963
14.1.27. stdalign.h	963
14.1.28. stdarg.h	963
14.1.29. stdatomic.h	964
14.1.30. stdbool.h	968
14.1.31. stddef.h	968
14.1.32. stdint.h	968
14.1.33. stdio.h and wchar.h	968
14.1.34. stdlib.h and wchar.h	976
14.1.35. stdnoreturn.h	979
14.1.36. string.h and wchar.h	980
14.1.37. time.h and wchar.h	981
14.1.38. typeinfo.h	984
14.1.39. uchar.h	984
14.1.40. unistd.h	984
14.1.41. vt100.h	985
14.1.42. wchar.h	985
14.1.43. wctype.h	986
14.2. C Library Reentrancy	987
15. List File Formats	999
15.1. Assembler List File Format	999
15.2. Linker Map File Format	1000
16. Object File Formats	1011
16.1. ELF/DWARF Object Format	1011
16.2. Intel Hex Record Format	1011
16.3. Motorola S-Record Format	1014
16.4. Binary Object Format	1016
17. Linker Script Language (LSL)	1017
17.1. Structure of a Linker Script File	1017
17.2. Syntax of the Linker Script Language	1019
17.2.1. Preprocessing	1019
17.2.2. Lexical Syntax	1020
17.2.3. Identifiers and Tags	1021
17.2.4. Expressions	1021
17.2.5. Built-in Functions	1022
17.2.6. LSL Definitions in the Linker Script File	1024
17.2.7. Memory and Bus Definitions	1025
17.2.8. Architecture Definition	1027
17.2.9. Derivative Definition	1030
17.2.10. Processor Definition and Board Specification	1031
17.2.11. Section Setup	1031
17.2.12. Section Layout Definition	1032
17.2.13. Veneer Layout Definition	1037
17.3. Expression Evaluation	1037
17.4. Semantics of the Architecture Definition	1038
17.4.1. Defining an Architecture	1039

17.4.2. Defining Internal Buses	1039
17.4.3. Defining Address Spaces	1040
17.4.4. Mappings	1044
17.4.5. Defining the MPU Layout	1047
17.5. Semantics of the Derivative Definition	1047
17.5.1. Defining a Derivative	1048
17.5.2. Instantiating Core Architectures	1048
17.5.3. Defining Internal Memory and Buses	1050
17.6. Semantics of the Board Specification	1051
17.6.1. Defining a Processor	1052
17.6.2. Instantiating Derivatives	1052
17.6.3. Defining External Memory and Buses	1053
17.7. Semantics of the Section Setup Definition	1053
17.7.1. Setting up a Section	1054
17.8. Semantics of the Section Layout Definition	1057
17.8.1. Defining a Section Layout	1058
17.8.2. Creating and Locating Groups of Sections	1059
17.8.3. Creating or Modifying Special Sections	1065
17.8.4. Creating Symbols	1070
17.8.5. Conditional Group Statements	1070
17.9. Semantics of the Veneer Layout Definition	1071
17.9.1. Veneer Placement	1071
18. Debug Target Configuration Files	1073
18.1. Custom Board Support	1073
18.2. Description of DTC Elements and Attributes	1074
18.3. Special Resource Identifiers	1077
18.4. Initialize Elements	1078
19. CPU Problem Bypasses and Checks	1079
20. CERT C Secure Coding Standard	1107
20.1. Preprocessor (PRE)	1107
20.2. Declarations and Initialization (DCL)	1108
20.3. Expressions (EXP)	1109
20.4. Integers (INT)	1110
20.5. Floating Point (FLP)	1110
20.6. Arrays (ARR)	1111
20.7. Characters and Strings (STR)	1111
20.8. Memory Management (MEM)	1111
20.9. Environment (ENV)	1112
20.10. Signals (SIG)	1112
20.11. Miscellaneous (MSC)	1113
21. MISRA C Rules	1115
21.1. MISRA C:1998	1115
21.2. MISRA C:2004	1119
21.3. MISRA C:2012	1127
22. C Implementation-defined Behavior	1135
22.1. C99 Implementation-defined Behavior	1135
22.1.1. Translation	1135
22.1.2. Environment	1136
22.1.3. Identifiers	1137
22.1.4. Characters	1137

22.1.5. Integers	1139
22.1.6. Floating-Point	1139
22.1.7. Arrays and Pointers	1141
22.1.8. Hints	1141
22.1.9. Structures, Unions, Enumerations, and Bit-fields	1141
22.1.10. Qualifiers	1142
22.1.11. Preprocessing Directives	1142
22.1.12. Library Functions	1143
22.1.13. Architecture	1148
22.2. C99 Locale-specific Behavior	1152
22.3. C11 Implementation-defined Behavior	1154
22.3.1. Translation	1154
22.3.2. Environment	1154
22.3.3. Identifiers	1156
22.3.4. Characters	1156
22.3.5. Integers	1158
22.3.6. Floating-Point	1158
22.3.7. Arrays and Pointers	1159
22.3.8. Hints	1160
22.3.9. Structures, Unions, Enumerations, and Bit-fields	1160
22.3.10. Qualifiers	1161
22.3.11. Preprocessing Directives	1161
22.3.12. Library Functions	1162
22.3.13. Architecture	1168
22.4. C11 Locale-specific Behavior	1172

Chapter 1. C Language

This chapter describes the target specific features of the C language, including language extensions that are not standard in ISO-C. For example, pragmas are a way to control the compiler from within the C source.

The TASKING VX-toolset for TriCore[®] C compiler fully supports the ISO C99 standard and supports all mandatory language features of the C11 standard, and adds extra possibilities to program the special functions of the target. C11 is the default of the C compiler.

C11 language features

All mandatory ISO C11 language features are supported (ISO/IEC 9899:2011 section 6.10.8.1 Mandatory macros). Furthermore the C compiler supports the following conditional features (ISO/IEC 9899:2011 section 6.10.8.3 Conditional feature macros):

- atomic types (including the `_Atomic` type qualifier) and the `<stdatomic.h>` header file (`__STDC_NO_ATOMICS__` is 0 for TriCore 1.6.x and 1.6.2)
- complex types and the `<complex.h>` header file
- variable length arrays and variably modified types

Other conditional language features such as threads, as mentioned in section 6.10.8.3 Conditional feature macros and section 6.10.8.2 Environment macros of the ISO/IEC 9899:2011 standard, are not supported. `__STDC_NO_THREADS__` is defined as 1.

Additional language features

In addition to the standard C language, the compiler supports the following:

- extra data types, like `__fract`, `__laccum` and `__packb`
- keywords to specify memory types for data and functions
- attribute to specify alignment and absolute addresses
- intrinsic (built-in) functions that result in target specific assembly instructions
- pragmas to control the compiler from within the C source
- predefined macros
- the possibility to use assembly instructions in the C source
- keywords for inlining functions and programming interrupt routines
- libraries

All non-standard keywords have two leading underscores (`__`).

In this chapter the target specific characteristics of the C language are described, including the above mentioned extensions.

1.1. Data Types

The C compiler supports the ISO C11 defined data types, and additionally fractional types and packed data types. The sizes of these types are shown in the following table.

C Type	Size	Align	Limits
_Bool	1	8	0 or 1
signed char	8	8	$[-2^7, 2^7-1]$
unsigned char	8	8	$[0, 2^8-1]$
short	16	16	$[-2^{15}, 2^{15}-1]$
unsigned short	16	16	$[0, 2^{16}-1]$
int	32	16	$[-2^{31}, 2^{31}-1]$
unsigned int	32	16	$[0, 2^{32}-1]$
enum ¹	8 16 32	8 16	$[-2^7, 2^7-1]$ or $[0, 2^8-1]$ $[-2^{15}, 2^{15}-1]$ or $[0, 2^{16}-1]$ $[-2^{31}, 2^{31}-1]$
long	32	16	$[-2^{31}, 2^{31}-1]$
unsigned long	32	16	$[0, 2^{32}-1]$
long long	64	32	$[-2^{63}, 2^{63}-1]$
unsigned long long	64	32	$[0, 2^{64}-1]$
_Float16 (10-bit mantissa) ²	16	16	$[-65504.0F, -6.103515625E-05]$ $[+6.103515625E-05, +65504.0F]$
float (23-bit mantissa)	32	16	$[-3.402E+38, -1.175E-38]$ $[+1.175E-38, +3.402E+38]$
double long double (52-bit mantissa)	64	32	$[-1.797E+308, -2.225E-308]$ $[+2.225E-308, +1.797E+308]$
_Imaginary float	32	16	$[-3.402E+38i, -1.175E-38i]$ $[+1.175E-38i, +3.402E+38i]$
_Imaginary double _Imaginary long double	64	32	$[-1.797E+308i, -2.225E-308i]$ $[+2.225E-308i, +1.797E+308i]$
_Complex float	64	32	real part + imaginary part
_Complex double _Complex long double	128	32	real part + imaginary part
pointer to data or function	32	32	$[0, 2^{32}-1]$
struct/union ³	≥ 64	32	$[0, 2^{32}-1]$
__sfract	16	16	$[-1, 1>$
__fract	32	32	$[-1, 1>$

C Type	Size	Align	Limits
<code>__laccum</code>	64	32	$[-131072, 131072>$
<code>__packb</code> <code>signed __packb</code>	32	16	$4x: [-2^7, 2^7-1]$
<code>unsigned __packb</code>	32	16	$4x: [0, 2^8-1]$
<code>__packhw</code> <code>signed __packhw</code>	32	16	$2x: [-2^{15}, 2^{15}-1]$
<code>unsigned __packhw</code>	32	16	$2x: [0, 2^{16}-1]$

¹ When you use the enum type, the compiler will use the smallest suitable integer type (char, unsigned char, short, unsigned short or int), unless you use [C compiler option --integer-enumeration](#) (always use 32-bit integers for enumeration).

² The C compiler supports half-precision (16-bit) floating-point via the `_Float16` type using the binary16 interchange format. See also [Section 1.1.1, Half Precision Floating-Point](#).

³ Structures and unions that are equal to or larger than 64-bit, are word aligned to allow efficient copy through LD.D and ST.D instructions. See also [Section 1.1.3, Packed Data Types](#).

`__bitsizeof()` operator

The `sizeof` operator always returns the size in bytes. Use the `__bitsizeof` operator in a similar way to return the size of an object or type in bits.

```
__bitsizeof( object | type )
```

`_Atomic` type qualifier

The `_Atomic` type qualifier is supported for the TriCore 1.6.x and 1.6.2 and designates an atomic type as specified in the C11 standard.

1.1.1. Half Precision Floating-Point

The TASKING C compiler supports half precision (16-bit) floating-point via the `_Float16` type using the binary16 interchange format. The binary16 interchange format is defined in IEEE Std 754-2008 IEEE Standard for Floating-Point Arithmetic. The `_Float16` type is defined in *ISO/IEC TS 18661-3 Draft Technical Specification – December 4, 2014 WG14 N1896*.

The `_Float16` type with binary16 format can represent normalized values in the range of 2^{-14} to 65504. There are 11 bits of significant precision, approximately 3 decimal digits. Also subnormal values are supported, as defined by `FLT16_HAS_SUBNORM` in `float.h`.

The `_Float16` type is a storage format only. For purposes of arithmetic and other operations, `_Float16` values in C expressions are automatically promoted to `float`.

Note that all conversions from and to `_Float16` involve an intermediate conversion to `float`. Because of rounding, this can sometimes produce a different result than a direct conversion.

When you specify C compiler option `--fp-model=soft`, the C compiler generates hardware floating-point instructions for conversions between `_Float16` and `float` for AURIX PLUS only.

Language-level support for the `_Float16` data type is independent of whether the C compiler generates code using hardware floating-point instructions or not. In cases where hardware support is not specified or not available for the selected core, the C compiler implements conversions between `_Float16` and `float` values as run-time library calls. These run-time functions are called `__f_ftohp` and `__f_hptof`.

```
_Float16 __f_ftohp( float f ); // single precision to half precision
float __f_hptof( _Float16 f ); // half precision to single precision
```

1.1.2. Fractional Types

The TASKING C compiler fully supports fractional data types which allow you to use normal expressions:

```
__fract f, f1, f2; /* Declaration of fractional variables */

f1 = 0.5;          /* Assignment of a fractional constants */
f2 = 0.242;

f = f1 * f2;       /* Multiplication of two fractionals */
```

The `__sfract` type has 1 sign bit + 15 mantissa bits. The `__fract` type has 1 sign bit + 31 mantissa bits. The `__laccum` type has 1 sign bit + 17 integral bits + 46 mantissa bits.

The `__accum` type is only included for compatibility reasons and is mapped to `__laccum`.

The TriCore instruction set supports most basic operations on fractional types directly. To obtain more portable code, you can use several intrinsic functions that use fractional types. Fractional values are automatically saturated.

[Section 1.11.5, *Intrinsic Functions*](#) explains intrinsic functions. [Section 1.11.5.2, *Fractional Arithmetic Support*](#) lists the intrinsic functions.

Promotion rules

For the three fractional types, the promotion rules are similar to the promotion rules for `char`, `short`, `int`, `long` and `long long`. This means that for an operation on two different fractional types, the smaller type is promoted to the larger type before the operation is performed.

When you mix a fractional type with a `float` or `double` type, the fractional number is first promoted to `float` respectively `double`.

When you mix an integer type with the `__laccum` type, the integer is first promoted to `__laccum`.

Because of the limited range of `__sfract` and `__fract`, only a few operations make sense when combining an integer with an `__sfract` or `__fract`. Therefore, the C compiler only supports the following operations for integers combined with fractional types:

left	operand	right	result
fractional	*	integer	fractional
integer	*	fractional	fractional
fractional	/	integer	fractional
fractional	<<	integer	fractional
fractional	>>	integer	fractional
fractional: __sfract, __fract			
integer: char, short, int, long, long long			

1.1.3. Packed Data Types

The TASKING C compiler additionally supports the packed types `__packb` and `__packhw`.

A `__packb` value consists of four signed or unsigned `char` values. A `__packhw` value consists of two signed or unsigned `short` values.

The TriCore instruction set supports a number of arithmetic operations on packed data types directly. For example, the following function:

```
__packb add4 ( __packb a, __packb b )
{
    return a + b;
}
```

results into the following assembly code:

```
add4:
    add.b    d2, d4, d5
    ret16
```

[Section 1.11.5, Intrinsic Functions](#) explains intrinsic functions. [Section 1.11.5.3, Packed Data Type Support](#) lists the intrinsic functions.

Halfword packed unions and structures

To minimize space consumed by alignment padding with unions and structures, elements follow the minimum alignment requirements imposed by the architecture. The TriCore architecture supports access to 32-bit integer variables on halfword boundaries.

Because only doubles, circular buffers, `__laccum` or pointers require the full word access, structures that do not contain members of these types are automatically halfword (2 bytes) packed.

Structures and unions with a size divisible by 64-bit or larger or structures and unions that contain members with a size divisible by 64-bit or larger, are word packed to allow efficient access through `LD.D` and `ST.D` instructions. These load and store operations require word aligned structures with a size divisible by 64-bit or larger. If necessary, 64-bit (or larger) divisible structure elements are aligned or padded to make the structure 64-bit accessible. Whether the `LD.D`/`ST.D` instructions are used or not depends on the tradeoff value ([C compiler option `--tradeoff \(-t\)`](#)): only for tradeoff values 0, 1 or 2 these instructions are used.

You can see the difference by using the following code (struct.c):

```
typedef struct
{
    char a;
    char b;
    char c;
    char d;

    char e;
    char f;
    char g;
    char h;

    char j;
} ST_64;

ST_64 st_64_1;
ST_64 st_64_2;

void main( void )
{
    st_64_1 = st_64_2;
}
```

and use the following invocations:

```
ctc struct.c -t0
ctc struct.c
```

With `#pragma pack 2` you can disable the LD.D/ST.D structure and union copy optimization to ensure halfword structure and union packing when possible. This "limited" halfword packing only supports structures and unions that do not contain double, circular buffer, `__laccum` or pointer type members and that are not qualified with `#pragma align` to get an alignment larger than two bytes. With `#pragma pack 0` you turn off halfword packing again.

```
#pragma pack 2
typedef struct {
    unsigned char  uc1;
    unsigned int   ui1;
    unsigned short us1;
    unsigned int   ui2;
    unsigned short us2;
} packed_struct;
#pragma pack 0
```

When you place a `#pragma pack 0` before a structure or union, its alignment will not be changed:

```
#pragma pack 0
packed_struct pstruct;
```


The alignment of data sections and stack can affect the alignment of the base address of a halfword packed structure. A halfword packed structure can be aligned on a halfword boundary or larger. When located on the stack or at the beginning of a section, the alignment becomes a word, because of the minimum required alignment of data sections and stack objects. A stack or data section can contain any type of object. To avoid wrong word alignment of objects in the section, the section base is also word aligned.

1.1.1.4. Changing the Alignment: `__unaligned`, `__packed__` and `__align()`

Normally data, pointers and structure members are aligned according to the table in [Section 1.1, Data Types](#).

Suppress alignment

With the type qualifier `__unaligned` you can specify to suppress the alignment of objects or structure members. This can be useful to create compact data structures. In this case the alignment will be one byte for objects and non-bit-field structure members.

At the left side of a pointer declaration you can use the type qualifier `__unaligned` to mark the pointer value as potentially unaligned. This can be useful to access externally defined data. However the compiler can generate less efficient instructions to dereference such a pointer, to avoid unaligned memory access.

You can always convert a normal pointer to an unaligned pointer. Conversions from an unaligned pointer to an aligned pointer are also possible. However, the compiler will generate a warning in this situation, with the exception of the following case: when the logical type of the destination pointer is `char` or `void`, no warning will be generated.

Example:

```
struct
{
    char c;
    __unaligned int i;    /* aligned at offset 1 ! */
} s;

__unaligned int * up = & s.i;
```

Packed structures

To prevent alignment gaps in structures, you can use the attribute `__packed__`. When you use the attribute `__packed__` directly after the keyword `struct`, all structure members are marked `__unaligned`. For example the following two declarations are the same:

```
struct __packed__
{
    char c;
    int * i;
} s1;

struct
{
```

```
char __unaligned c;  
int * __unaligned i; /* __unaligned at right side of '*'  
                      to pack pointer member          */  
} s2;
```

The attribute `__packed__` has the same effect as adding the type qualifier `__unaligned` to the declaration to suppress the standard alignment.

You can also use `__packed__` in a pointer declaration. In that case it affects the alignment of the pointer itself, not the value of the pointer. The following two declarations are the same:

```
int * __unaligned p;  
int * p __packed__;
```

Increasing the alignment: `__align()`

By default the TriCore compiler aligns variables, functions and structure members to the minimum alignment required by the architecture. See [Section 1.1, Data Types](#). With the attribute `__align(n)` you can increase the default alignment of variables, functions or structure members to *n* bytes. If you apply an alignment with a value lower than the default alignment of the variable, function or structure member, this has no effect on the alignment of the variable, function or structure member. The C compiler issues a warning in that case. The alignment must be a power of two.

Note that unlike variables and functions, structure member alignment is not affected by the [C compiler option `--align`](#) and the `#pragma align`.

When a function is inlined the attribute `__align()` has no effect on the inlined code, the alignment attribute is ignored.

Example:

```
__align(4) int globalvar; /* changed to 4 bytes alignment  
                          instead of default 2 bytes */
```

Instead of the attribute `__align(n)` you can also use `__attribute__((__align(n)))` or `__attribute__((aligned(n)))`.

1.2. Accessing Memory

You can use static memory qualifiers to allocate static objects in a particular part of the addressing space of the processor.

In addition, you can place variables at absolute addresses with the keyword `__at()`.

1.2.1. Memory Qualifiers

In the C language you can specify that a variable must lie in a specific part of memory. You can do this with a *memory qualifier*.

You can specify the following memory qualifiers:

Qualifier	Description	Location	Maximum object size	Pointer size	Section types
<code>__near</code> *	Near data, direct addressable	First 16 kB of a 256 MB block	16 kB	32-bit	<code>neardata</code> , <code>nearrom</code> , <code>nearbss</code> , <code>nearnoclear</code>
<code>__far</code> *	Far data, indirect addressable	Anywhere	no limit	32-bit	<code>fardata</code> , <code>farrom</code> , <code>farbss</code> , <code>farnoclear</code>
<code>__a0</code>	Uninitialized / constant / cleared data	Sign-extended 16-bit offset from address register A0.	64 kB	32-bit	<code>a0data</code> , <code>a0rom</code> , <code>a0bss</code>
<code>__a1</code>	Uninitialized / constant / cleared data	Sign-extended 16-bit offset from address register A1.	64 kB	32-bit	<code>a1data</code> , <code>a1rom</code> , <code>a1bss</code>
<code>__a8</code>	Uninitialized / constant / cleared data	Sign-extended 16-bit offset from address register A8.	64 kB	32-bit	<code>a8data</code> , <code>a8rom</code> , <code>a8bss</code>
<code>__a9</code>	Uninitialized / constant / cleared data	Sign-extended 16-bit offset from address register A9.	64 kB	32-bit	<code>a9data</code> , <code>a9rom</code> , <code>a9bss</code>

* If you do not specify `__near` or `__far`, the compiler chooses where to place the declared object. With the C compiler option `--default-near-size` (maximum size in bytes for data elements that are by default located in `__near` sections) you can specify the size of data objects which the compiler then by default places in near memory.

All these memory qualifiers (`__near`, `__far`, `__a0`, `__a1`, `__a8` and `__a9`) are related to the object being defined, they influence where the object will be located in memory. They are not part of the type of the object defined. Therefore, you cannot use these qualifiers in typedefs, type casts or for members of a struct or union.

Address registers A0, A1, A8, and A9 are designated as system global registers. They are not part of either context partition and are not saved/restored across calls. They can be protected against write access by user applications. A0, A1, A8 and A9 are freely to use on any type of data.

It is not allowed to assign ROM and RAM data to the same group. With the address registers A0, A1, A8 and A9 you can only access ROM or RAM but not both. Mixed ROM and RAM section types are not allowed for qualifiers `__a0`, `__a1`, `__a8` and `__a9`. When this happens the linker issues the message:

```
Section .rodata_ax in group ax has memory type ROM but
expected memory type RAM like section .data_ax in the same group
```

where x is one of 0, 1, 8, 9.

For example, it is not allowed to have variable `a0rom` and `a0ram` in a single application.

```
__a0 const int a0rom=1;
__a0      int a0ram=1; // Mixing ROM/RAM is not allowed
```

For this example the linker reports that the section `.rodata_a0.a0.a0rom` in group `a0` has memory type ROM but expected memory type RAM like section `.data_a0.a0.a0ram` in the same group.

Examples using memory qualifiers

To declare a fast accessible integer in directly addressable memory:

```
int __near Var_in_near;
```

To allocate a pointer in far memory (the compiler will not use absolute addressing mode):

```
__far int * Ptr_in_far;
```

To declare and initialize a string in A0 memory:

```
char __a0 string[] = "TriCore";
```

If you use the `__near` memory qualifier, the compiler generates faster access code for those (frequently used) variables. Pointers are always 32-bit.

Functions are by default allocated in ROM. In this case you can omit the memory qualifier. You cannot use memory qualifiers for function return values.

Some examples of using pointers with memory qualifiers:

```
int __near * p;    /* pointer to int in __near memory
                  (pointer has 32-bit size)      */
int __far * g;     /* pointer to int in __far memory
                  (pointer has 32-bit size)      */
void main(void)
{
    g = p;         /* allowed because pointers are 32-bit */
}
```

You cannot use memory qualifiers in structure declarations:

```
struct S {
    __near int i; /* put an integer in near
                  memory: Incorrect !      */
    __far int * p; /* put an integer pointer in
                  far memory: Incorrect !  */
}
```

If a library function declares a variable in near memory and you try to redeclare the variable in far memory, the linker issues an error:

```
extern int __near foo; /* extern int in near memory */

int __far foo;         /* int in far memory      */
```

The usage of the variables is always without a storage specifier:

```
char __near example;
example = 2;
```

The generated assembly would be:

```
mov16 d15,2
st.b example,d15
```

All allocations with the same storage specifiers are collected in units called 'sections'. The section with the `__near` attribute will be located within the first 16 kB of each 256 MB block.

1.2.2. Placing an Object at an Absolute Address: `__at()`

Just like you can declare a variable in a specific part of memory (using memory qualifiers), you can also place an object or a function at an absolute address in memory.

With the attribute `__at ( )` you can specify an absolute address. The address is a 32-bit linear address.

Examples

```
unsigned char Display[80*24] __at( 0x2000 );
```

The array `Display` is placed at address `0x2000`. In the generated assembly, an absolute section is created. On this position space is reserved for the variable `Display`.

```
int i __at(0x1000) = 1;
```

```
void f(void) __at( 0xa0001000 ) { __nop(); }
```

The function `f` is placed at address `0xa0001000`.

Restrictions

Take note of the following restrictions if you place a variable at an absolute address:

- The argument of the `__at ( )` attribute must be a constant address expression. Otherwise the compiler generates an error.
- You can place only global variables at absolute addresses. Parameters of functions, or automatic variables within functions cannot be placed at absolute addresses. If they are, the compiler generates an error.
- A variable that is declared `extern`, is not allocated by the compiler in the current module. Hence you should not use the keyword `__at ( )` on an external variable. If you do, the compiler ignores the keyword `__at ( )` without generating an error. Use `__at ( )` at the definition of the variable.
- You cannot place structure members at an absolute address. If you do, the compiler ignores the keyword `__at ( )` and generates a warning.
- Absolute variables cannot overlap each other. If you declare two absolute variables at the same address, the assembler and/or linker issues an error. The compiler does not check this.

1.2.3. Accessing Bits

There are several methods to access single bits in a variable. The compiler generates efficient bit operations where possible.

Masking and shifting

The classic method to extract a single bit in C is masking and shifting.

```
unsigned int bitword;
void foo( void )
{
    if( bitword & 0x0004 )    // bit 2 set?
    {
        bitword &= ~0x0004;  // clear bit 2
    }
    bitword |= 0x0001;        // set bit 0;
}
```

Built-in macros `__getbit()` and `__putbit()`

The compiler has the built-in macros `__getbit()` and `__putbit()`. These macros expand to the `__extru()` and `__imaskldmst()` and intrinsic functions to perform the required result.

```
int * bw;
void foo( void )
{
    if( __getbit( bw, 2 ) )
    {
        __putbit( 0, bw, 2 );
    }
    __putbit( 1, bw, 0 );
}
```

Accessing bits using a struct/union combination

```
typedef union
{
    unsigned int word;
    struct
    {
        int  b0 : 1;
        int  b1 : 1;
        int  b2 : 1;
        int  b3 : 1;
        int  b4 : 1;
        int  b5 : 1;
        int  b6 : 1;
        int  b7 : 1;
        int  b8 : 1;
    }
}
```

```

        int  b9 : 1;
        int  b10: 1;
        int  b11: 1;
        int  b12: 1;
        int  b13: 1;
        int  b14: 1;
        int  b15: 1;
    } bits;
} bitword_t;

bitword_t bw;

void foo( void )
{
    if( bw.bits.b3 )
    {
        bw.bits.b3 = 0;
    }
    bw.bits.b0 = 1;
}

void reset( void )
{
    bw.word = 0;
}

```

1.3. Data Type Qualifiers

1.3.1. Circular Buffers: `__circ`

The TriCore core has support for implementing specific DSP tasks, such as finite impulse response (FIR) and infinite impulse response (IIR) filters. For the FIR and IIR filters the TriCore architecture supports the circular addressing mode. The TriCore C compiler supports *circular buffers* for these DSP tasks. This way, the TriCore C compiler makes hardware features available at C source level instead of at assembly level only.

A *circular buffer* is a linear (one dimensional) array that you can access by moving a pointer through the data. The pointer can jump from the last location in the array to the first, or vice-versa (the pointer wraps-around). This way the buffer appears to be continuous. The TriCore C compiler supports the keyword `__circ` (circular addressing mode) for this type of buffer.

Example:

```

__fract __circ circbuf[10];
__fract __circ * ptr_to_circbuf = circbuf;

```

Here, `circbuf` is declared as a circular buffer. The compiler aligns the base address of the buffer on the access width (in this example an int, so 4 bytes). The compiler keeps the buffer size and uses it to control pointer arithmetic of pointers that are assigned to the buffer later.

Note that it is not allowed to declare an automatic circular buffer, because circular buffers require an alignment of 64-bit, but the TriCore stack uses a 32-bit alignment. Use keyword `static` for local circular buffers.

Circular pointers

You can perform operations on circular pointers with the usual C pointer arithmetic with the difference that the pointer will wrap. When you access the circular buffer with a circular pointer, it wraps at the buffer limits. Circular pointer variables are 64 bits in size.

Example:

```
while( *Pptr_to_circbuf++ );
```

Indexing

Indexing in the circular buffer, using an integer index, is treated equally to indexing in a non-circular array.

Example:

```
int i = circbuf[3];
```

The index is not calculated modulo; indexing outside the array boundaries yields undefined results.

Intrinsic function `__initcirc()`

If you want to initialize a circular pointer with a dynamically allocated buffer at run-time, you should use the intrinsic function `__initcirc()`:

```
#define N 100
unsigned short s = sizeof(__fract);
__fract * buf = calloc( N, s );
__fract __circ * ptr_to_circbuf = __initcirc( buf, N * s, 0 * s );
```

1.3.2. Accessing Hardware from C

Using Special Function Registers

It is easy to access Special Function Registers (SFRs) that relate to peripherals from C. The SFRs are defined in a special function register file (*.sfr) as symbol names for use with the compiler. An SFR file contains the names of the SFRs and the bits in the SFRs.

Example use in C (SFRs from `sfr/regtc1796b.sfr`):

```
void set_sfr(void)
{
    SBCU_SRC.I |= 0xb32a; /* access SBCU Service Request
                          Control register as a whole */

    SBCU_SRC.B.SRE = 0x1; /* access SRE bit-field of SBCU
```



```

        Service Request Control register */
}

```

You can find a list of defined SFRs and defined bits by inspecting the SFR file for a specific processor. The files are located in the `sfr` subdirectory of the standard `include` directory. The files are named `regcpu.sfr`, where *cpu* is the CPU specified with the [control program option --cpu](#). The compiler includes this register file if you specify [option --include-file=sfr/regtc1796b.sfr](#).

Defining Special Function Registers: `__sfrbit16`, `__sfrbit32`

SFRs are defined in SFR files and are written in C. With the data type qualifiers `__sfrbit16` and `__sfrbit32` you can declare bit-fields in special function registers.

According to the *TriCore Embedded Applications Binary Interface*, 'normal' bit-fields are accessed as `char`, `short` or `int`. Bit-fields are aligned according to the table in [Section 1.1, Data Types](#).

If you declare bit-fields in special function registers, this behavior is not always desired: some special function registers require 16-bit or 32-bit access. To force 16-bit or 32-bit access, you can use the data type qualifiers `__sfrbit16` and `__sfrbit32`.

When the SFR contains fields, the layout of the SFR is defined by a typedef-ed union. The following example is part of an SFR file and illustrates the declaration of a special function register using the data type qualifier `__sfrbit32`:

```

typedef volatile union
{
    struct
    {
        unsigned __sfrbit32 SRPN : 8; /* Service Priority Number */
        unsigned __sfrbit32      : 2;
        unsigned __sfrbit32 TOS  : 2; /* Type-of-Service Control */
        unsigned __sfrbit32 SRE  : 1; /* Service Request Enable Control */
        unsigned __sfrbit32 SRR  : 1; /* Service Request Flag */
        unsigned __sfrbit32 CLRR : 1; /* Request Flag Clear Bit */
        unsigned __sfrbit32 SETR : 1; /* Request Flag Set Bit */
        unsigned __sfrbit32      : 16;
    } B;

    int I;
    unsigned int U;
} LBCU_SRC_type;

```

Read-only fields can be marked by using the `const` keyword.

The SFR is defined by a cast to a 'typedef-ed union' pointer. The SFR address is given in parenthesis. Read-only SFRs are marked by using the `const` keyword in the macro definition.

```

#define LBCU_SRC (*(LBCU_SRC_type*)(0xF87FFEFCu))
                /* LBCU Service Control Register */

```

Restrictions

- You can use the `__sfrbit32` data type qualifier only on `int` bit-field types. The compiler issues an error if you use for example `__sfrbit32 char x : 8;`
- You can use the `__sfrbit16` data type qualifier only on `int` or `short` bit-field types. The compiler issues an error if you use for example `__sfrbit16 char x : 8;`
- When you use the `__sfrbit32` and `__sfrbit16` data type qualifiers on other types than a bit-field, the compiler ignores this without a warning. For example, `__sfrbit32 int global;` is equal to `int global;`.
- Structures or unions that contain a member qualified with `__sfrbit16`, are zero padded to complete a halfword if necessary. The structure or union will be halfword aligned. Structures or unions that contain a member qualified with `__sfrbit32`, are zero padded to complete a full word if necessary. The structure or union will be word aligned.

1.3.3. Saturation: `__sat`

When a variable is declared with the type qualifier `__sat`, all operations on that variable will be performed using saturating arithmetic. When an operation is performed on a plain variable and a `__sat` variable, the `__sat` takes precedence, and the operation is done using saturating arithmetic. The type of the result of such an operation also includes the qualifier `__sat`, so that another operation on the result will also be saturated. In this respect, the behavior of the type qualifier `__sat` is comparable to the `unsigned` keyword. You can overrule this behavior by inserting type casts with or without the type qualifier `__sat` in an expression.

You can only use the type qualifier `__sat` on type `int` (fractional types are always saturated).

Examples:

```
__sat int si      = 0x7FFFFFFF;
int i            = 0x12345;
unsigned int ui   = 0xFFFFFFFF;
```

```
si + i // a saturated addition is performed,
       // yielding a saturated int
```

```
si + ui // a saturated unsigned addition is performed,
        // yielding a saturated unsigned int
```

```
i + ui // a normal unsigned addition is performed,
        // yielding an unsigned int
```

1.3.4. External MCS RAM Data References: `__mcsram`

You can reference external MCS RAM data from the TriCore with the keyword `__mcsram`. Only external variables can be qualified with the keyword `__mcsram`.

Global MCS RAM variables can only be defined in the MCS application. Only types with a size of 32-bit can have the keyword `__mcsram`, because only 32-bit types are supported by the MCS in the MCS RAM space.

To refer to a global variable *name* in a specific MCS core, you need to prefix the variable name with `core_`

For example, if in two MCS cores a global variable `count` is defined, you can reference them externally by the TriCore:

```
extern volatile int __mcsram mcs00_count; /* variable count in mcs00 */
extern volatile int __mcsram mcs01_count; /* variable count in mcs01 */

__mcsram external variables get the _lc_t_ linker prefix. _lc_t_mcs00_count and
_lc_t_mcs01_count for the example above.
```

1.3.5. External PCP PRAM Data References: `__pram`

You can reference external PCP PRAM data from the TriCore with the keyword `__pram`. Only external variables can be qualified with the keyword `__pram`.

Global PRAM variables can only be defined in the PCP application. Only types with a size of 32-bit can have the keyword `__pram`, because only 32-bit types are supported by the PCP in the PRAM space.

For example, in the PCP source `channel3.c` of the `pcp-multi-ch3-ch4` example delivered with the product, the following variable is defined:

```
int channel3_count = 0;
```

In the TriCore source `tc_main.c` of the `pcp-multi-start` example delivered with the product, this global variable defined in a PCP channel is referenced externally by the TriCore. The symbol prefix used by the PCP for this channel is `_PCP_`.

```
extern volatile int __pram _PCP_channel3_count;
```

`__pram` external variables get the `_lc_s_` linker prefix.

When `__pram` is used for a TriCore 1.6.x or 1.6.2 core, a warning is generated and the keyword is ignored. The warning generated is:

```
W757: bad symbol attribute __pram for object "_PCP_channel3_count" -- ignored
```

1.3.6. Shared Data between TriCore and PCP: `__share_pcp`

Global data can be shared between the TriCore and PCP with the keyword `__share_pcp`. Only global and external variables can be qualified with the keyword `__share_pcp`. `__share_pcp` global variables get the `_lc_` linker prefix.

External reference of TriCore global variables from PCP

For example, in the TriCore source `tc_main.c` of the `pcp-multi-start` example delivered with the product, the following variable is defined:

```
volatile int __far __share_pcp shared_CPU_FPI;
```

In the PCP source `channel1.c` of the `pcp-multi-ch1` example delivered with the product, this variable is referenced as:

```
extern int __far __share_pcp shared_CPU_FPI;
```

When `__share_pcp` is used for a TriCore 1.6.x or 1.6.2 core, a warning is generated and the keyword is ignored. The warning generated is:

```
W757: bad symbol attribute __share_pcp for object "shared_CPU_FPI" -- ignored
```

External reference of PCP global variables from TriCore

For example, in the PCP source `channel1.c` of the `pcp-multi-ch1` example delivered with the product, the following variable is defined:

```
int __far __share_pcp channel2_shared_PCP_FPI = 0;
```

In the TriCore source `tc_main.c` of the `pcp-multi-start` example delivered with the product, this variable is referenced as:

```
extern volatile int __far __share_pcp channel2_shared_PCP_FPI;
```

External reference of PCP PRAM variables that have application scope

To externally reference PCP PRAM variables that have application scope instead of channels scope, you have to combine the keywords `__share_pcp` and `__pram`. Global PCP variables that are being shared between PCP channels do not use a PCP channel symbol prefix. `__pram __share_pcp` external variables get the `_lc_s__lc_` linker prefix.

For example, in the PCP source `channel1.c` of the `pcp-multi-ch1` example delivered with the product, the following variable is shared between different PCP channels:

```
int __share_pcp channel1_shared_PCP_PRAM = 0;
```

In the TriCore source `tc_main.c` of the `pcp-multi-start` example delivered with the product, this variable is referenced as:

```
extern volatile int __pram __share_pcp channel1_shared_PCP_PRAM;
```

1.4. Multi-Core Support

The TASKING VX-toolset for TriCore has support for multi-core versions of the TriCore. For example, the TC27x contains three TriCore cores (core 0, core 1 and core 2).

Note that multi-core is supported for the TriCore 1.6.x and 1.6.2 architectures only. This is automatically selected when you select, for example, the TC27x in Eclipse. If you build your sources on the command line with the control program, you have to specify **control program option** `--cpu=tc27x`. The control program will call the C compiler, assembler and linker with the correct options.

If you want to build an application for a single-core configuration, you also need to select the specific core in Eclipse (for example, Core 0 (tc0)). If you build your sources on the command line with the control program, you also have to specify **control program option** `--lsl-core=tc0`.

1.4.1. Scratchpad RAM

Each core has local memory for data (data scratchpad RAM, DSPR n) and code (program scratchpad RAM, PSPR n). This local memory is mirrored and can also be accessed by the other cores. Code can be shared between binary compatible cores. Data can be shared between cores when the memory system provides access to the data. To define how code and data is accessible from one or multiple cores you can specify code core associations and data core associations, as explained in the following sections.

Note that accessing data or code scratch pad memory from another core (which is only possible via global addresses) may result in an extra instruction cycle penalty depending on read/write, DSPR/PSPR, while the core itself can access its own data or code scratchpad memory (via local and global addressing modes) without this penalty. The following table describes the CPU access times in CPU clock cycles. The access times are described as maximum CPU stall cycles where e.g. a data access to the local DSPR results in zero stall cycles. Note that the CPU does not always immediately stall after the start of a data read from another SPR due to instruction pipelining effects. This means that the average number will be lower than the shown numbers.

CPU Access Mode	CPU clock cycles	
	TC26x / TC27x	TC29x
Data read access to own DSPR	0	0
Data write access to own DSPR	0	0
Data read access to own or other PSPR	5	8
Data write access to own or other PSPR	0	0
Data read access to other DSPR	5	8
Data write access to other DSPR	0	0
Instruction fetch from own PSPR	0	0
Instruction fetch from other PSPR (critical word)	5	8
Instruction fetch from other PSPR (any remaining words)	0	0
Instruction fetch from other DSPR (critical word)	5	8
Instruction fetch from other DSPR (any remaining words)	0	0

1.4.2. Program Flash Memory (PFLASH)

AURIX TC3xx derivatives support a Program Flash Memory per core (PFLASH0 .. PFLASH n). Where n is the number of cores minus 1. PFLASH is located in segment 8 cached and segment A non-cached. PFLASH memories are located in 3 MB pages. PFLASH core 0 starts at offset 0. For each next core its PFLASH offset is at the next 3 MB offset.

The total maximum of all PFLASHes is smaller than or equal to 16 MB. The size of PFLASH for core 0..4 is 3 MB each and the size of PFLASH for core 5 is 1 MB. With a maximum of six cores the total PFLASH size is 16 MB.

The target address range for a call or jump (j) instruction is +16 MB or -16 MB relative to the current PC (disp24). All inter-core calls are always in range of a disp24 addressing mode.

A PFLASH associated with a specific core is defined as a CPU local resource. All cores can access all PFLASHes but with an access latency penalty.

CPU Access Mode	CPU clock cycles TC3xx
Data read access to local PFLASH	5 (note 1)
Instruction fetch from local PFLASH (buffer miss)	2 (note 1)
Instruction fetch from local PFLASH (buffer hit)	4
Data read access to other PFLASH	10 (note 1)
Instruction fetch from other PFLASH (buffer miss)	9 (note 1)
Instruction fetch from other PFLASH (buffer hit)	6

Note 1: plus configured PFLASH Corrected Wait States

You can locate code and ROM data per core by using compile time section renaming and link time core association.

For example for core 0:

```
#pragma section farrom pflash0
static volatile const __far int rom_core0 = 0;
#pragma section farrom restore

#pragma section code pflash0
static __noinline int core0( void )
{
    return rom_core0;
}
#pragma section code restore
```

This is demonstrated in the pflash example delivered with the product. For more information about section renaming see [Section 1.12.1, Rename Sections](#). For information about creating a link time core association see [Section 7.9.10, Locating in a Multi-core Processor Environment](#).

1.4.3. Distributed Local Memory Unit SRAM (DLMU)

AURIX TC3xx derivatives support a Distributed Local Memory Unit SRAM per core (DLMU0 .. DLMUn). Where n is the number of cores minus 1. A portion of the LMU memory (called DLMU) is distributed between the processors to provide high performance access to global SRAM. DLMU is located in segment 0x9 cached and segment 0xB non-cached.

The DLMU memory is mapped at a different offset in these segments for each core. The DLMU size is 64 KB per core for all TC3xx derivatives. It is contiguous with other LMU memory in these segments.

Distributed LMU (DLMU) is single cycle memory for local data operations. There is an access latency penalty for a core that performs a non-local data operation on the DLMU memory.

CPU Access Mode	CPU clock cycles TC3xx
Data read access to local DLMU	0
Data write access to local DLMU	2
Instruction fetch from local DLMU	7
Data read access to other DLMU	7
Data write access to other DLMU	5 (+3 with pipelining)
Instruction fetch from other DLMU	7

DLMU has a data write access latency for a core local resource but DSPR has no latency.

You can locate code and ROM data per core by using compile time section renaming and link time core association.

For example for core 0:

```
#pragma section fardata dlmuo
__far int dlmuo_core0 = 0;
#pragma section fardata restore
```

This is demonstrated in the dlmuo example delivered with the product. For more information about section renaming see [Section 1.12.1, Rename Sections](#). For information about creating a link time core association see [Section 7.9.10, Locating in a Multi-core Processor Environment](#).

1.4.4. Compile Time Core Association

Code and data can be shared, private or cloned:

- **Shared.** In the default situation all code and data are accessible by all cores. The symbols are located in shared memory.
- **Private.** Private means that the code and/or data is copied to, and accessed from, the local scratchpad memory of one particular core.
- **Cloned.** Cloned means that code or data is copied to the local scratchpad memory of each binary compatible core, or a specific core. The core then treats the code/data as if it were private.

Compile time core association

You can determine at compile time, with memory qualifiers or pragmas or options, whether data or code objects are private or cloned instead of shared for local scratch pad RAM (DSPR/PSPR). This is explained in the following sections.

Link time core association

Instead of at compile time, you can determine in which memory objects should be located at link time. This is necessary to locate private objects in PFLASH memories for AURIX 2G derivatives such as the TC39x. It is also necessary when you want to restrict a clone section to a subset of the available cores. This is explained in [Section 7.9.10, Locating in a Multi-core Processor Environment](#).

1.4.4.1. Data Core Association

The term "data core association" (DCA) is used to define:

- whether a data object is accessible from one or multiple cores
- the type of memory where the data will be allocated
- the number of memory instances of the data object

You can use a memory qualifier (`__share`, `__privaten` or `__clone`) or pragma (`#pragma data_core_association`) to qualify individual data objects, or you can use an option (`C compiler option --data-core-association`) to specify the default data core association. The default data core association is "share". This means that when you do not explicitly specify a memory qualifier or a pragma, all data can be accessed by all cores.

Data core association	Memory qualifier	Accessible from *	Number of instances	Allocation in
Share	<code>__share</code>	All cores	One instance. The data object is shared between cores.	Global RAM or core-local RAM Note: when allocated in core-local RAM the shared object is accessed via the mirror page.
Private	<code>__privaten</code>	Core <i>n</i> only	One instance. For one specific core: core <i>n</i> .	Preferably in core <i>n</i> local RAM. If the memory is full, other memory is used.
Clone	<code>__clone</code>	All cores	Multiple instances. For each core one instance is allocated. All instances will be located at identical addresses.	Core-local RAM

* Note that `__privaten` data objects are private for core *n* from a compiler point of view, where *n* depends on the number of cores supported by the derivative. For example, for a derivative with three cores, *n* can be 0, 1 or 2. At run-time, the MPU (Memory Protection Unit) must be configured to protect memory from being accessed from other cores.

Instead of a memory qualifier you can also use a pragma:

```
#pragma data_core_association share | privaten | clone | default | restore
```

With default you switch back to the default behavior. With restore you restore the previous value of the pragma.

Based on the specified data core association the compiler stores the data object in a section with the following naming convention:

```
section_type_prefix.share|privaten|clone.module_name.symbol_name
```

Note however that when you do not specify a memory qualifier or a pragma or when you use `#pragma data_core_association default`, the data can be accessed by all cores, but the resulting sections do not have `.share` in the section name. This is the default behavior. The `.share` is only added to the name if the section was explicitly designated as shared.

Example:

```
#pragma data_core_association clone
__near int var_1; // var_1 ends up in section .zbss.clone.file_1.var_1

#pragma data_core_association default
__near int var_2; // var_2 ends up in section .zbss.file_1.var_2
```

For more information on section names see [Section 1.12, Compiler Generated Sections](#).

The linker recognizes the section names, duplicates clone sections for each binary compatible core and locates core specific code and data in the scratchpad memory of each core, resulting in one absolute object file (ELF) for each binary compatible set of cores.

1.4.4.2. Code Core Association

The term "code core association" (CCA) is used to define:

- the core or cores that are allowed to execute a function
- the type of memory where the function will be allocated
- the number of instances that are copied to local scratchpad RAM, i.e. the number of entries in the copy table
- a restriction on the type of data (defined by a data core association) the code may access

You can use a memory qualifier (`__share`, `__privaten` or `__clone`) or pragma (`#pragma code_core_association`) to qualify individual functions, or you can use an option ([C compiler option `--code-core-association`](#)) to change the default code core association. The default code core association is "share". This means that when you do not explicitly specify a memory qualifier or a pragma, all code can be executed by all cores.

Code core association	Memory qualifier	Executes on *	Number of instances	Allowed access of DCA qualified data	Allocation in
Share	__share	Any core	One instance. The code is shared between cores.	Shared Cloned (of executing core)	PFLASH_0, PFLASH_1 or core-local RAM
Private	__privaten	Core <i>n</i> only	One instance.	Shared Private Cloned	Preferably in core <i>n</i> local RAM. If the memory is full, other memory is used.
Clone	__clone	Any core	Multiple instances. Each code instance is executed by one core.	Shared Cloned	Core-local RAM

* Note that `__privaten` code is private for core *n* from a compiler point of view, where *n* depends on the number of cores supported by the derivative. For example, for a derivative with three cores, *n* can be 0, 1 or 2. At run-time, the MPU (Memory Protection Unit) must be configured to protect code on one core from being executed by other cores.

Instead of a memory qualifier you can also use a pragma:

```
#pragma code_core_association share | privaten | clone | default | restore
```

With `default` you switch back to the default behavior. With `restore` you restore the previous value of the pragma.

Based on the specified code core association the compiler stores the code object in a section with the following naming convention:

```
section_type_prefix.share|privaten|clone.module_name.symbol_name
```

Note however that when you do not specify a memory qualifier or a pragma or when you use `#pragma code_core_association default`, the code can be executed by all cores, but the resulting sections do not have `.share` in the section name. The `.share` is only added to the name if the section was explicitly designated as shared.

For more information on section names see [Section 1.12, Compiler Generated Sections](#).

The linker recognizes the section names, duplicates clone sections for each binary compatible core and locates core specific code and data in the scratchpad memory of each core, resulting in one absolute object file (ELF) for each binary compatible set of cores.

1.4.4.3. Core Association Restrictions

The following restriction apply to core associations:

Run-time bounds data is shared by all cores

For run-time bounds checking, bounds data is generated in a section declared with a fixed section name 'bounds'. No data core association is applied to this section. The linker uses the default core association share. The bounds data is shared among all cores.

Example (bounds.c):

```
typedef struct
{
    int i;
} s_t;

s_t s;
const int ci1 =55;
const int ci2 =55;

int main(void)
{
    s.i = 42;

    return ci1+ci2;
}
```

Use the following command to see the result:

```
ctc bounds.c --runtime --core=tc1.6.x --data-core-association=share
--code-core-association=share
```

Profile data is shared by all cores

For dynamic profiling, profiling data is generated in sections declared with fixed section names '*__prof_'. No data core association is applied to these sections. The linker uses the default core association share. The profiling data is shared among all cores.

Example (prof.c):

```
void f1(int i);
void f2(int i);
void f3(int i);

void f3(int i)
{
    if ( i )
    {
        f1(i-1);
        f2(i-1);
        f3(i-1);
    }
}
```

```
void f2(int i)
{
    if ( i )
    {
        f1(i-1);
        f2(i-1);
        f3(i-1);
    }
}

void f1(int i)
{
    if ( i )
    {
        f1(i-1);
        f2(i-1);
        f3(i-1);
    }
}

void main(void)
{
    f1(3);
    f2(3);
    f3(3);
}
```

When you compile with:

```
ctc prof.c --profile --core=tc1.6.x --data-core-association=share
--code-core-association=share
```

This results in the following section declarations:

```
.sdecl '.zbss.prof._999001__prof_counter_0',data
.sdecl '.zbss.prof._999002__prof_counter_0',data
.sdecl '.zbss.prof._999003__prof_counter_0',data
.sdecl '.zbss.prof._999004__prof_counter_0',data
```

Predefined identifier `__func__` is shared by all cores

You cannot use the data core association symbol qualifiers or pragmas to associate a core with predefined identifier `__func__`. The linker uses the default core association share. `__func__` is shared among all cores.

Example (func.c):

```
char funcname[10];

void function( void )
{
```

```

    for(int i = 0; i < 6; i++ ) {
        funcname[i] = __func__[i];
    }
}

```

When you compile with:

```

ctc func.c --core=tc1.6.x --data-core-association=share
--code-core-association=share

```

This results in the following section declaration:

```

.sdecl '.rodata.func._999001__func__',data,rom

```

No core is associated to a section renamed with the attribute section

No section prefixing is supported on sections that are renamed with attribute section. The linker uses the default core association share. Of course it is still possible to use the core association section naming convention in the section attribute to do the core association manually.

Example (section.c):

```

int function( void ) __section__("fixed_section_name")
{
    return 0;
}

```

When you compile with:

```

ctc section.c --core=tc1.6.x --data-core-association=share
--code-core-association=share

```

This results in the following section declaration:

```

.sdecl 'fixed_section_name',code

```

Renaming with #pragma section supports the normal section prefixing.

```

#pragma section code "myname"

```

```

int function( void )
{
    return 0;
}

```

results in the following section declaration:

```

.sdecl '.text.share.myname',code

```

For more details see [Section 1.12, Compiler Generated Sections](#).

1.4.4.4. Core Association and Addressing Modes

You can combine the data core associations with the memory qualifiers from [Section 1.2.1, Memory Qualifiers](#). The most efficient way is to qualify cloned and private data objects as `__near`. You can use the qualifier `__near` explicitly or you can use the C compiler option `--default-near-size=value`. All data objects with a size less than or equal to *value* are located in `__near` sections.

Data objects located in scratchpad RAM of core *N* can be treated as `__near` by core *N*. Other cores need to access the data object through the mirror pages and you have to use `__far` addressing. This results into the following scheme where `__near` means that `__near` access *may* be used, and `__far` means that `__far` access must be used:

Code core association	Data core association	Applicable addressing mode(s)
Share	Share	<code>__far</code> , <code>__a[0 1 8 9]</code>
	Private	<code>__near</code>
	Clone	<code>__near</code>
Private (RAM)	Share	<code>__far</code> , <code>__a[0 1 8 9]</code>
	Private	<code>__near</code>
	Clone	<code>__near</code>
Clone	Share	<code>__far</code> , <code>__a[0 1 8 9]</code>
	Clone	<code>__near</code>

As a consequence shared data located in scratchpad memory of core *N* is accessed via `__far` addressing, also by the code executing on core *N*.

1.4.4.5. Core Association and Function Calls

The code core association affects caller-callee relations. Private functions are not accessible by each core. Therefore, calling a private function is illegal unless it is guaranteed that the code that contains the call can only be executed by the core associated with the private function.

Both the C compiler and linker check for illegal function calls. However, the C compiler and linker cannot check indirect calls and the C compiler cannot check calls to external functions, due to lack of type information.

The following table shows the relation between function calls and code core associations.

Code core association of caller	Code core association of callee	Issues
Share	Share	No issues.
	Private	No issues.
	Clone	No issues.
Private	Share	No issues.

Code core association of caller	Code core association of callee	Issues
	Private	Caller and callee must be associated with the same core.
	Clone	No issues.
Clone	Share	No issues.
	Private	Illegal call. A cloned function is not allowed to call a private function.
	Clone	No issues.

1.5. Shift JIS Kanji Support

In order to allow for Japanese character support on non-Japanese systems (like PCs), you can use the Shift JIS Kanji Code standard. This standard combines two successive ASCII characters to represent one Kanji character. A valid Kanji combination is only possible within the following ranges:

- First (high) byte is in the range 0x81-0x9f or 0xe0-0xef.
- Second (low) byte is in the range 0x40-0x7e or 0x80-0xfc

Compiler option **-Ak** enables support for Shift JIS encoded Kanji multi-byte characters in strings and (wide) character constants. Without this option, encodings with 0x5c as the second byte conflict with the use of the backslash ('\') as an escape character. Shift JIS in comments is supported regardless of this option.

Note that Shift JIS also includes Katakana and Hiragana.

Example:

```
// Example usage of Shift JIS Kanji
// Do not switch off option -Ak
// At the position of the italic text you can
// put your Shift JIS Kanji code
int i; // put Shift JIS Kanji here
char c1;
char c2;
unsigned int ui;
const char mes[]="put Shift JIS Kanji here";
const unsigned int ar[5]={'K','a','n',
                          'j','i'};
                          // 5 Japanese array

void main(void)
{
    i=(int)c1;
    i++; /* put Shift JIS Kanji here\
        continuous comment */
    c2=mes[9];
```

```

    ui=ar[0];
}

```

1.6. Using Assembly in the C Source: `__asm()`

With the keyword `__asm()` you can use assembly instructions in the C source and pass C variables as operands to the assembly code.

It is recommended to use constructs in C or use [intrinsic functions](#) instead of `__asm()`. Be aware that C modules that contain assembly are not portable and harder to compile in other environments.

The compiler does not interpret assembly blocks but passes the assembly code to the assembly source file; they are regarded as a black box. So, it is your responsibility to make sure that the assembly block is syntactically correct. Possible errors can only be detected by the assembler.

You need to tell the compiler exactly what happens in the inline assembly code because it uses that for code generation and optimization. The compiler needs to know exactly which registers are written and which registers are only read. For example, if the inline assembly writes to a register from which the compiler assumes that it is only read, the generated code after the inline assembly is based on the fact that the register still contains the same value as before the inline assembly. If that is not the case the results may be unexpected. Also, an inline assembly statement using multiple input parameters may be assigned the same register if the compiler finds that the input parameters contain the same value. As long as this register is only read this is not a problem.

General syntax of the `__asm` keyword

```

__asm( "instruction_template"
      [ : output_param_list
        [ : input_param_list
          [ : register_reserve_list]] ] );

```

<i>instruction_template</i>	Assembly instructions that may contain parameters from the input list or output list in the form: <code>%parm_nr[.regnum]</code>
<code>%parm_nr[.regnum]</code>	Parameter number in the range 0 .. 9. With the optional <code>.regnum</code> you can access an individual register from a register pair. For example, with register pair d0/d1, <code>.0</code> selects register d0.
<i>output_param_list</i>	<code>[["&]constraint_char"(C_expression)],...</code>
<i>input_param_list</i>	<code>[["constraint_char"(C_expression)],...</code>
<code>&</code>	Says that an output operand is written to before the inputs are read, so this output must not be the same register as any input.
<i>constraint_char</i>	Constraint character: the type of register to be used for the <i>C_expression</i> . See the table below.
<i>C_expression</i>	Any C expression. For output parameters it must be an lvalue, that is, something that is legal to have on the left side of an assignment.
<i>register_reserve_list</i>	<code>[["register_name"],...</code>

register_name

Name of the register you want to reserve. For example because this register gets clobbered by the assembly code. The compiler will not use this register for inputs or outputs. Note that reserving too many registers can make register allocation impossible.

Specifying registers for C variables

With a *constraint character* you specify the register type for a parameter.

You can reserve the registers that are used in the assembly instructions, either in the parameter lists or in the reserved register list (*register_reserve_list*). The compiler takes account of these lists, so no unnecessary register saves and restores are placed around the inline assembly instructions.

Constraint character	Type	Operand	Remark
a	address register	a0 .. a15	
b	address register pair	b2, b4, b6, b12, b14	b2 = pair a2/a3, b4 = a4/a5, ...
d	data register	d0 .. d15	
e	data register pair	e0, e2, ..., e14	e0 = pair d0/d1, e2 = d2/d3, ...
m	memory	<i>variable</i>	memory operand
i	immediate value	<i>value</i>	
<i>number</i>	type of operand it is associated with	same as <i>%number</i>	Input constraint only. The <i>number</i> must refer to an output parameter. Indicates that <i>%number</i> and <i>number</i> are the same register.

If an input parameter is modified by the inline assembly then this input parameter must also be added to the list of output parameters (see [Example 7](#)). If this is not the case, the resulting code may behave differently than expected since the compiler assumes that an input parameter is not being changed by the inline assembly.

Loops and conditional jumps

The compiler does not detect loops with multiple `__asm( )` statements or (conditional) jumps across `__asm( )` statements and will generate incorrect code for the registers involved.

If you want to create a loop with `__asm( )`, the whole loop must be contained in a single `__asm( )` statement. The same counts for (conditional) jumps. As a rule of thumb, all references to a label in an `__asm( )` statement must be in that same statement. You can use numeric labels for these purposes.

Example 1: no input or output

A simple example without input or output parameters. You can use any instruction or label. When it is required that a sequence of `__asm( )` statements generates a contiguous sequence of instructions, then they can be best combined to a single `__asm( )` statement. Compiler optimizations can insert instruction(s)

in between `__asm( )` statements. Use newline characters `'\n'` to continue on a new line in a `__asm( )` statement. For multi-line output, use tab characters `'\t'` to indent instructions.

```
__asm( "nop\n"
      "\tnop" );
```

Example 2: using output parameters

Assign the result of inline assembly to a variable. With the constraint `m` memory is chosen for the parameter; the compiler decides where to put the variable. The `%0` in the instruction template is replaced with the name of the variable.

```
__near int out;
void func( void )
{
    __asm( "mov    d15,#1234\n"
          "\tst.w  %0,d15"
          : "=m" (out) );
}
```

Generated assembly code:

```
mov    d15,#1234
st.w   out,d15
```

Example 3: using input parameters

Assign a variable to a memory location. A data register is chosen for the parameter because of the constraint `d`; the compiler decides which register is best to use. The `%0` in the instruction template is replaced with the name of this register. The compiler generates code to move the input variable to the input register. Because there are no output parameters, the output parameter list is empty. Only the colon has to be present.

```
int in;
void initmem( void )
{
    __asm( "ST.W   0xa0000000,%0"
          :
          : "d" (in) );
}
```

Generated assembly code:

```
ld.w   d15,in
ST.W   0xa0000000,d15
```

Example 4: using input and output parameters

Multiply two C variables and assign the result to a third C variable. Data type registers are necessary for the input and output parameters (constraint `d`, `%0` for `out`, `%1` for `in1`, `%2` for `in2` in the instruction

template). The compiler generates code to move the input expressions into the input registers and to assign the result to the output variable.

```
int in1, in2, out;

void multiply32( void )
{
    __asm( "mul %0, %1, %2"
           : "=d" (out)
           : "d" (in1), "d" (in2) );
}
```

Generated assembly code:

```
ld.w    d15,in1
ld.w    d0,in2
mul      d15, d15, d0
st.w     out,d15
```

Example 5: reserving registers

Sometimes an instruction knocks out certain specific registers. The most common example of this is a function call, where the called function is allowed to do whatever it likes with some registers. If this is the case, you can list specific registers that get clobbered by an operation after the inputs.

Same as *Example 4*, but now register d0 is a reserved register. You can do this by adding a reserved register list (: "d0"). As you can see in the generated assembly code, register d0 is not used (the first register used is d1).

```
int in1, in2, out;

void multiply32( void )
{
    __asm( "mul %0, %1, %2"
           : "=d" (out)
           : "d" (in1), "d" (in2)
           : "d0" );
}
```

Generated assembly code:

```
ld.w    d15,in1
ld.w    d1,in2
mul      d15, d15, d1
st.w     out,d15
```

Example 6: use a register for an intermediate result

The following example demonstrates the use of an intermediate result.

```

int in1, in2, out;

void test( void )
{
    int temp_result;

    __asm("extr %0, %1, #31, #1" : "=d"(temp_result) : "d"(in1));
    __asm("sel %0, %1, %2, #0" : "=d"(out) : "d"(temp_result), "d"(in2));
}

```

Generated assembly code:

```

ld.w    d15,in1
extr    d15, d15, #31, #1
ld.w    d0,in2
sel     d15, d15, d0, #0
st.w    out,d15

```

Example 7: use the same register for input and output

As input constraint you can use a number to refer to an output parameter. This tells the compiler that the same register can be used for the input and output parameter. When the input and output parameter are the same C expression, these will effectively be treated as if the input parameter is also used as output. In that case it is allowed to write to this register. For example:

```

inline int foo(int par1, int par2, int * par3)
{
    int retvalue;

    __asm(
        "sh      %1,#-2\n\t"
        "add     %2,%1\n\t"
        "st.w    [%5],%2\n\t"
        "mov     %0,%2"
        : "=&d" (retvalue), "=d" (par1), "=d" (par2)
        : "1" (par1), "2" (par2), "a" (par3)
    );
    return retvalue;
}

int result, parm;

void func(void)
{
    result = foo(1000,1000,&parm);
}

```

In this example the "1" constraint for the input parameter par1 refers to the output parameter par1, and similar for the "2" constraint and par2. In the inline assembly %1 (par1) and %2 (par2) are written. This is allowed because the compiler is aware of this.

This results in the following generated assembly code:

```

mov    d15,#1000
lea    a15,parm
mov    d0,d15

sh     d15,#-2
add    d0,d15
st.w   [a15],d0
mov    d1,d0

st.w   result,d1

```

However, when the inline assembly would have been as given below, the compiler would have assumed that %1 (par1) and %2 (par2) were read-only. Because of the `inline` keyword the compiler knows that par1 and par2 both contain 1000. Therefore the compiler can optimize and assign the same register to %1 and %2. This would have given an unexpected result.

```

__asm(
    "sh      %1,#-2\n\t"
    "add     %2,%1\n\t"
    "st.w    [%3],%2\n\t"
    "mov     %0,%2"
    : "=&d" (retvalue)
    : "d" (par1), "d" (par2), "a" (par3)
);

```

Generated assembly code:

```

mov    d15,#1000
lea    a15,parm

sh     d15,#-2
add    d15,d15      ; same register, but is expected read-only
st.w   [a15],d15
mov    d0,d15

st.w   result,d0    ; contains unexpected result

```

Example 8: accessing individual registers in a register pair

You can access the individual registers in a register pair by adding a '.' after the operand specifier in the assembly part, followed by the index in the register pair.

```

int out1, out2;

void foo(double din)
{
    __asm ("ld.w %0, %2.0\n"
          "\tld.w %1, %2.1": "=&d"(out1), "=d"(out2): "e"(din) );
}

```

The first `ld.w` instruction uses index #0 of argument 2 (which is a double placed in a DxDx register) and the second `ld.w` instruction uses index #1. The input operand is located in register pair d4/d5. The assembly output becomes:

```
ld.w    d15, d4
ld.w    d0, e4,1 ; note that e4,1 corresponds to d5
st.w    out1,d15
st.w    out2,d0
```

If the index is not a valid index (for example, the register is not a register pair, or the argument has not a register constraint), the `'.'` is passed into the assembly output. This way you can still use the `'.'` in assembly instructions.

1.7. Attributes

You can use the keyword `__attribute__` to specify special attributes on declarations of variables, functions, types, and fields.

Syntax:

```
__attribute__((name,...))
```

or:

```
__name__
```

The second syntax allows you to use attributes in header files without being concerned about a possible macro of the same name. This second syntax is only possible on attributes that do not already start with an underscore. For example, you may use `__noreturn__` instead of `__attribute__((noreturn))`.

alias("symbol")

You can use `__attribute__((alias("symbol")))` to specify that the function declaration appears in the object file as an alias for another symbol. For example:

```
void __f() { /* function body */; }
void f() __attribute__((weak, alias("__f")));
```

declares 'f' to be a weak alias for '__f'.

__align(value), aligned(value)

You can use `__attribute__((__align(n)))` to increase the alignment of variables or functions. If you apply an alignment with a value lower than the default alignment of the variable or function, this has no effect on the alignment of the variable or function. The C compiler issues a warning in that case. The alignment must be a power of two and larger than or equal to 2. When a function is inlined the attribute has no effect on the inlined code, the attribute is ignored. See also [Section 1.1.4, Changing the Alignment: __unaligned, __packed and __align\(\)](#).

Instead of `__align()` you can also use the GNU compatible attribute `aligned()`.

const

You can use `__attribute__((const))` to specify that a function has no side effects and will not access global data. This can help the compiler to optimize code. See also attribute [pure](#).

The following kinds of functions should not be declared `__const__`:

- A function with pointer arguments which examines the data pointed to.
- A function that calls a non-const function.

export

You can use `__attribute__((export))` to specify that a variable/function has external linkage and should not be removed. During MIL linking, the compiler treats external definitions at file scope as if they were declared `static`. As a result, unused variables/functions will be eliminated, and the alias checking algorithm assumes that objects with static storage cannot be referenced from functions outside the current module. During MIL linking not all uses of a variable/function can be known to the compiler. For example when a variable is referenced in an assembly file or a (third-party) library. With the `export` attribute the compiler will not perform optimizations that affect the unknown code.

```
int i __attribute__((export)); /* 'i' has external linkage */
```

flatten

You can use `__attribute__((flatten))` to force inlining of all function calls in a function, including nested function calls.

Unless inlining is impossible or disabled by `__attribute__((noinline))` for one of the calls, the generated code for the function will not contain any function calls.

format(type,arg_string_index,arg_check_start)

You can use `__attribute__((format(type,arg_string_index,arg_check_start)))` to specify that functions take `printf`, `scanf`, `strftime` or `strfmon` style arguments and that calls to these functions must be type-checked against the corresponding format string specification.

type determines how the format string is interpreted, and should be `printf`, `scanf`, `strftime` or `strfmon`.

arg_string_index is a constant integral expression that specifies which argument in the declaration of the user function is the format string argument.

arg_check_start is a constant integral expression that specifies the first argument to check against the format string. If there are no arguments to check against the format string (that is, diagnostics should only be performed on the format string syntax and semantics), *arg_check_start* should have a value of 0. For `strftime`-style formats, *arg_check_start* must be 0.

Example:

```
int foo(int i, const char * my_format, ...) __attribute__((format(printf, 2, 3)));
```

The format string is the second argument of the function `foo` and the arguments to check start with the third argument.

jump

You can use `__attribute__((jump))` to specify that a function can only be jumped to. The compiler generates a jump instruction instead of a call instruction. This is for example used in the startup code generation:

```
static void __noinline__ __noreturn__ __jump__ _start( void );
```

When you call `_start()` in your C source, the compiler generates:

```
j _start
```

leaf

You can use `__attribute__((leaf))` to specify that a function is a leaf function. A leaf function is an external function that does not call a function in the current compilation unit, directly or indirectly. The attribute is intended for library functions to improve dataflow analysis. The attribute has no effect on functions defined within the current compilation unit.

malloc

You can use `__attribute__((malloc))` to improve optimization and error checking by telling the compiler that:

- The return value of a call to such a function points to a memory location or can be a null pointer.
- On return of such a call (before the return value is assigned to another variable in the caller), the memory location mentioned above can be referenced only through the function return value; e.g., if the pointer value is saved into another global variable in the call, the function is not qualified for the `malloc` attribute.
- The lifetime of the memory location returned by such a function is defined as the period of program execution between a) the point at which the call returns and b) the point at which the memory pointer is passed to the corresponding deallocation function. Within the lifetime of the memory object, no other calls to `malloc` routines should return the address of the same object or any address pointing into that object.

noinline

You can use `__attribute__((noinline))` to prevent a function from being considered for inlining. Same as keyword `__noinline` or `#pragma noinline`.

always_inline

With `__attribute__((always_inline))` you force the compiler to inline the specified function, regardless of the optimization strategy of the compiler itself. Same as keyword `inline` or `#pragma inline`.

noreturn

Some standard C function, such as `abort` and `exit` cannot return. The C compiler knows this automatically. You can use `__attribute__((noreturn))` to tell the compiler that a function never returns. For example:

```
void fatal() __attribute__((noreturn));

void fatal( /* ... */ )
{
    /* Print error message */
    exit(1);
}
```

The function `fatal` cannot return. The compiler can optimize without regard to what would happen if `fatal` ever did return. This can produce slightly better code and it helps to avoid warnings of uninitialized variables.

protect

You can use `__attribute__((protect))` to exclude a variable/function from the duplicate/unreferenced section removal optimization in the linker. When you use this attribute, the compiler will add the "protect" section attribute to the symbol's section. Example:

```
int i __attribute__((protect));
```

Note that the `protect` attribute will not prevent the compiler from removing an unused variable/function (see the `used` symbol attribute).

This attribute is the same as `#pragma protect/endprotect`.

pure

You can use `__attribute__((pure))` to specify that a function has no side effects, although it may read global data. Such pure functions can be subject to common subexpression elimination and loop optimization. See also attribute `const`.

section("section_name")

You can use `__attribute__((section("name")))` to specify that a function must appear in the object file in a particular section. For example:

```
extern void foobar(void) __attribute__((section("bar")));
```

puts the function `foobar` in the section named `bar`.

Note that this is a GNU style attribute. It does not follow the TriCore EABI guidelines. It does not add the section prefix as with `#pragma section`. It gives you full control over the section name. So, to be EABI compliant make sure you provide the correct section prefix.

See also [#pragma section](#) and [Section 1.12, Compiler Generated Sections](#).

uncached

You can use `__attribute__((uncached))` for variables to instruct the linker to allocate the corresponding variable in a non-cached memory segment. If a variable is declared with the attribute `uncached`, its corresponding section name includes a `'.uncached'` prefix, for example, `.zbss.uncached.my_module.my_atomic_var`. Atomic variables automatically imply the attribute `uncached`. Attribute `uncached` has no effect on local variables. The C compiler issues a warning that the attribute `uncached` is ignored for the automatic object.

used

You can use `__attribute__((used))` to prevent an unused symbol from being removed by the compiler. Example:

```
static const char copyright[] __attribute__((used)) = "Copyright 2019 TASKING BV";
```

When there is no C code referring to the `copyright` variable, the compiler will normally remove it. The `__attribute__((used))` symbol attribute prevents this. Because the linker should also not remove this symbol, you should consider to use `__attribute__((used, protect))`.

unused

You can use `__attribute__((unused))` to specify that a variable or function is possibly unused. The compiler will not issue warning messages about unused variables or functions.

weak

You can use `__attribute__((weak))` to specify that the symbol resulting from the function declaration or variable must appear in the object file as a weak symbol, rather than a global one. This is primarily useful when you are writing library functions which can be overwritten in user code without causing duplicate name errors.

See also [#pragma weak](#).

1.8. Pragmas to Control the Compiler

Pragmas are keywords in the C source that control the behavior of the compiler. Pragmas overrule compiler options. Put pragmas in your C source where you want them to take effect. Unless stated otherwise, a pragma is in effect from the point where it is included to the end of the compilation unit or until another pragma changes its status.

The syntax is:

```
#pragma [label:]pragma-spec pragma-arguments [on | off | default | restore]
```

or:

```
_Pragma( "[label:]pragma-spec pragma-arguments [on | off | default | restore]" )
```

Some pragmas can accept the following special arguments:

on	switch the flag on (same as without argument)
off	switch the flag off
default	set the pragma to the initial value
restore	restore the previous value of the pragma

Examples:

```
// by default all warnings are shown
```

```
#pragma warning 535          // disable W535
#pragma warning 530          // also disable W530
const char var_1 = 0x5678;    // W530 is not shown
var_2;                       // W535 is not shown
#pragma warning restore      // restore one level, only W535 is disabled
const char var_3 = 0x56789;   // W530 is shown
#pragma warning default      // back to default, all warnings are shown
var_4;                       // W535 is shown
```

Label pragmas

Some pragmas support a label prefix of the form "*label*:" between #pragma and the pragma name. Such a label prefix limits the effect of the pragma to the statement following a label with the specified name. The restore argument on a pragma with a label prefix has a special meaning: it removes the most recent definition of the pragma for that label.

You can see a label pragma as a kind of macro mechanism that inserts a pragma in front of the statement after the label, and that adds a corresponding #pragma ... restore after the statement.

Compared to regular pragmas, label pragmas offer the following advantages:

- The pragma text does not clutter the code, it can be defined anywhere before a function, or even in a header file. So, the pragma setting and the source code are uncoupled. When you use different header files, you can experiment with a different set of pragmas without altering the source code.

- The pragma has an implicit end: the end of the statement (can be a loop) or block. So, no need for pragma restore / endoptimize etc.

Example:

```
#pragma lab1:optimize P
```

```
volatile int v;
```

```
void f( void )
```

```
{
```

```
    int i, a;
```

```
    a = 42;
```

```
lab1: for( i=1; i<10; i++ )
```

```
{
```

```
    /* the entire for loop is part of the pragma optimize */
```

```
    a += i;
```

```
}
```

```
    v = a;
```

```
}
```

Supported pragmas

The compiler recognizes the following pragmas, other pragmas are ignored. On the command line you can use **ctc --help=pragmas** to get a list of all supported pragmas. Pragmas marked with (*) support a label prefix.

STDC FP_CONTRACT [on | off | default | restore] (*)

This pragma is defined in ISO C99/C11. With this pragma you can control the **+contract** flag of [C compiler option --fp-model](#).

alias symbol=defined_symbol

Define *symbol* as an alias for *defined_symbol*. It corresponds to a [.ALIAS](#) directive at assembly level. The *symbol* should not be defined elsewhere, and *defined_symbol* should be defined with static storage duration (not extern or automatic).

align {value | default | restore} (*)

Increase the alignment of variables or functions. If you apply an alignment with a value lower than the default alignment of the variable or function, this has no effect on the alignment of the variable or function. The C compiler issues a warning in that case. When a function is inlined the pragma has no effect on the inlined code, the pragma is ignored. The alignment value must be a power of two or 0. Value 0 defaults to the compiler natural object alignment.

See [C compiler option --align](#).

`assume_if {value | default | restore} (*)`

Assume that the expression of the `if` statement is true or false. You can use this pragma to give branch prediction hints to the C compiler for the branch target alignment optimization. Branch target alignment is one of the execution speed optimizations. Reasonable alignment is based on branch prediction. This pragma affects the subsequent controlling expression of `if` statements from this pragma occurrence until another pragma `assume_if` is encountered. Possible values are:

`true`, the `if` expression is likely to be true

`false`, the `if` expression is unlikely to be true

`undef`, no hints for the `if` expression. This is the default value.

Depending on the branch prediction hints, the C compiler generates alignment (`.align 8` directive) for the respective branch target when the [C compiler option `--branch-target-align`](#) is specified. If the hint has value `undef` the option has no effect.

Example: (bta.c)

```
#if defined(__TASKING__)
    #define LIKELY(expr)    (_Pragma("assume_if true")(expr)_Pragma("assume_if restore"))
    #define UNLIKELY(expr) (_Pragma("assume_if false")(expr)_Pragma("assume_if restore"))
#else
    #define LIKELY(expr)    (expr)
    #define UNLIKELY(expr) (expr)
#endif

void f0(void);
void f1(void);
void f2(void);
void f3(void);

void f4(int n)
{
    f0();
    if (LIKELY(n == 0))
        f1();
    else
        f2();
    f3();
}

void f5(int n)
{
    f0();
    if (UNLIKELY(n == 0))
        f1();
    else
        f2();
    f3();
}
```

```
}  
  
void f6(int n)  
{  
    f0();  
    if (UNLIKELY(n == 0))  
        f1();  
    f3();  
}
```

Invoke the C compiler to see the differences, without branch target alignment:

```
ctc -s bta.c -o bta_off.src
```

or with branch target alignment:

```
ctc --branch-target-align -s bta.c -o bta_on.src
```

boolean [on | off | default | restore] (*)

This pragma is used to mark the macros "false" and "true" from the library header file `stdbool.h` as "essentially BOOLEAN", which is a concept from the MISRA C:2012 standard.

clear / noclear [on | off | default | restore] (*)

By default, uninitialized global or static variables are cleared to zero on startup. With `pragma noclear`, this step is skipped. `Pragma clear` resumes normal behavior. This pragma applies to constant data as well as non-constant data.

See [C compiler option --no-clear](#).

code_core_association {value | default | restore} (*)

Switch to another code core association, where *value* is one of `share`, `privaten` (for core *n*) or `clone`. The code core association of this pragma is assigned to the functions declarations or definitions that follow.

See [C compiler option --code-core-association](#).

This pragma is only supported for multi-core TriCore architectures (`--core=tc1.6.x` or `--core=tc1.6.2`).

compactmaxmatch {value | default | restore} (*)

With this pragma you can control the maximum size of a match.

See [C compiler option --compact-max-size](#).

CPU_TCnum [on | off | default | restore] (*)

Use software workarounds for the specified functional problem. For the list of functional problem numbers see [Chapter 19, CPU Problem Bypasses and Checks](#).

See also [C compiler option --silicon-bug](#).

For example, to enable workarounds for problem CPU_TC.013, specify the following pragma (without the dot):

```
#pragma CPU_TC013
```

data_core_association {*value* | default | restore} (*)

Switch to another data core association, where *value* is one of *share*, *privaten* (for core *n*) or *clone*. The data core association of this pragma is assigned to the data declarations that follow.

See [C compiler option --data-core-association](#).

This pragma is only supported for multi-core TriCore architectures ([--core=tc1.6.x](#) or [--core=tc1.6.2](#)).

default_a0_size [*value*] [default | restore] (*)

With this pragma you can specify a threshold value for `__a0` allocation.

See [C compiler option --default-a0-size \(-Z\)](#).

default_a1_size [*value*] [default | restore] (*)

With this pragma you can specify a threshold value for `__a1` allocation.

See [C compiler option --default-a1-size \(-Y\)](#).

default_near_size [*value*] [default | restore] (*)

With this pragma you can specify a threshold value for `__near` allocation.

See [C compiler option --default-near-size \(-N\)](#).

extension isuffix [on | off | default | restore] (*)

Enables a language extension to specify imaginary floating-point constants. With this extension, you can use an "i" suffix on a floating-point constant, to make the type `_Imaginary`.

```
float 0.5i
```

extern symbol

Normally, when you use the C keyword `extern`, the compiler generates an `.EXTERN` directive in the generated assembly source. However, if the compiler does not find any references to the `extern` symbol in the C module, it optimizes the assembly source by leaving the `.EXTERN` directive out.

With this pragma you can force an external reference (`.EXTERN` assembler directive), even when the *symbol* is not used in the module.

for_constant_data_use_memory *memory*
for_extern_data_use_memory *memory*
for_initialized_data_use_memory *memory*
for_uninitialized_data_use_memory *memory*

Use the specified memory for the type of data mentioned in the pragma name. You can specify the following memories: near, far, a0, a1, a8 or a9.

This pragma overrides the pragmas default_a0_size, default_a1_size, default_near_size.

fp_negzero [on | off | default | restore] (*)

With this pragma you can control the **+negzero** flag of C compiler option **--fp-model**.

fp_nonan [on | off | default | restore] (*)

With this pragma you can control the **+nonan** flag of C compiler option **--fp-model**.

fp_rewrite [on | off | default | restore] (*)

With this pragma you can control the **+rewrite** flag of C compiler option **--fp-model**.

immediate_in_code [on | off | default | restore] (*)

With this pragma you force the compiler to encode all immediate values into instructions.

See C compiler option **--immediate-in-code**.

indirect [on | off | default | restore] (*)

Generates code for indirect function calling.

See C compiler option **--indirect**.

indirect_runtime [on | off | default | restore] (*)

Generates code for indirect calls to run-time functions.

See C compiler option **--indirect-runtime**.

inline / noline / smartinline [default | restore] (*)

See Section 1.11.3, *Inlining Functions: inline*.

inline_max_incr {value | default | restore} (*)

inline_max_size {value | default | restore} (*)

With these pragmas you can control the automatic function inlining optimization process of the compiler. It has effect only when you have enable the inlining optimization ([C compiler option --optimize=+inline](#)).

See [C compiler options --inline-max-incr / --inline-max-size](#).

loop_alignment {value | default | restore} (*)

Specify the alignment loop bodies will get when **--loop=+value** is enabled. Loops are only aligned if the align-loop optimization is enabled and the tradeoff is set to speed (<=2).

See [C compiler option --loop-alignment](#).

macro / nomacro [on | off | default | restore] (*)

Turns macro expansion on or off. By default, macro expansion is enabled.

maxcalldepth {value | default | restore} (*)

With this pragma you can control the maximum call depth. Default is infinite (-1).

See [C compiler option --max-call-depth](#).

message "message" ...

Print the message string(s) on standard output.

nomisrac [nr,...] [default | restore] (*)

Without arguments, this pragma disables MISRA C checking. Alternatively, you can specify a comma-separated list of MISRA C rules to disable.

See [C compiler option --misrac](#) and [Section 4.7.2, C Code Checking: MISRA C](#).

object_comment "string" ... | default | restore (*)

This pragma generates a .comment section in the assembly file with the specified string. After assembling, this string appears in the generated .o or .elf object file. If you specify this pragma more than once in the same module, only the last pragma has effect.

See [C compiler option --object-comment](#).

optimize [flags] / endoptimize [default | restore] (*)

You can overrule the C compiler option **--optimize** for the code between the pragmas optimize and endoptimize. The pragma works the same as [C compiler option --optimize](#).

See [Section 4.6, Compiler Optimizations](#).

pack {0 | 2 | default | restore} (*)

Specifies packing of structures. See [Section 1.1.3, Packed Data Types](#).

profile [flags] / endprofile [default | restore] (*)

Control the profile settings. The pragma works the same as [C compiler option --profile](#). Note that this pragma will only be checked at the start of a function. `endprofile` switches back to the previous profiling settings.

profiling [on | off | default | restore] (*)

If profiling is enabled on the command line ([C compiler option --profile](#)), you can disable part of your source code for profiling with the pragmas `profiling off` and `profiling`.

protect / endprotect [on | off | default | restore] (*)

With these pragmas you can protect sections against linker optimizations. This excludes a section from unreferenced section removal and duplicate section removal by the linker. `endprotect` restores the default section protection.

runtime [flags | default | restore] (*)

With this pragma you can control the generation of additional code to check for a number of errors at run-time. The pragma argument syntax is the same as for the arguments of the [C compiler option --runtime](#). You can use this pragma to control the run-time checks for individual statements. In addition, objects declared when the "bounds" sub-option is disabled are not bounds checked. The "malloc" sub-option cannot be controlled at statement level, as it only extracts an alternative malloc implementation from the library.

section all ["name" | default | restore] (*)

section type ["name" | default | restore] (*)

section_name_with_module [on | off | default | restore] (*)

section_name_with_symbol [on | off | default | restore] (*)

Changes section names. See [Section 1.12, Compiler Generated Sections](#) and [C compiler option --rename-sections](#) for more information.

section code_init | const_init | vector_init

At startup copies corresponding sections to RAM for initialization. See [Section 1.12.2, Influence Section Definition](#).

section data_overlay

Allow overlaying data sections.

source / nosource [on | off | default | restore] (*)

With these pragmas you can choose which C source lines must be listed as comments in assembly output.

See [C compiler option --source](#).

stdinc [on | off | default | restore] (*)

This pragma changes the behavior of the `#include` directive. When set, the C compiler options `--include-directory` and `--no-stdinc` are ignored.

switch auto | jumptab | linear | lookup | default | restore (*)

With these pragmas you can overrule the C compiler chosen switch method.

See [Section 1.10, Switch Statement](#) and [C compiler option --switch](#).

tradeoff /level [default | restore] (*)

Specify tradeoff between speed (0) and size (4). See [C compiler option --tradeoff](#)

unroll_factor value / endunroll_factor [default | restore] (*)

Specify how many times the following loop should be unrolled, if possible. At the end of the loop use `endunroll_factor`.

See [C compiler option --unroll-factor](#).

user_mode user-0 | user-1 | kernel | default | restore (*)

With this pragma you specify the user mode (I/O privilege mode) the TriCore runs in.

See [C compiler option --user-mode](#).

warning [number,...] [default | restore] (*)

With this pragma you can disable warning messages. If you do not specify a warning number, all warnings will be suppressed.

weak symbol

Mark a symbol as "weak" (`.WEAK` assembler directive). The symbol must have external linkage, which means a global or external object or function. A static symbol cannot be declared weak.

A weak external reference is resolved by the linker when a global (or weak) definition is found in one of the object files. However, a weak reference will not cause the extraction of a module from a library to resolve the reference. When a weak external reference cannot be resolved, the null pointer is substituted.

A weak definition can be overruled by a normal global definition. The linker will not complain about the duplicate definition, and ignore the weak definition.

1.9. Predefined Preprocessor Macros

You can use the following predefined macros in your C source. The macros are useful to create conditional C code.

Macro	Description
__BUILD__	Identifies the build number of the compiler in the format yymmddqq (year, month, day and quarter in UTC).
__CORE_core__	A symbol is defined depending on the option --core=core . The <i>core</i> is converted to uppercase and '.' is removed. For example, if --core=tc1.3.1 is specified, the symbol <code>__CORE_TC131__</code> is defined. When no --core is supplied, the compiler defines <code>__CORE_TC13__</code> .
__CTC__	Identifies the compiler. You can use this symbol to flag parts of the source which must be recognized by the TASKING ctc compiler only. It expands to 1.
__CPU__	Expands to the name of the CPU supplied with the control program option --cpu=cpu . When no --cpu is supplied, or when you do not use the control program, this symbol is not defined. For example, if --cpu=tc1796b is specified to the control program, the symbol <code>__CPU__</code> expands to <code>tc1796b</code> .
__CPU_cpu__	A symbol is defined depending on the control program option --cpu=cpu . The <i>cpu</i> is converted to uppercase. For example, if --cpu=tc1796b is specified to the control program, the symbol <code>__CPU_TC1796B__</code> is defined. When no --cpu is supplied, or when you do not use the control program, this symbol is not defined.
__DATE__	Expands to the compilation date: "mmm dd yyyy".
__DOUBLE_FP__	Expands to 1 if you used option --fp-model=float , otherwise unrecognized as macro.
__DSPC__	Indicates conformation to the DSP-C standard. It expands to 1.
__DSPC_VERSION__	Expands to the decimal constant 200001L.
__FILE__	Expands to the current source file name.
__FPU__	Expands to 1 when the option --fp-model=+soft is not used. Otherwise unrecognized as a macro.
__LINE__	Expands to the line number of the line where this macro is called.
__MISRAC_VERSION__	Expands to the MISRA C version used 1998, 2004 or 2012 (option --misrac-version). The default is 2004.
__PRECISE_LIB_FP__	Expands to 1 when the option --fp-model=-fastlib is used. The compiler uses precise library functions for certain floating-point operations. Otherwise unrecognized as a macro.
__PROF_ENABLE__	Expands to 1 if profiling is enabled, otherwise expands to 0.
__REVISION__	Expands to the revision number of the compiler. Digits are represented as they are; characters (for prototypes, alphas, betas) are represented by -1. Examples: v1.0r1 -> 1, v1.0rb -> -1

Macro	Description
<code>__SFRFILE__(cpu)</code>	If control program option <code>--cpu=cpu</code> is specified, this macro expands to the filename of the used SFR file, including the pathname and the < >. The <i>cpu</i> is the argument of the macro. For example, if <code>--cpu=tc1796b</code> is specified, the macro <code>__SFRFILE__(__CPU__)</code> expands to <code>__SFRFILE__(tc1796b)</code> , which expands to <code><sfr/regtc1796b.sfr></code> .
<code>__SINGLE_FP__</code>	Expands to 1 if you used option <code>--fp-model=+float</code> (Treat 'double' as 'float'), otherwise unrecognized as macro.
<code>__STDC__</code>	Identifies the level of ANSI standard. The macro expands to 1 if you set option <code>--language</code> (Control language extensions), otherwise expands to 0.
<code>__STDC_HOSTED__</code>	Always expands to 0, indicating the implementation is not a hosted implementation.
<code>__STDC_NO_ATOMICS__</code>	(C11 only) Expands to 0 for TriCore1.6.x (<code>--core=tc1.6.x</code>) and TriCore1.6.2 (<code>--core=tc1.6.2</code>) to indicate that this implementation supports atomic types and the <code>stdatomic.h</code> header file. Expands to 1 for other TriCore cores, indicating that atomic is not supported.
<code>__STDC_NO_THREADS__</code>	(C11 only) Expands to 1 to indicate that this implementation does not support the <code>threads.h</code> header file.
<code>__STDC_VERSION__</code>	Identifies the ISO-C version number. Expands to 201112L for ISO C11, 199901L for ISO C99 or 199409L for ISO C90.
<code>__TASKING__</code>	Identifies the compiler as a TASKING compiler. Expands to 1 if a TASKING compiler is used.
<code>__TIME__</code>	Expands to the compilation time: "hh:mm:ss"
<code>__VERSION__</code>	Identifies the version number of the compiler. For example, if you use version 6.3r1 of the compiler, <code>__VERSION__</code> expands to 6003 (dot and revision number are omitted, minor version number in 3 digits).

Example

```
#ifdef __FPU__
/* this part is only valid if an FPU is present */
...
#endif
```

1.10. Switch Statement

The TASKING C compiler supports three ways of code generation for a switch statement: a jump chain (linear switch), a jump table or a lookup table.

A *jump chain* is comparable with an if/else-if/else-if/else construction. A *jump table* table filled with target addresses for each possible switch value. The switch argument is used as an index within this table. A *lookup table* is a table filled with a value to compare the switch argument with and a target address to jump to. A binary search lookup is performed to select the correct target address.

By default, the compiler will automatically choose the most efficient switch implementation based on code and data size and execution speed. With the [C compiler option `--tradeoff`](#) you can tell the compiler to put more emphasis on speed than on ROM size.

Especially for large switch statements, the jump table approach executes faster than the lookup table approach. Also the jump table has a predictable behavior in execution speed: independent of the switch argument, every case is reached in the same execution time. However, when the case labels are distributed far apart, the jump table becomes sparse, wasting code memory. The compiler will not use the jump table method when the waste becomes excessive.

With a small number of cases, the jump chain method can be faster in execution and shorter in size.

Note that a jump table or lookup table is part of a function and as such is considered code instead of data.

How to overrule the default switch method

You can overrule the compiler chosen switch method by using a pragma:

```
#pragma switch linear    force jump chain code
#pragma switch jumptab   force jump table code
#pragma switch lookup    force lookup table code
#pragma switch auto      let the compiler decide the switch method used (this is the default)
#pragma switch restore   restore previous switch method
```

The switch pragmas must be placed before the switch statement. Nested switch statements use the same switch method, unless the nested switch is implemented in a separate function which is preceded by a different switch pragma.

Example:

```
/* place pragma before function body */

#pragma switch jumptab

void test(unsigned char val)
{ /* function containing the switch */
    switch (val)
    {
        /* use jump table */
    }
}
```

On the command line you can use [C compiler option `--switch`](#).

1.11. Functions

1.11.1. Calling Convention

Parameter passing

A lot of execution time of an application is spent transferring parameters between functions. The fastest parameter transport is via registers. Therefore, function parameters are first passed via registers. If no more registers are available for a parameter, the compiler pushes parameters on the stack.

Registers available for parameter passing are D4, D5, E4, D6, D7, E6, A4, A5, A6, A7. Up to 4 arithmetic types and 4 pointers can be passed this way. A 64-bit argument is passed in an even/odd data register pair. Parameter registers skipped because of alignment for a 64-bit argument are used by subsequent 32-bit arguments. Any remaining function arguments are passed on the stack. Stack arguments are pushed in reversed order, so that the first one is at the lowest address. On function entry, the first stack parameter is at the address (SP+0).

Structures and unions up to eight bytes are passed via a data register or data register pair. Larger structures/unions are passed via the stack.

All function arguments passed on the stack are aligned on a multiple of 4 bytes. As a result, the stack offsets for all types except float are compatible with the stack offsets used by a function declared without a prototype.

Examples:

```
void func1( int i, char * p, char c );      /* D4 A4 D5 */
void func2( int i1, double d, int i2 );    /* D4 E6 D5 */
void func3( char c1, char c2, char c3[] ); /* D4 D5 A4 */
void func4( double d1, int i1, double d2, int i2 );
                                           /* E4 D6 stack D7 */
```

Function return values

The C compiler uses registers to store C function return values, depending on the function return types.

Return Type	Register
Arithmetic, structure or union <= 32 bits	D2
Arithmetic, structure or union <= 64 bits	D2/D3 (E2)
Pointer	A2
Circular pointer	A2/A3

When the function returns an arithmetic, structure or union type larger than 64 bits, it is copied to a "return area" that is allocated by the caller. The address of this area is passed as an implicit first argument in A4.

Stack usage

The user stack on TriCore derivatives is used for parameter passing and the allocation of automatic and temporary storage. The stack grows from higher addresses to lower addresses. The stack pointer (SP)

points to the bottom (low address) of the stack frame. The stack pointer alignment is 8 bytes. For more information about the stack and frame layout refer to section 2.2.2 *Stack Frame Management* in the EABI.

Stack model: `__stackparm`

The function qualifier `__stackparm` changes the standard calling convention of a function into a convention where all function arguments are passed via the stack, conforming a so-called stack model. This qualifier is only needed for situations where you need to use an indirect call to a function for which you do not have a valid prototype.

Note that the TASKING TriCore compiler deviates from the EABI at this point. The EABI states that objects larger than 64 bits must be passed via a pointer and a copy of the object is not necessary. This is dangerous, because the user is then responsible for the copy object (if required). Therefore, the TASKING TriCore compiler places ALL arguments on the stack.

The compiler sets the least significant bit of the function pointer when you take the address of a function declared with the `__stackparm` qualifier, so that these function pointers can be identified at run-time. The least significant bit of a function pointer address is ignored by the hardware.

Example:

```
void          plain_func ( int );
void __stackparm stack_func ( int );

void call_indirect ( unsigned int fp, int arg )
{
    typedef __stackparm void (*SFP)( int );
    typedef void (*RFP)( int );

    SFP    fp_stack;
    RFP    fp_reg;

    if ( fp & 1 )
    {
        fp_stack = (SFP) fp;
        fp_stack( arg );
    }
    else
    {
        fp_reg = (RFP) fp;
        fp_reg( arg );
    }
}

void main ( void )
{
    call_indirect( (unsigned int) plain_func, 1 );
    call_indirect( (unsigned int) stack_func, 2 );
}
```


Function calling modes: `__indirect`

Functions are by default called with a single word direct call. However, when you link the application and the target address appears to be out of reach (+/- 16 MB from the `call` or `j` instruction), the linker generates an error. In this case you can use the `__indirect` keyword to force the less efficient, two and a half word indirect call to the function:

```
int __indirect foo( void )
{
    ...
}
```

With C compiler option `--indirect` you tell the C compiler to generate far calls for *all* functions.

1.11.2. Register Usage

The C compiler uses the data registers and address registers according to the convention given in the following table.

Register		Usage	Register	Usage
D0	E0	scratch	A0	global
D1		scratch	A1	global
D2	E2	return register for arithmetic types and struct/union	A2	return register for pointers
D3		most significant part of 64 bit result	A3	scratch
D4	E4	parameter passing	A4	parameter passing
D5		parameter passing	A5	parameter passing
D6	E6	parameter passing	A6	parameter passing
D7		parameter passing	A7	parameter passing
D8	E8	saved register	A8	global
D9		saved register	A9	global
D10	E10	saved register	A10	stack pointer
D11		saved register	A11	link register
D12	E12	saved register	A12	saved register
D13		saved register	A13	saved register
D14	E14	saved register	A14	saved register
D15		saved register, implicit pointer	A15	saved register, implicit pointer

1.11.3. Inlining Functions: `inline`

With the C compiler option `--optimize=+inline`, the C compiler automatically inlines small functions in order to reduce execution time (smart inlining). The compiler inserts the function body at the place the function is called. The C compiler decides which functions will be inlined. You can overrule this behavior with the two keywords `inline` (ISO-C) and `__noinline`.

With the `inline` keyword you force the compiler to inline the specified function, regardless of the optimization strategy of the compiler itself:

```
inline unsigned int abs(int val)
{
    unsigned int abs_val = val;
    if (val < 0) abs_val = -val;
    return abs_val;
}
```

If a function with the keyword `inline` is not called at all, the compiler does not generate code for it.

You must define inline functions in the same source module as in which you call the function, because the compiler only inlines a function in the module that contains the function definition. When you need to call the inline function from several source modules, you must include the definition of the inline function in each module (for example using a header file).

With the `__noinline` keyword, you prevent a function from being inlined:

```
__noinline unsigned int abs(int val)
{
    unsigned int abs_val = val;
    if (val < 0) abs_val = -val;
    return abs_val;
}
```

Using pragmas: `inline`, `noinline`, `smartinline`

Instead of the `inline` qualifier, you can also use `#pragma inline` and `#pragma noinline` to inline a function body:

```
#pragma inline
unsigned int abs(int val)
{
    unsigned int abs_val = val;
    if (val < 0) abs_val = -val;
    return abs_val;
}
#pragma noinline
void main( void )
{
    int i;
    i = abs(-1);
}
```

If a function has an `inline`/`__noinline` function qualifier, then this qualifier will overrule the current pragma setting.

With the `#pragma noinline`/`#pragma smartinline` you can temporarily disable the default behavior that the C compiler automatically inlines small functions when you turn on the [C compiler option `--optimize=+inline`](#).

With the C compiler options `--inline-max-incr` and `--inline-max-size` you have more control over the automatic function inlining process of the compiler.

Combining inline with `__asm` to create intrinsic functions

With the keyword `__asm` it is possible to use assembly instructions in the body of an inline function. Because the compiler inserts the (assembly) body at the place the function is called, you can create your own intrinsic function. See [Section 1.11.5, *Intrinsic Functions*](#).

1.11.4. Interrupt and Trap Functions

The TriCore C compiler supports a number of function qualifiers and keywords to program interrupt service routines (ISR) or trap handlers. Trap handlers may also be defined by the operating system if your target system uses one.

An *interrupt service routine* (or: interrupt function, or: interrupt handler) is called when an interrupt event (or: *service request*) occurs. This is always an external event; peripherals or external inputs can generate an interrupt signals to the CPU to request for service. Unlike other interrupt systems, each interrupt has a unique *interrupt request priority number* (IRPN). This number (0 to 255) is set as the pending interrupt priority number (PIPN) in the interrupt control register (ICR) by the interrupt control unit. If multiple interrupts occur at the same time, the priority number of the request with the highest priority is set, so this interrupt is handled.

The TriCore vector table provides an entry for each pending interrupt priority number, not for a specific interrupt source. A request is handled if the priority number is higher than the CPU priority number (CCPN). An interrupt service routine can be interrupted again by another interrupt request with a higher priority. Interrupts with priority number 0 are never handled.

A trap service routine (or: trap function, or: trap handler) is called when a trap event occurs. This is always an event generated within or by the application. For example, a divide by zero or an invalid memory access.

Overview of function qualifiers

With the following function qualifiers you can declare an interrupt handler or trap handler:

```
__interrupt()    __interrupt_fast()
__interrupt8()  __interrupt8_fast()
__trap()        __trap_fast()
__vector_table()
```

There is one special type of trap function which you can call manually, the system call exception (trap class 6). See [Section 1.11.4.3, *Defining a Trap Service Routine Class 6: `\_\_syscallfunc\(\)`*](#).

```
__syscallfunc()
```

During the execution of an interrupt service routine or trap service routine, the system blocks the CPU from taking further interrupt requests. With the following keywords you can enable interrupts again, immediately after an interrupt or trap function is called:

```
__enable_        __bistr_()
```

Multi-core interrupt/trap vector table number

For TriCore 1.6.x or 1.6.2 derivatives that support multiple cores, an interrupt vector table and trap vector table is present for each core. These vector tables are defined in the LSL files `inttab $nr$ .ls1` and `traptab $nr$ .ls1`. The core vector table number nr corresponds to the TriCore core used. The default core is `tc0` and therefore, the default vector table number is 0.

The compiler generates a vector table entry for an interrupt function or trap function. With the interrupt function qualifier `__vector_table` you can assign it to one or more core vector table.

Syntax:

```
__vector_table(vector_table_number, ...)
```

When you do not specify `__vector_table` for an interrupt function or trap function, the default vector table number 0 is used for functions with a clone or share code association. See also [Section 1.4.4.2, Code Core Association](#).

You do not have to specify a vector table number for an interrupt function or trap function with a private code core association, the vector table corresponds to the private core number association. When you do specify a vector table number for an interrupt function or trap function with a private code core association, the number must correspond to the private core number association.

Fast interrupt functions or fast trap functions are only allowed for functions that have a share code core association and can only be assigned to one vector table.

Restrictions of `__vector_table`:

- `__vector_table` is only allowed for multi-core TriCore derivatives.
- `__vector_table` is only allowed for (fast) interrupt functions or trap qualified functions.
- `__vector_table` does not accept more vector table numbers than defined by the number of cores in the TriCore architecture.
- `__vector_table` does not accept duplicate vector table numbers.

1.11.4.1. Defining an Interrupt Service Routine: `__interrupt()`, `__interrupt_fast()`, `__interrupt8()`, `__interrupt8_fast()`

With the function type qualifier `__interrupt()` you can declare a function as an interrupt service routine. The function type qualifier `__interrupt()` takes one interrupt vector (0..255) as argument.

Interrupt functions cannot return anything and must have a void argument type list:

```
void __interrupt(vector) [__vector_table(nr, ...)]
isr( void )
{
    ...
}
```

The argument *vector* identifies the entry into the interrupt vector table (0..255). Unlike other interrupt systems, the priority number (PIN) of the interrupt now being serviced by the CPU identifies the entry into the vector table.

For an extensive description of the TriCore interrupt system, see the *TriCore 1 Unified Processor Core v1.3 Architecture Manual, Doc v1.3.3* [2002-09, Infineon].

The compiler generates an interrupt service frame for interrupts. The difference between a normal function and an interrupt function is that an interrupt function ends with an RFE instruction instead of a RET, and that the lower context is saved and restored with a pair of SVLCX / RSLCX instructions when one of the lower context registers is used in the interrupt handler.

When you define an interrupt service routine with the `__interrupt()` qualifier, the compiler generates an entry for the interrupt vector table. This vector jumps to the interrupt handler.

The compiler puts the interrupt vectors in sections with the following naming convention:

```
.text[.inttabnr].intvec.vector
```

The optional `.inttabnr` is generated when one of the cores of the TriCore 1.6.x or 1.6.2 is selected. The core vector table number *nr* corresponds to the TriCore core used. You can specify a core vector table by using the interrupt function qualifier `__vector_table`.

The following example illustrates the function definition for a function for a software interrupt with vector number 0x30:

```
int c;

void __interrupt( 0x30 ) __vector_table( 1 ) transmit( void )
{
    c = 1;
}
```

This results in a section called `".text.inttab1.intvec.030"`.

8 byte vector table entry support (TriCore 1.6.x and 1.6.2 only)

For the TriCore 1.6.x and 1.6.2 an entry in the vector table can be 32 bytes or 8 bytes. For 32 byte entries you can use `__interrupt()` as explained above. For 8 byte vector table entries you can use function type qualifier `__interrupt8()`.

```
void __interrupt8(vector) [__vector_table(nr,...)]
isr8( void )
{
    ...
}
```

An absolute jump instruction is generated to the interrupt service routine, which restricts the address range to absolute 24. Loading a 32-bit address and jumping indirectly does not fit in an 8 byte vector.

The compiler puts the 8 byte interrupt vectors in sections with the following naming convention:

```
.text[.inttabnr].intvec8.vector
```

8 byte and 32 byte spacing is available at the same time, no LSL configuration is required. Mixing 8 byte and 32 byte spacing on the same core is not possible, but different cores can use different spacings. You define at compile which kind of spacing is required.

The vector spacing is configured at startup per core in the Base Interrupt Vector (BIV) with startup macro `__BIV_8BYTE_INIT` (see `cstart.c`). It is your responsibility that this is conform the spacing required by the interrupt functions, because the compiler cannot check if usage of interrupt functions qualifiers corresponds with the BIV configuration. In Eclipse you can set this macro as follows:

1. From the **Project** menu, select **Properties for**

The Properties dialog appears.

2. In the left pane, expand **C/C++ Build** and select **Startup Configuration**.

In the right pane the Startup Configuration page appears.

3. Enable the option **Initialize 8 byte spacing interrupt vector table**.

4. Click **OK**.

The file `cstart.h` in your project is updated with the new value.

Fast interrupts

When you define an interrupt service routine with the `__interrupt_fast()` qualifier, the interrupt handler is directly placed in the interrupt vector table, thereby eliminating the jump code. There is only 32 bytes of space available for an entry in the vector table, but the compiler does not check this restriction. Fast interrupts can span more than one vector. Fast interrupts are only restricted to one entry when the next interrupt vector is also occupied. The linker generates an error when the fast interrupt does not fit or overlaps with another vector or interrupt.

For the TriCore 1.6.x and 1.6.2 an entry in the vector table can be 32 bytes or 8 bytes using Base Interrupt Vector 0 (BIV[0]). For 8 byte fast interrupts you use the `__interrupt8_fast()` qualifier.

1.11.4.2. Defining a Trap Service Routine: `__trap()`, `__trap_fast()`

The definition of a trap service routine is similar to the definition of an interrupt service routine. Trap functions cannot accept arguments and do not return anything:

```
void __trap( class ) [__vector_table(nr,...)]
tsr( void )
{
    ...
}
```

The argument `class` identifies the entry into the trap vector table. TriCore defines eight classes of trap functions. Each class has its own trap handler.

When a trap service routine is called, the d15 register contains the so-called *Trap Identification Number* (TIN). This number identifies the cause of the trap. In the trap service routine you can test and branch on the value in d15 to reach the sub-handler for a specific TIN. With the intrinsic function `__get_tin()` you can use the TIN anywhere in your code.

The following table shows the classes supported by TriCore.

Class	Description
Class 0	Reset
Class 1	Internal Protection Traps
Class 2	Instruction Errors
Class 3	Context Management
Class 4	System Bus and Peripheral Errors
Class 5	Assertion Traps
Class 6	System Call
Class 7	Non-Maskable Interrupt

For a complete overview of the trap system and the meaning of the trap identification numbers, see the *TriCore 1 Unified Processor Core v1.3 Architecture Manual, Doc v1.3.3* [2002-09, Infineon]

Analogous to interrupt service routines, the compiler generates a trap service frame for interrupts.

When you define a trap service routine with the `__trap()` qualifier, the compiler generates an entry for the interrupt vector table. This vector jumps to the trap handler.

The compiler puts the trap vectors in sections with the following naming convention:

```
.text[.traptab{0|1|2}].trapvec.class
```

The optional `.traptab0`, `.traptab1` or `.traptab2` is generated when one of the cores of the TriCore 1.6.x or 1.6.2 is associated with the trap vector. You can specify a core vector table by using the interrupt function qualifier `__vector_table`.

The following example illustrates the function definition for a reset trap:

```
int c;

void __trap( 0 ) __vector_table( 1 ) rst( void )
{
    c = 1;
}
```

This results in a section called `".text.traptab1.trapvec.000"`.

Fast traps

When you define a trap service routine with the `__trap_fast()` qualifier, the trap handler is directly placed in the trap vector table, thereby eliminating the jump code. You should only use this when the trap

handler is very small, as there is only 32 bytes of space available in the vector table. The compiler does not check this restriction.

1.11.4.3. Defining a Trap Service Routine Class 6: `__syscallfunc()`

A special kind of trap service routine is the system call trap. With a system call the trap service routine of class 6 is called. For the system call trap, the trap identification number (TIN) is taken from the immediate constant specified with the function qualifier `__syscallfunc()`:

```
__syscallfunc(TIN)
```

The TIN is a value in the range 0 and 255. You can only use `__syscallfunc()` in the function declaration. A function body is useless, because when you call the function declared with `__syscallfunc()`, a trap class 6 occurs which calls the corresponding trap service routine.

In case of the other traps, when a trap service routine is called, the system places a trap identification number in d15.

Unlike the other traps, a class 6 trap service routine can contain arguments and return a value (the lower context is not saved and restored). Arguments that are passed via the stack, remain on the stack of the caller because it is not possible to pass arguments from the user stack to the interrupt stack on a system call. This restriction, caused by the TriCore's run-time behavior, cannot be checked by the compiler.

Example

The following example illustrates the definition of a class 6 trap service routine and the corresponding system call:

```
#pragma alias syscall_a=trap6
#pragma alias syscall_b=trap6

__syscallfunc(1) int syscall_a( int, int );
__syscallfunc(2) int syscall_b( int, int );

int x;

void main( void )
{
    x = syscall_a(1,2);    // causes a trap class 6 with TIN = 1
    x = syscall_b(4,3);    // causes a trap class 6 with TIN = 2
}

int __trap( 6 ) trap6( int a, int b ) // trap class 6 handler
{
    int tin;
    tin = __get_tin(); // get the TIN

    switch( tin )
    {
    case 1:
```



```

        a += b;
        break;
case 2:
    a -= b;
    break;
default:
    break;
}
return a;
}

```

1.11.4.4. Enabling Interrupt Requests: `__enable_`, `__bistr_()`

Enabling interrupt service requests

During the execution of an interrupt service routine or trap service routine, the system blocks the CPU from taking further interrupt requests. You can immediately re-enable the system to accept interrupt requests:

```

__interrupt(vector) __enable_ isr( void )
__trap(class) __enable_ tsr( void )

```

The compiler generates an `enable` instruction as the first instruction in the routine. The `enable` instruction sets the interrupt enable bit (ICR.IE) in the interrupt control register.

You can also generate the `enable` instruction with the intrinsic function `__enable()`, but it is not guaranteed that it will be the first instruction in the routine.

Enabling interrupt service requests and setting CPU priority number

The function qualifier `__bistr_()` also re-enables the system to accept interrupt requests. In addition, the *current CPU priority number* (CCPN) in the interrupt control register is set:

```

__interrupt(vector) __bistr_(CCPN) isr( void )
__trap(class) __bistr_(CCPN) tsr( void )

```

The argument *CCPN* is a number between 0 and 255. The system accepts all interrupt requests that have a higher pending interrupt priority number (PIP_N) than the current CPU priority number. So, if the CPU priority number is set to 0, the system accepts all interrupts. If it is set to 255, no interrupts are accepted.

The compiler generates a `bistr` instruction as the first instruction in the routine. The `bistr` instruction sets the interrupt enable bit (ICR.IE) and the current CPU priority number (ICR.CCPN) in the interrupt control register.

You can also generate the `bistr` instruction with the intrinsic function `__bistr()`, but it is not guaranteed that it will be the first instruction in the routine.

Setting the CPU priority number in a Class 6 trap service routine

The `bistr` instruction saves the lower context so passing and returning arguments is not possible. Therefore, you cannot use the function qualifier `__bistr_()` for class 6 traps.

Instead, you can use the function qualifier `__enable__` to set the ICR.IE bit, and the intrinsic function `__mtcr( int, int )` to set the ICR.CCPN value at the beginning of a class 6 trap service routine (or use the intrinsic function `__mtcr( )` to set both the ICR.IE bit and the ICR.CCPN value).

1.11.4.5. Single Entry Vector Table for TriCore 1.6.x and 1.6.2

For the TriCore 1.6.x and 1.6.2 you can reduce the vector table to a single entry by masking the PIPN.

A minimum vector table can be configured if the BIV masks the PIPN so that any interrupt address calculation results in the same address.

For example in `cstart.c`:

```
__mtcr(BIV, (unsigned int)(_lc_u_int_tab) | (0xff<<3) | 1 );
```

This configures the BIV register to use a common, single entry where a function interrupt handler is located to branch to the specific interrupt routine by using an array of function pointers. A pointer to an array is used to switch the array quickly.

The C library contains functions to support Single Entry Vector Table (SEVT) for TriCore 1.6.x and 1.6.2. Interrupt Service Routines can be installed in the SEVT ISR array for each core, using `_sevt_isr_install`. For example, install C function `blink( )` with Interrupt Request Priority Number (IRPN) 1 on core `tc0`.

```
#include <sevt.h>
```

```
extern void blink( void );
_sevt_isr_install( 1, &blink, 0 );
```

The SEVT ISR handler indirectly calls the functions installed in the SEVT data array. The SEVT ISR handler is located at interrupt vector table entry 64. The SEVT ISR handler and SEVT data array are supported by `_sevt_isr_tc0|1|2( )` and `_sevt_isr_tc0|1|2[ ]` in C library module `sevt.c`. The SEVT data array can be switched with `_sevt_isr_install_array( )`. SEVT can be enabled by `cstart` macro `__BIV_SINGLE_INIT` (see files `cstart*.h`).

1.11.5. Intrinsic Functions

Some specific assembly instructions have no equivalence in C. *Intrinsic functions* give the possibility to use these instructions. Intrinsic functions are predefined functions that are recognized by the compiler. The compiler generates the most efficient assembly code for these functions.

The compiler always inlines the corresponding assembly instructions in the assembly source (rather than calling it as a function). This avoids parameter passing and register saving instructions which are normally necessary during function calls.

Intrinsic functions produce very efficient assembly code. Though it is possible to inline assembly code by hand, intrinsic functions use registers even more efficiently. At the same time your C source remains very readable.

You can use intrinsic functions in C as if they were ordinary C (library) functions. All intrinsics begin with a double underscore character (`__`).

The following example illustrates the use of an intrinsic function and its resulting assembly code.

```
x = __min( 4,5 );
```

The resulting assembly code is inlined rather than being called:

```
mov16    d2, #4
min      d2, d2, #5
```

The intrinsics cover the following subjects:

- Minimum and maximum of (short) integers
- Fractional data type support
- Packed data type support
- Interrupt handling
- Insert single assembly instruction
- Register handling
- Insert / extract bit-fields and bits
- Atomic support
- Miscellaneous

Writing your own intrinsic function

Because you can use any assembly instruction with the `__asm( )` keyword, you can use the `__asm( )` keyword to create your own intrinsic functions. The essence of an intrinsic function is that it is inlined.

1. First write a function with assembly in the body using the keyword `__asm( )`. See [Section 1.6, Using Assembly in the C Source: `\_\_asm\(\)`](#)
2. Next make sure that the function is inlined rather than being called. You can do this with the function qualifier `inline`. This qualifier is discussed in more detail in [Section 1.11.3, Inlining Functions: `inline`](#).

```
int a, b, result;
```

```
inline void __my_mul( void )
{
    __asm( "mul %0, %1, %2": "=d"(result): "d"(a), "d"(b) );
}
```

```
void main(void)
{
    // call to function __my_mul
    __my_mul();
}
```

Generated assembly code:

```
main:
    ; __my_mul code is inlined here
    ld.w  d15, a
    ld.w  d0, b
    mul   d15, d15, d0
    st.w  result, d15
```

As you can see, the generated assembly code for the function `__my_mul` is inlined rather than called.

1.11.5.1. Minimum and Maximum of (Short) Integers

The following table provides an overview of the intrinsic functions that return the minimum or maximum of a signed integer, unsigned integer or short integer.

Intrinsic Function	Description
<code>int __min(int, int);</code>	Return minimum of two integers
<code>short __mins(short, short);</code>	Return minimum of two short integers
<code>unsigned int __minu(unsigned int, unsigned int);</code>	Return minimum of two unsigned integers
<code>int __max(int, int);</code>	Return maximum of two integers
<code>short __maxs(short, short);</code>	Return maximum of two short integers
<code>unsigned int __maxu(unsigned int, unsigned int);</code>	Return maximum of two unsigned integers

1.11.5.2. Fractional Arithmetic Support

The following table provides an overview of intrinsic functions to convert fractional values. Note that the TASKING VX-toolset C compiler for TriCore fully supports the fractional type so normally you should not need these intrinsic functions (except for `__mulfractlong`). For compatibility reasons the TASKING C compiler does support these functions.

Conversion of fractional values

Intrinsic Function	Description
<code>long __mulfractlong(__fract, long);</code>	Integer part of the multiplication of a <code>__fract</code> and a <code>long</code>
<code>__sfract __round16(__fract);</code>	Convert <code>__fract</code> to <code>__sfract</code>
<code>__fract __getfract(__laccum);</code>	Convert <code>__laccum</code> to <code>__fract</code>
<code>short __clssf(__sfract);</code>	Count the consecutive number of bits that have the same value as bit 15 of an <code>__sfract</code>
<code>__sfract __shasfracts(__sfract, int);</code>	Left/right shift of an <code>__sfract</code>
<code>__fract __shafracts(__fract, int);</code>	Left/right shift of an <code>__fract</code>
<code>__laccum __shaaccum(__laccum, int);</code>	Left/right shift of an <code>__laccum</code>

Intrinsic Function	Description
<code>__sfract __mac_sf(__sfract a, __sfract b, __sfract c);</code>	Multiply-add <code>__sfract</code> . Returns $(a + b * c)$
<code>__sfract __mac_r_sf(__sfract, __sfract, __sfract);</code>	Multiply-add with rounding. Returns the rounded result of $(a + b * c)$
<code>__sfract __u16_to_sfract(unsigned short integer);</code>	Convert unsigned short to <code>__sfract</code>
<code>__sfract __s16_to_sfract(signed short integer);</code>	Convert signed short to <code>__sfract</code>
<code>unsigned short int __sfract_to_u16(__sfract);</code>	Convert <code>__sfract</code> to unsigned short
<code>signed short int __sfract_to_s16(__sfract);</code>	Convert <code>__sfract</code> to signed short

1.11.5.3. Packed Data Type Support

The following table provides an overview of the intrinsic functions for initialization of packed data type.

Initialize packed data types

Intrinsic Function	Description
<code>__packb __initpackbl(long);</code>	Initialize <code>__packb</code> with a long integer
<code>__packb __initpackb(int, int, int, int);</code>	Initialize <code>__packb</code> with four integers
<code>unsigned __packb __initupackb(unsigned, unsigned, unsigned, unsigned);</code>	Idem, but unsigned
<code>__packhw __initpackhl(long);</code>	Initialize <code>__packhw</code> with a long integer
<code>__packhw __initpackhw(short, short);</code>	Initialize <code>__packhw</code> with two integers
<code>unsigned __packhw __initupackhw(unsigned short, unsigned short);</code>	Idem, but unsigned

Extract values from packed data types

The following table provides an overview of the intrinsic functions to extract a single byte or halfword from a `__packb` or `__packhw` data type.

Intrinsic Function	Description
<code>char __extractbyte1(__packb);</code>	Extract first byte from a <code>__packb</code>
<code>unsigned char __extractbyte1(__unsigned packb);</code>	Idem, but unsigned
<code>char __extractbyte2(__packb);</code>	Extract second byte from a <code>__packb</code>
<code>unsigned char __extractbyte2(__unsigned packb);</code>	Idem, but unsigned
<code>char __extractbyte3(__packb);</code>	Extract third byte from a <code>__packb</code>
<code>unsigned char __extractbyte3(__unsigned packb);</code>	Idem, but unsigned
<code>char __extractbyte4(__packb);</code>	Extract fourth byte from a <code>__packb</code>
<code>unsigned char __extractbyte4(__unsigned packb);</code>	Idem, but unsigned
<code>short __extracthw1(__packhw);</code>	Extract first short from a <code>__packhw</code>

Intrinsic Function	Description
unsigned short __extractuhw1(unsigned __packhw);	Idem, but unsigned
short __extracthw2(__packhw);	Extract second short from a __packhw
unsigned short __extractuhw2(unsigned __packhw);	Idem, but unsigned
volatile char __getbyte1(__packb *);	Extract first byte from a __packb
volatile unsigned char __getubyte1(__unsigned packb *);	Idem, but unsigned
volatile char __getbyte2(__packb *);	Extract second byte from a __packb
volatile unsigned char __getubyte2(__unsigned packb *);	Idem, but unsigned
volatile char __getbyte3(__packb *);	Extract third byte from a __packb
volatile unsigned char __getubyte3(__unsigned packb *);	Idem, but unsigned
volatile char __getbyte4(__packb *);	Extract fourth byte from a __packb
volatile unsigned char __getubyte4(__unsigned packb *);	Idem, but unsigned
volatile short __gethw1(__packhw *);	Extract first short from a __packhw
volatile unsigned short __getuhw1(unsigned __packhw *);	Idem, but unsigned
volatile short __gethw2(__packhw *);	Extract second short from a __packhw
volatile unsigned short __getuhw2(unsigned __packhw *);	Idem, but unsigned

Insert values into packed data types

The following table provides an overview of the intrinsic functions to insert a single byte or halfword into a __packb or __packhw data type.

Intrinsic Function	Description
__packb __insertbyte1(__packb, char);	Insert char into first byte of a __packb
unsigned __packb __insertubyte1(unsigned __packb, unsigned char);	Idem, but unsigned
__packb __insertbyte2(__packb, char);	Insert char into second byte of a __packb
unsigned __packb __insertubyte2(unsigned __packb, unsigned char);	Idem, but unsigned
__packb __insertbyte3(__packb, char);	Insert char into third byte of a __packb
unsigned __packb __insertubyte3(unsigned __packb, unsigned char);	Idem, but unsigned
__packb __insertbyte4(__packb, char);	Insert char into fourth byte of a __packb
unsigned __packb __insertubyte4(unsigned __packb, unsigned char);	Idem, but unsigned
__packhw __inserthw1(__packhw, short);	Insert short into first halfword of a __packhw
unsigned __packhw __insertuhw1(unsigned __packhw, unsigned short);	Idem, but unsigned

Intrinsic Function	Description
__packhw __insertw2(__packhw, short);	Insert short into second halfword of a __packhw
unsigned __packhw __insertuhw2(unsigned __packhw, unsigned short);	Idem, but unsigned
void __setbyte1(__packb *, char);	Insert char into first byte of a __packb
void __setubyte1(unsigned __packb *, unsigned char);	Idem, but unsigned
void __setbyte2(__packb *, char);	Insert char into second byte of a __packb
void __setubyte2(unsigned __packb *, unsigned char);	Idem, but unsigned
void __setbyte3(__packb *, char);	Insert char into third byte of a __packb
void __setubyte3(unsigned __packb *, unsigned char);	Idem, but unsigned
void __setbyte4(__packb *, char);	Insert char into fourth byte of a __packb
void __setubyte4(unsigned __packb *, unsigned char);	Idem, but unsigned
void __sethw1(__packhw *, short);	Insert short into first halfword of a __packhw
void __setuhw1(unsigned __packhw *, unsigned short);	Idem, but unsigned
void __sethw2(__packhw *, short);	Insert short into second halfword of a __packhw
void __setuhw2(unsigned __packhw *, unsigned short);	Idem, but unsigned

Calculate absolute values of packed data type values

The following table provides an overview of the intrinsic functions to calculate the absolute value of packed data type values.

Intrinsic Function	Description
__packb __absb(__packb);	Absolute value of __packb
__packhw __absh(__packhw);	Absolute value of __packhw
__sat __packhw __abssh(__sat __packhw);	Absolute value of __packhw using saturation

Calculate minimum packed data type values

The following table provides an overview of the intrinsic functions to calculate the minimum from two packed data type values.

Intrinsic Function	Description
__packb __minb(__packb, __packb);	Minimum of two __packb values
unsigned __packb __minbu(unsigned __packb, unsigned __packb);	Minimum of two unsigned __packb values
__packhw __minh(__packhw, __packhw);	Minimum of two __packhw values
unsigned __packhw __minhu(unsigned __packhw, unsigned __packhw);	Minimum of two unsigned __packhw values

1.11.5.4. Interrupt Handling

The following table provides an overview of the intrinsic functions to read or set interrupt handling.

Intrinsic Function	Description
volatile void __enable (void);	Enable interrupts immediately at function entry
volatile void __disable (void);	Disable interrupts.
volatile int __disable_and_save (void);	Disable interrupts and return previous interrupt state (enabled or disabled). Only supported for TriCore 1.6, 1.6.x and 1.6.2 (--core=tc1.6 , --core=tc1.6.x , --core=tc1.6.2).
unsigned int __get_tin(void);	Get the Trap Identification Number (TIN). See Section 1.11.4.2, Defining a Trap Service Routine: __trap(), __trap_fast() .
volatile void __restore (int);	Restore interrupt state. Only supported for TriCore 1.6, 1.6.x and 1.6.2 (--core=tc1.6 , --core=tc1.6.x , --core=tc1.6.2).
volatile void __bistr (const unsigned int);	Set CPU priority number [0..255] and enable interrupts immediately at function entry
volatile void __syscall (int);	Call a system call function number

1.11.5.5. Insert Single Assembly Instruction

The following table provides an overview of the intrinsic functions that you can use to insert a single assembly instruction. You can also use inline assembly but these intrinsics provide a shorthand for frequently used assembly instructions.

See [Section 1.6, Using Assembly in the C Source: __asm\(\)](#).

Intrinsic Function	Description
volatile void __debug(void);	Insert DEBUG instruction
volatile void __dsync(void);	Insert DSYNC instruction
volatile void __isync(void);	Insert ISYNC instruction
volatile void __svlcx(void);	Insert SVLCX instruction
volatile void __rslcx(void);	Insert RSLCX instruction
volatile void __nop(void);	Insert NOP instruction
volatile void __ldmst(unsigned int * <i>address</i> , unsigned int <i>mask</i> , unsigned int <i>value</i>);	Insert LDMST instruction. Note that <i>address</i> must be word-aligned.
volatile unsigned int __swap(unsigned int * <i>place</i> , unsigned int <i>value</i>);	Insert SWAP instruction. Note that <i>place</i> must be word-aligned.

1.11.5.6. Register Handling

Access control registers

The following table provides an overview of the intrinsic functions that you can use to access control registers.

Intrinsic Function	Description
<code>volatile int __mfcr(int csfr);</code>	Move contents of the addressed Core Special Function Register (CSFR) into a data register
<code>volatile void __mtcr (int csfr, int val);</code>	Move contents of a data register (second int) to the addressed CSFR (first int) and generate an ISYNC instruction. The ISYNC instruction ensures that the effects of the CSFR update are correctly seen by all following instructions.

See the `.sfr` files in the `include\sfr` directory for a list of the 16-bit CSFRs.

For example:

```
#include "sfr/regtc1796b.sfr"

int get_cpu_id( void )
{
    return __mfcr( CPU_ID ); // return contents of CSFR CPU_ID
}
```

This results in the assembly instruction:

```
mfcr d2,#65048
```

Note that if you want to set a single bit in a CSFR you have to create a bit mask. For example in `cstart.c`:

```
__mtcr(BIV, (unsigned int)(_lc_u_int_tab) | (0xff<<3) | 1 );
```

Perform register value operations

The following table provides an overview of the intrinsic functions that operate on a register and return a value in another register.

Intrinsic Function	Description
<code>int __clz (int);</code>	Count leading zeros in int
<code>int __clo (int);</code>	Count leading ones in int
<code>int __cls (int);</code>	Count number of redundant sign bits (all consecutive bits with the same value as bit 31)
<code>signed char __satb (int);</code>	Return saturated byte
<code>unsigned char __satbu (int);</code>	Return saturated unsigned byte
<code>short __sath (int);</code>	Return saturated halfword
<code>unsigned short __sathu (int);</code>	Return saturated unsigned halfword
<code>int __abs (int);</code>	Return absolute value
<code>int __abss (int);</code>	Return absolute value with saturation
<code>float __fabsf (float f);</code>	Return absolute floating-point value
<code>double __fabs (double d);</code>	Return absolute double precision floating-point value

Intrinsic Function	Description
int __parity (int);	Return parity

Get or set stack pointer register A10

The following table provides an overview of the intrinsic functions to set or get the stack pointer register A10.

Intrinsic Function	Description
void __set_sp(void *);	Set stack pointer register
void * __get_sp(void);	Get stack pointer register

Example:

```
# define STACK_ALIGN 0xffffffff8
void set( void )
{
    void * sp = (void *)((unsigned int)(_lc_ue_ustack) & STACK_ALIGN);

    __set_sp( sp );    /* load user stack pointer */
}

void * get( void )
{
    return __get_sp();
}
```

1.11.5.7. Insert / Extract Bit-fields and Bits

Insert / extract bit-fields

The following table provides an overview of the intrinsic functions to insert or extract a bit-field.

Intrinsic Function	Description
int __extr (int <i>value</i> , int <i>pos</i> , int <i>width</i>);	Extract a bit-field (bit <i>pos</i> to bit <i>pos+width</i>) from <i>value</i>
unsigned int __extru (int <i>value</i> , int <i>pos</i> , int <i>width</i>);	Same as __extr () but return bit-field as unsigned integer
int __insert (int <i>trg</i> , int <i>src</i> , int <i>pos</i> , int <i>width</i>);	Extract bit-field (<i>width</i> bits starting at bit 0) from <i>src</i> and insert it in <i>trg</i> at <i>pos</i> .
int __ins(int <i>trg</i> , int <i>trgbit</i> , int <i>src</i> , int <i>srcbit</i>);	Return <i>trg</i> but replace <i>trgbit</i> by <i>srcbit</i> in <i>src</i> .
int __insn(int <i>trg</i> , int <i>trgbit</i> , int <i>src</i> , int <i>srcbit</i>);	Return <i>trg</i> but replace <i>trgbit</i> by inverse of <i>srcbit</i> in <i>src</i> .

Atomic load-modify-store

With the following intrinsic function you can perform atomic Load-Modify-Store of a bit-field from an integer value. This function uses the IMASK and LDMST instruction. The intrinsic writes the number of *bits* of an

integer *value* at a certain *address* location in memory with a *bitoffset*. The number of bits must be a constant value. Note that all operands must be word-aligned.

Intrinsic Function	Description
void __imaskldmst(int* <i>address</i> , int <i>value</i> , int <i>bitoffset</i> , int <i>bits</i>);	Atomic load-modify-store

Store a single bit

With the intrinsic macro `__putbit()` you can store a single bit atomically in memory at a specified bit offset. The bit at offset 0 in *value* is stored at an *address* location in memory with a *bitoffset*.

This intrinsic is implemented as a macro definition which uses the `__imaskldmst()` intrinsic:

```
#define __putbit( value, address, bitoffset ) __imaskldmst
( address, value, bitoffset, 1 )
```

Note that all operands must be word-aligned.

Intrinsic Function	Description
void __putbit(int <i>value</i> , int* <i>address</i> , int <i>bitoffset</i>);	Store a single bit

Load a single bit

With the intrinsic macro `__getbit()` you can load a single bit from memory at a specified bit offset. A bit value is loaded from an *address* location in memory with a *bitoffset* and returned as an unsigned integer value.

This intrinsic is implemented as a macro definition which uses the `__extru()` intrinsic:

```
#define __getbit( address, bitoffset ) __extru( *(address), bitoffset, 1 )
```

Intrinsic Function	Description
unsigned int __getbit(int * <i>address</i> , int <i>bitoffset</i>);	Load a single bit

1.11.5.8. Atomic Intrinsic Functions

Initialization

Intrinsic Function	Description
volatile void __c11_atomic_init(volatile _Atomic type * <i>obj</i> , type <i>value</i>)	Initializes the atomic object pointed to by <i>obj</i> to the value <i>value</i> .

Fences

Intrinsic Function	Description
volatile void __c11_atomic_thread_fence(memory_order order)	Depending on the memory order, this operation has either no effects, is an acquire fence, a release fence, both acquire and release fence, or sequentially consistent acquire and release fence.
volatile void __c11_atomic_signal_fence(memory_order order)	Equivalent to __c11_atomic_thread_fence, except that the resulting ordering constraints are established only between a thread and a signal handler executed in the same thread.

Lock-free property

Intrinsic Function	Description
volatile __Bool __c11_atomic_is_lock_free(volatile _Atomic type * obj)	Indicates whether or not the object pointed to by <i>obj</i> is lock-free.

Operations on atomic types

Intrinsic Function	Description
volatile void __c11_atomic_store(volatile _Atomic type * obj, type desired, memory_order order)	Atomically replaces the value pointed to by <i>obj</i> with the value of <i>desired</i> . Memory is affected according to the value of <i>order</i> .
volatile type __c11_atomic_load(volatile _Atomic type * obj, memory_order order)	Atomically returns the value pointed to by <i>obj</i> .
volatile type __c11_atomic_exchange(volatile _Atomic type * obj, type desired, memory_order order)	Atomically replaces the value pointed to by <i>obj</i> with the value of <i>desired</i> . Memory is affected according to the value of <i>order</i> . These operations are atomic read-modify-write operations.
volatile __Bool __c11_atomic_compare_exchange_strong(volatile _Atomic type * obj, type * expected, type desired, memory_order success, memory_order failure)	Atomically compares the value pointed to by <i>obj</i> for equality with that in <i>expected</i> . If the comparison is true, it replaces the value pointed to by <i>obj</i> with <i>desired</i> , and memory is affected according to the value of <i>success</i> . If the comparison is false, it updates the value in <i>expected</i> with the value pointed to by <i>obj</i> , and memory is affected according to the value of <i>failure</i> . These operations are atomic read-modify-write operations.
volatile __Bool __c11_atomic_compare_exchange_weak(volatile _Atomic type * obj, type * expected, type desired, memory_order success, memory_order failure)	

Intrinsic Function	Description
volatile type __c11_atomic_fetch_key(volatile _Atomic type * <i>obj</i> , type <i>operand</i> , memory_order <i>order</i>)	Atomically replaces the value pointed to by <i>obj</i> with the result of the computation applied to the value pointed to by <i>obj</i> and the given <i>operand</i> . Memory is affected according to the value of <i>order</i> . The key values are: add (+, addition) sub (-, subtraction) or (, bitwise inclusive or) xor (^, bitwise exclusive or) and (&, bitwise and)

Atomic flag type and operations

Intrinsic Function	Description
volatile _Bool __c11_atomic_flag_test_and_set(volatile atomic_flag * <i>obj</i> , memory_order <i>order</i>)	Atomically sets the value pointed to by <i>obj</i> to true. Memory is affected according to the value of <i>order</i> .
volatile void __c11_atomic_flag_clear(volatile atomic_flag * <i>obj</i> , memory_order <i>order</i>)	Atomically sets the value pointed to by <i>obj</i> to false. Memory is affected according to the value of <i>order</i> .

1.11.5.9. Miscellaneous Intrinsic Functions

Multiply and scale back

Intrinsic Function	Description
int __mulsc(int a, int b, int <i>offset</i>);	Multiply two 32-bit numbers to an intermediate 64-bit result, and scale back the result to 32 bits. To scale back the result, 32 bits are extracted from the intermediate 64-bit result: bit 63- <i>offset</i> to bit 31- <i>offset</i> .

Cache write back and invalidation

To support write back and invalidation of cache address or cache index the following intrinsics are available:

Intrinsic Function	Description
volatile void __cacheawi(unsigned char * p);	Write back and invalidate cache address "p". Generates CACHEA.WI [Ab].
volatile void __cacheiwi(unsigned char * p);	Write back and invalidate cache index "p". Generates CACHEI.WI [Ab].
unsigned char * __cacheawi_bo_post_inc(unsigned char * p);	Write back and invalidate cache address "p" and return post incremented value of "p". Generates CACHEA.WI [Ab+].

See `sync_on_halt.c` for some examples.

Swap

Intrinsic Function	Description
<code>volatile unsigned int __swapmskw(unsigned int * <i>memory</i>, unsigned int <i>value</i>, unsigned int <i>mask</i>);</code>	Swap under mask. Exchanges the values of <i>value</i> and <i>memory</i> , but only those bits that are allowed by <i>mask</i> . Before the <code>swapmsk.w</code> instruction is generated, the parameters <i>value</i> and <i>mask</i> are moved into a double register. Only supported for TriCore1.6.x (--core=tc1.6.x) and TriCore1.6.2 (--core=tc1.6.2). Note that <i>memory</i> must be word-aligned.
<code>volatile unsigned int __cmpswapw(unsigned int * <i>memory</i>, unsigned int <i>value</i>, unsigned int <i>compare</i>);</code>	Compare and swap. Exchanges the values of <i>value</i> and <i>memory</i> if the contents of <i>memory</i> equals <i>compare</i> . Generates the <code>cmpswap.w</code> instruction. Only supported for TriCore1.6.x (--core=tc1.6.x) and TriCore1.6.2 (--core=tc1.6.2). Note that <i>memory</i> must be word-aligned.

CRC generate

Intrinsic Function	Description
<code>unsigned int __crc32(unsigned int b, unsigned int a);</code>	Calculate the CRC32 checksum of a and inverse of b and return the result. Generates the <code>crc32</code> instruction. Only supported for TriCore1.6.x (--core=tc1.6.x) and TriCore1.6.2 (--core=tc1.6.2). For example: <pre>ld.w d15, b ld.w d0, a crc32 d2, d15, d0</pre>
<code>unsigned int __crc32b(unsigned int b, unsigned int a);</code>	Calculate the CRC of 8 bits of a and return the result. The first argument b contains either an initial seed value, or the cumulative CRC result from a previous sequence of data. Generates the <code>crc32.b</code> instruction. Only supported for TriCore1.6.2 (--core=tc1.6.2).
<code>unsigned int __crc32bw(unsigned int b, unsigned int a);</code>	Calculate the CRC of four bytes in big-endian order of a and return the result. The first argument b contains either an initial seed value, or the cumulative CRC result from a previous sequence of data. Generates the <code>crc32b.w</code> instruction. The intrinsic <code>__crc32bw</code> is an alias for intrinsic <code>__crc32</code> . The instructions generated for the <code>__crc32</code> and <code>__crc32bw</code> use the same instruction encoding. The intrinsic <code>__crc32</code> generates instruction <code>crc32b.w</code> for TriCore1.6.2, otherwise it generates instruction <code>crc32</code> . Only supported for TriCore1.6.x (--core=tc1.6.x) and TriCore1.6.2 (--core=tc1.6.2).
<code>unsigned int __crc32lw(unsigned int b, unsigned int a);</code>	Calculate the CRC of four bytes in little-endian order of a and return the result. The first argument b contains either an initial seed value, or the cumulative CRC result from a previous sequence of data. Generates the <code>crc32l.w</code> instruction. Only supported for TriCore1.6.2 (--core=tc1.6.2).

Intrinsic Function	Description
<pre>unsigned int __crcn(unsigned int d, unsigned int a, unsigned int b);</pre>	Calculate the CRC value for 1 to 8 bits of b using a user-defined CRC algorithm with a CRC width from 1 up to 16 bits and return the result. The first argument d contains an initial seed value, or the cumulative CRC result from a previous sequence of data. The second argument a specifies all parameters of the CRC algorithm. Generates the <code>crcn</code> instruction. Only supported for TriCore1.6.2 (--core=tc1.6.2). The bit-fields of a are: a[2:0] contains M-1, where M is the input data width of b. a[7:3] must be zero. a[8] if set input data bit order of b is treated as little-endian, otherwise input data bit order is treated as big-endian. a[9] if set a bit-wise logical inversion is applied to both the result and seed values. a[11:10] must be zero. a[15:12] contains N-1, where N is the CRC width in the range[1,16] a[16+N-1:16] encodes the coefficients of the generator polynomial. a[32:16+N-1] must be zero.

SHUFFLE generate

Intrinsic Function	Description
<pre>unsigned int __shuffle(unsigned int a, int const9);</pre>	Shuffle the order of the bytes of a according to const9 and return the result. The value const9 contains four 2-byte fields which specify which bytes from a are chosen to return. The value of each 2-bit byte select field specifies the index of the source byte from a. Bit 8 of const9 reverses bits in the bytes of a. Generates the <code>shuffle</code> instruction when const9 is an immediate value, otherwise generates a call to a run-time shuffle function that implements the shuffle algorithm: <pre>unsigned int __rt_shuffle(unsigned int a, int const9);</pre> Bits const9[31:9] are ignored, no error is generated. The shuffle instruction is only generated for TriCore1.6.2 (--core=tc1.6.2) when const9 is an immediate value.

POPCNTW generate

Intrinsic Function	Description
<pre>unsigned int __popcntw(unsigned int a);</pre>	Count the total number of ones in a and return the result. Generates the <code>popcnt.w</code> instruction. Only supported for TriCore1.6.2 (--core=tc1.6.2).

LHA generate

Intrinsic Function	Description
<code>void * __lha(unsigned int off18);</code>	Compute the 32-bit effective address (EA) of absolute address offset of <code>off18</code> and return the EA. $EA = \{off18[17:0], 14b'0\}$; Generates the <code>lha</code> instruction when <code>off18</code> is an immediate value and the selected core is TriCore1.6.2 (--core=tc1.6.2), otherwise generates a shift left 14 and move address instruction (<code>sh dx, dy, #14 mov.a ax, dx</code>).

Wait for asynchronous event

Intrinsic Function	Description
<code>void volatile __wait(void);</code>	The processor suspends execution until the next enabled interrupt or asynchronous trap event is detected. Generates the <code>wait</code> instruction. Only supported for TriCore1.6.x (--core=tc1.6.x) and TriCore1.6.2 (--core=tc1.6.2).

Initialize circular pointer

Intrinsic Function	Description
<code>__circ void * __initcirc(void * buf, unsigned short bufsize, unsigned short byteindex);</code>	Initialize a circular pointer with a dynamically allocated buffer at run-time. See also Section 1.3.1, Circular Buffers: __circ .

Rotate left/right

Intrinsic Function	Description
<code>unsigned int __rol(unsigned int operand, unsigned int count)</code>	Rotate <i>operand</i> left <i>count</i> times. The bits that are shifted out are inserted at the right side (bit 31 is shifted to bit 0).
<code>unsigned int __ror(unsigned int operand, unsigned int count)</code>	Rotate <i>operand</i> right <i>count</i> times. The bits that are shifted out are inserted at the left side (bit 0 is shifted to bit 31).

Floating-point rounding direction

Intrinsic Function	Description
<code>volatile void __fesetround(int round);</code>	Set the floating-point rounding direction using the <code>updf l</code> instruction. <i>round</i> must be one of the rounding direction macros <code>FE_TONEAREST</code>, <code>FE_UPWARD</code>, <code>FE_DOWNWARD</code>, or <code>FE_TOWARDZERO</code>. If <i>round</i> is not equal to one of the rounding direction macros, the rounding direction is not changed. This intrinsic function is used in the startup code. For TriCore1.3 (--core=tc1.3) or when TriCore1.3 compatibility mode (COMPAT) is enabled the rounding direction is restored on a RET (Return From Call) instruction.

Intrinsics used by compiler/libraries

Intrinsic Function	Description
<code>void * volatile __alloc(__size_t size);</code>	Allocate memory. Returns a pointer to memory of <i>size</i> bytes length. Returns NULL if there is not enough space left. This function is used internally for variable length arrays, it is not to be used by end users.
<code>void * __dotdotdot__(void);</code>	Variable argument '...' operator. Used in library function <code>va_start()</code> . Returns the stack offset to the variable argument list.
<code>volatile void __free(void *p);</code>	Deallocates the memory pointed to by <i>p</i> . <i>p</i> must point to memory earlier allocated by a call to <code>__alloc()</code> .
<code>__codeptr __get_return_address(void);</code>	Used by the compiler for profiling when you compile with the option --profile . Returns the return address of a function.

1.12. Compiler Generated Sections

The compiler generates code and data in several types of sections. By default the C compiler generates sections with the following names:

```
section_type_prefix[.uncached][.core_association].module_name.symbol_name
```

`uncached` is only added for atomic variables (see [Section 1.7, “uncached”](#)). A core association, `share`, `private` or `clone`, is only present when this is specified for the TriCore 1.6.x or 1.6.2. See [Section 1.4, Multi-Core Support](#).

For interrupt vectors and trap vectors the C compiler generates special section names, where the number *n* refers to core *n*:

```
.text[.inttabn].intvec.vector_number
```

```
.text[.traptabn].trapvec.vector_number
```

When you use a section renaming pragma, the compiler uses the following section naming convention:

```
section_type_prefix[.uncached][.core_association][.module_name][.symbol_name][.pragma_value]
```

The prefix depends on the type of the section and determines if the section is initialized, constant or uninitialized and which addressing mode is used. The *symbol_name* is either the name of an object or the name of a function.

The following table lists the section types and name prefixes.

Section type	Name prefix	Description
code	<code>.text</code>	program code
neardata	<code>.zdata</code>	initialized <code>__near</code> data
fardata	<code>.data</code>	initialized <code>__far</code> data
nearrom	<code>.zrodata</code>	constant <code>__near</code> data

Section type	Name prefix	Description
farrom	.rodata	constant __far data
nearbss	.zbss	uninitialized __near data (cleared)
farbss	.bss	uninitialized __far data (cleared)
nearnoclear	.zbss	uninitialized __near data
farnoclear	.bss	uninitialized __far data
a0data	.data_a0	initialized __a0 data
a0rom	.rodata_a0	constant __a0 data
a0bss	.bss_a0	uninitialized __a0 data (cleared)
a1data	.data_a1	initialized __a1 data
a1rom	.rodata_a1	constant __a1 data
a1bss	.bss_a1	uninitialized __a1 data (cleared)
a8data	.data_a8	initialized __a8 data
a8rom	.rodata_a8	constant __a8 data
a8bss	.bss_a8	uninitialized __a8 data (cleared)
a9data	.data_a9	initialized __a9 data
a9rom	.rodata_a9	constant __a9 data
a9bss	.bss_a9	uninitialized __a9 data (cleared)

1.12.1. Rename Sections

You can change the default section names with one of the following pragmas. The naming convention for the renamed section is:

```
section_type_prefix[.uncached][.core_association][.module_name][.symbol_name][.pragma_value]
```

Note however that a symbol at an absolute address (__at) is located in a section that always uses the default section name.

#pragma section type ["name" | default | restore]

With this pragma all sections of the specified *type* will be named "*prefix.name*". For example,

```
#pragma section neardata "where"
```

all sections of type neardata have the name ".zdata.where".

#pragma section *type* will set section naming for sections of this type conform its name "*prefix*".

#pragma section *type* default will restore the default section naming for sections of this type.

#pragma section *type* restore will restore the previous setting of #pragma section *type*.

#pragma section all ["name" | default | restore]

With this pragma all sections will be named "*prefix.name*", unless you use a type specific renaming pragma. For example,

```
#pragma section all "here"
```

all sections have the syntax "*prefix*[.uncached].*here*". For example, sections of type neardata have the name ".zdata.here"

#pragma section all will set section naming conform its name "*prefix*".

#pragma section all default will restore the default section naming (not for sections that have a type specific renaming pragma).

#pragma section all restore will restore the previous setting of #pragma section all.

On the command line you can use the [C compiler option --rename-sections\[=name\]](#).

Note that when you use one of the above section renaming pragmas, the module name and symbol name are no longer part of the section name. Use one or both of the following pragmas to influence the section naming convention.

#pragma section_name_with_module [on | off | default | restore]

With this pragma all section renaming pragmas will use a renaming scheme like:

```
section_type_prefix[.uncached].module_name.pragma_value
```

#pragma section_name_with_symbol [on | off | default | restore]

With this pragma all section renaming pragmas will use a renaming scheme like:

```
section_type_prefix[.uncached].symbol_name.pragma_value
```

See also [C compiler option --section-name-with-symbol](#).

Examples

```
#pragma section all "rename_1"
// .text.rename_1
// .data.rename_1

#pragma section code "rename_2"
// .text.rename_2
// .data.rename_1

#pragma section code
// .text
// .data.rename_1
```

1.12.2. Influence Section Definition

The following pragmas also influence the section definition:

#pragma section code_init

Code sections are copied from ROM to RAM at program startup.

#pragma section const_init

Sections with constant data are copied from ROM to RAM at program startup.

#pragma section vector_init

Sections with interrupts and trap vectors are copied from ROM to RAM at program startup.

#pragma section data_overlay

The `nearnoclear` and `farnoclear` sections can be overlaid by other sections with the same name. Since by default section naming never leads to sections with the same name, you must force the same name by using one of the section renaming pragmas. To get `noclear` sections instead of `BSS` sections you must also use `#pragma noclear`.

Chapter 3. Assembly Language

This chapter describes the most important aspects of the TASKING assembly language for TriCore. For a complete overview of the architecture you are using, refer to the target's core Architecture Manual.

3.1. Assembly Syntax

An assembly program consists of statements. A statement may optionally be followed by a comment. Any source statement can be extended to more lines by including the line continuation character (\) as the last character on the line. The length of a source statement (first line and continuation lines) is only limited by the amount of available memory.

Mnemonics, directives and other keywords are case insensitive. Labels, symbols, directive arguments, and literal strings are case sensitive.

The syntax of an assembly statement is:

```
[label[:]] [instruction | directive | macro_call] [;comment]
```

label

A label is a special symbol which is assigned the value and type of the current program location counter. A label can consist of letters, digits and underscore characters (_). The first character cannot be a digit. The label can also be a *number*. A label which is prefixed by whitespace (spaces or tabs) has to be followed by a colon (:). The size of an identifier is only limited by the amount of available memory.

number is a number ranging from 1 to 255. This type of label is called a *numeric label* or *local label*. To refer to a numeric label, you must put an **n** (next) or **p** (previous) immediately after the label. This is required because the same label number may be used repeatedly.

Examples:

```
LAB1: ; This label is followed by a colon and
      ; can be prefixed by whitespace
LAB1  ; This label has to start at the beginning
      ; of a line
1: j 1p ; This is an endless loop
      ; using numeric labels
```

instruction

An instruction consists of a mnemonic and zero, one or more operands. It must not start in the first column.

Operands are described in [Section 3.3, Operands of an Assembly Instruction](#). The instructions are described in the target's core Architecture Manual.

directive

With directives you can control the assembler from within the assembly source. Except for preprocessing directives, these must not start in the first column. Directives are described in [Section 3.9, Assembler Directives and Controls](#).

<i>macro_call</i>	A call to a previously defined macro. It must not start in the first column. See Section 3.10, Macro Operations .
<i>comment</i>	Comment, preceded by a ; (semicolon).

You can use empty lines or lines with only comments.

Apart from the assembly statements as described above, you can put a so-called 'control line' in your assembly source file. These lines start with a \$ in the first column and alter the default behavior of the assembler.

\$control

For more information on controls see [Section 3.9, Assembler Directives and Controls](#).

3.1.1. Deviations from the Instruction Set Manual

.T bit instructions

With respect to all bit variants of branching and arithmetic instructions carrying the .T extension, the TASKING assembler for TriCore accepts an extra instruction format in addition to the *TriCore V1.6 Instruction Set User Manual (Volume 2)* .

For example, the instruction set manual specifies

```
xor . t  d0, d5, 31, d1, #31
```

The TASKING assembler accepts this, but also accepts the following syntax (which is generated by the C compiler):

```
xor . t  d0, d5:31, d1:31
```

A colon instead of a comma is accepted as delimiter for the bit position.

Both notations generate the same instruction encoding.

Instructions with the SLRO/SSRO opcode formats (LD/ST instructions) expect byte addressing

The Infineon TriCore instruction set manuals are ambiguous about the addressing used in instructions that support the SLRO/SSRO opcode formats. According to the instruction set manuals, 4-bit offsets can be byte, half-word or word addressed (depending on the type of instruction), but implementing the instruction that way would lead to inconsistencies with other variations of the same instructions that support a 16-bit offset operand as well (which always expects byte addressing). To prevent this ambiguity, the TASKING assembler always expects byte addressing and selects the shortest instruction encoding in which the address (converted to a half-word or word address when necessary) can be fitted.

Example:

```
ld.h, [a15]0x10 ; 0x10 is always interpreted as a byte address
```

3.2. Assembler Significant Characters

You can use all ASCII characters in the assembly source both in strings and in comments. Also the extended characters from the ISO 8859-1 (Latin-1) set are allowed.

Some characters have a special meaning to the assembler. Special characters associated with expression evaluation are described in [Section 3.6.3, Expression Operators](#). Other special assembler characters are:

Character	Description
;	Start of a comment
\	Line continuation character or macro operator: argument concatenation
?	Macro operator: return decimal value of a symbol
%	Macro operator: return hex value of a symbol
^	Macro operator: override local label
"	Macro string delimiter or quoted string .DEFINE expansion character
'	String constants delimiter
@	Start of a built-in assembly function
*	Location counter substitution
#	Constant number
++	String concatenation operator
[]	Substring delimiter

3.3. Operands of an Assembly Instruction

In an instruction, the mnemonic is followed by zero, one or more operands. An operand has one of the following types:

Operand	Description
<i>symbol</i>	A symbolic name as described in Section 3.4, Symbol Names . Symbols can also occur in expressions.
<i>register</i>	Any valid register as listed in Section 3.5, Registers .
<i>expression</i>	Any valid expression as described in Section 3.6, Assembly Expressions .
<i>address</i>	A combination of <i>expression</i> , <i>register</i> and <i>symbol</i> .

Addressing modes

The TriCore assembly language has several addressing modes. These are described in detail in the target's core Architecture Manual.

3.4. Symbol Names

User-defined symbols

A user-defined *symbol* can consist of letters, digits and underscore characters (`_`). The first character cannot be a digit. The size of an identifier is only limited by the amount of available memory. The case of these characters is significant. You can define a symbol by means of a label declaration or an equate or set directive.

Predefined preprocessor symbols

These symbols start and end with two underscore characters, `__symbol__`, and you can use them in your assembly source to create conditional assembly. See [Section 3.4.1, Predefined Preprocessor Symbols](#).

Labels

Symbols used for memory locations are referred to as labels. It is allowed to use reserved symbols as labels as long as the label is followed by a colon.

Reserved symbols

Symbol names and other identifiers starting with a period (`.`) are reserved for the system (for example for directives or section names). Identifiers starting with an at sign (`@`) are reserved for built-in assembler functions. Instructions are also reserved. The case of these built-in symbols is insignificant.

Examples

Valid symbol names:

```
loop_1
ENTRY
a_B_c
_aBC
```

Invalid symbol names:

```
1_loop      ; starts with a number
d15         ; reserved register name
.DEFINE     ; reserved directive name
```

3.4.1. Predefined Preprocessor Symbols

The TASKING assembler knows the predefined symbols as defined in the table below. The symbols are useful to create conditional assembly.

Symbol	Description
<code>__ASTC__</code>	Identifies the assembler. You can use this symbol to flag parts of the source which must be recognized by the astc assembler only. It expands to 1.

Symbol	Description
__BUILD__	Identifies the build number of the assembler in the format yymmddqq (year, month, day and quarter in UTC).
__CORE_core__	A symbol is defined depending on the option option --core=core . The <i>core</i> is converted to uppercase and '.' is removed. For example, if --core=tc1.3.1 is specified, the symbol <code>__CORE_TC131__</code> is defined. When no --core is supplied, the assembler defines <code>__CORE_TC13__</code> .
__CPU_cpu__	A symbol is defined depending on the control program option --cpu=cpu . The <i>cpu</i> is converted to uppercase. For example, if --cpu=tc1796b is specified to the control program, the symbol <code>__CPU_TC1796B__</code> is defined. When no --cpu is supplied, this symbol is not defined.
__CPU_TCnum__	The corresponding symbol is defined if the CPU functional problem <i>cpu-tcnum</i> is specified with the option --silicon-bug .
__FPU__	Expands to 0 if you used option --no-fpu (Disable FPU instructions), otherwise expands to 1.
__MMU__	Expands to 1 if you used option --mmu-present (allow use of MMU instructions), otherwise expands to 0.
__REVISION__	Expands to the revision number of the assembler. Digits are represented as they are; characters (for prototypes, alphas, betas) are represented by -1. Examples: <code>v1.0r1 -> 1</code> , <code>v1.0rb -> -1</code>
__TASKING__	Identifies the assembler as a TASKING assembler. Expands to 1 if a TASKING assembler is used.
__UM_KERNEL__	Expands to 1 if the TriCore runs in kernel/supervisor mode (option --user-mode=kernel).
__UM_USER_1__	Expands to 1 if the TriCore runs in User-1 mode (option --user-mode=user-1).
__VERSION__	Identifies the version number of the assembler. For example, if you use version 2.1r1 of the assembler, <code>__VERSION__</code> expands to 2001 (dot and revision number are omitted, minor version number in 3 digits).

Example

```
.if @def('__ASTC__')
; this part is only for the astc assembler
...
.endif
```

3.5. Registers

The following register names, either uppercase or lowercase, should not be used for user-defined symbol names in an assembly language source file:

D0 .. D15 (data registers)
E0 .. E14 (data register pairs, only the even numbers)
A0 .. A15 (address registers)

3.5.1. Special Function Registers

It is easy to access Special Function Registers (SFRs) that relate to peripherals from assembly. The SFRs are defined in a special function register definition file (*.def) as symbol names for use by the assembler. The assembler can include the SFR definition file with the command line option **--include-file (-H)**. SFRs are defined with **.EQU** directives.

For example (from regtc1796b.def):

```
PSW      .equ  0xFE04
```

Example use in assembly:

```
mfcrr    d0, #PSW
andn     d0, d0, #0x7f          ; reset counter
insert   d0, d0, #1, #7, #1     ; enable
insert   d0, d0, #1, #8, #1     ; set GW bit
mtcr     #PSW, d0
isync
```

Without an SFR file the assembler only knows the general purpose registers D0-D15 and A0-A15.

3.6. Assembly Expressions

An expression is a combination of symbols, constants, operators, and parentheses which represent a value that is used as an operand of an assembler instruction (or directive).

Expressions can contain user-defined labels (and their associated integer or floating-point values), and any combination of integers, floating-point numbers, or ASCII literal strings.

Expressions follow the conventional rules of algebra and boolean arithmetic.

Expressions that can be evaluated at assembly time are called *absolute expressions*. Expressions where the result is unknown until all sections have been combined and located, are called *relocatable* or *relative expressions*.

When any operand of an expression is relocatable, the entire expression is relocatable. Relocatable expressions are emitted in the object file and evaluated by the linker. Relocatable expressions can only contain integral functions; floating-point functions and numbers are not supported by the ELF/DWARF object format.

The assembler evaluates expressions with 64-bit precision in two's complement.

The syntax of an *expression* can be any of the following:

- *numeric constant*

- *string*
- *symbol*
- *expression binary_operator expression*
- *unary_operator expression*
- *(expression)*
- *function call*

All types of expressions are explained in separate sections.

3.6.1. Numeric Constants

Numeric constants can be used in expressions. If there is no prefix, by default the assembler assumes the number is a decimal number. Prefixes can be used in either lowercase or uppercase.

Base	Description	Example
Binary	A 0b or 0B prefix followed by binary digits (0,1).	0B1101 0b11001010
Hexadecimal	A 0x or 0X prefix followed by hexadecimal digits (0-9, A-F, a-f).	0X12FF 0x45 0xfa10
Decimal integer	Decimal digits (0-9).	12 1245
Decimal floating-point	Decimal digits (0-9), includes a decimal point, or an 'E' or 'e' followed by the exponent.	6E10 .6 3.14 2.7e10

3.6.2. Strings

ASCII characters, enclosed in single (') or double (") quotes constitute an ASCII string. Strings between double quotes allow symbol substitution by a `.DEFINE` directive, whereas strings between single quotes are always literal strings. Both types of strings can contain escape characters.

Strings constants in expressions are evaluated to a number (each character is replaced by its ASCII value). Strings in expressions can have a size of up to 4 characters or less depending on the operand of an instruction or directive; any subsequent characters in the string are ignored. In this case the assembler issues a warning. An exception to this rule is when a string is used in a `.BYTE` assembler directive; in that case all characters result in a constant value of the specified size. Null strings have a value of 0.

Square brackets ([]) delimit a substring operation in the form:

[*string*, *offset*, *length*]

offset is the start position within *string*. *length* is the length of the desired substring. Both values may not exceed the size of *string*.

Examples

```
'ABCD'           ; (0x41424344)
'''79'          ; to enclose a quote double it
"A\"BC"         ; or to enclose a quote escape it
'AB'+1          ; (0x4143) string used in expression
''             ; null string
.word 'abcdef'   ; (0x64636261) 'ef' are ignored
                ; warning: string value truncated
'abc'++'de'      ; you can concatenate
                ; two strings with the '++' operator.
                ; This results in 'abcde'
['TASKING',0,4]  ; results in the substring 'TASK'
```

3.6.3. Expression Operators

The next table shows the assembler operators. They are ordered according to their precedence. Operators of the same precedence are evaluated left to right. Parenthetical expressions have the highest priority (innermost first).

Valid operands include numeric constants, literal ASCII strings and symbols.

Most assembler operators can be used with both integer and floating-point values. If one operand has an integer value and the other operand has a floating-point value, the integer is converted to a floating-point value before the operator is applied. The result is a floating-point value.

Type	Operator	Name	Description
	()	parenthesis	Expressions enclosed by parenthesis are evaluated first.
Unary	+	plus	Returns the value of its operand.
	-	minus	Returns the negative of its operand.
	~	one's complement	Integer only. Returns the one's complement of its operand. It cannot be used with a floating-point operand.
	!	logical negate	Returns 1 if the operands' value is 0; otherwise 0. For example, if buf is 0 then !buf is 1. If buf has a value of 1000 then !buf is 0.
Arithmetic	*	multiplication	Yields the product of its operands.
	/	division	Yields the quotient of the division of the first operand by the second. For integer operands, the divide operation produces a truncated integer result.

Type	Operator	Name	Description
	%	modulo	Integer only. This operator yields the remainder from the division of the first operand by the second.
	+	addition	Yields the sum of its operands.
	-	subtraction	Yields the difference of its operands.
Shift	<<	shift left	Integer only. Causes the left operand to be shifted to the left (and zero-filled) by the number of bits specified by the right operand.
	>>	shift right	Integer only. Causes the left operand to be shifted to the right by the number of bits specified by the right operand. The sign bit will be extended.
Relational	<	less than	Returns an integer 1 if the indicated condition is TRUE or an integer 0 if the indicated condition is FALSE.
	<=	less than or equal	
	>	greater than	
	>=	greater than or equal	For example, if D has a value of 3 and E has a value of 5, then the result of the expression D<E is 1, and the result of the expression D>E is 0.
	==	equal	
	!=	not equal	Use tests for equality involving floating-point values with caution, since rounding errors could cause unexpected results.
Bit and Bitwise	:	bit position	Specify bit position (right operand) in a data register (left operand) for use in bit operations (instructions that have the .T data type modifier).
	&	AND	Integer only. Yields the bitwise AND function of its operand.
		OR	Integer only. Yields the bitwise OR function of its operand.
	^	exclusive OR	Integer only. Yields the bitwise exclusive OR function of its operands.
Logical	&&	logical AND	Returns an integer 1 if both operands are non-zero; otherwise, it returns an integer 0.
		logical OR	Returns an integer 1 if either of the operands is non-zero; otherwise, it returns an integer 1

The relational operators and logical operators are intended primarily for use with the conditional assembly .if directive, but can be used in any expression.

3.7. Working with Sections

Sections are absolute or relocatable blocks of contiguous memory that can contain code or data. Some sections contain code or data that your program declared and uses directly, while other sections are created by the compiler or linker and contain debug information or code or data to initialize your application.

These sections can be named in such a way that different modules can implement different parts of these sections. These sections are located in memory by the linker (using the linker script language, LSL) so that concerns about memory placement are postponed until after the assembly process.

All instructions and directives which generate data or code must be within an active section. The assembler emits a warning if code or data starts without a section definition and activation. The compiler automatically generates sections. If you program in assembly you have to define sections yourself.

For more information about locating sections see [Section 7.9.9, The Section Layout Definition: Locating Sections](#).

Section definition

Sections are defined with the `.SDECL` directive and have a name. A section may have attributes to instruct the linker to place it on a predefined starting address, or that it may be overlaid with another section.

```
.SDECL "name", type [, attribute ]... [AT address]
```

See the description of the [.SDECL directive](#) for a complete description of all possible attributes.

Section activation

Sections are defined once and are activated with the `.SECT` directive.

```
.SECT "name"
```

The linker will check between different modules and emits an error message if the section attributes do not match. The linker will also concatenate all matching section definitions into one section. So, all "code" sections generated by the compiler will be linked into one big "code" chunk which will be located in one piece. A `.SECT` directive referring to an earlier defined section is called a *continuation*. Only the name can be specified.

Examples

```
.SDECL ".text.hello.main", CODE
.SECT ".text.hello.main"
```

Defines and activates a relocatable section in CODE memory. Other parts of this section, with the same name, may be defined in the same module or any other module. Other modules should use the same `.SDECL` statement. When necessary, it is possible to give the section an absolute starting address.

```
.SDECL ".CONST", CODE AT 0x1000
.SECT ".CONST"
```

Defines and activates an absolute section named `.CONST` starting at address 0x1000.

```
.SDECL ".fardata", DATA, CLEAR
.SECT ".fardata"
```

Defines a relocatable named section in DATA memory. The CLEAR attribute instructs the linker to clear the memory located to this section. When this section is used in another module it must be defined identically. Continuations of this section in the same module are as follows:

```
.SECT    ".fardata"
```

3.8. Built-in Assembly Functions

The TASKING assembler has several built-in functions to support data conversion, string comparison, and math computations. You can use functions as terms in any expression.

Syntax of an assembly function

```
@function_name([argument[, argument]...])
```

Functions start with the '@' character and have zero or more arguments, and are always followed by opening and closing parentheses. White space (a blank or tab) is not allowed between the function name and the opening parenthesis and between the (comma-separated) arguments.

The names of assembly functions are case insensitive.

Overview of mathematical functions

Try to avoid usage of assembler functions that work with float values. The assembler uses IEEE floating-point routines of the host on which the assembler runs to calculate some fixed floating-point values. Because of the fact that there are differences between hosts (Windows, Linux and Solaris) with respect to the number of bits used and rounding mechanism (although all claim to be IEEE compliant) it is possible that some internal assembler functions return a slightly different value depending on the input. The difference is usually at position 16 behind the comma.

Function	Description
@ABS(<i>expr</i>)	Absolute value
@ACS(<i>expr</i>)	Arc cosine
@ASN(<i>expr</i>)	Arc sine
@AT2(<i>expr1</i> , <i>expr2</i>)	Arc tangent of <i>expr1</i> / <i>expr2</i>
@ATN(<i>expr</i>)	Arc tangent
@CEL(<i>expr</i>)	Ceiling function
@COH(<i>expr</i>)	Hyperbolic cosine
@COS(<i>expr</i>)	Cosine
@FLR(<i>expr</i>)	Floor function
@L10(<i>expr</i>)	Log base 10
@LOG(<i>expr</i>)	Natural logarithm
@MAX(<i>expr1</i> [, ... , <i>exprN</i>])	Maximum value

Function	Description
@MIN(<i>expr1</i> [, . . . , <i>exprN</i>])	Minimum value
@POW(<i>expr1</i> , <i>expr2</i>)	Raise to a power
@RND ()	Random value
@SGN(<i>expr</i>)	Returns the sign of an expression as -1, 0 or 1
@SIN(<i>expr</i>)	Sine
@SNH(<i>expr</i>)	Hyperbolic sine
@SQT(<i>expr</i>)	Square root
@TAN(<i>expr</i>)	Tangent
@TNH(<i>expr</i>)	Hyperbolic tangent
@XPN(<i>expr</i>)	Exponential function (raise e to a power)

Overview of conversion functions

Function	Description
@CVF(<i>expr</i>)	Convert integer to floating-point
@CVI(<i>expr</i>)	Convert floating-point to integer
@FLD(<i>base</i> , <i>value</i> , <i>width</i> [, <i>start</i>])	Shift and mask operation
@FRACT(<i>expr</i>)	Convert floating-point to 32-bit fractional
@SFRACT(<i>expr</i>)	Convert floating-point to 16-bit fractional
@LNG(<i>expr1</i> , <i>expr2</i>)	Concatenate to double word
@LUN(<i>expr</i>)	Convert long fractional to floating-point
@RVB(<i>expr</i> [, <i>exprN</i>])	Reverse order of bits in field
@UNF(<i>expr</i>)	Convert fractional to floating-point

Overview of string functions

Function	Description
@CAT(<i>str1</i> , <i>str2</i>)	Concatenate <i>str1</i> and <i>str2</i>
@LEN(<i>string</i>)	Length of string
@POS(<i>str1</i> , <i>str2</i> [, <i>start</i>])	Position of <i>str2</i> in <i>str1</i>
@SCP(<i>str1</i> , <i>str2</i>)	Compare <i>str1</i> with <i>str2</i>
@SUB(<i>str</i> , <i>expr1</i> , <i>expr2</i>)	Return substring

Overview of macro functions

Function	Description
@ARG('symbol' <i>expr</i>)	Test if macro argument is present

Function	Description
@CNT ()	Return number of macro arguments
@MAC (<i>symbol</i>)	Test if macro is defined
@MXP ()	Test if macro expansion is active

Overview of address calculation functions

Function	Description
@HI (<i>expr</i>)	Returns upper 16 bits of expression value
@HIS (<i>expr</i>)	Returns upper 16 bits of expression value, adjusted for signed addition
@LO (<i>expr</i>)	Returns lower 16 bits of expression value
@LOS (<i>expr</i>)	Returns lower 16 bits of expression value, adjusted for signed addition
@LSB (<i>expr</i>)	Least significant byte of the expression
@MSB (<i>expr</i>)	Most significant byte of the expression

Overview of assembler mode functions

Function	Description
@ASTC ()	Returns the name of the assembler executable
@DEF (' <i>symbol</i> ' <i>symbol</i>)	Returns 1 if symbol has been defined
@EXP (<i>expr</i>)	Expression check
@INT (<i>expr</i>)	Integer check
@LST ()	LIST control flag value

Detailed Description of Built-in Assembly Functions

@ABS(*expression*)

Returns the absolute value of the expression as an integer value.

Example:

```
AVAL .SET @ABS( -2.1 ) ; AVAL = 2
```

@ACS(*expression*)

Returns the arc cosine of *expression* as a floating-point value in the range zero to pi. The result of *expression* must be between -1 and 1.

Example:

```
ACOS .SET @ACS(-1.0) ;ACOS = 3.1415926535897931
```

@ARG(*symbol* | *expression*)

Returns integer 1 if the macro argument represented by *symbol* or *expression* is present, 0 otherwise.

You can specify the argument with a *symbol* name (the name of a macro argument enclosed in single quotes) or with *expression* (the ordinal number of the argument in the macro formal argument list). If you use this function when macro expansion is not active, the assembler issues a warning.

Example:

```
.IF @ARG('TWIDDLE') ;is argument twiddle present?  
.IF @ARG(1)          ;is first argument present?
```

@ASN(*expression*)

Returns the arc sine of *expression* as a floating-point value in the range $-\pi/2$ to $\pi/2$. The result of *expression* must be between -1 and 1.

Example:

```
ARCSINE .SET @ASN(-1.0) ;ARCSINE = -1.570796
```

@ASTC()

Returns the name of the assembler executable. This is 'astc' for the TriCore assembler.

Example:

```
ANAME: .byte @ASTC() ;ANAME = 'astc'
```

@AT2(*expression1*,*expression2*)

Returns the arc tangent of *expression1*/*expression2* as a floating-point value in the range $-\pi$ to π . *expression1* and *expression2* must be separated by a comma.

Example:

```
ATAN2 .EQU @AT2(-1.0,1.0) ;ATAN2 = -0.7853982
```

@ATN(*expression*)

Returns the arc tangent of *expression* as a floating-point value in the range $-\pi/2$ to $\pi/2$.

Example:

```
ATAN .SET @ATN(1.0) ;ATAN = 0.78539816339744828
```

@CAT(string1,string2)

Concatenates the two strings into one string. The two strings must be enclosed in single or double quotes.

Example:

```
.DEFINE ID "@CAT('TASK','ING')";ID = 'TASKING'
```

@CEL(expression)

Returns a floating-point value which represents the smallest integer greater than or equal to *expression*.

Example:

```
CEIL .SET @CEL(-1.05);CEIL = -1.0
```

@CNT()

Returns the number of macro arguments of the current macro expansion as an integer. If you use this function when macro expansion is not active, the assembler issues a warning.

Example:

```
ARGCOUNT .SET @CNT(); reserve argument count
```

@COH(expression)

Returns the hyperbolic cosine of *expression* as a floating-point value.

Example:

```
HYCOS .EQU @COH(VAL);compute hyperbolic cosine
```

@COS(expression)

Returns the cosine of *expression* as a floating-point value.

Example:

```
.WORD -@COS(@CVF(COUNT)*FREQ);compute cosine value
```

@CVF(expression)

Converts the result of *expression* to a floating-point value.

Example:

```
FLOAT .SET @CVF(5);FLOAT = 5.0
```

@CVI(*expression*)

Converts the result of *expression* to an integer value. This function should be used with caution since the conversions can be inexact (e.g., floating-point values are truncated).

Example:

```
INT    .SET    @CVI(-1.05)        ;INT = -1
```

@DEF('symbol' | *symbol*)

Returns 1 if *symbol* has been defined, 0 otherwise. *symbol* can be any symbol or label not associated with a `.MACRO` or `.SDECL` directive. If *symbol* is quoted, it is looked up as a `.DEFINE` symbol; if it is not quoted, it is looked up as an ordinary symbol or label.

Example:

```
.IF @DEF('ANGLE')        ;is symbol ANGLE defined?
.IF @DEF(ANGLE)           ;does label ANGLE exist?
```

@EXP(*expression*)

Returns 0 if the evaluation of *expression* would normally result in an error. Returns 1 if the expression can be evaluated correctly. With the @EXP function, you prevent the assembler from generating an error if the expression contains an error. No test is made by the assembler for warnings. The expression may be relative or absolute.

Example:

```
.IF    !@EXP(3/0)        ;Do the IF on error
                        ;assembler generates no error

.IF    !(3/0)            ;assembler generates an error
```

@FLD(*base,value,width[,start]*)

Shift and mask *value* into *base* for *width* bits beginning at bit *start*. If *start* is omitted, zero (least significant bit) is assumed. All arguments must be positive integers and none may be greater than the target word size. Returns the shifted and masked value.

Example:

```
VAR1 .EQU @FLD(0,1,1)    ;turn bit 0 on, VAR1=1
VAR2 .EQU @FLD(0,3,1)    ;turn bit 0 on, VAR2=1
VAR3 .EQU @FLD(0,3,2)    ;turn bits 0 and 1 on, VAR3=3
VAR4 .EQU @FLD(0,3,2,1)  ;turn bits 1 and 2 on, VAR4=6
VAR5 .EQU @FLD(0,1,1,7)  ;turn eighth bit on, VAR5=0x80
```

@FLR(*expression*)

Returns a floating-point value which represents the largest integer less than or equal to *expression*.

Example:

```
FLOOR    .SET    @FLR(2.5)          ;FLOOR = 2.0
```

@FRACT(expression)

Returns the 32-bit fractional representation of the floating-point *expression*. The expression must be in the range [-1,+1>.

Example:

```
.WORD    @FRACT(0.1), @FRACT(1.0)
```

@HI(expression)

Returns the upper 16 bits of a value. @HI(expression) is equivalent to ((expression>>16) & 0xffff).

Example:

```
mov.u    d2, #@LO(COUNT)
addih    d2, d2, #@HI(COUNT)        ;upper 16 bits of COUNT
```

@HIS(expression)

Returns the upper 16 bits of a value, adjusted for a signed addition. @HIS(expression) is equivalent to (((expression+0x8000)>>16) & 0xffff).

Example:

```
movh.a   a3, #@HIS(label)
lea      a3, [a3]@LOS(label)
```

@INT(expression)

Returns integer 1 if *expression* has an integer result; otherwise, it returns a 0. The expression may be relative or absolute.

Example:

```
.IF    @INT(TERM)          ;Test if result is an integer
```

@L10(expression)

Returns the base 10 logarithm of *expression* as a floating-point value. *expression* must be greater than zero.

Example:

```
LOG      .EQU    @L10(100.0)        ;LOG = 2
```

@LEN(string)

Returns the length of *string* as an integer.

Example:

```
SLEN    .SET    @LEN('string')    ;SLEN = 6
```

@LNG(expression1,expression2)

Concatenates the 16-bit *expression1* and *expression2* into a 32-bit word value such that *expression1* is the high half and *expression2* is the low half.

Example:

```
LWORD    .WORD    @LNG(HI,LO)        ;build long word
```

@LO(expression)

Returns the lower 16 bits of a value. @LO(*expression*) is equivalent to (*expression* & 0xffff).

Example:

```
mov.u    d2, #@LO(COUNT)              ;lower 16 bits of COUNT
addih    d2, d2, #@HI(COUNT)
```

@LOG(expression)

Returns the natural logarithm of *expression* as a floating-point value. *expression* must be greater than zero.

Example:

```
LOG      .EQU    @LOG(100.0)          ;LOG = 4.605170
```

@LOS(expression)

Returns the lower 16 bits of a value, adjusted for a signed addition. @LOS(*expression*) is equivalent to (((*expression*+0x8000) & 0xffff) - 0x8000).

Example:

```
movh.a    a3, #@HIS(label)
lea        a3, [a3]@LOS(label)
```

@LSB(expression)

Returns the least significant byte of the result of the *expression*. The result of the expression is calculated as 16 bit.

Example:

```
VAR1    .SET @LSB(0x34)           ;VAR1 = 0x34
VAR2    .SET @LSB(0x1234)         ;VAR2 = 0x34
VAR3    .SET @LSB(0x654321)       ;VAR3 = 0x21
```

@LST()

Returns the value of the **\$LIST ON/OFF control** flag as an integer. Whenever a **\$LIST ON** control is encountered in the assembler source, the flag is incremented; when a **\$LIST OFF** control is encountered, the flag is decremented.

Example:

```
.DUP    @ABS(@LST())              ;list unconditionally
```

@LUN(expression)

Converts the 32-bit *expression* to a floating-point value. *expression* should represent a binary fraction.

Example:

```
DBLFRC1 .EQU @LUN(0x40000000)     ;DBLFRC1 = 0.5
DBLFRC2 .EQU @LUN(3928472)        ;DBLFRC2 = 0.007354736
DBLFRC3 .EQU @LUN(0xE0000000)     ;DBLFRC3 = -0.75
```

@MAC(symbol)

Returns integer 1 if *symbol* has been defined as a macro name, 0 otherwise.

Example:

```
.IF    @MAC(DOMUL)                ;does macro DOMUL exist?
```

@MAX(expression1[,expressionN],...)

Returns the maximum value of *expression1*, ..., *expressionN* as a floating-point value.

Example:

```
MAX:    .BYTE @MAX(1, -2.137, 3.5) ;MAX = 3.5
```

@MIN(expression1[,expressionN],...)

Returns the minimum value of *expression1*, ..., *expressionN* as a floating-point value.

Example:

```
MIN:    .BYTE @MIN(1, -2.137, 3.5) ;MIN = -2.137
```

@MSB(*expression*)

Returns the most significant byte of the result of the *expression*. The result of the expression is calculated as 16 bit.

Example:

```
VAR1  .SET @MSB(0x34)           ;VAR1 = 0x00
VAR2  .SET @MSB(0x1234)         ;VAR2 = 0x12
VAR3  .SET @MSB(0x654321)       ;VAR3 = 0x43
```

@MXP()

Returns integer 1 if the assembler is expanding a macro, 0 otherwise.

Example:

```
.IF    @MXP()                   ;macro expansion active?
```

@POS(*string1*,*string2*[,*start*])

Returns the position of *string2* in *string1* as an integer. If *string2* does not occur in *string1*, the last string position + 1 is returned.

With *start* you can specify the starting position of the search. If you do not specify *start*, the search is started from the beginning of *string1*. Note that the first position in a string is position 0.

Example:

```
ID1  .EQU @POS('TASKING','ASK') ; ID1 = 1
ID2  .EQU @POS('ABCDABCD','B',2) ; ID2 = 5
ID3  .EQU @POS('TASKING','BUG')  ; ID3 = 7
```

@POW(*expression1*,*expression2*)

Returns *expression1* raised to the power *expression2* as a floating-point value. *expression1* and *expression2* must be separated by a comma.

Example:

```
BUF  .EQU @CVI(@POW(2.0,3.0))    ;BUF = 8
```

@RND()

Returns a random value in the range 0.0 to 1.0.

Example:

```
SEED  .EQU @RND()                ;save initial SEED value
```


@RVB(*expression1*,*expression2*)

Reverse the order of bits in *expression1* delimited by the number of bits in *expression2*. If *expression2* is omitted the field is bounded by the target word size. Both expressions must be 16-bit integer values.

Example:

```
VAR1 .SET @RVB(0x200)    ;reverse all bits, VAR1=0x40
VAR2 .SET @RVB(0xB02)    ;reverse all bits, VAR2=0x40D0
VAR3 .SET @RVB(0xB02,2)  ;reverse bits 0 and 1,
                        ;VAR3=0xB01
```

@SCP(*string1*,*string2*)

Returns integer 1 if the two strings compare, 0 otherwise. The two strings must be separated by a comma.

Example:

```
.IF @SCP(STR, 'MAIN')   ; does STR equal 'MAIN'?
```

@SFRACT(*expression*)

This function returns the 16-bit fractional representation of the floating-point expression. The expression must be in the range [-1,+1].

Example:

```
.WORD @SFRACT(0.1), @SFRACT(1.0)
```

@SGN(*expression*)

Returns the sign of *expression* as an integer: -1 if the argument is negative, 0 if zero, 1 if positive. The expression may be relative or absolute.

Example:

```
VAR1 .SET @SGN(-1.2e-92) ;VAR1 = -1
VAR2 .SET @SGN(0)        ;VAR2 = 0
VAR3 .SET @SGN(28.382)    ;VAR3 = 1
```

@SIN(*expression*)

Returns the sine of *expression* as a floating-point value.

Example:

```
.WORD @SIN(@CVF(COUNT)*FREQ) ;compute sine value
```

@SNH(*expression*)

Returns the hyperbolic sine of *expression* as a floating-point value.

Example:

```
HSINE .EQU @SNH(VAL) ;hyperbolic sine
```

@SQT(expression)

Returns the square root of *expression* as a floating-point value. *expression* must be positive.

Example:

```
SQRT1 .EQU @SQT(3.5) ;SQRT1 = 1.870829
SQRT2 .EQU @SQT(16) ;SQRT2 = 4
```

@SUB(string,expression1,expression2)

Returns the substring from *string* as a string. *expression1* is the starting position within *string*, and *expression2* is the length of the desired string. The assembler issues an error if either *expression1* or *expression2* exceeds the length of string. Note that the first position in a string is position 0.

Example:

```
.DEFINE ID "@SUB('TASKING',3,4)" ;ID = 'KING'
```

@TAN(expression)

Returns the tangent of *expression* as a floating-point value.

Example:

```
TANGENT .SET @TAN(1.0) ;TANGENT = 1.5574077
```

@TNH(expression)

Returns the hyperbolic tangent of *expression* as a floating-point value.

Example:

```
HTAN .SET @TNH(1) ;HTAN = 0.76159415595
```

@UNF(expression)

Converts *expression* to a floating-point value. *expression* should represent a 16-bit binary fraction.

Example:

```
FRC .EQU @UNF(0x4000) ;FRC = 0.5
```

@XPN(expression)

Returns the exponential function (base e raised to the power of *expression*) as a floating-point value.

Example:

```
EXP      .EQU    @XPN(1.0)           ;EXP = 2.718282
```

3.9. Assembler Directives and Controls

An assembler directive is simply a message to the assembler. Assembler directives are not translated into machine instructions. There are three main groups of assembler directives.

- Assembler directives that tell the assembler how to go about translating instructions into machine code. This is the most typical form of assembly directives. Typically they tell the assembler where to put a program in memory, what space to allocate for variables, and allow you to initialize memory with data. When the assembly source is assembled, a location counter in the assembler keeps track of where the code and data is to go in memory.

The following directives fall under this group:

- Assembly control directives
- Symbol definition and section directives
- Data definition / Storage allocation directives
- High Level Language (HLL) directives
- Directives that are interpreted by the macro preprocessor. These directives tell the macro preprocessor how to manipulate your assembly code before it is actually being assembled. You can use these directives to write macros and to write conditional source code. Parts of the code that do not match the condition, will not be assembled at all.
- Some directives act as assembler options and most of them indeed do have an equivalent assembler (command line) option. The advantage of using a directive is that with such a directive you can overrule the assembler option for a particular part of the code. Directives of this kind are called *controls*. A typical example is to tell the assembler with an option to generate a list file while with the controls `$LIST ON` and `$LIST OFF` you overrule this option for a part of the code that you do not want to appear in the list file. Controls always appear on a separate line and start with a '\$' sign in the first column.

The following controls are available:

- Assembly listing controls
- Miscellaneous controls

Each assembler directive or control has its own syntax. You can use assembler directives and controls in the assembly code as pseudo instructions.

Some assembler directives can be preceded with a label. If you do not precede an assembler directive with a label, you must use white space instead (spaces or tabs). The assembler recognizes both uppercase and lowercase for directives.

3.9.1. Assembler Directives

Overview of assembly control directives

Directive	Description
<code>.COMMENT</code>	Start comment lines. You cannot use this directive in <code>.IF/.ELSE/.ENDIF</code> constructs and <code>.MACRO/.DUP</code> definitions.
<code>.END</code>	Indicates the end of an assembly module
<code>.FAIL</code>	Programmer generated error message
<code>.INCLUDE</code>	Include file
<code>.MESSAGE</code>	Programmer generated message
<code>.WARNING</code>	Programmer generated warning message

Overview of symbol definition and section directives

Directive	Description
<code>.ALIAS</code>	Create an alias for a symbol
<code>.EQU</code>	Set permanent value to a symbol
<code>.EXTERN</code>	Import global section symbol
<code>.GLOBAL</code>	Declare global section symbol
<code>.LOCAL</code>	Declare local section symbol
<code>.ORG</code>	Initialize memory space and location counters to create a nameless section
<code>.SDECL</code>	Declare a section with name, type and attributes
<code>.SECT</code>	Activate a declared section
<code>.SET</code>	Set temporary value to a symbol
<code>.SIZE</code>	Set size of symbol in the ELF symbol table
<code>.TYPE</code>	Set symbol type in the ELF symbol table
<code>.WEAK</code>	Mark a symbol as 'weak'

Overview of data definition / storage allocation directives

Directive	Description
<code>.ACCUM</code>	Define 64-bit constant of 18 + 46 bits format
<code>.ALIGN</code>	Align location counter
<code>.ASCII, .ASCIIZ</code>	Define ASCII string without / with ending NULL byte
<code>.BYTE</code>	Define byte
<code>.DOUBLE</code>	Define a 64-bit floating-point constant
<code>.FLOAT</code>	Define a 32-bit floating-point constant
<code>.FRACT</code>	Define a 32-bit constant fraction

Directive	Description
.HALF	Define half-word (16 bits)
.SFRACT	Define a 16-bit constant fraction
.SPACE	Define storage
.WORD	Define word (32 bits)

Overview of macro preprocessor directives

Directive	Description
.DEFINE	Define substitution string
.DUP , .ENDM	Duplicate sequence of source lines
.DUPA , .ENDM	Duplicate sequence with arguments
.DUPC , .ENDM	Duplicate sequence with characters
.DUPF , .ENDM	Duplicate sequence in loop
.IF , .ELIF , .ELSE	Conditional assembly directive
.ENDIF	End of conditional assembly directive
.EXITM	Exit macro
.MACRO , .ENDM	Define macro
.PMACRO	Undefine (purge) macro
.UNDEF	Undefine .DEFINE symbol

Overview of HLL directives

Directive	Description
.CALLS	Pass call tree information and/or stack usage information
.COMPILER_INVOCATION	Pass C compiler invocation
.COMPILER_NAME	Pass C compiler name
.COMPILER_VERSION	Pass C compiler version header
.MISRA C	Pass MISRA C information

.ACCUM

Syntax

[*label*:].ACCUM *expression*[,*expression*]. . .

Description

With the .ACCUM directive the assembler allocates and initializes two words of memory (64 bits) for each argument. Use commas to separate multiple arguments.

An *expression* can be:

- a fractional fixed point expression (range $[-2^{17}, 2^{17}]$)
- NULL (indicated by two adjacent commas: ,,)

Multiple arguments are stored in successive address locations in sets of two words. If an argument is NULL its corresponding address location is filled with zeros.

If the evaluated expression is out of the range $[-2^{17}, 2^{17}]$, the assembler issues a warning and saturates the fractional value.

Example

```
ACC:  .ACCUM  0.1, 0.2, 0.3
```

Related Information

[.FRACT](#), [.SFRACT](#) (Define 32-bit/16-bit constant fraction)

[.SPACE](#) (Define Storage)

.ALIAS

Syntax

alias-name **.ALIAS** *symbol-name*

Description

With the **.ALIAS** directive you can create an alias of a symbol. The C compiler generates this directive when you use the `#pragma alias`.

Example

```
exit .ALIAS _Exit
```

Related information

[Pragma alias](#)

.ALIGN

Syntax

.ALIGN *expression*

Description

With the `.ALIGN` directive you instruct the assembler to align the location counter. By default the assembler aligns on one byte.

When the assembler encounters the `.ALIGN` directive, it advances the location counter to an address that is aligned as specified by *expression* and places the next instruction or directive on that address. The alignment is in minimal addressable units (MAUs). The assembler fills the 'gap' with NOP instructions for code sections or with zeros for data sections. If the location counter is already aligned on the specified alignment, it remains unchanged. The location of absolute sections will not be changed.

The *expression* must be a power of two: 2, 4, 8, 16, ... If you specify another value, the assembler changes the alignment to the next higher power of two and issues a warning.

The assembler aligns sections automatically to the largest alignment value occurring in that section.

A label is not allowed before this directive.

Example

```

.sdecl '.text.mod.csec',code
.sect  '.text.mod.csec'
.ALIGN 16      ; the assembler aligns
instruction    ; this instruction at 16 MAUs and
               ; fills the 'gap' with NOP instructions.

.sdecl '.text.mod.csec2',code
.sect  '.text.mod.csec2'
.ALIGN 12      ; WRONG: not a power of two, the
instruction    ; assembler aligns this instruction at
               ; 16 MAUs and issues a warning.

```

.ASCII, .ASCIIZ

Syntax

```
[label:] .ASCII  string[,string]...
```

```
[label:] .ASCIIZ string[,string]...
```

Description

With the `.ASCII` or `.ASCIIZ` directive the assembler allocates and initializes memory for each *string* argument.

The `.ASCII` directive does not add a NULL byte to the end of the string. The `.ASCIIZ` directive does add a NULL byte to the end of the string. The "z" in `.ASCIIZ` stands for "zero". Use commas to separate multiple strings.

Example

```
STRING:  .ASCII  "Hello world"  
STRINGZ: .ASCIIZ "Hello world"
```

Note that with the `.BYTE` directive you can obtain exactly the same effect:

```
STRING:  .BYTE  "Hello world"      ; without a NULL byte  
STRINGZ: .BYTE  "Hello world",0    ; with a NULL byte
```

Related Information

[.BYTE](#) (Define a constant byte)

[.SPACE](#) (Define Storage)

.BYTE

Syntax

```
[label:] .BYTE argument[,argument]...
```

Description

With the `.BYTE` directive the assembler allocates and initializes a byte of memory for each argument.

An *argument* can be:

- a single or multiple character string constant
- an integer expression
- NULL (indicated by two adjacent commas: ,,)

Multiple arguments are stored in successive byte locations. If an argument is NULL its corresponding byte location is filled with zeros.

If you specify *label*, it gets the value of the location counter at the start of the directive processing.

Integer arguments are stored as is, but must be byte values (within the range 0-255); floating-point numbers are not allowed. If the evaluated expression is out of the range [-256, +255] the assembler issues an error. For negative values within that range, the assembler adds 256 to the specified value (for example, -254 is stored as 2).

String constants

Single-character strings are stored in a byte whose lower seven bits represent the ASCII value of the character, for example:

```
.BYTE 'R'          ; = 0x52
```

Multiple-character strings are stored in consecutive byte addresses, as shown below. The standard C language escape characters like '\n' are permitted.

```
.BYTE 'AB',, 'C'    ; = 0x41420043 (second argument is empty)
```

Example

```
TABLE .BYTE 'two',0,'strings',0
CHARS .BYTE 'A','B','C','D'
```

Related Information

[.ASCII](#), [.ASCIIZ](#) (Define ASCII string without/with ending NULL)

[.WORD](#), [.HALF](#) (Define a word / halfword)

[.SPACE](#) (Define Storage)

.CALLS

Syntax

```
.CALLS 'caller','callee'
```

or

```
.CALLS 'caller','',stack_usage
```

Description

The first syntax creates a call graph reference between *caller* and *callee*. The linker needs this information to build a call graph. *caller* and *callee* are names of functions.

The second syntax specifies stack information. When *callee* is an empty name, this means we define the stack usage of the function itself. The value specified is the stack usage in bytes at the time of the call including the return address.

This information is used by the linker to compute the used stack within the application. The information is found in the generated linker map file within the Memory Usage.

This directive is generated by the C compiler. Normally you will not use it in hand-coded assembly.

A label is not allowed before this directive.

Example

```
.CALLS 'main','nfunc'
```

Indicates that the function `main` calls the function `nfunc`.

```
.CALLS 'main','',8
```

The function `main` uses 8 bytes on the stack.

.COMMENT

Syntax

```
.COMMENT  delimiter  
.  
.  
delimiter
```

Description

With the `.COMMENT` directive you can define one or more lines as comments. The first non-blank character after the `.COMMENT` directive is the comment delimiter. The two delimiters are used to define the comment text. The line containing the second comment delimiter will be considered the last line of the comment. The comment text can include any printable characters and the comment text will be produced in the source listing as it appears in the source file.

A label is not allowed before this directive.

Example

```
.COMMENT  + This is a one line comment +  
.COMMENT  * This is a multiple line  
           comment. Any number of lines  
           can be placed between the two  
           delimiters.  
           *
```

.COMPILER_INVOCATION, .COMPILER_NAME, .COMPILER_VERSION

Syntax

```
.COMPILER_VERSION "version_header"  
.COMPILER_INVOCATION "invocation"  
.COMPILER_NAME "name"
```

Description

The C compiler generates information about itself and the invocation at the start of the assembly source. This way you can always see how the assembly source file was generated. When you assemble the source file, this information will appear in `.note` sections in the object file.

A label is not allowed before these directives.

Example

```
.COMPILER_VERSION "TASKING VX-toolset for TriCore: C compiler vx.yrz Build yymddq"  
.COMPILER_INVOCATION "ctc --core=tc1.3 --fp-model=+float test.c"  
.COMPILER_NAME "ctc"
```

.DEFINE

Syntax

```
.DEFINE symbol string
```

Description

With the `.DEFINE` directive you define a substitution string that you can use on all following source lines. The assembler searches all succeeding lines for an occurrence of *symbol*, and replaces it with *string*. If the *symbol* occurs in a double quoted string it is also replaced. Strings between single quotes are not expanded.

This directive is useful for providing better documentation in the source program. A *symbol* can consist of letters, digits and underscore characters (`_`), and the first character cannot be a digit.

Macros represent a special case. `.DEFINE` directive translations will be applied to the macro definition as it is encountered. When the macro is expanded, any active `.DEFINE` directive translations will again be applied.

The assembler issues a warning if you redefine an existing symbol.

A label is not allowed before this directive.

Example

Suppose you defined the symbol `LEN` with the substitution string `"32"`:

```
.DEFINE LEN "32"
```

Then you can use the symbol `LEN` for example as follows:

```
.SPACE LEN  
.MESSAGE "The length is: LEN"
```

The assembler preprocessor replaces `LEN` with `"32"` and assembles the following lines:

```
.SPACE 32  
.MESSAGE "The length is: 32"
```

Related Information

`.UNDEF` (Undefine a `.DEFINE` symbol)

`.MACRO`, `.ENDM` (Define a macro)

.DUP, .ENDM

Syntax

```
[label:] .DUP expression  
    . . . .  
    .ENDM
```

Description

With the .DUP/.ENDM directive you can duplicate a sequence of assembly source lines. With *expression* you specify the number of duplications. If the *expression* evaluates to a number less than or equal to 0, the sequence of lines will not be included in the assembler output. The *expression* result must be an absolute integer and cannot contain any forward references (symbols that have not already been defined). The .DUP directive may be nested to any level.

If you specify *label*, it gets the value of the location counter at the start of the directive processing.

Example

In this example the loop is repeated three times. Effectively, the preprocessor repeats the source lines (.BYTE 10) three times, then the assembler assembles the result:

```
.DUP 3  
.BYTE 10 ; assembly source lines  
.ENDM
```

Related Information

- .DUPA, .ENDM (Duplicate sequence with arguments)
- .DUPC, .ENDM (Duplicate sequence with characters)
- .DUPF, .ENDM (Duplicate sequence in loop)
- .MACRO, .ENDM (Define a macro)

.DUPA, .ENDM

Syntax

```
[label:] .DUPA formal_arg, argument [, argument] ...
      . . .
      .ENDM
```

Description

With the `.DUPA/.ENDM` directive you can repeat a block of source statements for each *argument*. For each repetition, every occurrence of the *formal_arg* parameter within the block is replaced with each succeeding *argument* string. If an argument includes an embedded blank or other assembler-significant character, it must be enclosed with single quotes.

If you specify *label*, it gets the value of the location counter at the start of the directive processing.

Example

Consider the following source input statements,

```
.DUPA  VALUE, 12, , 32, 34
.BYTE  VALUE
.ENDM
```

This is expanded as follows:

```
.BYTE  12
.BYTE  VALUE ; results in a warning
.BYTE  32
.BYTE  34
```

The second statement results in a warning of the assembler that the local symbol `VALUE` is not defined in this module and is made external.

Related Information

- `.DUP, .ENDM` (Duplicate sequence of source lines)
- `.DUPC, .ENDM` (Duplicate sequence with characters)
- `.DUPF, .ENDM` (Duplicate sequence in loop)
- `.MACRO, .ENDM` (Define a macro)

.DUPC, .ENDM

Syntax

```
[label:] .DUPC formal_arg,string  
    ....  
    .ENDM
```

Description

With the .DUPC/.ENDM directive you can repeat a block of source statements for each character within *string*. For each character in the string, the *formal_arg* parameter within the block is replaced with that character. If the string is empty, then the block is skipped.

If you specify *label*, it gets the value of the location counter at the start of the directive processing.

Example

Consider the following source input statements,

```
.DUPC  VALUE, '123'  
.BYTE  VALUE  
.ENDM
```

This is expanded as follows:

```
.BYTE  1  
.BYTE  2  
.BYTE  3
```

Related Information

- .DUP, .ENDM (Duplicate sequence of source lines)
- .DUPA, .ENDM (Duplicate sequence with arguments)
- .DUPF, .ENDM (Duplicate sequence in loop)
- .MACRO, .ENDM (Define a macro)

.DUPF, .ENDM

Syntax

```
[label:] .DUPF formal_arg, [start], end[, increment]
      . . .
      .ENDM
```

Description

With the `.DUPF/.ENDM` directive you can repeat a block of source statements $(end - start) + 1 / increment$ times. *start* is the starting value for the loop index; *end* represents the final value. *increment* is the increment for the loop index; it defaults to 1 if omitted (as does the *start* value). The *formal_arg* parameter holds the loop index value and may be used within the body of instructions.

If you specify *label*, it gets the value of the location counter at the start of the directive processing.

Example

Consider the following source input statements,

```
.DUPF      NUM, 0, 7
MOV  D\NUM, #0
.ENDM
```

This is expanded as follows:

```
MOV  D0, #0
MOV  D1, #0
MOV  D2, #0
MOV  D3, #0
MOV  D4, #0
MOV  D5, #0
MOV  D6, #0
MOV  D7, #0
```

Related Information

- `.DUP, .ENDM` (Duplicate sequence of source lines)
- `.DUPA, .ENDM` (Duplicate sequence with arguments)
- `.DUPC, .ENDM` (Duplicate sequence with characters)
- `.MACRO, .ENDM` (Define a macro)

.END

Syntax

.END

Description

With the optional `.END` directive you tell the assembler that the end of the module is reached. If the assembler finds assembly source lines beyond the `.END` directive, it ignores those lines and issues a warning.

You cannot use the `.END` directive in a macro expansion.

The assembler does not allow a label with this directive.

When you use [assembler option `--require-end`](#), the `.END` directive is mandatory.

Example

```
        ; source lines
.END          ; End of assembly module
```

Related Information

-

.EQU

Syntax

symbol **.EQU** *expression*

Description

With the **.EQU** directive you assign the value of *expression* to *symbol* permanently. The expression can be relocatable or absolute and forward references are allowed. Once defined, you cannot redefine the symbol. With the **.GLOBAL** directive you can declare the symbol global.

Example

To assign the value 0x400 permanently to the symbol MYSYMBOL:

```
MYSYMBOL .EQU 0x4000
```

You cannot redefine the symbol MYSYMBOL after this.

Related Information

.SET (Set temporary value to a symbol)

.EXITM

Syntax

.EXITM

Description

With the **.EXITM** directive the assembler will immediately terminate a macro expansion. It is useful when you use it with the conditional assembly directive **.IF** to terminate macro expansion when, for example, error conditions are detected.

A label is not allowed before this directive.

Example

```
CALC  .MACRO  XVAL,YVAL
      .IF    XVAL<0
      .FAIL  'Macro parameter value out of range'
      .EXITM ;Exit macro
      .ENDIF
      .
      .
      .
      .ENDM
```

Related Information

- .DUP**, **.ENDM** (Duplicate sequence of source lines)
- .DUPA**, **.ENDM** (Duplicate sequence with arguments)
- .DUPC**, **.ENDM** (Duplicate sequence with characters)
- .DUPF**, **.ENDM** (Duplicate sequence in loop)
- .MACRO**, **.ENDM** (Define a macro)

.EXTERN

Syntax

.EXTERN *symbol* [, *symbol*] . . .

Description

With the `.EXTERN` directive you define an *external* symbol. It means that the specified symbol is referenced in the current module, but is not defined within the current module. This symbol must either have been defined outside of any module or declared as globally accessible within another module with the `.GLOBAL` directive.

If you do not use the `.EXTERN` directive and the symbol is not defined within the current module, the assembler issues a warning and inserts the `.EXTERN` directive.

A label is not allowed with this directive.

Example

```

.EXTERN AA,CC,DD      ;defined elsewhere
.sdecl ".text.code", code
.sect ".text.code"
.
.
MOV D0, #AA          ; AA is used here
.

```

Related Information

[.GLOBAL](#) (Declare global section symbol)

[.LOCAL](#) (Declare local section symbol)

.FAIL

Syntax

```
.FAIL {str|exp}[,{str|exp]}...
```

Description

With the `.FAIL` directive you tell the assembler to print an error message to `stderr` during the assembling process.

An arbitrary number of strings and expressions, in any order but separated by commas with no intervening white space, can be specified to describe the nature of the generated error. If you use expressions, the assembler outputs the result. The assembler outputs a space between each argument.

The total error count will be incremented as with any other error. The `.FAIL` directive is for example useful in combination with conditional assembly for exceptional condition checking. The assembly process proceeds normally after the error has been printed.

With this directive the assembler exits with exit code 1 (an error).

A label is not allowed with this directive.

Example

```
.FAIL 'Parameter out of range'
```

This results in the error:

```
E143: ["filename" line] Parameter out of range
```

Related Information

[.MESSAGE](#) (Programmer generated message)

[.WARNING](#) (Programmer generated warning)

.FLOAT, .DOUBLE

Syntax

`[label:] .FLOAT expression[, expression]...`

`[label:] .DOUBLE expression[, expression]...`

Description

With the `.FLOAT` or `.DOUBLE` directive the assembler allocates and initializes a floating-point number (32 bits) or a double (64 bits) in memory for each argument.

An *expression* can be:

- a floating-point expression
- NULL (indicated by two adjacent commas: ,,)

You can represent a constant as a signed whole number with fraction or with the 'e' format as used in the C language. For example, `12.457` and `+0.27E-13` are legal floating-point constants.

If the evaluated argument is too large to be represented in a single word / double-word, the assembler issues an error and truncates the value.

If you specify *label*, it gets the value of the location counter at the start of the directive processing.

Example

```
FLT:  .FLOAT   12.457, +0.27E-13
DBL:  .DOUBLE  12.457, +0.27E-13
```

Related Information

[.SPACE](#) (Define Storage)

.FRACT, .SFRACT

Syntax

`[label:] .FRACT expression[,expression]...`

`[label:] .SFRACT expression[,expression]...`

Description

With the `.FRACT` or `.SFRACT` directive the assembler allocates and initializes a 32-bit or 16-bit constant fraction in memory for each argument. Use commas to separate multiple arguments.

An *expression* can be:

- a fractional fixed point expression (range [-1, +1>)
- NULL (indicated by two adjacent commas: ,,)

Multiple arguments are stored in successive address locations in sets of two bytes. If an argument is NULL its corresponding address location is filled with zeros.

If the evaluated expression is out of the range [-1, +1>, the assembler issues a warning and saturates the fractional value.

Example

```
FRCT:    .FRACT    0.1,0.2,0.3
SFRCT:    .SFRACT   0.1,0.2,0.3
```

Related Information

[.ACCUM](#) (Define 64-bit constant fraction in 18+46 bits format)

[.SPACE](#) (Define Storage)

.GLOBAL

Syntax

```
.GLOBAL symbol [, symbol] . . .
```

Description

All symbols or labels defined in the current section or module are local to the module by default. You can change this default behavior with [assembler option --symbol-scope=global](#).

With the `.GLOBAL` directive you declare one or more symbols as global. It means that the specified symbols are defined within the current section or module, and that those definitions should be accessible by all modules.

To access a symbol, defined with `.GLOBAL`, from another module, use the `.EXTERN` directive.

Only program labels and symbols defined with `.EQU` can be made global.

If the symbols that appear in the operand field are not used in the module, the assembler gives a warning.

The assembler does not allow a label with this directive.

Example

```
LOOPA .EQU 1          ; definition of symbol LOOPA
      .GLOBAL  LOOPA  ; LOOPA will be globally
                      ; accessible by other modules
```

Related Information

[.EXTERN](#) (Import global section symbol)

[.LOCAL](#) (Declare local section symbol)

.IF, .ELIF, .ELSE, .ENDIF

Syntax

```
.IF expression
.
.
[.ELIF expression] ; the .ELIF directive is optional
.
.
[.ELSE]           ; the .ELSE directive is optional
.
.
.ENDIF
```

Description

With the .IF/.ENDIF directives you can create a part of conditional assembly code. The assembler assembles only the code that matches a specified condition.

The *expression* must evaluate to an absolute integer and cannot contain forward references. If *expression* evaluates to zero, the IF-condition is considered FALSE, any non-zero result of *expression* is considered as TRUE.

If the optional .ELSE and/or .ELIF directives are not present, then the source statements following the .IF directive and up to the next .ENDIF directive will be included as part of the source file being assembled only if the *expression* had a non-zero result.

If the *expression* has a value of zero, the source file will be assembled as if those statements between the .IF and the .ENDIF directives were never encountered.

If the .ELSE directive is present and *expression* has a nonzero result, then the statements between the .IF and .ELSE directives will be assembled, and the statement between the .ELSE and .ENDIF directives will be skipped. Alternatively, if *expression* has a value of zero, then the statements between the .IF and .ELSE directives will be skipped, and the statements between the .ELSE and .ENDIF directives will be assembled.

You can nest .IF directives to any level. The .ELSE and .ELIF directive always refer to the nearest previous .IF directive.

A label is not allowed with this directive.

Example

Suppose you have an assemble source file with specific code for a test version, for a demo version and for the final version. Within the assembly source you define this code conditionally as follows:

```
.IF TEST
... ; code for the test version
.ELIF DEMO
... ; code for the demo version
.ELSE
```

```
... ; code for the final version  
.ENDIF
```

Before assembling the file you can set the values of the symbols TEST and DEMO in the assembly source before the .IF directive is reached. For example, to assemble the demo version:

```
TEST .SET 0  
DEMO .SET 1
```

You can also define the symbols on the command line with the [assembler option --define \(-D\)](#):

```
astc --define=DEMO --define=TEST=0 test.asm
```

.INCLUDE

Syntax

.INCLUDE "*filename*" | <*filename*>

Description

With the `.INCLUDE` directive you include another file at the exact location where the `.INCLUDE` occurs. This happens before the resulting file is assembled. The `.INCLUDE` directive works similarly to the `#include` statement in C. The source from the include file is assembled as if it followed the point of the `.INCLUDE` directive. When the end of the included file is reached, assembly of the original file continues.

The string specifies the filename of the file to be included. The filename must be compatible with the operating system (forward/backward slashes) and can contain a directory specification.

If an absolute pathname is specified, the assembler searches for that file. If a relative path is specified or just a filename, the order in which the assembler searches for include files is:

1. The current directory if you use the "*filename*" construction.

The current directory is not searched if you use the <*filename*> syntax.

2. The path that is specified with the [assembler option --include-directory](#).
3. The path that is specified in the environment variable `ASTCINC` when the product was installed.
4. The default `include` directory in the installation directory.

The assembler does not allow a label with this directive.

Example

```
.INCLUDE 'storage\mem.asm'    ; include file
.INCLUDE <data.asm>          ; Do not look in
                             ; current directory
```

.LOCAL

Syntax

.LOCAL *symbol* [, *symbol*] ...

Description

All symbols or labels defined in the current section or module are local to the module by default. You can change this default behavior with [assembler option --symbol-scope=global](#).

With the `.LOCAL` directive you declare one or more symbols as local. It means that the specified symbols are explicitly local to the module in which you define them.

If the symbols that appear in the operand field are not used in the module, the assembler gives a warning.

The assembler does not allow a label with this directive.

Example

```

.SDECL ".data.io",DATA
.SECT ".data.io"
.LOCAL LOOPA ; LOOPA is local to this section

LOOPA .HALF 0x100 ; assigns the value 0x100 to LOOPA

```

Related Information

[.EXTERN](#) (Import global section symbol)

[.GLOBAL](#) (Declare global section symbol)

.MACRO, .ENDM

Syntax

```
macro_name .MACRO [argument[,argument]...]
...
macro_definition_statements
...
.ENDM
```

Description

With the `.MACRO` directive you define a macro. Macros provide a shorthand method for handling a repeated pattern of code or group of instructions. You can define the pattern as a macro, and then call the macro at the points in the program where the pattern would repeat.

The definition of a macro consists of three parts:

- *Header*, which assigns a name to the macro and defines the arguments (`.MACRO` directive).
- *Body*, which contains the code or instructions to be inserted when the macro is called.
- *Terminator*, which indicates the end of the macro definition (`.ENDM` directive).

The arguments are symbolic names that the macro processor replaces with the literal arguments when the macro is expanded (called). Each formal *argument* must follow the same rules as symbol names: the name can consist of letters, digits and underscore characters (`_`). The first character cannot be a digit. Argument names cannot start with a percent sign (`%`).

Macro definitions can be nested but the nested macro will not be defined until the primary macro is expanded.

You can use the following operators in macro definition statements:

Operator	Name	Description
\	Macro argument concatenation	Concatenates a macro argument with adjacent alphanumeric characters.
?	Return decimal value of symbol	Substitutes the <code>?symbol</code> sequence with a character string that represents the decimal value of the symbol.
%	Return hex value of symbol	Substitutes the <code>%symbol</code> sequence with a character string that represents the hexadecimal value of the symbol.
"	Macro string delimiter	Allows the use of macro arguments as literal strings.
^	Macro local label override	Prevents name mangling on labels in macros.

Example

The macro definition:

```
CONST.D .MACRO dx,v ;header
movh dx, #@his(v) ;body
```



```

    addi    dx, dx, #@los(v)
    .ENDM                                     ;terminator

```

The macro call:

```

    .SDECL  ".text", code
    .SECT   ".text"
    CONST.D d4, 0x12345678

```

The macro expands as follows:

```

    movh    d4, #@his(0x12345678)
    addi    d4, d4, #@los(0x12345678)

```

Related Information

[Section 3.10, *Macro Operations*](#)

- [.DUP, .ENDM](#) (Duplicate sequence of source lines)
- [.DUPA, .ENDM](#) (Duplicate sequence with arguments)
- [.DUPC, .ENDM](#) (Duplicate sequence with characters)
- [.DUPF, .ENDM](#) (Duplicate sequence in loop)
- [.PMACRO](#) (Undefine macro)
- [.DEFINE](#) (Define a substitution string)

.MESSAGE

Syntax

```
.MESSAGE {str|exp}[,{str|exp}]...
```

Description

With the `.MESSAGE` directive you tell the assembler to print a message to `stderr` during the assembling process.

An arbitrary number of strings and expressions, in any order but separated by commas with no intervening white space, can be specified to describe the nature of the generated message. If you use expressions, the assembler outputs the result. The assembler outputs a space between each argument.

The error and warning counts will not be affected. The `.MESSAGE` directive is for example useful in combination with conditional assembly to indicate which part is assembled. The assembling process proceeds normally after the message has been printed.

This directive has no effect on the exit code of the assembler.

A label is not allowed with this directive.

Example

```
.DEFINE LONG "SHORT"  
.MESSAGE 'This is a LONG string'  
.MESSAGE "This is a LONG string"
```

Within single quotes, the defined symbol `LONG` is not expanded. Within double quotes the symbol `LONG` is expanded so the actual message is printed as:

```
This is a LONG string  
This is a SHORT string
```

Related Information

`.FAIL` (Programmer generated error)

`.WARNING` (Programmer generated warning)

.MISRAC

Syntax

.MISRAC *string*

Description

The C compiler can generate the **.MISRAC** directive to pass the compiler's MISRA C settings to the object file. The linker performs checks on these settings and can generate a report. It is not recommended to use this directive in hand-coded assembly.

Example

```
.MISRAC 'MISRA-C:2004,64,e2,0b,e,e11,27,6,ef83,e1,  
ef,66,cb75,af1,eff,e7,e7f,8d,63,87ff7,6ff3,4'
```

Related Information

[Section 4.7.2, C Code Checking: MISRA C](#)

C compiler option **--misrac**

.ORG

Syntax

```
.ORG [abs-loc][,sect_type][,attribute]. . .
```

Description

With the `.ORG` directive you can specify an absolute location (*abs_loc*) in memory of a section. This is the same as a `.SDECL`/`.SECT` without a section name.

This directive uses the following arguments:

<i>abs-loc</i>	Initial value to assign to the run-time location counter. <i>abs-loc</i> must be an absolute expression. If <i>abs_loc</i> is not specified, then the value is zero.
<i>sect_type</i>	An optional section type: code or data
<i>attribute</i>	An optional section attribute: init, noread, noclear, max, rom, group(<i>string</i>), cluster(<i>string</i>), protect

For more information about the section types and attributes see the [assembler directive .SDECL](#).

The section type and attributes are case insensitive. A label is not allowed with this directive.

Example

```
; define a section at location 100 decimal
.org    100

; define a relocatable nameless section
.org

; define a relocatable data section
.org    ,data

; define a data section at 0x8000
.org    0x8000,data
```

Related Information

[.SDECL](#) (Declare section name and attributes)

[.SECT](#) (Activate a declared section)

.PMACRO

Syntax

.PMACRO *symbol* [, *symbol*] . . .

Description

With the **.PMACRO** directive you tell the assembler to undefine the specified macro, so that later uses of the symbol will not be expanded.

The assembler does not allow a label with this directive.

Example

```
.PMACRO MAC1, MAC2
```

This statement causes the macros named MAC1 and MAC2 to be undefined.

Related Information

.MACRO, **.ENDM** (Define a macro)

.SDECL

Syntax

```
.SDECL "name", type[, attribute]... [AT address]
```

Description

With the .SDECL directive you can define a section with a *name*, *type* and optional *attributes*. Before any code or data can be placed in a section, you must use the .SECT directive to activate the section.

The *name* specifies the name of the section. The *type* operand specifies the section's type and must be one of:

Type	Description
CODE	Code section.
DATA	Data section.
DEBUG	Debug section.

The section type and attributes are case insensitive.

The defined *attributes* are:

Attribute	Description	Allowed on type
AT <i>address</i>	Locate the section at the given <i>address</i> .	CODE, DATA
CLEAR	Sections are zeroed at startup.	DATA
CLUSTER(' <i>name</i> ')	Cluster code sections with companion debug sections. Used by the linker during removal of unreferenced sections. The name must be unique for this module (not for the application). To prevent naming conflicts with other symbols, the prefix ".cluster." is added to the cluster name during object file generation.	CODE, DATA, DEBUG
CONCAT	Concatenate sections. Used by the linker to merge sections with the same name.	DATA
INIT	Defines that the section contains initialization data, which is copied from ROM to RAM at program startup.	CODE, DATA
LINKONCE ' <i>tag</i> '	For internal use only.	
MAX	When data sections with the same name occur in different object modules with the MAX attribute, the linker generates a section of which the size is the maximum of the sizes in the individual object modules.	DATA
NOCLEAR	Sections are not zeroed at startup. This is a default attribute for data sections. This attribute is only useful with BSS sections, which are cleared at startup by default.	DATA
NOREAD	Defines that the section can be executed from but not read.	CODE

Attribute	Description	Allowed on type
PROTECT	Tells the linker to exclude a section from unreferenced section removal and duplicate section removal.	CODE, DATA
ROM	Section contains data to be placed in ROM. This ROM area is not executable.	CODE, DATA

Section names

The *name* of a section can have a special meaning for locating sections. The name of code sections should always start with ".text". With data sections, the prefix in the name is important. The prefix determines if the section is initialized, constant or uninitialized and which addressing mode is used. See the following table.

Name prefix	Type of section
.bss	uninitialized data
.bss_a0	uninitialized data, a0 addressing
.bss_a1	uninitialized data, a1 addressing
.bss_a8	uninitialized data, a8 addressing
.bss_a9	uninitialized data, a9 addressing
.data	initialized data
.data_a0	initialized data, a0 addressing
.data_a1	initialized data, a1 addressing
.data_a8	initialized data, a8 addressing
.data_a9	initialized data, a9 addressing
.ldata	constant data, a1 addressing (read only constants, literal data)
.rodata	constant data
.rodata_a0	constant data, a0 addressing
.rodata_a1	constant data, a1 addressing
.rodata_a8	constant data, a8 addressing
.rodata_a9	constant data, a9 addressing
.sbss	uninitialized data, a0 addressing
.sdata	initialized data, a0 addressing
.text	program code
.zbss	uninitialized data, abs 18 addressing
.zdata	initialized data, abs 18 addressing
.zrodata	constant data, abs 18 addressing

Note that the compiler uses the following name convention:

prefix.module_name.function_or_object_name

Also note that you cannot use the @ sign in section names. The assembler strips the @ sign and any following characters from the section name.

Example

```
.sdecl ".text.t.main", CODE    ; declare code section
.sect  ".text.t.main"          ; activate section

.sdecl ".data.t.var1", DATA   ; declare data section
.sect  ".data.t.var1"          ; activate section

.sdecl ".text.intvec.00a", CODE ; declare interrupt
                                ; vector table entry for interrupt 10
.sect  ".text.intvec.00a"      ; activate section

.sdecl ".data.t.abssec",data at 0x100
                                ; absolute section
.sect  ".data.t.abssec"       ; activate section
```

Related Information

[.SECT](#) (Activate a declared section)

[.ORG](#) (Initialize a nameless section)

.SECT

Syntax

```
.SECT "name" [, RESET]
```

Description

With the `.SECT` directive you activate a previously declared section with the name *name*. Before you can activate a section, you must define the section with the `.SDECL` directive. You can activate a section as many times as you need.

With the attribute `RESET` you can reset counting storage allocation in data sections that have section attribute `MAX`.

Example

```
.sdecl ".zdata.t.var2", DATA ; declare data section  
.sect ".zdata.t.var2"        ; activate section
```

Related Information

[.SDECL](#) (Declare section name and attributes)

[.ORG](#) (Initialize a nameless section)

.SET

Syntax

```
symbol .SET expression  
  
      .SET symbol expression
```

Description

With the .SET directive you assign the value of *expression* to symbol *temporarily*. If a symbol was defined with the .SET directive, you can redefine that symbol in another part of the assembly source, using the .SET directive again. Symbols that you define with the .SET directive are always local: you cannot define the symbol global with the .GLOBAL directive.

The .SET directive is useful in establishing temporary or reusable counters within macros. *expression* must be absolute and forward references are allowed.

Example

```
COUNT .SET 0 ; Initialize count. Later on you can  
          ; assign other values to the symbol
```

Related Information

[.EQU](#) (Set permanent value to a symbol)

.SIZE

Syntax

.SIZE *symbol, expression*

Description

With the .SIZE directive you set the size of the specified *symbol* to the value represented by *expression*.

The .SIZE directive may occur anywhere in the source file unless the specified symbol is a function. In this case, the .SIZE directive must occur after the function has been defined.

Example

```
main: .type    func
      .        ; function main
      .
      ret16
main_function_end:
      .size    main,main_function_end-main
```

Related Information

[.TYPE](#) (Set symbol type)

.SPACE

Syntax

[*label*:] **.SPACE** *expression*

Description

The **.SPACE** directive reserves a block in memory. The reserved block of memory is not initialized to any value.

If you specify the optional *label*, it gets the value of the location counter at the start of the directive processing.

The *expression* specifies the number of MAUs (Minimal Addressable Units) to be reserved, and how much the location counter will advance. The expression must evaluate to an integer greater than zero and cannot contain any forward references (symbols that have not yet been defined). For the TriCore the MAU size is 8 (1 byte).

If you specify *label*, it gets the value of the location counter at the start of the directive processing.

Example

To reserve 12 bytes (not initialized) of memory in a TriCore data section:

```
        .sdecl  ".zbss.tst.uninit",DATA
        .sect   ".zbss.tst.uninit"
uninit  .SPACE  12      ; Sample buffer
```

Related Information

.BYTE (Define a constant byte)

.TYPE

Syntax

symbol .TYPE *typeid*

Description

With the .TYPE directive you set a *symbol*'s type to the specified value in the ELF symbol table. Valid symbol types are:

FUNC The symbol is associated with a function or other executable code.

OBJECT The symbol is associated with an object such as a variable, an array, or a structure.

FILE The symbol name represents the filename of the compilation unit.

Labels in code sections have the default type FUNC. Labels in data sections have the default type OBJECT.

Example

```
Afunc: .type func
```

Related Information

[.SIZE](#) (Set symbol size)

.UNDEF

Syntax

.UNDEF *symbol*

Description

With the **.UNDEF** directive you can undefine a substitution string that was previously defined with the **.DEFINE** directive. The substitution string associated with *symbol* is released, and *symbol* will no longer represent a valid **.DEFINE** substitution or macro.

The assembler issues a warning if you undefine a non-existing symbol.

The assembler does not allow a label with this directive.

Example

The following example undefines the LEN substitution string that was previously defined with the **.DEFINE** directive:

```
.UNDEF LEN
```

Related Information

[.DEFINE](#) (Define a substitution string)

.WARNING

Syntax

.WARNING {*str|exp*}[,{*str|exp*}]...

Description

With the **.WARNING** directive you tell the assembler to print a warning message to `stderr` during the assembling process.

An arbitrary number of strings and expressions, in any order but separated by commas with no intervening white space, can be specified to describe the nature of the generated warning. If you use expressions, the assembler outputs the result. The assembler outputs a space between each argument.

The total warning count will be incremented as with any other warning. The **.WARNING** directive is for example useful in combination with conditional assembly to indicate which part is assembled. The assembling process proceeds normally after the message has been printed.

This directive has no effect on the exit code of the assembler, unless you use the [assembler option `--warnings-as-errors`](#). In that case the assembler exits with exit code 1 (an error).

A label is not allowed with this directive.

Example

```
.WARNING 'Parameter out of range'
```

This results in the warning:

```
W144: ["filename" line] Parameter out of range
```

Related Information

[.FAIL](#) (Programmer generated error)

[.MESSAGE](#) (Programmer generated message)

.WEAK

Syntax

```
.WEAK symbol [, symbol] ...
```

Description

With the `.WEAK` directive you mark one or more symbols as 'weak'. The *symbol* can be defined in the same module with the `.GLOBAL` directive or the `.EXTERN` directive. If the symbol does not already exist, it will be created.

A 'weak' external reference is resolved by the linker when a global (or weak) definition is found in one of the object files. However, a weak reference will not cause the extraction of a module from a library to resolve the reference.

You can overrule a weak definition with a `.GLOBAL` definition in another module. The linker will not complain about the duplicate definition, and ignore the weak definition.

Only program labels and symbols defined with `.EQU` can be made weak.

Example

```
LOOPA .EQU 1          ; definition of symbol LOOPA
      .GLOBAL  LOOPA  ; LOOPA will be globally
                      ; accessible by other modules
      .WEAK  LOOPA    ; mark symbol LOOPA as weak
```

Related Information

[.EXTERN](#) (Import global section symbol)

[.GLOBAL](#) (Declare global section symbol)

.WORD, .HALF

Syntax

```
[label:] .WORD argument[,argument]...
[label:] .HALF argument[,argument]...
```

Description

With the .WORD or .HALF directive the assembler allocates and initializes one word (32 bits) or a halfword (16 bits) of memory for each *argument*.

If you specify the optional *label*, it gets the value of the location counter at the start of the directive processing.

An *argument* can be a single- or multiple-character string constant, an expression or empty.

Multiple arguments are stored in sets of four or two bytes. One or more arguments can be null (indicated by two adjacent commas), in which case the corresponding byte location will be filled with zeros.

The value of the arguments must be in range with the size of the directive; floating-point numbers are not allowed. If the evaluated argument is too large to be represented in a word / halfword, the assembler issues a warning and truncates the value.

String constants

Single-character strings are stored in the most significant byte of a word / halfword, where the lower seven bits in that byte represent the ASCII value of the character, for example:

```
.WORD 'R'          ; = 0x52000000
.HALF 'R'          ; = 0x5200
```

Multiple-character strings are stored in consecutive byte addresses, as shown below. The standard C language escape characters like '\n' are permitted.

```
.WORD 'ABCD'       ; = 0x44434241
```

Example

When a string is supplied as argument of a directive that initializes multiple bytes, each character in the string is stored in consecutive bytes whose lower seven bits represent the ASCII value of the character. For example:

```
HTBL: .HALF 'ABC',, 'D' ; results in 0x424100004400 , the 'C' is truncated
WTBL: .WORD 'ABC'      ; results in 0x43424100
```

Related Information

[.BYTE](#) (Define a constant byte)

[.SPACE](#) (Define Storage)

3.9.2. Assembler Controls

Controls start with a \$ as the first character on the line. Unknown controls are ignored after a warning is issued.

Overview of assembler listing controls

Control	Description
\$LIST ON/OFF	Print / do not print source lines to list file
\$PAGE	Generate form feed in list file
\$PAGE <i>settings</i>	Define page layout for assembly list file
\$PRCTL	Send control string to printer
\$STITLE	Set program subtitle in header of assembly list file
\$TITLE	Set program title in header of assembly list file

Overview of miscellaneous assembler controls

Control	Description
\$CASE ON/OFF	Case sensitive user names ON/OFF
\$CPU_TCnum ON	Enable/disable assembler check for specified functional problem
\$DEBUG ON/OFF	Generation of symbolic debug ON/OFF
\$DEBUG " <i>flags</i> "	Select debug information
\$HW_ONLY	Prevent substitution of assembly instructions by smaller or faster instructions
\$IDENT LOCAL/GLOBAL	Assembler treats labels by default as local or global
\$MMU	Allow memory management instructions
\$NO_FPU	Do not allow single precision floating-point instructions
\$OBJECT	Alternative name for the generated object file
\$TC131 / \$TC16 / \$TC16X / \$TC162	Allow TriCore 1.3.1, 1.6, 1.6.x or 1.6.2 instructions
\$WARNING OFF [<i>num</i>]	Suppress all or some warnings

\$CASE

Syntax

```
$CASE  ON
$CASE  OFF
```

Default

```
$CASE  ON
```

Description

With the `$CASE ON` and `$CASE OFF` controls you specify whether the assembler operates in case sensitive mode or not. By default the assembler operates in case sensitive mode. This means that all user-defined symbols and labels are treated case sensitive, so `LAB` and `Lab` are distinct.

Note that the instruction mnemonics, register names, directives and controls are always treated case insensitive.

Example

```
;begin of source
$CASE OFF    ; assembler in case insensitive mode
```

Related Information

Assembler option `--case-insensitive`

\$CPU_TCnum

Syntax

\$CPU_TCnum ON
\$CPU_TCnum OFF

Description

With these controls you can enable or disable specific CPU functional problem checks.

When you use this control, the define `__CPU_TCnum__` is set to 1.

Example

```
$CPU_TC018 ON    ; enable assembler check for CPU  
                 ; functional problem CPU_TC.018,  
                 ; __CPU_TC018__ is defined
```

Related Information

Assembler option **--silicon-bug**

Chapter 19, *CPU Problem Bypasses and Checks*

\$DEBUG

Syntax

```

$DEBUG  ON
$DEBUG  OFF
$DEBUG  "flags"

```

Default

```
$DEBUG "AhLS"
```

Description

With the `$DEBUG ON` and `$DEBUG OFF` controls you turn the generation of debug information on or off. (`$DEBUG ON` is similar to the assembler option `--debug-info=+local (-gl)`).

If you use the `$DEBUG` control with flags, you can set the following flags:

a/A	Assembly source line information
h/H	Pass high level language debug information (HLL)
l/L	Assembler local symbols debug information
s/S	Smart debug information

You cannot specify `$DEBUG "ah"`. Either the assembler generates assembly source line information, or it passes HLL debug information.

Debug information that is generated by the C compiler, is always passed to the object file.

Example

```

;begin of source
$DEBUG ON ; generate local symbols debug information

```

Related Information

Assembler option `--debug-info`

\$HW_ONLY

Syntax

\$HW_ONLY

Description

Normally the assembler replaces instructions by other, smaller or faster instructions. For example, the instruction `jeq d0, #0, label1` is replaced by `jz d0, label1`.

With the **\$HW_ONLY** control you instruct the assembler to encode all instruction as they are. The assembler does not substitute instructions with other, faster or smaller instructions.

Example

```
;begin of source
$HW_ONLY    ; the assembler does not substitute
             ; instructions with other, smaller or
             ; faster instructions.
```

Related Information

Assembler option **--optimize=+generics**

\$IDENT

Syntax

\$IDENT LOCAL
\$IDENT GLOBAL

Default

\$IDENT LOCAL

Description

With the controls **\$IDENT LOCAL** and **\$IDENT GLOBAL** you tell the assembler how to treat symbols that you have not specified explicitly as local or global with the assembler directives **.LOCAL** or **.GLOBAL**.

By default the assembler treats all symbols as local symbols unless you have defined them to be global explicitly.

Example

```
;begin of source  
$IDENT GLOBAL ; assembly labels are global by default
```

Related Information

Assembler directive **.GLOBAL**

Assembler directive **.LOCAL**

Assembler option **--symbol-scope**

\$LIST ON/OFF

Syntax

\$LIST ON
\$LIST OFF

Default

\$LIST ON

Description

If you generate a list file with the assembler option **--list-file**, you can use the **\$LIST ON** and **\$LIST OFF** controls to specify which source lines the assembler must write to the list file. Without the assembler option **--list-file** these controls have no effect. The controls take effect starting at the next line.

The **\$LIST ON** control actually increments a counter that is checked for a positive value and is symmetrical with respect to the **\$LIST OFF** control. Note the following sequence:

```
; Counter value currently 1
$LIST ON           ; Counter value = 2
$LIST ON           ; Counter value = 3
$LIST OFF          ; Counter value = 2
$LIST OFF          ; Counter value = 1
```

The listing still would not be disabled until another **\$LIST OFF** control was issued.

Example

```
... ; source line in list file
$LIST OFF
... ; source line not in list file
$LIST ON
... ; source line also in list file
```

Related Information

Assembler option **--list-file**

Assembler function **@LST()**

\$MMU

Syntax

\$MMU

Description

With the \$MMU control you instruct the assembler to accept and encode memory management instructions in the assembly source file.

When you use this control, the define `__MMU__` is set to 1.

Example

```
;begin of source
$MMU      ; the use of memory management instructions
          ; in this source is allowed.
```

Related Information

Assembler option `--mmu-present`

\$NO_FPU

Syntax

\$NO_FPU

Description

By default, the assembler accepts and encodes single precision floating-point (FPU) instructions in the assembly source file. With the \$NO_FPU control you tell the assembler that FPU instructions are not allowed in the assembly source file.

When you use this control, the define `__FPU__` is set to 0. By default the define `__FPU__` is set to 1 which tells the assembler to accept single precision floating-point instructions.

Example

```
;begin of source
$NO_FPU      ; the use of single precision FPU instructions
              ; in this source is not allowed.
```

Related Information

Assembler option **--no-fpu**

\$OBJECT

Syntax

```
$OBJECT "file"  
$OBJECT OFF
```

Default

\$OBJECT

Description

With the \$OBJECT control you can specify an alternative name for the generated object file. With the \$OBJECT OFF control, the assembler does not generate an object file at all.

Example

```
;Begin of source  
$object "x1.o"          ; generate object file x1.o
```

Related Information

Assembler option **--output**

\$PAGE

Syntax

```
$PAGE [pagewidth[,pagelength[,blankleft[,blanktop[,blankbtm]]]]
```

Default

```
$PAGE 132,72,0,0,0
```

Description

If you generate a list file with the assembler option **--list-file**, you can use the \$PAGE control to format the generated list file.

The arguments may be any positive absolute integer expression, and must be separated by commas.

<i>pagewidth</i>	Number of columns per line. The default is 132, the minimum is 40.
<i>pagelength</i>	Total number of lines per page. The default is 72, the minimum is 10. As a special case, a page length of 0 turns off page breaks.
<i>blankleft</i>	Number of blank columns at the left of the page. The default is 0, the minimum is 0, and the maximum must maintain the relationship: <i>blankleft</i> < <i>pagewidth</i> .
<i>blanktop</i>	Number of blank lines at the top of the page. The default is 0, the minimum is 0 and the maximum must be a value so that $(blanktop + blankbtm) \leq (pagelength - 10)$.
<i>blankbtm</i>	Number of blank lines at the bottom of the page. The default is 0, the minimum is 0 and the maximum must be a value so that $(blanktop + blankbtm) \leq (pagelength - 10)$.

If you use the \$PAGE control without arguments, it causes a 'formfeed': the next source line is printed on the next page in the list file. The \$PAGE control itself is not printed.

Example

```
$PAGE          ; formfeed, the next source line is printed
                ; on the next page in the list file.
```

```
$PAGE 96       ; set page width to 96. Note that you can
                ; omit the last four arguments.
```

```
$PAGE ,,,3,3   ; use 3 line top/bottom margins.
```

Related Information

Assembler option **--list-file**

\$PRCTL

Syntax

\$PRCTL *exp|string[,exp|string]...*

Description

If you generate a list file with the assembler option **--list-file**, you can use the \$PRCTL control to send control strings to the printer.

The \$PRCTL control simply concatenates its arguments and sends them to the listing file (the control line itself is not printed unless there is an error).

You can specify the following arguments:

expr A byte expression which may be used to encode non-printing control characters, such as ESC.

string An assembler string, which may be of arbitrary length, up to the maximum assembler-defined limits.

The \$PRCTL control can appear anywhere in the source file; the assembler sends out the control string at the corresponding place in the listing file.

If a \$PRCTL control is the last line in the last input file to be processed, the assembler insures that all error summaries, symbol tables, and cross-references have been printed before sending out the control string. In this manner, you can use a \$PRCTL control to restore a printer to a previous mode after printing is done.

Similarly, if the \$PRCTL control appears as the first line in the first input file, the assembler sends out the control string before page headings or titles.

Example

```
$PRCTL $1B, 'E' ; Reset HP LaserJet printer
```

Related Information

Assembler option **--list-file**

\$STITLE

Syntax

\$STITLE *"string"*

Default

\$STITLE ""

Description

If you generate a list file with the assembler option **--list-file**, you can use the \$STITLE control to specify the program subtitle which is printed at the top of all succeeding pages in the assembler list file below the title.

The specified subtitle is valid until the assembler encounters a new \$STITLE control. By default, the subtitle is empty.

The \$STITLE control itself will not be printed in the source listing.

If the page width is too small for the title to fit in the header, it will be truncated.

Example

```
$TITLE    'This is the title'
$STITLE   'This is the subtitle'
```

Related Information

Assembler option **--list-file**

Assembler control **\$TITLE**

\$TC131 / \$TC16 / \$TC16X / \$TC162**Syntax**

\$TC131
\$TC16
\$TC16X
\$TC162

Description

With the \$TC131, \$TC16,\$TC16X or \$TC162 control you instruct the assembler to accept and encode TriCore 1.3.1, 1.6, 1.6.x or 1.6.2 instructions respectively in the assembly source file.

When you use one of these controls, the define `__CORE_TC131__`, `__CORE_TC16__`, `__CORE_TC16X__` or `__CORE_TC162__` is set to 1 respectively. When no control and no **--core** option is given, the default core is TC1.3 and the define `__CORE_TC13__` is set to 1.

Example

```
;begin of source
$TC162    ; the use of TriCore 1.6.2 instructions
          ; in this source is allowed.
```

Related Information

Assembler option **--core**

\$TITLE

Syntax

\$TITLE *"string"*

Default

\$TITLE ""

Description

If you generate a list file with the assembler option **--list-file**, you can use the **\$TITLE** control to specify the program title which is printed at the top of each page in the assembler list file.

The specified title is valid until the assembler encounters a new **\$TITLE** control. By default, the title is empty.

The **\$TITLE** control itself will not be printed in the source listing.

If the page width is too small for the title to fit in the header, it will be truncated.

Example

```
$TITLE 'This is the title'
```

Related Information

Assembler option **--list-file**

Assembler control **\$STITLE**

\$WARNING OFF

Syntax

\$WARNING OFF [*number*]

Default

All warnings are reported.

Description

This control allows you to disable all or individual warnings. The *number* argument must be a valid warning message number.

Example

```
$WARNING OFF      ; all warning messages are suppressed
```

```
$WARNING OFF 135  ; suppress warning message 135
```

Related Information

Assembler option **--no-warnings**

3.10. Macro Operations

Macros provide a shorthand method for inserting a repeated pattern of code or group of instructions. You can define the pattern as a macro, and then call the macro at the points in the program where the pattern would repeat.

Some patterns contain variable entries which change for each repetition of the pattern. Others are subject to conditional assembly.

When a macro is called, the assembler executes the macro and replaces the call by the resulting in-line source statements. 'In-line' means that all replacements act as if they are on the same line as the macro call. The generated statements may contain substitutable arguments. The statements produced by a macro can be any processor instruction, almost any assembler directive, or any previously-defined macro. Source statements resulting from a macro call are subject to the same conditions and restrictions as any other statements.

Macros can be nested. The assembler processes nested macros when the outer macro is expanded.

3.10.1. Defining a Macro

The first step in using a macro is to define it.

The definition of a macro consists of three parts:

- *Header*, which assigns a name to the macro and defines the arguments (`.MACRO` directive).
- *Body*, which contains the code or instructions to be inserted when the macro is called.
- *Terminator*, which indicates the end of the macro definition (`.ENDM` directive).

A macro definition takes the following form:

```
macro_name .MACRO [argument[,argument]...]
...
macro_definition_statements
...
.ENDM
```

For more information on the definition see the description of the [.MACRO directive](#).

The `.DUP`, `.DUPA`, `.DUPC`, and `.DUPF` directives are specialized macro forms to repeat a block of source statements. You can think of them as a simultaneous definition and call of an unnamed macro. The source statements between the `.DUP`, `.DUPA`, `.DUPC`, and `.DUPF` directives and the `.ENDM` directive follow the same rules as macro definitions.

3.10.2. Calling a Macro

To invoke a macro, construct a source statement with the following format:

```
[label] macro_name [argument[,argument]...]  [; comment]
```

where,

<i>label</i>	An optional label that corresponds to the value of the location counter at the start of the macro expansion.
<i>macro_name</i>	The name of the macro. This may not start in the first column.
<i>argument</i>	One or more optional, substitutable arguments. Multiple arguments must be separated by commas.
<i>comment</i>	An optional comment.

The following applies to macro arguments:

- Each argument must correspond one-to-one with the formal arguments of the macro definition. If the macro call does not contain the same number of arguments as the macro definition, the assembler issues a warning.
- If an argument has an embedded comma or space, you must surround the argument by single quotes (').
- You can declare a macro call argument as null in three ways:

- enter delimiting commas in succession with no intervening spaces

macroname ARG1,,ARG3 ; the second argument is a null argument

- terminate the argument list with a comma, the arguments that normally would follow, are now considered null

macroname ARG1, ; the second and all following arguments are null

- declare the argument as a null string
- No character is substituted in the generated statements that reference a null argument.

3.10.3. Using Operators for Macro Arguments

The assembler recognizes certain text operators within macro definitions which allow text substitution of arguments during macro expansion. You can use these operators for text concatenation, numeric conversion, and string handling.

Operator	Name	Description
\	Macro argument concatenation	Concatenates a macro argument with adjacent alphanumeric characters.
?	Return decimal value of symbol	Substitutes the <i>?symbol</i> sequence with a character string that represents the decimal value of the symbol.
%	Return hex value of symbol	Substitutes the <i>%symbol</i> sequence with a character string that represents the hexadecimal value of the symbol.
"	Macro string delimiter	Allows the use of macro arguments as literal strings.
^	Macro local label override	Prevents name mangling on labels in macros.

**Example: Argument Concatenation Operator - **

Consider the following macro definition:

```
SWAP_MEM .MACRO REG1,REG2          ;swap memory contents
    LD.W  D0,[A\REG1]              ;use D0 as temp
    LD.W  D1,[A\REG2]              ;use D1 as temp
    ST.W  [A\REG1],D1
    ST.W  [A\REG2],D0
.ENDM
```

The macro is called as follows:

```
SWAP_MEM 0,1
```

The macro expands as follows:

```
LD.W  D0,[A0]
LD.W  D1,[A1]
ST.W  [A0],D1
ST.W  [A1],D0
```

The macro preprocessor substitutes the character '0' for the argument REG1, and the character '1' for the argument REG2. The concatenation operator (\) indicates to the macro preprocessor that the substitution characters for the arguments are to be concatenated with the character 'A'.

Without the '\' operator the macro would expand as:

```
LD.W  D0,[AREG1]
LD.W  D1,[AREG2]
ST.W  [AREG1],D1
ST.W  [AREG2],D0
```

which results in an assembler error (invalid operand).

Example: Decimal Value Operator - ?

Instead of substituting the formal arguments with the actual macro call arguments, you can also use the value of the macro call arguments.

Consider the following source code that calls the macro SWAP_SYM after the argument AREG has been set to 0 and BREG has been set to 1.

```
AREG .SET      0
BREG .SET      1
SWAP_SYM AREG,BREG
```

If you want to replace the arguments with the value of AREG and BREG rather than with the literal strings 'AREG' and 'BREG', you can use the ? operator and modify the macro as follows:

```
SWAP_SYM .MACRO REG1,REG2          ;swap memory contents
    LD.W  D0,_lab\?REG1            ;use D0 as temp
```

```
LD.W  D1,_lab\?REG2           ;use D1 as temp
ST.W  _lab\?REG1,D1
ST.W  _lab\?REG2,D0
.ENDM
```

The macro first expands as follows:

```
LD.W  D0,_lab\?AREG
LD.W  D1,_lab\?BREG
ST.W  _lab\?AREG,D1
ST.W  _lab\?BREG,D0
```

Then ?AREG is replaced by '0' and ?BREG is replaced by '1':

```
LD.W  D0,_lab\1
LD.W  D1,_lab\2
ST.W  _lab\1,D1
ST.W  _lab\2,D0
```

Because of the concatenation operator '\' the strings are concatenated:

```
LD.W  D0,_lab1
LD.W  D1,_lab2
ST.W  _lab1,D1
ST.W  _lab2,D0
```

Example: Hex Value Operator - %

The percent sign (%) is similar to the standard decimal value operator (?) except that it returns the hexadecimal value of a symbol.

Consider the following macro definition:

```
GEN_LAB  .MACRO  LAB, VAL, STMT
LAB%VAL  STMT
.ENDM
```

The macro is called after NUM has been set to 10:

```
NUM  .SET      10
GEN_LAB  HEX, NUM, NOP
```

The macro expands as follows:

```
HEXA NOP
```

The %VAL argument is replaced by the character 'A' which represents the hexadecimal value 10 of the argument VAL.

Example: Argument String Operator - "

To generate a literal string, enclosed by single quotes ('), you must use the argument string operator (") in the macro definition.

Consider the following macro definition:

```
STR_MAC      .MACRO  STRING
               .BYTE  "STRING"
               .ENDM
```

The macro is called as follows:

```
STR_MAC  ABCD
```

The macro expands as follows:

```
.BYTE    'ABCD'
```

Within double quotes .DEFINE directive definitions can be expanded. Take care when using constructions with single quotes and double quotes to avoid inappropriate expansions. Since .DEFINE expansion occurs before macro substitution, any .DEFINE symbols are replaced first within a macro argument string:

```
        .DEFINE LONG 'short'
STR_MAC      .MACRO  STRING
               .MESSAGE 'This is a LONG STRING'
               .MESSAGE "This is a LONG STRING"
               .ENDM
```

If the macro is called as follows:

```
STR_MAC  sentence
```

it expands as:

```
.MESSAGE 'This is a LONG STRING'
.MESSAGE 'This is a short sentence'
```

Macro Local Label Override Operator - ^

If you use labels in macros, the assembler normally generates another unique name for the labels (such as LAB__M_L000001).

The macro ^-operator prevents name mangling on macro local labels.

Consider the following macro definition:

```
INIT      .MACRO  addr
LAB:      LD.W    D0, ^addr
               .ENDM
```

The macro is called as follows:

```
LAB:
    INIT LAB
```

The macro expands as:

```
LAB__M_L0000001: LD.W  D0, LAB
```

If you would have omitted the ^ operator, the macro preprocessor would choose another name for LAB because the label already exists. The macro would expand like:

```
LAB__M_L0000001: LD.W  D0, LAB__M_L0000001
```

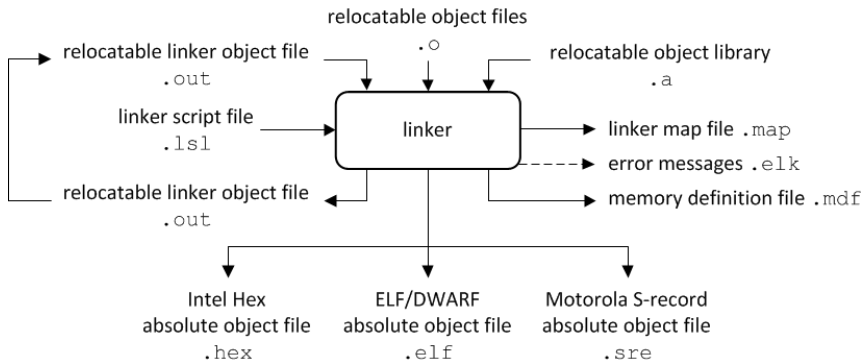

Chapter 7. Using the Linker

This chapter describes the linking process, how to call the linker and how to control the linker with a script file.

The TASKING linker is a combined linker/locator. The linker phase combines relocatable object files (.o files, generated by the assembler), and libraries into a single relocatable linker object file (.out). The locator phase assigns absolute addresses to the linker object file and creates an absolute object file which you can load into a target processor. From this point the term linker is used for the combined linker/locator.

The linker can simultaneously link and locate all programs for all cores available on a target board. The target board may be of arbitrary complexity. A simple target board may contain one standard processor with some external memory that executes one task. A complex target board may contain multiple standard processors and DSPs combined with configurable IP-cores loaded in an FPGA. Each core may execute a different program, and external memory may be shared by multiple cores.

The linker takes the following files for input and output:



This chapter first describes the linking process. Then it describes how to call the linker and how to use its options. An extensive list of all options and their descriptions is included in [Section 10.5, *Linker Options*](#).

To control the link process, you can write a script for the linker. This chapter shortly describes the purpose and basic principles of the Linker Script Language (LSL) on the basis of an example. A complete description of the LSL is included in Linker Script Language.

7.1. Linking Process

The linker combines and transforms relocatable object files (.o) into a single absolute object file. This process consists of two phases: the linking phase and the locating phase.

In the first phase the linker combines the supplied relocatable object files and libraries into a single relocatable object file. In the second phase, the linker assigns absolute addresses to the object file so it can actually be loaded into a target.

Terms used in the linking process

Term	Definition
Absolute object file	Object code in which addresses have fixed absolute values, ready to load into a target.
Address	A specification of a location in an address space.
Address space	The set of possible addresses. A core can support multiple spaces, for example in a Harvard architecture the addresses that identify the location of an instruction refer to <i>code</i> space, whereas addresses that identify the location of a data object refer to a <i>data</i> space.
Architecture	A description of the characteristics of a core that are of interest for the linker. This encompasses the address space(s) and the internal bus structure. Given this information the linker can convert logical addresses into physical addresses.
Copy table	A section created by the linker. This section contains data that specifies how the startup code initializes the data and BSS sections. For each section the copy table contains the following fields: • action: defines whether a section is copied or zeroed • destination: defines the section's address in RAM • source: defines the sections address in ROM, zero for BSS sections • length: defines the size of the section in MAUs of the destination space
Core	An instance of an architecture.
Derivative	The design of a processor. A description of one or more cores including internal memory and any number of buses.
Library	Collection of relocatable object files. Usually each object file in a library contains one symbol definition (for example, a function).

Term	Definition
Link task	A scope for linking: resolving symbols from object files and libraries. Such a task is associated with one core in the LSL file(s). Other LSL cores may be imported into this core, associating two or more hardware cores with one link task.
Logical address	An address as encoded in an instruction word, an address generated by a core (CPU).
LSL file	The set of linker script files that are passed to the linker.
MAU	Minimum Addressable Unit. For a given processor the number of bits between an address and the next address. This is not necessarily a byte or a word.
Object code	The binary machine language representation of the C source.
Physical address	An address generated by the memory system.
Processor	An instance of a derivative. Usually implemented as a (custom) chip, but can also be implemented in an FPGA, in which case the derivative can be designed by the developer.
Relocatable object file	Object code in which addresses are represented by symbols and thus relocatable.
Relocation	The process of assigning absolute addresses.
Relocation information	Information about how the linker must modify the machine code instructions when it relocates addresses.
Section	A group of instructions and/or data objects that occupy a contiguous range of addresses.
Section attributes	Attributes that define how the section should be linked or located.
Target	The hardware board on which an application is executing. A board contains at least one processor. However, a complex target may contain multiple processors and external memory and may be shared between processors.
Unresolved reference	A reference to a symbol for which the linker did not find a definition yet.

7.1.1. Phase 1: Linking

The linker takes one or more relocatable object files and/or libraries as input. A relocatable object file, as generated by the assembler, contains the following information:

- *Header information:* Overall information about the file, such as the code size, name of the source file it was assembled from, and creation date.
- *Object code:* Binary code and data, divided into various named sections. Sections are contiguous chunks of code that have to be placed in specific parts of the memory. The program addresses start at zero for each section in the object file.
- *Symbols:* Some symbols are exported - defined within the file for use in other files. Other symbols are imported - used in the file but not defined (external symbols). Generally these symbols are names of routines or names of data objects.

- *Relocation information*: A list of places with symbolic references that the linker has to replace with actual addresses. When in the code an external symbol (a symbol defined in another file or in a library) is referenced, the assembler does not know the symbol's size and address. Instead, the assembler generates a call to a preliminary relocatable address (usually 0000), while stating the symbol name.
- *Debug information*: Other information about the object code that is used by a debugger. The assembler optionally generates this information and can consist of line numbers, C source code, local symbols and descriptions of data structures.

The linker resolves the external references between the supplied relocatable object files and/or libraries and combines the files into a single relocatable linker object file.

The linker starts its task by scanning all specified relocatable object files and libraries. If the linker encounters an unresolved symbol, it remembers its name and continues scanning. The symbol may be defined elsewhere in the same file, or in one of the other files or libraries that you specified to the linker. If the symbol is defined in a library, the linker extracts the object file with the symbol definition from the library. This way the linker collects all definitions and references of all of the symbols.

Next, the linker combines sections with the same section name and attributes into single sections. The linker also substitutes (external) symbol references by (relocatable) numerical addresses where possible. At the end of the linking phase, the linker either writes the results to a file (a single relocatable object file) or keeps the results in memory for further processing during the locating phase.

The resulting file of the linking phase is a single relocatable object file (.out). If this file contains unresolved references, you can link this file with other relocatable object files (.o) or libraries (.a) to resolve the remaining unresolved references.

With the linker command line option **--link-only**, you can tell the linker to only perform this linking phase and skip the locating phase. The linker complains if any unresolved references are left.

7.1.2. Phase 2: Locating

In the locating phase, the linker assigns absolute addresses to the object code, placing each section in a specific part of the target memory. The linker also replaces references to symbols by the actual address of those symbols. The resulting file is an absolute object file which you can actually load into a target memory. Optionally, when the resulting file should be loaded into a ROM device the linker creates a so-called copy table section which is used by the startup code to initialize the data and BSS sections.

Code modification

When the linker assigns absolute addresses to the object code, it needs to modify this code according to certain rules or *relocation expressions* to reflect the new addresses. These relocation expressions are stored in the relocatable object file. Consider the following snippet of x86 code that moves the contents of variable a to variable b via the eax register:

```
A1 3412 0000 mov a,%eax    (a defined at 0x1234, byte reversed)
A3 0000 0000 mov %eax,b    (b is imported so the instruction refers to
                           0x0000 since its location is unknown)
```

Now assume that the linker links this code so that the section in which a is located is relocated by 0x10000 bytes, and b turns out to be at 0x9A12. The linker modifies the code to be:

```
A1 3412 0100 mov a,%eax    (0x10000 added to the address)
A3 129A 0000 mov %eax,b    (0x9A12 patched in for b)
```

These adjustments affect instructions, but keep in mind that any pointers in the data part of a relocatable object file have to be modified as well.

Output formats

The linker can produce its output in different file formats. The default ELF/DWARF format (`.elf`) contains an image of the executable code and data, and can contain additional debug information. The Intel-Hex format (`.hex`) and Motorola S-record format (`.sre`) only contain an image of the executable code and data. You can specify a format with the options **--output (-o)** and **--chip-output (-c)**.

Controlling the linker

Via a so-called *linker script file* you can gain complete control over the linker. The script language is called the *Linker Script Language* (LSL). Using LSL you can define:

- The memory installed in the embedded target system:

To assign locations to code and data sections, the linker must know what memory devices are actually installed in the embedded target system. For each physical memory device the linker must know its start-address, its size, and whether the memory is read-write accessible (RAM) or read-only accessible (ROM).

- How and where code and data should be placed in the physical memory:

Embedded systems can have complex memory systems. If for example on-chip and off-chip memory devices are available, the code and data located in internal memory is typically accessed faster and with dissipating less power. To improve the performance of an application, specific code and data sections should be located in on-chip memory. By writing your own LSL file, you gain full control over the locating process.

- The underlying hardware architecture of the target processor.

To perform its task the linker must have a model of the underlying hardware architecture of the processor you are using. For example the linker must know how to translate an address used within the object file (a logical address) into an offset in a particular memory device (a physical address). In most linkers this model is hard coded in the executable and can not be modified. For the TASKING linker this hardware model is described in the linker script file. This solution is chosen to support configurable cores that are used in system-on-chip designs.

When you want to write your own linker script file, you can use the standard linker script files with architecture descriptions delivered with the product.

See also [Section 7.9, Controlling the Linker with a Script](#).

7.2. Calling the Linker

In Eclipse you can set options specific for the linker. After you have built your project, the output files are available in a subdirectory of your project directory, depending on the active configuration you have set in the **C/C++ Build » Settings** page of the **Project » Properties for** dialog.

Building a project under Eclipse

You have several ways of building your project:

- Build Individual Project ()

To build individual projects incrementally, select **Project » Build project**.

- Rebuild Project () This builds every file in the project whether or not a file has been modified since the last build. A rebuild is a clean followed by a build.

1. Select **Project » Clean...**
2. Enable the option **Start a build immediately** and click **OK**.

- Build Automatically. This performs a build of all projects whenever any project file is saved, such as your makefile.

This way of building is not recommended, but to enable this feature select **Project » Build Automatically** and ensure there is a check mark beside the **Build Automatically** menu item. In order for this option to work, you must also enable option **Build on resource save (Auto build)** on the **Behavior** tab of the **C/C++ Build** page of the **Project » Properties for** dialog.

To access the linker options

1. From the **Project** menu, select **Properties for**
The Properties dialog appears.
2. In the left pane, expand **C/C++ Build** and select **Settings**.
In the right pane the Settings appear.
3. From the **Configuration** list, select a configuration or select [All configurations].
4. On the Tool Settings tab, select **Linker**.
5. Select the sub-entries and set the options in the various pages.

Note that when you click **Restore Defaults** to restore the default tool options, as a side effect the processor is also reset to its default value on the **Processor** page (**C/C++ Build » Processor**).

You can find a detailed description of all linker options in [Section 10.5, Linker Options](#).

Invocation syntax on the command line:

```
ltdc [ [option]... [file]... ]...
```

When you are linking multiple files, either relocatable object files (.o) or libraries (.a), it is important to specify the files in the right order. This is explained in [Section 7.3, Linking with Libraries](#).

Example:

```
ltdc -dtdc1796b.lsl test.o
```

This links and locates the file test.o and generates the file test.elf.

7.3. Linking with Libraries

There are two kinds of libraries: system libraries and user libraries.

System library

System libraries are stored in the directories:

<TriCore installation path>\lib\tc13	(TriCore 1.3 libraries)
<TriCore installation path>\lib\tc1130_mmu	(TC1130 MMU variant)
<TriCore installation path>\lib\tc131	(TriCore 1.3.1 libraries)
<TriCore installation path>\lib\tc16	(TriCore 1.6 libraries)
<TriCore installation path>\lib\tc16x	(TriCore 1.6.x libraries)
<TriCore installation path>\lib\tc162	(TriCore 1.6.2 libraries)
<TriCore installation path>\lib\p	(protected libraries)

The p directory contains subdirectories with the protected libraries for CPU functional problems.

An overview of the system libraries is given in the following table:

Libraries	Description
libc[s][w].a libc[s][w]_fpu.a	C libraries Optional letter: s = single precision floating-point (control program option --fp-model=+float) w = wide-character support (control program option --c++=11, --io-streams) _fpu = with FPU instructions (default, control program option --fp-model=-soft)
libfp[t].a libfp[t]_fpu.a	Floating-point libraries Optional letter: t = trapping (control program option --fp-model=+trap) _fpu = with FPU instructions (default, control program option --fp-model=-soft)
librt.a	Run-time library

Libraries	Description
libpb.a libpc.a libpct.a libpd.a libpt.a	Profiling libraries pb = block/function counter pc = call graph pct = call graph and timing pd = dummy pt = function timing
libcp[s][x].a libcp[s][x]_fpu.a	C++ libraries Optional letter: s = single precision floating-point x = exception handling _fpu = with FPU instructions (default, control program option --fp-model=soft)
libstl[s]x.a libstl[s]x_fpu.a	STLport C++ libraries (exception handling variants only) Optional letter: s = single precision floating-point _fpu = with FPU instructions (default, control program option --fp-model=soft)

Sources for the libraries are present in the directories `lib\src`, `lib\src.*` in the form of an executable. If you run the executable it will extract the sources in the corresponding directory.

To link the default C (system) libraries

- From the **Project** menu, select **Properties for**
The Properties dialog appears.
- In the left pane, expand **C/C++ Build** and select **Settings**.
In the right pane the Settings appear.
- On the Tool Settings tab, select **Linker » Libraries**.
- Enable the option **Link default libraries**.

When you want to link system libraries from the command line, you must specify this with the option **--library (-l)**. For example, to specify the system library `libc.a`, type:

```
ltc --library=c test.o
```

User library

You can create your own libraries. [Section 8.5, Archiver](#) describes how you can use the archiver to create your own library with object modules.

To link user libraries

- From the **Project** menu, select **Properties for**

The Properties dialog appears.

2. In the left pane, expand **C/C++ Build** and select **Settings**.

In the right pane the Settings appear.

3. On the Tool Settings tab, select **Linker » Libraries**.
4. Add your libraries to the **Libraries** box.

When you want to link user libraries from the command line, you must specify their filenames on the command line:

```
ltd start.o mylib.a
```

If the library resides in a sub-directory, specify that directory with the library name:

```
ltd start.o mylibs\mylib.a
```

If you do not specify a directory, the linker searches the library in the current directory only.

Library order

The order in which libraries appear on the command line is important. By default the linker processes object files and libraries in the order in which they appear at the command line. Therefore, when you use a weak symbol construction, like `printf`, in an object file or your own library, you must position this object/library before the C library.

With the option **--first-library-first** you can tell the linker to scan the libraries from left to right, and extract symbols from the first library where the linker finds it. This can be useful when you want to use newer versions of a library routine:

```
ltd --first-library-first a.a test.o b.a
```

If the file `test.o` calls a function which is both present in `a.a` and `b.a`, normally the function in `b.a` would be extracted. With this option the linker first tries to extract the symbol from the first library `a.a`.

Note that routines in `b.a` that call other routines that are present in both `a.a` and `b.a` are now also resolved from `a.a`.

7.3.1. How the Linker Searches Libraries

System libraries

You can specify the location of system libraries in several ways. The linker searches the specified locations in the following order:

1. The linker first looks in the **Library search path** that are specified in the **Linker » Libraries** page in the **C/C++ Build » Settings » Tool Settings** tab of the Project Properties dialog (equivalent to the option **--library-directory (-L)**). If you specify the **-L** option without a pathname, the linker stops searching after this step.

2. When the linker did not find the library (because it is not in the specified library directory or because no directory is specified), it looks in the path(s) specified in the environment variables `LIBTC1V1_3` / `LIBTC1V1_3_1` / `LIBTC1V1_6` / `LIBTC1V1_6_X` / `LIBTC1V1_6_2`.
3. When the linker did not find the library, it tries the default `lib` directory relative to the installation directory (or a processor specific sub-directory).

User library

If you use your own library, the linker searches the library in the current directory only.

7.3.2. How the Linker Extracts Objects from Libraries

A library built with the TASKING archiver **artc** always contains an index part at the beginning of the library. The linker scans this index while searching for unresolved externals. However, to keep the index as small as possible, only the defined symbols of the library members are recorded in this area.

When the linker finds a symbol that matches an unresolved external, the corresponding object file is extracted from the library and is processed. After processing the object file, the remaining library index is searched. If after a complete search of the library unresolved externals are introduced, the library index will be scanned again. After all files and libraries are processed, and there are still unresolved externals and you did not specify the linker option **--no-rescan**, all libraries are rescanned again. This way you do not have to worry about the library order on the command line and the order of the object files in the libraries. However, this rescanning does not work for 'weak symbols'. If you use a weak symbol construction, like `printf`, in an object file or your own library, you must position this object/library before the C library.

The option **--verbose (-v)** shows how libraries have been searched and which objects have been extracted.

Resolving symbols

If you are linking from libraries, only the objects that contain symbols to which you refer, are extracted from the library. This implies that if you invoke the linker like:

```
ltc mylib.a
```

nothing is linked and no output file will be produced, because there are no unresolved symbols when the linker searches through `mylib.a`.

It is possible to force a symbol as external (unresolved symbol) with the option **--extern (-e)**:

```
ltc --extern=main mylib.a
```

In this case the linker searches for the symbol `main` in the library and (if found) extracts the object that contains `main`.

If this module contains new unresolved symbols, the linker looks again in `mylib.a`. This process repeats until no new unresolved symbols are found.

7.4. Incremental Linking

With the TASKING linker it is possible to link incrementally. Incremental linking means that you link some, but not all .o modules to a relocatable object file .out. In this case the linker does not perform the locating phase. With the second invocation, you specify both new .o files as the .out file you had created with the first invocation.

Incremental linking is only possible on the command line.

```
ltdc --incremental test1.o -otest.out  
ltdc test2.o test.out
```

This links the file test1.o and generates the file test.out. This file is used again and linked together with test2.o to create the file test.elf (the default name if no output filename is given in the default ELF/DWARF format).

With incremental linking it is normal to have unresolved references in the output file until all .o files are linked and the final .out or .elf file has been reached. The [option --incremental \(-r\)](#) for incremental linking also suppresses warnings and errors because of unresolved symbols.

7.5. Cross-Linking

Cross-linking allows linking of (for example, validated) object files built with a certain TASKING VX-toolset for TriCore release into a project that is being developed with a newer toolset release. For version v6.3r1 of the TASKING VX-toolset for TriCore, Altium guarantees that object files developed with TASKING VX-toolset for TriCore versions v4.2r2, v5.0r2, v6.0r1, v6.1r1, v6.2r1 or v6.2r2 of the product can be linked to v6.3r1 object files with the v6.3r1 linker¹, including patches created for those versions, unless explicitly stated otherwise for a specific patch, under the following conditions:

- The following options of the TriCore compiler have the same values for the whole application:

```
--eabi=+bitfield-align
--eabi=+char-bitfield
--eabi=+half-word-align
--eabi=+word-struct-align
--fp-model=+float
--integer-enumeration
--mmu-on and --mmu-present
--signed-bitfields
--uchar
```

Problems caused by using different settings for those options can be detected with global type checking, by specifying the [C compiler option --global-type-checking](#) or [C compiler option --debug-info](#) and the [linker option --global-type-checking](#) (or when you use MIL linking). But only if the older objects were also compiled with [--global-type-checking](#) or [--debug-info](#).

Also keep in mind that the option [--eabi-compliant](#) of the compiler is an alias for a set of [--eabi](#) option flags. To ensure compatibility, the [--eabi](#) option flags [char-bitfield](#) (introduced in v6.1r1), [word-struct-align](#) (introduced in v6.2r1) and [bitfield-align](#) (introduced in v6.3r1) should not be disabled when you are cross-linking objects from older releases, neither directly, nor through the option [--eabi-compliant](#).

Furthermore it is recommended that the following options of the TriCore compiler have the same values for the whole application or PIC module (this includes the corresponding pragmas):

```
--core
--default-a0-size
--default-a1-size
--default-near-size
--fp-model=+soft
--fp-model=+trap
--pic
--silicon-bug
```

- The code in the object files does not rely on undefined or unspecified behavior of the compiler, as defined by the relevant C language standard.
- The code in the object files only relies on implementation-defined behavior that is documented to be identical between the two product versions (see [Chapter 22, C Implementation-defined Behavior](#)). For a list of differences in implementation-defined behavior you can contact TASKING Support.

- The code in the object files does not trigger any of the known bugs in the relevant compiler versions.
- The object files are linked with the linker and libraries supplied with version v6.3r1 of the toolset.
- The code in the object files does not use the C functions `setjmp()` and `longjmp()` to transfer control from code compiled with one version of the compiler to code compiled with another version of the compiler.
- The code in the object files does not use old-style function declarations with an empty parameter list; for example:

```
extern int f(); /* empty parameter list */
```

```
int f( int x ) { ... }
```

Functions with an explicitly empty parameter list, such as `int f(void)`, are supported.

- The code in the object files does not use old-style formal parameter list notation:

```
int f( x ) int x; { ... }
```

- Instances of flexible array members and bit-fields in structures are only accessed by code in object files that have been compiled with the same version of the compiler.

¹ Note that a v4.2r2 object file cannot be linked to a v5.0r2 object file by means of the v6.3r1 linker.

7.6. Linking Core-Specific Projects into a Multi-Core Application

The TASKING VX-toolset for TriCore has support for multi-core versions of the TriCore. Multi-core is supported for the TriCore 1.6.x and 1.6.2 architectures only. To build an application for such a multi-core processor it is sufficient to specify the correct processor in Eclipse or to the control program ([control program option `--cpu=tc27x`](#)).

By default, all cores share code and data, although actual use of functions and variables is determined by the application. If, instead, you want to build an application for a specific core with its own code and data (a separate namespace), you need to select the specific core in Eclipse (for example, TriCore core 0) for the core-specific project. If you build your sources on the command line with the control program, apart from `--cpu` you also have to specify [control program option `--lsl-core=tcn`](#) for core *n*. Your main project should always be a core 0 project that has project references to core-specific projects.

How to share code and data

When sharing data, make sure it is not a0/a1/a8/a9 data if the specific register does not have the same value on the different cores. In the application that references the variable, declare the variable with prefix `"_lc_s_name"` or `"_lc_t_core_name"`. See [Section 7.10, Linker Labels](#).

When sharing code, make sure the code does not use a0/a1/a8/a9 data if the specific register does not have the same value on the different cores. The same symbol prefixes are used as for variables. Calls

from the shared code will use functions from the application where the code is defined, so e.g. a call to C library function `malloc()` from shared code would reference the "wrong" heap.

To select a single-core configuration

1. From the **Project** menu, select **Properties for**
The Properties dialog appears.
2. In the left pane, expand **C/C++ Build** and select **Processor**.
In the right pane the Processor settings appear.
3. From the **Processor Selection** list, select a processor or select **User defined TriCore ...**.
4. From the **Multi-core configuration** list, select a TriCore single-core.

Add the core-specific projects to the main core 0 project

1. In the C/C++ Projects view, right-click on the name of main core 0 TriCore project and select **Properties**.
The Properties dialog appears.
2. In the left pane, select **Project References**.
3. In the right pane, select the core-specific projects that must be part of the TriCore project and click **OK**.

Build the project

1. Make the core 0 project the active project.
2. From the **Project** menu, select **Build project**.

Eclipse will create linker `.out` files for the core-specific projects other than core 0. They will be linked to the main core 0 project to produce the final ELF absolute object file.

When you build your project, the linker is called with linker option `--core=mpe:tcn`, where n is the core number, and the macro `__NO_VTC` is defined with linker option `-D`. The macro `__NO_VTC` must also be defined with C compiler option `-D` when compiling the startup code. The control program passes the proper defines to the tools when you use control program option `--lsl-core=tcn`.

Using the control program

1. Build the core projects other than core 0 to `.out` files (option `.`). For example,

```
cctc -Ctc27x --lsl-core=tc1 source_tc1.c cstart_tc1.c --link-only -t -o tc1.out
cctc -Ctc27x --lsl-core=tc2 source_tc2.c cstart_tc2.c --link-only -t -o tc2.out
```
2. Build the core 0 project with link tasks for each core. For example,

```
cctc -Ctc27x --lsl-core=tc0 mtpproject.c cstart.c --new-task=tc1,tc1.out --new-task=tc2,tc2.out
```

Example project

The `tc_multitask_tc0` and `tc_multitask_tc1` projects are included in the product as an example.

7.7. Importing Binary Files

With the TASKING linker it is possible to add a binary file to your absolute output file. In an embedded application you usually do not have a file system where you can get your data from.

Add a data object in Eclipse

1. Select **Linker » Data Objects**.

The Data objects box shows the list of object files that are imported.

2. To add a data object, click on the **Add** button in the **Data objects** box.
3. Type or select a binary file (including its path).

On the command line you can add raw data to your application with the linker option **--import-object**.

This makes it possible for example to display images on a device or play audio. The linker puts the raw data from the binary file in a section. The section is aligned on a 4-byte boundary. The section name is derived from the filename, in which dots are replaced by an underscore. So, when importing a file called `my.mp3`, a section with the name `my_mp3` is created. In your application you can refer to the created section by using linker labels.

For example:

```
#include <stdio.h>
__far extern char _lc_ub_my_mp3; /* linker labels */
__far extern char _lc_ue_my_mp3;
char* mp3 = &_amp;lc_ub_my_mp3;

void main(void)
{
    int size = &_amp;lc_ue_my_mp3 - &_amp;lc_ub_my_mp3;
    int i;
    for (i=0;i<size;i++)
        putchar(mp3[i]);
}
```

Because the compiler does not know in which space the linker will locate the imported binary, you have to make sure the symbols refer to the same space in which the linker will place the imported binary. You do this by using the [memory qualifier](#) `__far`, otherwise the linker cannot bind your linker symbols.

Also note that if you want to use the export functionality of Eclipse, the binary file has to be part of your project.

7.8. Linker Optimizations

During the linking and locating phase, the linker looks for opportunities to optimize the object code. Both code size and execution speed can be optimized.

To enable or disable optimizations

1. From the **Project** menu, select **Properties for**
The Properties dialog appears.
2. In the left pane, expand **C/C++ Build** and select **Settings**.
In the right pane the Settings appear.
3. On the Tool Settings tab, select **Linker » Optimization**.
4. Enable one or more optimizations.

You can enable or disable the optimizations described below. The command line option for each optimization is given in brackets.

Delete unreferenced sections (option -Oc/-OC)

This optimization removes unreferenced sections from the resulting object file.

This optimization considers a section referenced if either of the following two conditions is true:

1. The section is protected from unreferenced section removal, which can be one of:
 - the section is assigned an absolute address, either in the object file or in LSL
 - the section is selected by exact name in LSL (no wildcard pattern) *
 - a symbol defined in the section is referenced in LSL
 - the section has the 'protected' section flag set, either in the object file or in LSL
2. The section is referenced via a relocation by another section that is considered referenced.

* If multiple sections of a specific name are created by using section renaming, all of these sections are protected against unreferenced section removal. With a selection using wildcards, matching sections are

selected, but matching sections that are unreferenced may be removed. See [Selecting sections for a group](#) in [Section 17.8.2, Creating and Locating Groups of Sections](#).

First fit decreasing (option -OI/-OL)

When the physical memory is fragmented or when address spaces are nested it may be possible that a given application cannot be located although the size of the available physical memory is larger than the sum of the section sizes. Enable the first-fit-decreasing optimization when this occurs and re-link your application.

The linker's default behavior is to place sections in the order that is specified in the LSL file (that is, working from low to high memory addresses or vice versa). This also applies to sections within an unrestricted group. If a memory range is partially filled and a section must be located that is larger than the remainder of this range, then the section and all subsequent sections are placed in a next memory range. As a result of this gaps occur at the end of a memory range.

When the first-fit-decreasing optimization is enabled the linker will first place the largest sections in the smallest memory ranges that can contain the section. Small sections are located last and can likely fit in the remaining gaps.

Compress copy table (option -Ot/-OT)

The startup code initializes the application's data areas. The information about which memory addresses should be zeroed and which memory ranges should be copied from ROM to RAM is stored in the copy table.

When this optimization is enabled the linker will try to locate sections in such a way that the copy table is as small as possible thereby reducing the application's ROM image.

Note that this optimization only affects unrestricted sections that require an initialization action in the copy table. The affected sections get a clustered restriction. Unrestricted sections are sections that do not have their absolute location or their relative location to other sections restricted. See also [Define the mutual order of sections in an LSL group](#) in [Section 17.8.2, Creating and Locating Groups of Sections](#).

Delete duplicate code (option -Ox/-OX)

Delete duplicate constant data (option -Oy/-OY)

These two optimizations remove code and constant data that is defined more than once, from the resulting object file.

Note that when these linker optimizations are enabled, different C objects or functions may have identical addresses. This means that you cannot distinguish these objects or functions with a pointer comparison as described in the ISO C standard (C99/C11 6.5.9p6). If your application relies on pointer comparisons to distinguish different objects and/or functions, disable these linker optimizations.

7.9. Controlling the Linker with a Script

With the options on the command line you can control the linker's behavior to a certain degree. From Eclipse it is also possible to determine where your sections will be located, how much memory is available, which sorts of memory are available, and so on. Eclipse passes these locating directions to the linker via a script file. If you want even more control over the locating process you can supply your own script.

The language for the script is called the *Linker Script Language*, or shortly LSL. You can specify the script file to the linker, which reads it and locates your application exactly as defined in the script. If you do not specify your own script file, the linker always reads a standard script file which is supplied with the toolset.

7.9.1. Purpose of the Linker Script Language

The Linker Script Language (LSL) serves three purposes:

1. It provides the linker with a definition of the target's core architecture. This definition is supplied with the toolset.
2. It provides the linker with a specification of the memory attached to the target processor.
3. It provides the linker with information on how your application should be located in memory. This gives you, for example, the possibility to create overlaying sections.

The linker accepts multiple LSL files. You can use the specifications of the core architectures that Altium has supplied in the `include.lsl` directory. Do not change these files.

If you use a different memory layout than described in the LSL file supplied for the target core, you must specify this in a separate LSL file and pass both the LSL file that describes the core architecture and your LSL file that contains the memory specification to the linker. Next you may want to specify how sections should be located and overlaid. You can do this in the same file or in another LSL file.

LSL has its own syntax. In addition, you can use the standard C preprocessor keywords, such as `#include` and `#define`, because the linker sends the script file first to the C preprocessor before it starts interpreting the script.

The complete LSL syntax is described in [Chapter 17, Linker Script Language \(LSL\)](#).

7.9.2. Eclipse and LSL

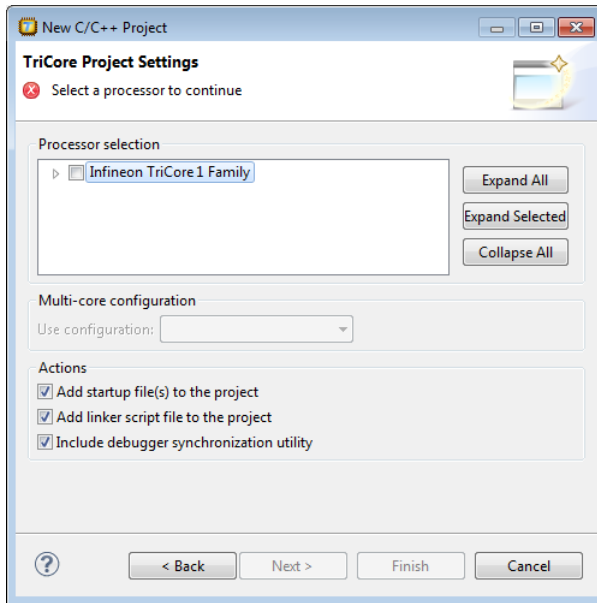
In Eclipse you can specify the size of the stack and heap; the physical memory attached to the processor; identify that particular address ranges are reserved; and specify which sections are located where in memory. Eclipse translates your input into an LSL file that is stored in the project directory under the name `project_name.lsl` and passes this file to the linker. If you want to learn more about LSL you can inspect the generated file `project_name.lsl`.

To add a generated Linker Script File to your project

1. From the **File** menu, select **File » New » TASKING TriCore C/C++ Project**.

The New C/C++ Project wizard appears.

2. Fill in the project settings in each dialog and click **Next >** until the following dialog appears.



3. Enable the option **Add linker script file to the project** and click **Finish**.

Eclipse creates your project and the file "project_name.lsl" in the project directory.

If you do not add the linker script file here, you can always add it later with **File » New » Linker Script File (LSL)**.

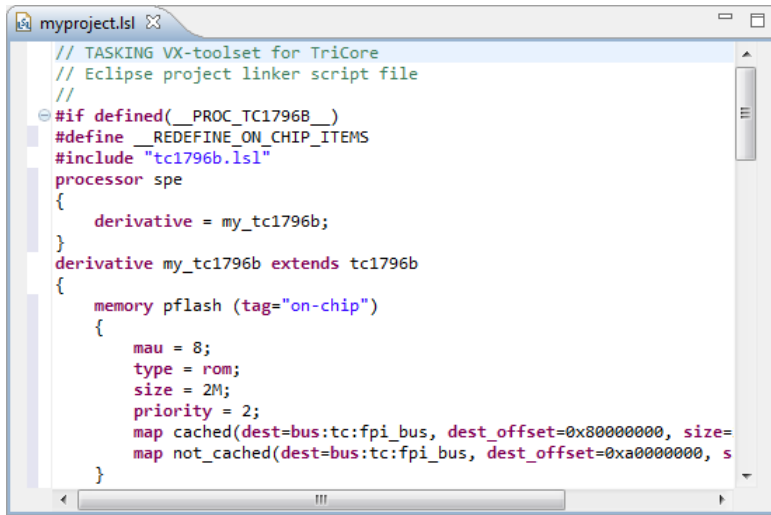
To change the Linker Script File in Eclipse

There are two ways of changing the LSL file in Eclipse.

- You can change the LSL file directly in an editor.

1. Double-click on the file `project_name.lsl`.

The project LSL file opens in the editor area.



2. You can edit the LSL file directly in the `project_name.lsl` editor.

*A * appears in front of the name of the LSL file to indicate that the file has changes.*

3. Click  or select **File » Save** to save the changes.

- You can also make changes to the property pages Memory and Stack/Heap.

1. From the **Project** menu, select **Properties for**

The Properties dialog appears.

2. In the left pane, expand **C/C++ Build** and select **Memory** or **Stack/Heap**.

In the right pane the corresponding property page appears.

3. Make changes to memory and/or stack/heap and click **OK**.

The project LSL file is updated automatically according to the changes you make in the pages.

You can quickly navigate through the LSL file by using the Outline view (**Window » Show View » Outline**).

7.9.3. Preprocessor Macros in the Linker Script Files

The linker script files contain several predefined preprocessor macros. If for some reason you need to change a default value, you can use Eclipse or the [linker option --define](#) to define a new value, or add this option via the control program to the linker.

For example, to set the user stack size from the command line to 24k, enter:

```
cctc -Wl--define=USTACK=24k test.c
```

With option **-WI** the control program passes the macro definition to the linker.

The following macros are available in the linker script files.

Macro	Description
A0_START / A1_START/ A8_START / A9_START	Specifies the fixed address of the A0/A1/A8/A9-addressable segment for all cores.
CSA	Specifies the size of the context save area (same as CSA_TC0).
CSA_TC <i>n</i>	Specifies the size of the context save area for the specified TriCore core <i>n</i> .
CSA_START	Specifies the start address of the context save area (same as CSA_START_TC0).
CSA_START_TC <i>n</i>	Specifies the start address of the context save area for the specified TriCore core <i>n</i> .
HEAP	Specifies the size of the heap.
INTTAB	Specifies the start address of the interrupt table (same as INTTAB0).
INTTAB <i>n</i>	Specifies the start address of the interrupt table for the specified TriCore core <i>n</i> .
ISTACK	Specifies the size of the interrupt stack (same as ISTACK_TC0)..
ISTACK_TC <i>n</i>	Specifies the size of the interrupt stack for the specified TriCore core <i>n</i> .
MCS00_0RAM .. MCSxx_0RAM	Identifies the offset of the memory for core MCS00 .. MCSxx from the GTM base address.
RESET	Specifies the reset address.
TRAPTAB	Specifies the start address of the trap table (same as TRAPTAB0).
TRAPTAB <i>n</i>	Specifies the start address of the trap table for the specified TriCore core <i>n</i> .
USTACK	Specifies the size of the user stack (same as USTACK_TC0).
USTACK_TC <i>n</i>	Specifies the size of the user stack for the specified TriCore core <i>n</i> .
__DISABLE_SCR_BOOT_MAGIC	If defined as 1, no SCR boot magic section will be generated.
__ISTACK_ENTRY_POINTS	Specifies the entry points for stack estimation of the interrupt stack.
__ISTACK <i>n</i> _ENTRY_POINTS	Specifies the entry points for stack estimation of the interrupt stack for the specified TriCore core <i>n</i> . Multiple entry points are separated by commas and enclosed in square brackets [].
__MAX_CONCURRENT_HANDLERS	Specifies the number of interrupt and trap handlers that should contribute to stack size estimation. Defining this macro adds a threads keyword to <code>ustack</code> , with the macro value increased by one.
__NO_VTC	When this macro is defined, the virtual core <code>vtc</code> is not available, so a separate link task is needed for each TriCore core. By default this macro is undefined.

Macro	Description
<code>__REDEFINE_ON_CHIP_ITEMS</code>	If defined as 1, no on-chip memories are defined.
<code>__TCn_Ax_START</code>	Specifies the fixed address of the Ax-addressable segment for core <i>n</i> , available when macro <code>__NO_VTC</code> is defined. This macro overrides the value set with <code>Ax_START</code> for core <i>n</i> . By default this macro is not set.
<code>__USTACK_ENTRY_POINTS</code>	Specifies the entry points for stack estimation of the user stack.
<code>__USTACKn_ENTRY_POINTS</code>	Specifies the entry points for stack estimation of the user stack for the specified TriCore core <i>n</i> . Multiple entry points are separated by commas and enclosed in square brackets [].

7.9.4. Structure of a Linker Script File

A script file consists of several definitions. The definitions can appear in any order.

The architecture definition (required)

In essence an *architecture definition* describes how the linker should convert logical addresses into physical addresses for a given type of core. If the core supports multiple address spaces, then for each space the linker must know how to perform this conversion. In this context a physical address is an offset on a given internal or external bus. Additionally the architecture definition contains information about items such as the (hardware) stack and the interrupt vector table.

This specification is normally written by Altium. Altium supplies LSL files in the `include.lsl` directory. The file `tc_arch.lsl` defines the base architecture for all generic TriCore cores and includes an interrupt vector table (`inttab.lsl`) and an trap vector table (`traptab.lsl`). The file `tc_mc_arch.lsl` defines the base architecture for all multi-core TriCore cores. The files `tc1v1_3.lsl`, `tc1v1_3_1.lsl`, `tc1v1_6.lsl`, `tc1v1_6_x.lsl` and `tc1v1_6_2.lsl` extend the base architecture for each TriCore core.

The architecture definition of the LSL file should not be changed by you unless you also modify the core's hardware architecture. If the LSL file describes a multi-core system an architecture definition must be available for each different type of core.

The linker uses the architecture name in the LSL file to identify the target. For example, the default library search path can be different for each core architecture.

The derivative definition

The *derivative definition* describes the configuration of the internal (on-chip) bus and memory system. Basically it tells the linker how to convert offsets on the buses specified in the architecture definition into offsets in internal memory. Microcontrollers and DSPs often have internal memory and I/O sub-systems apart from one or more cores. The design of such a chip is called a *derivative*.

When you want to use multiple cores of the same type, you must instantiate the cores in a derivative definition, since the linker automatically instantiates only a single core for an unused architecture.

Altium supplies LSL files for each derivative (`derivative.lsl`), along with "SFR files", which provide easy access to registers in I/O sub-systems from C and assembly programs. When you build an ASIC

or use a derivative that is not (yet) supported by the TASKING tools, you may have to write a derivative definition.

The processor definition

The *processor definition* describes an instance of a derivative. A processor definition is only needed in a multi-processor embedded system. It allows you to define multiple processors of the same type.

If for a derivative 'A' no processor is defined in the LSL file, the linker automatically creates a processor named 'A' of derivative 'A'. This is why for single-processor applications it is enough to specify the derivative in the LSL file.

The memory and bus definitions (optional)

Memory and bus definitions are used within the context of a derivative definition to specify internal memory and on-chip buses. In the context of a board specification the memory and bus definitions are used to define external (off-chip) memory and buses. Given the above definitions the linker can convert a logical address into an offset into an on-chip or off-chip memory device.

The board specification

The processor definition and memory and bus definitions together form a board specification. LSL provides language constructs to easily describe single-core and heterogeneous or homogeneous multi-core systems. The board specification describes all characteristics of your target board's system buses, memory devices, I/O sub-systems, and cores that are of interest to the linker. Based on the information provided in the board specification the linker can for each core:

- convert a logical address to an offset within a memory device
- locate sections in physical memory
- maintain an overall view of the used and free physical memory within the whole system while locating

The section layout definition (optional)

The optional section layout definition enables you to exactly control where input sections are located. Features are provided such as: the ability to place sections at a given address, to place sections in a given order, and to overlay code and/or data sections.

Example: Skeleton of a Linker Script File

A linker script file that defines a derivative "X" based on the TC1V1.3 architecture, its external memory and how sections are located in memory, may have the following skeleton:

```
architecture TC1V1.3
{
    // Specification of the TC1V1.3 core architecture.
    // Written by Altium.
}

derivative X // derivative name is arbitrary
```

```

{
    // Specification of the derivative.
    // Written by Altium.
    core tc          // always specify the core
    {
        architecture = TC1V1.3;
    }

    bus fpi_bus      // internal bus
    {
        // maps to bus "fpi_bus" in "tc" core
    }

    // internal memory
}

processor spe        // processor name is arbitrary
{
    derivative = X;

    // You can omit this part, except if you use a
    // multi-core system.
}

memory ext_name
{
    // external memory definition
}

section_layout spe:tc:linear // section layout
{
    // section placement statements

    // sections are located in address space 'linear'
    // of core 'tc' of processor 'spe'
}

```

Overview of LSL files delivered by Altium

Altium supplies the following LSL files in the directory `include.lsl`.

LSL file	Description
<code>tc_arch.lsl</code>	Defines the base architecture (TC) for all generic TriCore cores. It includes the files <code>base_address_groups.lsl</code> , <code>inttab.lsl</code> and <code>traptab.lsl</code> .
<code>tc_mc_arch.lsl</code>	Defines the base architecture (TC) for all multi-core TriCore cores.
<code>mcs_arch.lsl</code>	Defines the base architecture (MCS) for all MCS cores.
<code>base_address_groups.lsl</code>	Groups sections that belong to A0, A1, A8 or A9. It is included in the file <code>tc_arch.lsl</code> . See also linker option --auto-base-register

LSL file	Description
<code>inttab.lsl</code>	Defines the interrupt vector table. It is included in the file <code>tc_arch.lsl</code> .
<code>inttabn.lsl</code>	Defines a core <i>n</i> specific interrupt vector table. It is included in derivative LSL files that have multi-core support.
<code>traptab.lsl</code>	Defines the trap vector table. It is included in the file <code>tc_arch.lsl</code> .
<code>traptabn.lsl</code>	Defines a core <i>n</i> specific trap vector table. It is included in derivative LSL files that have multi-core support.
<code>tc1v1_3.lsl</code> <code>tc1v1_3_1.lsl</code> <code>tc1v1_6.lsl</code> <code>tc1v1_6_x.lsl</code> <code>tc1v1_6_2.lsl</code>	Extends the base architecture for cores TC1V1.3, TC1V1.3.1, TC1V1.6, TC1V1.6.X or TC1V1.6.2. It includes the file <code>tc_arch.lsl</code> or <code>tc_mc_arch.lsl</code> .
<code>derivative.lsl</code>	Defines the derivative and defines a single processor (spe) or a multi-core processor (mpe). Contains a memory definition and section layout. It includes one of the files <code>tcversion.lsl</code> . The selection of the derivative is based on your CPU selection (control program option <code>--cpu</code>). AURIX derivatives also include the file <code>tc1v1_6_x.bmhd.lsl</code> or <code>tc1v1_6_2.bmhd.lsl</code> .
<code>userdef13.lsl</code> <code>userdef131.lsl</code> <code>userdef16.lsl</code> <code>userdef16x.lsl</code> <code>userdef162.lsl</code>	Defines a user defined derivative for cores TC1V1.3, TC1V1.3.1, TC1V1.6, TC1V1.6.X or TC1V1.6.2 and defines a single processor for TC1V1.3, TC1V1.3.1 and TC1V1.6, and a multi-core processor for TC1V1.6.X and TC1V1.6.2, or a single core in a multi-core processor environment for TC1V1.6.X and TC1V1.6.2. <code>userdef16x.lsl</code> and <code>userdef162.lsl</code> also include the file <code>tc1v1_6_x.bmhd.lsl</code> or <code>tc1v1_6_2.bmhd.lsl</code> .
<code>template.lsl</code>	This file is used by Eclipse as a template for the project LSL file. It includes the file <code>cpu.lsl</code> .
<code>template_pic.lsl</code>	This file is used by Eclipse as a template for a PIC/PID project LSL file. It includes the file <code>pic.lsl</code> .
<code>cpu.lsl</code>	This file includes the file <code>derivative.lsl</code> based on your CPU selection. The CPU is specified by the <code>__CPU__</code> macro.
<code>default.lsl</code>	Contains a default memory definition and section layout based on the tc1796b derivative. This file is used on a command line invocation of the tools, when no CPU is selected (no option <code>--cpu</code>). It includes the file <code>extmem.lsl</code> .
<code>extmem.lsl</code>	Template file with a specification of the external memory attached to the target processor.
<code>pic.lsl</code>	This file contains definitions for the creation of position independent modules.
<code>tc1v1_6_x.bmhd.lsl</code> <code>tc1v1_6_2.bmhd.lsl</code>	AURIX Boot Mode Header definition for BMHD0 .. BMHD3. See Section 7.9.14, Boot Mode Headers .

When you select to add a linker script file when you create a project in Eclipse, Eclipse makes a copy of the file `template.lsl` and names it "*project_name.lsl*". On the command line, the linker uses the file `default.lsl`, unless you specify another file with the linker option `--lsl-file (-d)`.

7.9.5. The Architecture Definition

Although you will probably not need to write an architecture definition (unless you are building your own processor core) it helps to understand the Linker Script Language and how the definitions are interrelated.

Within an *architecture definition* the characteristics of a target processor core that are important for the linking process are defined. These include:

- space definitions: the logical address spaces and their properties
- bus definitions: the core local buses and I/O buses of the core architecture
- mappings: the address translations between logical address spaces, the connections between logical address spaces and buses and the address translations between buses
- Memory Protection Unit (MPU) layout: the number of memory protection register sets and the number of data protection ranges and code protection ranges.

Address spaces

A logical address space is a memory range for which the core has a separate way to encode an address into instructions. Most microcontrollers and DSPs support multiple address spaces. For example, the TriCore's 32-bit linear address space encloses 16 24-bit sub-spaces and 16 14-bit sub-spaces. See also the section "*Memory Model*" in the *TriCore Architecture Manual*. Normally, the size of an address space is 2^N , with N the number of bits used to encode the addresses.

The relation of an address space with another address space can be one of the following:

- one space is a subset of the other. These are often used for "small" absolute or relative addressing.
- the addresses in the two address spaces represent different locations so they do not overlap. This means the core must have separate sets of address lines for the address spaces. For example, in Harvard architectures we can identify at least a code and a data memory space.

Address spaces (even nested) can have different minimal addressable units (MAU), alignment restrictions, and page sizes. All address spaces have a number that identifies the logical space (id). The following table lists the different address spaces for the architecture TC as defined in `tc_arch.lsl`.

Space	Id	MAU	Description	ELF sections
linear	1	8	Linear address space	.text*, .data*, .sdata*, .ldata*, .rodata*, .bss*, .sbss*, table, istack, ustack
abs24	2	8	Absolute 24-bit addressable space	
abs18	3	8	Absolute 18-bit addressable space	.zdata, .zrodata, .zbss
csa	4	8	Context Save Area	csa.*
pcp_code	8	16	PCP code	.pcptext
pcp_data	9	32	PCP data	.pcpdata

The following table lists the different address spaces for the architecture TC as defined in `tc_mc_arch.lsl` for multi-core processors.

Space	Id	MAU	Description	ELF sections
linear	1	8	Linear address space	.text*, .data*, .sdata*, .ldata*, .rodata*, .bss*, .sbss*, table, istack, ustack
abs24	2	8	Absolute 24-bit addressable space	
abs18	3	8	Absolute 18-bit addressable space	.zdata, .zrodata, .zbss
csa	4	8	Context Save Area	csa.*

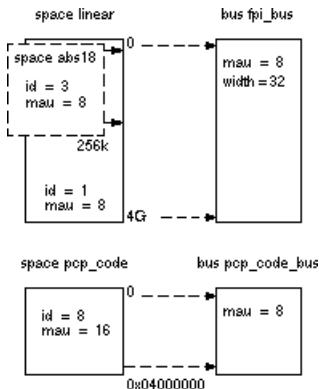
The MCS, which is part of TriCore v1.6.x and v1.6.2 derivatives, such as TC27x, has one address space for architecture MCS as defined in `mcs_arch.lsl`.

Space	Id	MAU	Description	ELF sections
mcs	1	8	MCS address space	.mcstext, .mcsdata

The MCS is described in a separate manual. See the *TASKING VX-toolset for MCS User Guide* for more information.

The TriCore architecture in LSL notation

The best way to write the architecture definition, is to start with a drawing. The following figure shows a part of the TriCore architecture:



The figure shows three address spaces called `linear`, `abs18` and `pcp_code`. The address space `abs18` is a subset of the address space `linear`. All address spaces have attributes like a number that identifies the logical space (`id`), a MAU and an alignment. In LSL notation the definition of these address spaces looks as follows:

```
space linear
{
    id = 1;
    mau = 8;

    map (src_offset=0x00000000, dest_offset=0x00000000,
        size=4G, dest=bus:fpi_bus);
}
```

```
space abs18
{
    id = 3;
    mau = 8;

    map (src_offset=0x00000000, dest_offset=0x00000000,
        size=16k, dest=space:linear);
    map (src_offset=0x10000000, dest_offset=0x10000000,
        size=16k, dest=space:linear);
    map (src_offset=0x20000000, dest_offset=0x20000000,
        size=16k, dest=space:linear);
    //...
}

space pcpcode
{
    id = 8;
    mau = 16;
    map (src_offset=0x00000000, dest_offset=0,
        size=0x04000000, dest=bus:pcpcode_bus);
}
```

Mappings

The keyword map corresponds with the arrows in the drawing. You can map:

- address space => address space
- address space => bus
- memory => bus (not shown in the drawing)
- bus => bus (not shown in the drawing)

Buses

Next the two internal buses, named `fpi_bus` and `pcpcode_bus` must be defined in LSL:

```
bus fpi_bus
{
    mau = 8;
    width = 32; // there are 32 data lines on the bus
}

bus pcpcode_bus
{
    mau = 8;
    width = 8;
}
```

MPU layout

For each AURIX architecture that supports it, the layout of the Memory Protection Unit (MPU) must be defined in LSL.

```
mpu_layout
{
    register_sets = 4;
    data_ranges = 16;
    code_ranges = 8;
}
```

Architecture definition

This completes the LSL code in the architecture definition. Note that all code above goes into the architecture definition, thus between:

```
architecture TC1V1.3
{
    // All code above goes here.
}
```

7.9.6. The Derivative Definition

Although you will probably not need to write a derivative definition (unless you are using multiple cores that both access the same memory device) it helps to understand the Linker Script Language and how the definitions are interrelated.

A *derivative* is the design of a processor, as implemented on a chip (or FPGA). It comprises one or more cores and on-chip memory. The derivative definition includes:

- core definition: an instance of a core architecture
- bus definition: the I/O buses of the core architecture
- memory definitions: internal (or on-chip) memory

Core

Each derivative must have at least one core and each core must have a specification of its core architecture. This core architecture must be defined somewhere in the LSL file(s).

```
core tc
{
    architecture = TC1V1.3;
}
```

A link task (resolving symbols from object files and libraries) is associated with one core in the LSL file(s). In a multi-core environment you can combine multiple cores with the same architecture into a single link task. This is done by importing one or more cores into a root core with an `import` statement. By importing a core the hardware resources of that core are made available to the link task associated with the core

that contains the `import` statement. The imported cores share a single symbol namespace. The address spaces in each imported core must have a unique ID in the link task. For each imported core is specified that the space IDs of the imported core start at a specific offset. If writable sections for a core must be initialized by using the copy table of a different core, this is specified by a `copytable_space`. The following example is part of `tc27x.lsl` delivered with the product.

```
core tc0 // core 0
{
    architecture = TC1V1.6.X;
    space_id_offset = 100; // add 100 to all space IDs in
                          // the architecture definition
    copytable_space = vtc:linear; // use copytable from core vtc
}
core tc1 // core 1
{
    architecture = TC1V1.6.X;
    space_id_offset = 200; // add 200 to all space IDs in
                          // the architecture definition
    copytable_space = vtc:linear; // use copytable from core vtc
}
core tc2 // core 2
{
    architecture = TC1V1.6.X;
    space_id_offset = 300; // add 300 to all space IDs in
                          // the architecture definition
    copytable_space = vtc:linear; // use copytable from core vtc
}
core vtc
{
    architecture = TC1V1.6.X;
    import tc0; // add all address spaces of tc0 for linking
    import tc1; // add all address spaces of tc1 for linking
    import tc2; // add all address spaces of tc2 for linking
}
```

So, for TriCore multi-core architectures, such as the AURIX derivatives, core `vtc` is used for resolving symbols, linking and locating all addresses of all TriCore cores, because the memory map is virtually the same for all cores.

Bus

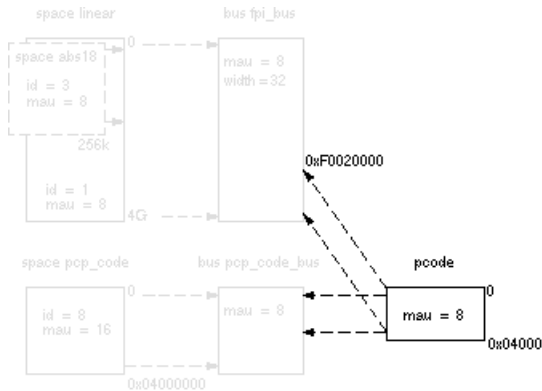
Each derivative can contain a bus definition for connecting external memory. In this example, the bus `fpi_bus` maps to the bus `fpi_bus` defined in the architecture definition of core `tc`:

```
bus fpi_bus
{
    mau = 8;
    width = 32;
```

```
map (dest=bus:tc:fpi_bus, dest_offset=0, size=4G);
}
```

Memory

External memory is usually described in a separate memory definition, but you can specify on-chip memory for a derivative. For example:



According to the drawing, the TriCore contains internal memory called pcode with a size 0x04000 (16 kB). This is physical memory which is mapped to the internal bus pcp_code_bus and to the fpi_bus, so both the tc unit and the PCP can access the memory:

```
memory pcode
{
    mau = 8;
    size = 16k;
    type = ram;
    map (dest=bus:tc:fpi_bus, dest_offset=0xF0020000,
        size=16k);
    map (dest=bus:tc:pcp_code_bus, size=16k);
}
```

This completes the LSL code in the derivative definition. Note that all code above goes into the derivative definition, thus between:

```
derivative X    // name of derivative
{
    // All code above goes here
}
```

7.9.7. The Processor Definition

The processor definition is only needed when you write an LSL file for a multi-processor embedded system. The processor definition explicitly instantiates a derivative, allowing multiple processors of the same type.

```
processor name
{
    derivative = derivative_name;
}
```

If no processor definition is available that instantiates a derivative, a processor is created with the same name as the derivative.

Altium defines a "single processor environment" (spe) in each *derivative.lsl* file. For example:

```
processor spe
{
    derivative = tc1796b;
}
```

For TriCore derivatives that have multiple processor cores, Altium defines a "multi-core processor environment" (mpe) in each *derivative.lsl* file. For example:

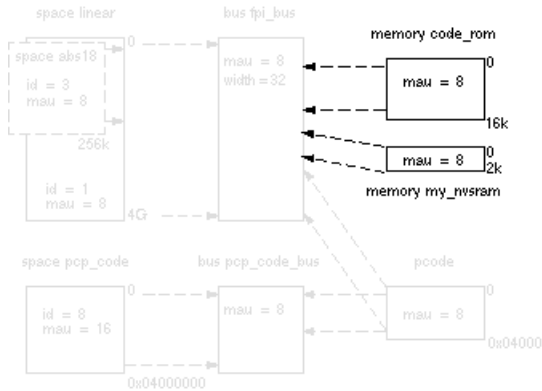
```
processor mpe
{
    derivative = tc27x;
}
```

7.9.8. The Memory Definition

Once the core architecture is defined in LSL, you may want to extend the processor with external (or off-chip) memory. You need to specify the location and size of the physical external memory devices in the target system.

The principle is the same as defining the core's architecture but now you need to fill the memory definition:

```
memory name
{
    // memory definitions
}
```

Suppose your embedded system has 16 kB of external ROM, named `code_rom` and 2 kB of external NVRAM, named `my_nvsram`. Both memories are connected to the bus `fpi_bus`. In LSL this looks like:

```
memory code_rom
{
    mau = 8;
    size = 16k;
    type = rom;
    map( dest=bus:spe:fpi_bus, dest_offset=0xa0000000, size=16k );
}

memory my_nvsram
{
    mau = 8;
    size = 2k;
    type = nvram;
    map( dest=bus:spe:fpi_bus, dest_offset=0xc0000000, size=2k );
}
```

If you use a different memory layout than described in the LSL file supplied for the target core, you can specify this in Eclipse or you can specify this in a separate LSL file and pass both the LSL file that describes the core architecture and your LSL file that contains the memory specification to the linker.

To add memory using Eclipse

1. From the **Project** menu, select **Properties for**

The Properties dialog appears.

2. In the left pane, expand **C/C++ Build** and select **Memory**.

In the right pane the Memory page appears.

3. Open the **Memory** tab and click on the **Add...** button.

The Add new memory dialog appears.

4. Enter the memory name (for example `my_nvram`), type (for example `nvr`) and size.
5. Click on the **Add...** button.

The Add new mapping dialog appears.

6. You have to specify at least one mapping. Enter the mapping name (optional), address, size and destination and click **OK**.

The new mapping is added to the list of mappings.

7. Click **OK**.

The new memory is added to the list of memories (user memory).

8. Click **OK** to close the Properties dialog.

The updated settings are stored in the project LSL file.

If you make changes to the on-chip memory as defined in the architecture LSL file, the memory is copied to your project LSL file and the line `#define __REDEFINE_ON_CHIP_ITEMS` is added. If you remove all the on-chip memory from your project LSL file, also make sure you remove this define.

7.9.9. The Section Layout Definition: Locating Sections

Once you have defined the internal core architecture and optional memory, you can actually define where your application must be located in the physical memory.

During compilation, the compiler divides the application into sections. Sections have a name, an indication (section type) in which address space it should be located and attributes like writable or read-only.

In the section layout definition you can exactly define how input sections are placed in address spaces, relative to each other, and what their absolute run-time and load-time addresses will be.

Example: section propagation through the toolset

To illustrate section placement, the following example of a C program (`bat.c`) is used. The program saves the number of times it has been executed in battery back-upped memory, and prints the number.

```
#define BATTERY_BACKUP_TAG 0xa5f0
#include <stdio.h>

int uninitialized_data;
int initialized_data = 1;
#pragma section all "non_volatile"
#pragma nclear
int battery_backup_tag;
int battery_backup_invok;
#pragma clear
#pragma section all

void main (void)
```

```

{
    if (battery_backup_tag != BATTERY_BACKUP_TAG )
    {
        // battery back-upped memory area contains invalid data
        // initialize the memory
        battery_backup_tag = BATTERY_BACKUP_TAG;
        battery_backup_invok = 0;
    }
    printf( "This application has been invoked %d times\n",
           battery_backup_invok++);
}

```

The compiler assigns names and attributes to sections. With the `#pragma section` the compiler's default section naming convention is overruled and a section with the name `non_volatile` is defined. In this section the battery back-upped data is stored.

By default the compiler creates a section with the name `".zbss.bat"` of section type `data` to store uninitialized data objects. The section prefix `".zbss"` tells the linker to locate the section in address space `abs18` and that the section content should be filled with zeros at startup.

As a result of the `#pragma section` all `"non_volatile"`, the data objects between the pragma pair are placed in a section with the name `".zbss.non_volatile"`. Note that `".zbss"` sections are cleared at startup. However, battery back-upped sections should not be cleared and therefore we used `#pragma nclear`.

Section placement

The number of invocations of the example program should be saved in non-volatile (battery back-upped) memory. This is the memory `my_nvram` from the example in [Section 7.9.8, The Memory Definition](#).

To control the locating of sections, you need to write one or more section definitions in the LSL file. At least one for each address space where you want to change the default behavior of the linker. In our example, we need to locate sections in the address space `abs18`:

```

section_layout ::abs18
{
    select "ELF sections";
    // Section placement statements
}

```

The space, in this case `abs18`, and the *ELF sections* must be a valid combination from the table in [Section 7.9.5, The Architecture Definition](#).

To locate sections, you must create a group in which you select sections from your program. For the battery back-up example, we need to define one group, which contains the section `.zbss.non_volatile`. All other sections are located using the defaults specified in the architecture definition. Section `.zbss.non_volatile` should be placed in non-volatile ram. To achieve this, the run address refers to our non-volatile memory called `my_nvram`.

```

group ( run_addr = mem:my_nvram )
{

```

```

    select ".zbss.non_volatile";
}

```

This completes the LSL file for the sample architecture and sample program. You can now invoke the linker with this file and the sample program to obtain an application that works for this architecture.

For a complete description of the Linker Script Language, refer to [Chapter 17, Linker Script Language \(LSL\)](#).

7.9.10. Locating in a Multi-core Processor Environment

Locating in core local RAM with link time core association

For TriCore derivatives that have multi-core support, such as the TC27x or TC39x, the preferred way of locating is to use the core `vtc` and just specify the addresses where you want to locate a section. Instead of determining the core at compile time, for example by using `__private0` in your C source, you can use link time core association: omit the keyword in your C source and locate the section directly at the correct location in RAM as follows:

```

section_layout mpe:vtc:linear // core vtc containing all address spaces
{
    // of all cores

    group pspr0 ( run_addr = mem:mpe:pspr0, copy ) // tc0 memory pspr0
    {
        select "*.pspr0"; // select sections for pspr0
    }
}

```

The `copy` keyword tells to copy the section from ROM to RAM at program startup.

If you do use `__private0` in your C source, you have to use `mpe:tc0` instead of `mpe:vtc`, because section selections are restricted to the address space of the section layout in which the group definition occurs.

Locating in program flash memory for TC39x

If you want to locate sections in `pflash0` for TC39x derivatives, it is advised not to use the `__private0` keyword in your C sources. You can locate the section in `pflash0` using link time core association as follows (note that the `copy` keyword is omitted in this case):

```

section_layout :vtc:linear // core vtc containing all address spaces
{
    // of all cores

    group pflash0 ( run_addr = mem:mpe:pflash0 ) // tc0 memory pflash0
    {
        select "*.pflash0"; // select sections for pflash0
    }
}

```

See also the `pflash` example project delivered with the product.

Locating in DLMU (Distributed Local Memory Unit SRAM) for TC39x

If you want to locate sections in DLMU per core for TC39x derivatives, use link time core association as follows (core 0 is taken as an example):

```
section_layout :vtc:linear // core vtc containing all address spaces
{
    // of all cores

    group dlmuo ( run_addr = mem:mpe:dlmucpu0 ) // tc0 memory dlmucpu0
    {
        select "*.dlmu0"; // select sections that end in dlmuo
    }
}
```

See also the dlmuo example project delivered with the product.

Locating clone sections

Instead of using the `__clone` keyword in C you can create a clone section in LSL with the `section_setup` keyword in combination with `modify input`. For more details about these LSL keywords, see [Section 17.7.1, Setting up a Section](#). The following example shows a definition for a multi-core TriCore with six cores with clone sections in address space `linear` in five of the available cores.

```
section_setup :vtc:linear
{
    modify input (space = mpe:vtc:mpe_tc0_linear|mpe_tc1_linear|mpe_tc2_linear|
                        mpe_tc4_linear|mpe_tc5_linear)
    {
        select ".text.file_1.func_1";
    }
}
```

Note that core 3 is not included in this example.

In order to locate this section e.g. at a dedicated address in the core local memory use an entry like:

```
section_layout :vtc:mpe_tc0_linear|mpe_tc1_linear|mpe_tc2_linear|
               mpe_tc4_linear|mpe_tc5_linear
{
    group MY_CLONE_FUNCTION (ordered, run_addr = 0xC0000800)
    {
        select ".text.file_1.func_1";
    }
}
```

Note that clone is not possible for DLMU memory, because the core local DLMU memories only have unique addresses.

7.9.11. Locating Private Sections in ROM

For TriCore derivatives that have multi-core support, such as the TC27x, private sections are by default located in core-local RAM. If however all core-local RAM is used, you can tell the linker to locate private sections in ROM. You can do this by adding the keywords `nocopy`, `attributes=rx` to the group specification in LSL. See also [Section 1.4, Multi-Core Support](#).

The following example shows the function `main()` in `main.c` that calls function `p0()` that is marked as `__private0` in `private0.c`.

```
/* main.c */
extern void __private0 p0( void );
extern int i;
int main( void )
{
    p0();
    return i;
}

/* private0.c */
int i;

void __private0 p0( void )
{
    i++;
}
```

To specify that the section `.text.private0.private0.p0` must be located in ROM instead of core-local RAM, you can specify the following LSL part:

```
// nocopy.lsl

section_layout mpe:tc0:code
{
    group PSPR ( run_addr = mem:mpe:pflash0, nocopy, attributes=rx )
    {
        select ".text.private0.private0.p0";
    }
}
```

The keyword `nocopy` specifies that the section is not copied from ROM to RAM at program startup and `attributes=rx` marks the section read-only and executable.

After the following invocation on the command line, you can inspect the resulting map file to see the results.

```
cctc -Ctc27x main.c private0.c -wl-dnocopy.lsl
```

Part of map file:

```
+ Space mpe:tc0:code (MAU = 8bit)
```

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Chip           | Group | Section                                     | Size (MAU) | ...
|=====|=====|=====|=====|=====|
| mpe:pflash0    | PSPR  | .text.private0.private0.p0 (2) | 0x0000000c | ...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

7.9.12. Stack Size Estimation

The TriCore architecture defines two stacks: the user stack (ustack) and the interrupt stack (istack). Several TriCore devices have more than one TriCore core, each of which has its own pair of ustack and istack. It is possible to calculate the stack usage for interrupt handlers and the stack usage for each core separately.

The TriCore architecture has one stack pointer register. The ustack/istack switch is done by loading a different value into the stack pointer register. This means the compiler does not know what stack a function will use, nor on which cores a function will run. Instead, the linker must associate code with stack areas. This can be done through the linker script language (LSL).

If a separate program is run on a specific core *n*, then the stack usage of this program can be computed separately by defining LSL macro `__USTACKn_ENTRY_POINTS` to the name of the symbol (between double quotes) that represents the main function for this program. [entry_points statements](#) are used for this. Multiple symbols can be specified by listing them between square brackets [], separated by commas. Each symbol name must correspond to the caller name of a `.CALLS` directive as generated by the compiler.

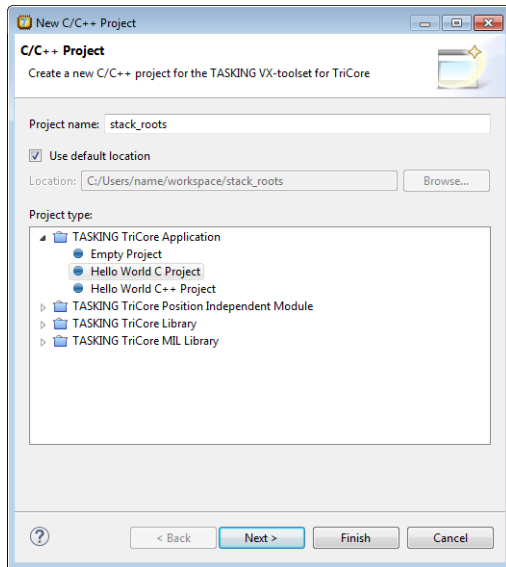
Note that pre-AURIX TriCore derivative LSL files use `__USTACK_ENTRY_POINTS` and `__ISTACK_ENTRY_POINTS`, whereas AURIX and AURIX 2G derivative LSL files use `__USTACKn_ENTRY_POINTS` or `__ISTACKn_ENTRY_POINTS` for core *n*.

Create a multi-core project and specify the stack entry points

The following example multi-core project shows you how to specify stack entry points for the core local user stacks `ustack_tc1` and `ustack_tc2`. In the TriCore Eclipse IDE perform the following steps:

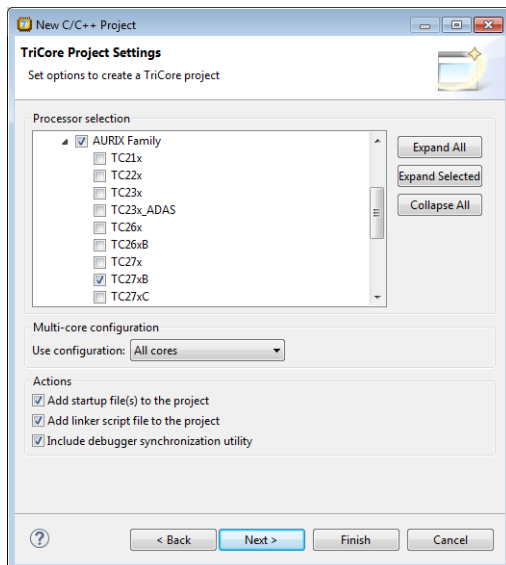
1. From the **File** menu, select **New » TASKING TriCore C/C++ Project**.

The New C/C++ Project wizard appears.



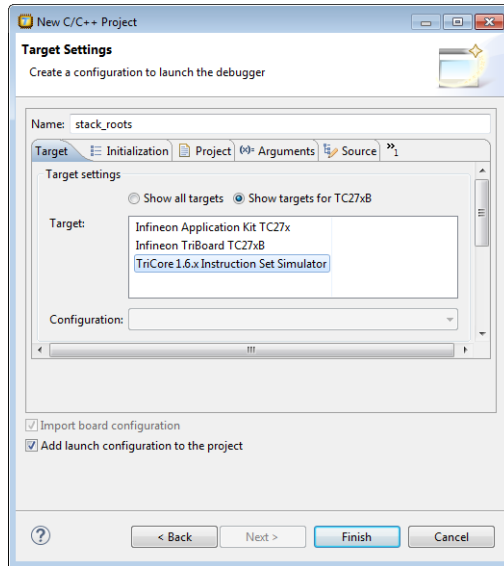
2. Enter a name for your project, for example `stack_roots`.
3. In the **Project type** box, expand **TASKING TriCore Application** and select **Hello World C Project**.
This creates the file `stack_roots.c` with a simple main function.
4. Click **Next**.

The TriCore Project Settings page appears.



5. Select a multi-core processor. In this example we choose the TC27xB.
6. In the **Multi-core configuration** select **All cores**.
7. Enable all **Actions** checkboxes and click **Next**.

The Target Settings page appears.



8. Select the simulator and click **Finish**.
9. Replace the contents of `stack_roots.c` with the following source:

```
#ifdef __CPU__
#include __SFRFILE__(__CPU__)
#endif

#define CORE __mfc(CORE_ID)

int f1(int n)
{
    return n * 7 + 29;
}

void main_tc0(void)
{
    int arr[28];
    int i;

    arr[0] = 8;
```

```
    for (i = 1; i < 28; ++i)
    {
        arr[i+1] = f1(arr[i]);
    }
}

int f2(int n)
{
    return n * 5 + 83;
}

void main_tc1(void)
{
    int arr[78];
    int i;

    arr[0] = 194;

    for (i = 1; i < 78; ++i)
    {
        arr[i+1] = f2(arr[i]);
    }
}

int f3(int n)
{
    return n * 5 + 83;
}

void main_tc2(void)
{
    int arr[23];
    int i;

    arr[0] = 14;

    for (i = 1; i < 23; ++i)
    {
        arr[i+1] = f3(arr[i]);
    }
}

int main(int argc, char ** argv)
{
    switch (CORE)
    {
        case 0:
            main_tc0();
            break;
        case 1:
```

```

    main_tc1();
    break;
case 2:
    main_tc2();
    break;
}
return 0;
}

```

- Open file `stack_roots.lsl` and add the following two lines at the beginning of the file

```

#define __USTACK1_ENTRY_POINTS "main_tc1"
#define __USTACK2_ENTRY_POINTS "main_tc2"

```

- From the **Project** menu, select **Properties for stack_roots**, select **C/C++ Build » Startup Configuration**, and in the **core tc0** tab enable **Start TC1** and **Start TC2**. and click **OK**.

This will start the other cores from the main core 0.

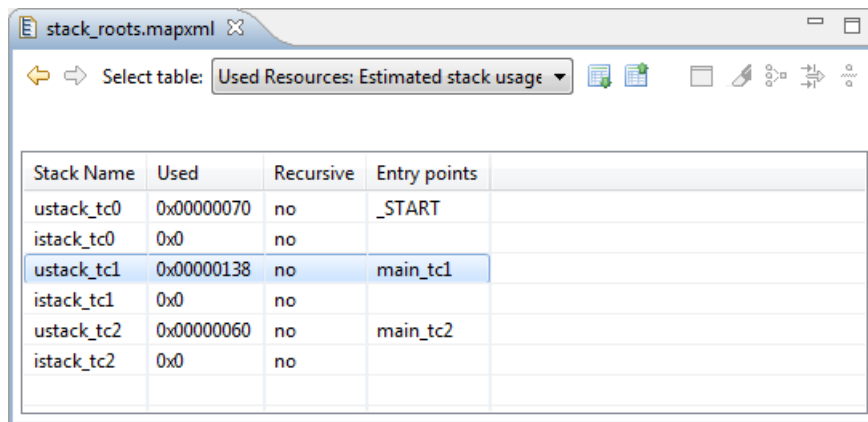
Build the project

- From the **Project** menu, select **Rebuild stack_roots**.

This creates files in the Debug folder of your project.

Examine the stack size estimation in the linker map file

- From the Debug folder in your project, double-click on `stack_roots.mapxml` to open the map file.
- From the **Select table** list, select **Used Resources: Estimated stack usage**. You will see results similar to this.



Stack Name	Used	Recursive	Entry points
ustack_tc0	0x00000070	no	_START
istack_tc0	0x0	no	
ustack_tc1	0x00000138	no	main_tc1
istack_tc1	0x0	no	
ustack_tc2	0x00000060	no	main_tc2
istack_tc2	0x0	no	

As you can see, apart from the default `ustack_tc0` and `istack_tc0`, there are now also stack estimations for the stacks `ustack_tc1` and `ustack_tc2`. These entry points are now also visible in the call graph as root functions.

The **Used** column contains an estimation of the stack usage. The linker calculates the required stack size by using information (`.CALLS` directives) generated by the compiler. If for example recursion is detected, the calculated stack size is inaccurate; therefore this is an estimation only. The calculated stack size is supposed to be smaller than the actual allocated stack size. If that is not the case, then a warning is given.

The **Entry Points** column contains a list of entry points used for estimation of the stack usage.

7.9.13. MPU Configuration Data

Within some multi-core automotive applications it is required to partition code and data into different safety groups based on ISO 26262 ASIL levels. Safety groups have to be isolated from each other: bugs in low-safety-level code should not effect high-safety-level data, and high-safety-level code should not depend on low-safety-level code. So, low-safety-level code is not allowed to write in high-safety-level data sections, and high-safety-level code is not allowed to call low-safety-level functions.

The partitioning of the application is done in LSL definitions. You can define different named safety groups by selecting either sections by name or by address ranges. The linker reads and interprets the LSL file and uses the locate result of the application to generate information to set up the Memory Protection Unit (MPU) for this specific application. For each TriCore AURIX architecture the layout of the MPU is defined in LSL (see [Section 17.4.5, Defining the MPU Layout](#)).

Safety class areas

In an LSL file you can define safety class areas with the LSL keyword `safety_class` in a group statement.

For example (`safety.lsl`):

```
#define SC_ASIL_A 3
#define SC_ASIL_D 7

section_layout :vtc:linear
{
    group ASIL_D (safety_class = SC_ASIL_D)
    {
        select ".text.safety.f*";
        // select f1(), f2(), f3()
    }
}
section_layout :vtc:abs18
{
    group ASIL_A (safety_class = SC_ASIL_A)
    {
        select ".zbss.safety.*";
        //select x, y, z
    }
}
```

```
    }
}
```

The group names will appear in the linker map file. See also [LSL keyword `safety\_class`](#).

Safety class access rights

In the `section_setup` part of an LSL file you can define the safety class access rights between safety class areas with the `section_reference_restriction` statement.

For example (`safety.lsl`):

```
section_setup :vtc:linear
{
    section_reference_restriction
    {
        safety_class = SC_ASIL_D;
        target_safety_class = SC_ASIL_A;
        attributes = r;
        // safety class '7' only has read access in safety class '3'
    }
}
```

See also [LSL keyword `section\_reference\_restriction`](#).

Generating MPU configuration data

With [linker option `--mpu-configuration-data`](#) the linker generates a section containing the MPU configuration data.

MPU configuration data location

A **`mpu_data_table`** statement in a `section_setup` definition specifies in which address space the linker should generate MPU configuration data. In a `section_layout` you can locate the MPU configuration data table.

For example,

```
section_setup ::linear
{
    mpu_data_table;
}

section_layout ::linear
{
    group select "mpu_data_table";
}
```

MPU configuration data table format

The section containing the MPU configuration data is organized as a table consisting of zero or more entries followed by a terminator.

The section start with two integer numbers that represent the number of data ranges and the number of code ranges, followed by the specified number of data ranges, as well as the specified number of code ranges, where a range consists of a lower bound address and an upper bound address.

Each safety class that is defined in LSL has a unique entry in the table. For the 'default' safety class (with safety class ID equal to zero) no entry in the table is emitted. You have to make sure to assign a safety class to all code in your application.

Each safety class entry in the table consists of the safety class ID and access permissions for code and data in the safety class. The access permissions are represented as three integer numbers that are interpreted as bit masks, for read access, write access, and execute access. If bit n is set in the read access mask, read access to data range n is allowed. If bit n is set in the write access mask, write access to data range n is allowed. If bit m is set in the execute access mask, execute access to code range m is allowed. The fields and the order they appear in within an entry are shown in the tables below.

Safety classes are identified by a positive integer number, or zero for the 'default' safety class. Therefore, safety class ID '-1' is used as terminator in the table.

MPU table:

Field	Description
Nr of data ranges	The number of data ranges specified in this table.
Nr of code ranges	The number of code ranges specified in this table.
Data range registers	The (values for the) data range registers, for each data range. A data range can be identified by its index in this list: $n = 0 \dots \text{Nr of data ranges} - 1$. Refer to the table 'Range registers' below.
Code range registers	The (values for the) code range registers, for each code range. A code range can be identified by its index in this list: $m = 0 \dots \text{Nr of code ranges} - 1$. Refer to the table 'Range registers' below.
Permissions	See the table 'MPU permissions entry' below.

MPU permissions entry:

Field	Description
Safety class ID	A positive integer number identifying the safety class.
Read permissions	A number that is interpreted as a bit mask, where each bit n enables read access to data range n .
Write permissions	A number that is interpreted as a bit mask, where each bit n enables write access to data range n .
Execute permissions	A number that is interpreted as a bit mask, where each bit m enables execute access to code range m .

Range registers:

Field	Description
Range lower bound	The lower boundary address of the memory range. The lower bound is inclusive.
Range upper bound	The upper boundary address of the memory range. The upper bound is exclusive.

7.9.14. Boot Mode Headers

The linker and LSL has support for the AURIX Boot Mode Headers BMHD0 .. BMHD3. For detailed information, for example for the TC27x, about Boot Mode Headers, Startup Software (SSW) and Bootstrap loaders refer to *chapter 5 TC27x BootROM Content* in the *AURIX TC27x User's Manual*.

Altium delivers AURIX LSL files, such as `tc27x.lsl`, in the `include.lsl` directory of the product. Support for the boot mode headers is present in two separate LSL files, `tc1v1_6_x.bmhd.lsl` and `tc1v1_6_2.bmhd.lsl`, which is included in the TriCore derivative LSL files.

Note that these system LSL files are not to be changed. Since v6.2r1 the boot mode header LSL filenames have changed (`bmhd.lsl` was changed into `tc1v1_6_x.bmhd.lsl` and `tc1v1_6_2.bmhd.lsl` was added). If you have a project from a previous version, make sure to use the latest *derivative.lsl* file from the `include.lsl` directory.

Boot Mode Header configurations

A Boot Mode Header (BMHD) is defined with a `struct` statement in `tc1v1_6_x.bmhd.lsl` and `tc1v1_6_2.bmhd.lsl`. Part of the BMHD structure is the Boot Mode Index (BMI), which holds information, for example, about the start-up mode of the device.

You can configure each of the Boot Mode Headers BMHD0 .. BMHD3. The system LSL files `tc1v1_6_x.bmhd.lsl` and `tc1v1_6_2.bmhd.lsl` contain macros for the configuration. These system LSL files are not to be changed. However, you can use macros in your project LSL file or you can use Eclipse to do this for you. When you make changes in Eclipse, macros are added to the project's LSL file. The following configurations are possible for each Boot Mode Header:

Boot Mode Header configuration	Description
Reserved Boot Mode Header	Only the memory areas for the Boot Mode Headers are reserved. This means it will not be flashed. This does not result in any settings in the project's LSL files, because this is the default in <code>tc1v1_6_x.bmhd.lsl</code> and <code>tc1v1_6_2.bmhd.lsl</code> .
Generate Boot Mode Header	The Boot Mode Header is generated with the values defined in Eclipse.
Invalidate Boot Mode Header	The Boot Mode Header is initialized with zeros (invalid data). This ensures that the processor will not use this Boot Mode Header.

Boot Mode Header configuration	Description
Disable Boot Mode Header	The Boot Mode Header is not generated and no memory area is reserved. This setting is not supported in Eclipse. This is for users who want to define everything themselves. To do this, set the macro <code>__BMHDx_CONFIG</code> to <code>__BMHD_DISABLE</code> in the project's LSL file.

Note that you cannot set all Boot Mode Headers to "invalidate". The linker generates an error because this combination might brick the device.

To configure Boot Mode headers using macros in LSL

If you do not use Eclipse, you can configure the Boot Mode Headers in your project LSL file. The following is an example of a configuration in project LSL file `myproject.lsl`.

```
/** start of myproject.lsl
/** define macros for Boot Mode Headers
#define __BMHD0_CONFIG      __BMHD_GENERATE
#define __BMHD0_STADABM    _START
#define __BMHD0_BMI        __BMHD_BMI_PINDIS|__BMHD_BMI_HWCFG_ABM
#define __BMHD0_CHKSTART   addressof(mem:flash)
#define __BMHD0_CHKEND     addressof(mem:flash) + sizeof(mem:flash)

/** include the TC27x system LSL file
#include "tc27x.lsl"
```

This example tells to generate Boot Mode Header BMHD0, use the Alternate Boot Mode (ABM) as start-up mode for the device and calculate a checksum over the specified flash memory range.

If the linker LSL macro `__BMHD_DISABLE_ALL__` is defined, boot mode header generation is disabled.

You can use the following macros to control the Boot Mode Headers. The x in the macro name can be 0, 1, 2 or 3.

Macro	Default value	Description
__BMHDx_CONFIG	__BMHD_RESERVE	The Boot Mode Header Configuration. This macro must be defined with one of the following values: __BMHD_RESERVE Only reserve memory areas for the Boot Mode Header. __BMHD_GENERATE Generate the Boot Mode Header with the values defined with the macros in this table. __BMHD_INVALIDATE Initialize the Boot Mode Header with zeros (invalid data). __BMHD_DISABLE Disable the Boot Mode Header generation and reservation.
__BMHDx_STADABM	__START	Defines the user code start address.
__BMHDx_BMI	__BMHD_BMI_HWCFG_INTERNAL	Defines the BMI value, the default is the factory settings. The various fields are described with macros in the table below.
__BMHDx_CHKSTART	"__START"	Defines the start address of the memory range to calculate the checksum of. TriCore 1.6.x only.
__BMHDx_CHKEND	"__START" + 4	Defines the end address of the memory range to calculate the checksum of. The specified address is the first address after the last byte that is part of the checksum. TriCore 1.6.x only.

Note that for the checksum memory range start address and end address you can specify labels, such as "__START", absolute addresses, [LSL built-in functions](#) or [linker labels](#).

The following macros are available for composing the value of the BMI field. These macros are the same for each Boot Mode Header.

TriCore 1.6.x Boot Mode Index values

BMI field macro	Value	Description
__BMHD_BMI_PINDIS	0x0008	Disable mode selection by HWCfg pins.
__BMHD_BMI_HWCfg_INTERNAL	0x0070	Hardware start-up mode: internal start from flash
__BMHD_BMI_HWCfg_ABM	0x0060	Hardware start-up mode: Alternate Boot Mode (ABM)
__BMHD_BMI_HWCfg_GENERIC_BSL	0x0040	Hardware start-up mode: Generic Bootstrap Loader
__BMHD_BMI_HWCfg_ASC_BSL	0x0030	Hardware start-up mode: ASC Bootstrap Loader

BMI field macro	Value	Description
__BMHD_BMI_LCL0LSEN	0x0100	Lockstep for CPU 0
__BMHD_BMI_LCL1LSEN	0x0200	Lockstep for CPU 1

TriCore 1.6.2 Boot Mode Index values

BMI field macro	Value	Description
__BMHD_BMI_PINDIS	0x0001	Disable mode selection by HWCFG pins.
__BMHD_BMI_HWCFG_INTERNAL	0x000E	Hardware start-up mode: internal start from flash
__BMHD_BMI_HWCFG_ABM	0x000C	Hardware start-up mode: Alternate Boot Mode (ABM)
__BMHD_BMI_HWCFG_GENERIC_BSL	0x0008	Hardware start-up mode: Generic Bootstrap Loader
__BMHD_BMI_HWCFG_ASC_BSL	0x0006	Hardware start-up mode: ASC Bootstrap Loader
__BMHD_BMI_LSENA0	0x0010	Lockstep for CPU 0
__BMHD_BMI_LSENA1	0x0020	Lockstep for CPU 1
__BMHD_BMI_LSENA2	0x0040	Lockstep for CPU 2
__BMHD_BMI_LSENA3	0x0080	Lockstep for CPU 3
__BMHD_BMI_LBISTENA	0x0100	LBIST execution start by Startup Software (SSW)

Additionally, the following macro is defined to control the Boot Mode Header alignment:

```
__BMHD_ALIGN( address, alignment )
```

Both the *address* and *alignment* arguments are expressions. Note that the expressions simply evaluate to a number. So, the address and alignment are also (integer) numbers. In practice those numbers always represent MAUs, though. When the *alignment* argument is a positive value the result will be the first aligned address higher than or equal to the specified *address* argument. When the *alignment* argument is a negative value the result will be the first aligned address lower than or equal to the specified *address* argument.

To configure Boot Mode Headers using Eclipse

1. From the **Project** menu, select **Properties for**

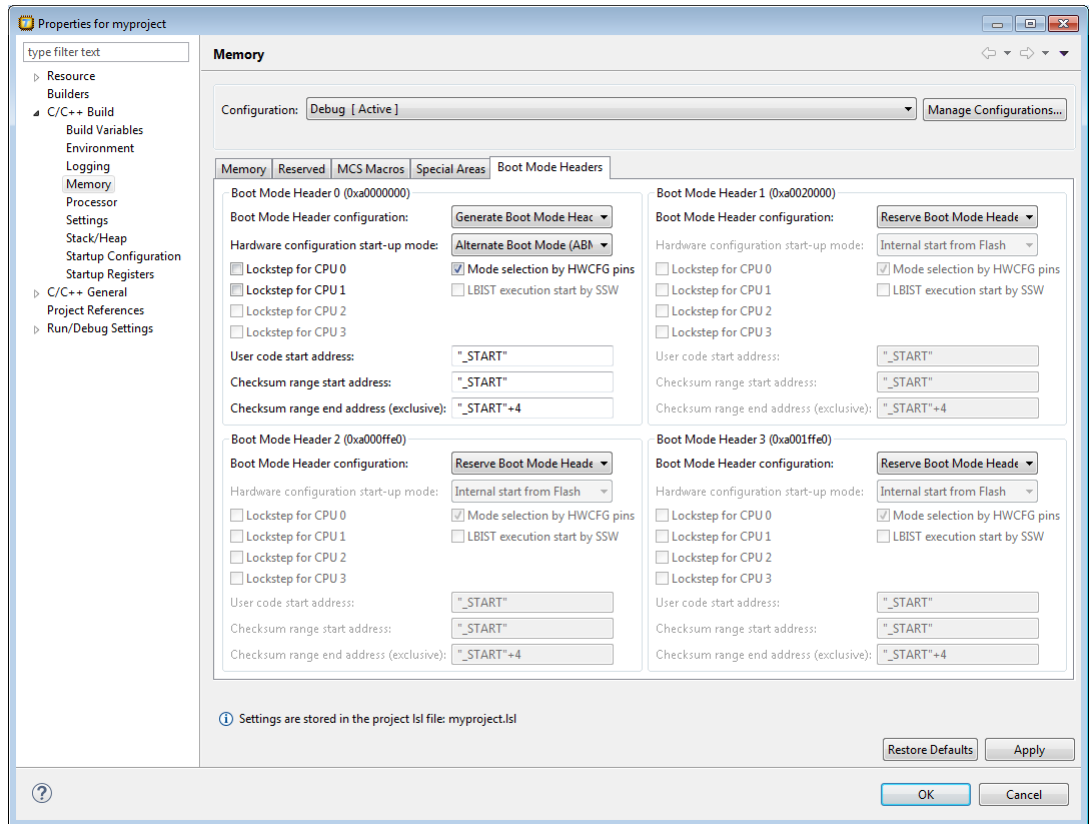
The Properties dialog appears.

2. In the left pane, expand **C/C++ Build** and select **Memory**.

In the right pane the Memory page appears.

3. Open the **Boot Mode Headers** tab.

Four group boxes are present, one for each Boot Mode Header.



4. Select a **Boot Mode Header configuration**.

When you select **Generate Boot Mode Header**, you can optionally specify additional information as specified below.

5. (Optionally) Specify one or more of the following options:

- **Hardware configuration start-up mode**

Here you can select the start-up mode configuration by the HWCFG bits of the BMI. Select one of:

- Internal start from Flash
- Alternate Boot Mode (ABM)
- Generic Bootstrap Loader
- ASC Bootstrap Loader

- **Mode selection by HWCFG pins**

When you disable this option, the Mode selection by HWCFG pins is disabled. This corresponds to setting the PINDIS field of the BMI to 1. This option is enabled by default (PINDIS=0).

- **Lockstep for CPU 0 / 1 / 2 / 3**

This is the Lockstep Comparator Logic control for CPU 0, CPU 1 (and/or CPU 2, CPU 3 for TriCore 1.6.2). These options correspond to the LCL0LSEN and LCL1LSEN fields of the BMI for TriCore 1.6.x or the LSENA0, LSENA1, LSENA2 or LSENA3 fields of the BMI for TriCore 1.6.2. By default these options are disabled.

- **LBIST execution start by SSW** (TriCore 1.6.2 only)

When this option is enabled the Logic Built-in-Self-Test execution is started by Startup Software (SSW). This option corresponds to the LBISTENA field of the BMI for TriCore 1.6.2. By default this option is disabled.

6. For the **Alternate Boot Mode (ABM)** start-up mode, specify the following information:

- **User code start address**

Enter the start address of your application. This field corresponds to the start address alternate boot mode field (STADABM) of the Boot Mode Header (BMHD) structure.

- **Checksum range start address / end address (exclusive)** (TriCore 1.6.x only)

Specify the memory address range that needs to be checked. A crc32 checksum is calculated over the specified address range. Note that in the end address you specify here is not part of the range. These fields correspond to the ChkStart and ChkEnd + 4 fields of the Boot Mode Header (BMHD) structure.

Example

You can specify labels, such as "_START", absolute addresses, [LSL built-in functions](#) or [linker labels](#). For example to use the start and end address of a group named crc32_bmhd0, you can specify:

Checksum range start address: "_lc_gb_crc32_bmhd0"

Checksum range end address (exclusive): "_lc_ge_crc32_bmhd0"

or:

Checksum range start address: `addressof(group:crc32_bmhd0)`

Checksum range end address (exclusive): `addressof(group:crc32_bmhd0)+sizeof(group:crc32_bmhd0)`

7. Click **OK** to close the Properties dialog.

The updated settings are stored in the project LSL file.

7.9.15. Booting AURIX SCR

The SCR in AURIX and AURIX 2G devices does not boot unless the XRAM memory contains a specific 8-byte sequence at the very end of the memory. The LSL files for AURIX and AURIX 2G devices that include an SCR module, specify a section with a size of 8 bytes with the name `scr_boot_magic` in a link task for the xc800 core. This section contains the code `0x55 0xAA 0x55 0xAA 0x55 0xAA 0x55 0xAA` and will be located at xdata address `0x1ff8`.

If you already have this 8-byte sequence in your own code, you can disable the generation of this SCR boot magic section by defining the linker macro `__DISABLE_SCR_BOOT_MAGIC`.

7.10. Linker Labels

The linker creates labels that you can use to refer to from within the application software. Some of these labels are real labels at the beginning or the end of a section. Other labels have a second function, these labels are used to address generated data in the locating phase. The data is only generated if the label is used.

Linker labels are labels starting with `_lc_`. The linker assigns addresses to the following labels when they are referenced:

Label	Description
<code>_lc_ub_name</code> <code>_lc_b_name</code>	Begin of section <i>name</i> . Also used to mark the lowest address of the stack or heap or copy table.
<code>_lc_ue_name</code> <code>_lc_e_name</code>	End of section <i>name</i> . Also used to mark the highest address of the stack or heap.
<code>_lc_cb_name</code>	Start address of an overlay section in ROM.
<code>_lc_ce_name</code>	End address of an overlay section in ROM.
<code>_lc_gb_name</code>	Begin of group <i>name</i> . This label appears in the output file even if no reference to the label exists in the input file.
<code>_lc_ge_name</code>	End of group <i>name</i> . This label appears in the output file even if no reference to the label exists in the input file.
<code>_lc_s_name</code>	Variable <i>name</i> is mapped through memory in shared memory situations.
<code>_lc_t_core_name</code>	Variable or linker label <i>name</i> in the specified <i>core</i> is mapped to the address space of the referred section. This way you can refer to a variable or linker label on a specific core on the same processor.

The linker only allocates space for the stack and/or heap when a reference to either of the section labels exists in one of the input object files.

Additionally, the linker script file defines the following symbols:

Symbol	Description
<code>_lc_cp</code>	Start of copy table. Same as <code>_lc_ub_table</code> . The copy table gives the source and destination addresses of sections to be copied. This table will be generated by the linker only if this label is used.
<code>_lc_bh</code>	Begin of heap. Same as <code>_lc_ub_heap</code> .
<code>_lc_eh</code>	End of heap. Same as <code>_lc_ue_heap</code> .

If you want to use linker labels in your C source for sections that have a dot (.) in the name, you have to replace all dots by underscores.

Example: refer to a label with section name with dots from C

Suppose the C source file `foo.c` contains the following:

```
int myfunc(int a)
{
    /* some source lines */
    return 1;
}
```

This results in a section with the name `.text.foo.myfunc`.

In the following source file `main.c` all dots of the section name are replaced by underscores:

```
#include <stdio.h>
extern char _lc_ub__text_foo_myfunc[];

void main(void)
{
    printf("The function myfunc is located at %p\n",
        _lc_ub__text_foo_myfunc);
}
```

Example: refer to a PCP variable from TriCore C source

When memory is shared between two or more cores, for instance TriCore and PCP, the addresses of variables (or functions) on that memory may be different for the cores. For the TriCore the variable will be defined and you can access it in the usual way. For the PCP, when you would use the variable directly in your TriCore source, this would use an incorrect address (PCP address). The linker can map the address of the variable from one space to another, if you prefix the variable name with `_lc_s_`.

When a symbol `foo` is defined in a PCP assembly source file, by default it gets the symbol name `foo`. To use this symbol from a TriCore C source file, write:

```
extern long _lc_s_foo;

void main(int argc, char **argv)
{
```

```

    _lc_s_foo = 7;
}

```

Example: refer to an MCS variable or linker label from TriCore C source

From within the TriCore source you can access MCS variables. The same symbol name can be defined in different MCS cores. To uniquely select a variable from a core, you prefix the variable name with `_lc_t_core_`. When the linker sees the `_lc_t_`, it removes the linker label prefix, and the core name prefix. The remainder is a symbol name, that has to be found inside the *core*.

For example, when a symbol count is defined in assembly sources of two different MCS cores, you can access them from a TriCore C source file as follows:

```

extern int _lc_t_mcs00_count; /* variable count in mcs00 */
extern int _lc_t_mcs01_count; /* variable count in mcs01 */

void main(int argc, char **argv)
{
    _lc_t_mcs00_count = 0;
    _lc_t_mcs01_count = 1;
}

```

Note that you can also refer to MCS linker labels from the TriCore C source. For example, to refer to the beginning of a group with the name `MY_MCS_CODE`, you can use:

```

/* linker label _lc_gb_MY_MCS_CODE in mcs00 */
extern char _lc_t_mcs00__lc_gb_MY_MCS_CODE[];

```

Example: refer to the stack

Suppose in an LSL file a stack section is defined with the name "ustack" (with the keyword `stack`). You can refer to the begin and end of the stack from your C source as follows:

```

#include <stdio.h>
extern char _lc_ub_ustack[];
extern char _lc_ue_ustack[];
void main()
{
    printf( "Size of stack is %d\n",
           _lc_ue_ustack - _lc_ub_ustack );
}

```

From assembly you can refer to the end of the stack with:

```

.extern _lc_ue_ustack    ; end of user stack

```

See [Section 1.11.1, Calling Convention](#) and [section 2.2.2 Stack Frame Management](#) in the TriCore EABI for more information about the stack.

7.11. Generating a Map File

The map file is an additional output file that contains information about the location of sections and symbols. You can customize the type of information that should be included in the map file.

To generate a map file

1. From the **Project** menu, select **Properties for**
The Properties dialog appears.
2. In the left pane, expand **C/C++ Build** and select **Settings**.
In the right pane the Settings appear.
3. On the Tool Settings tab, select **Linker » Map File**.
4. Enable the option **Generate XML map file format (.mapxml) for map file viewer**.
5. (Optional) Enable the option **Generate map file (.map)**.
6. (Optional) Enable the options to include that information in the map file.

Example on the command line

The following command generates the map file `test.map`:

```
ltc --map-file test.o
```

With this command the map file `test.map` is created.

See [Section 15.2, Linker Map File Format](#), for an explanation of the format of the map file.

7.12. Linker Error Messages

The linker reports the following types of error messages in the Problems view of Eclipse.

F (Fatal errors)

After a fatal error the linker immediately aborts the link/locate process.

E (Errors)

Errors are reported, but the linker continues linking and locating. No output files are produced unless you have set the [linker option --keep-output-files](#).

W (Warnings)

Warning messages do not result into an erroneous output file. They are meant to draw your attention to assumptions of the linker for a situation which may not be correct. You can control warnings in the **C/C++ Build » Settings » Tool Settings » Linker » Diagnostics** page of the **Project » Properties for** menu (linker option `--no-warnings`).

I (Information)

Verbose information messages do not indicate an error but tell something about a process or the state of the linker. To see verbose information, use the linker option `--verbose`.

S (System errors)

System errors occur when internal consistency checks fail and should never occur. When you still receive the system error message

`S6##: message`

please report the error number and as many details as possible about the context in which the error occurred.

Display detailed information on diagnostics

1. From the **Window** menu, select **Show View » Other » TASKING » Problems**.

The Problems view is added to the current perspective.

2. In the Problems view right-click on a message.

A popup menu appears.

3. Select **Detailed Diagnostics Info**.

A dialog box appears with additional information.

On the command line you can use the linker option `--diag` to see an explanation of a diagnostic message:

```
ltc --diag=[format:]{all | number, ...}
```


Chapter 17. Linker Script Language (LSL)

To make full use of the linker, you can write a script with information about the architecture of the target processor and locating information. The language for the script is called the *Linker Script Language (LSL)*. This chapter first describes the structure of an LSL file. The next section contains a summary of the LSL syntax. In the remaining sections, the semantics of the Linker Script Language is explained.

The TASKING linker is a target independent linker/locator that can simultaneously link and locate all programs for all cores available on a target board. The target board may be of arbitrary complexity. A simple target board may contain one standard processor with some external memory that executes one task. A complex target board may contain multiple standard processors and DSPs combined with configurable IP-cores loaded in an FPGA. Each core may execute a different program, and external memory may be shared by multiple cores.

LSL serves two purposes. First it enables you to specify the characteristics (that are of interest to the linker) of your specific target board and of the cores installed on the board. Second it enables you to specify how sections should be located in memory.

17.1. Structure of a Linker Script File

A script file consists of several definitions. The definitions can appear in any order.

The architecture definition (required)

In essence an *architecture definition* describes how the linker should convert logical addresses into physical addresses for a given type of core. If the core supports multiple address spaces, then for each space the linker must know how to perform this conversion. In this context a physical address is an offset on a given internal or external bus. Additionally the architecture definition contains information about items such as the (hardware) stack and the interrupt vector table.

This specification is normally written by Altium. Altium supplies LSL files in the `include.lsl` directory. The architecture definition of the LSL file should not be changed by you unless you also modify the core's hardware architecture. If the LSL file describes a multi-core system an architecture definition must be available for each different type of core.

See [Section 17.4, *Semantics of the Architecture Definition*](#) for detailed descriptions of LSL in the architecture definition.

The derivative definition

The *derivative definition* describes the configuration of the internal (on-chip) bus and memory system. Basically it tells the linker how to convert offsets on the buses specified in the architecture definition into offsets in internal memory. Microcontrollers and DSPs often have internal memory and I/O sub-systems apart from one or more cores. The design of such a chip is called a *derivative*.

Altium provides LSL descriptions of supported derivatives, along with "SFR files", which provide easy access to registers in I/O sub-systems from C and assembly programs. When you build an ASIC or use a derivative that is not (yet) supported by the TASKING tools, you may have to write a derivative definition.

When you want to use multiple cores of the same type, you must instantiate the cores in a derivative definition, since the linker automatically instantiates only a single core for an unused architecture.

See [Section 17.5, *Semantics of the Derivative Definition*](#) for a detailed description of LSL in the derivative definition.

The processor definition

The *processor definition* describes an instance of a derivative. Typically the processor definition instantiates one derivative only (single-core processor). A processor that contains multiple cores having the same (homogeneous) or different (heterogeneous) architecture can also be described by instantiating multiple derivatives of the same or different types in separate processor definitions.

See [Section 17.6, *Semantics of the Board Specification*](#) for a detailed description of LSL in the processor definition.

The memory and bus definitions (optional)

Memory and bus definitions are used within the context of a derivative definition to specify internal memory and on-chip buses. In the context of a board specification the memory and bus definitions are used to define external (off-chip) memory and buses. Given the above definitions the linker can convert a logical address into an offset into an on-chip or off-chip memory device.

See [Section 17.6.3, *Defining External Memory and Buses*](#), for more information on how to specify the external physical memory layout. *Internal* memory for a processor should be defined in the derivative definition for that processor.

The board specification

The processor definition and memory and bus definitions together form a *board specification*. LSL provides language constructs to easily describe single-core and heterogeneous or homogeneous multi-core systems. The board specification describes all characteristics of your target board's system buses, memory devices, I/O sub-systems, and cores that are of interest to the linker. Based on the information provided in the board specification the linker can for each core:

- convert a logical address to an offset within a memory device
- locate sections in physical memory
- maintain an overall view of the used and free physical memory within the whole system while locating

The section layout definition (optional)

The optional section layout definition enables you to exactly control where input sections are located. Features are provided such as: the ability to place sections at a given load-address or run-time address, to place sections in a given order, and to overlay code and/or data sections.

Which object files (sections) constitute the task that will run on a given core is specified on the command line when you invoke the linker. The linker will link and locate all sections of all tasks simultaneously. From the section layout definition the linker can deduce where a given section may be located in memory,

from the board specification the linker can deduce which physical memory is (still) available while locating the section.

See [Section 17.8, *Semantics of the Section Layout Definition*](#), for more information on how to locate a section at a specific place in memory.

Skeleton of a Linker Script File

```
architecture architecture_name
{
    // Specification core architecture
}

derivative derivative_name
{
    // Derivative definition
}

processor processor_name
{
    // Processor definition
}

memory and/or bus definitions

section_layout space_name
{
    // section placement statements
}
```

17.2. Syntax of the Linker Script Language

This section describes what the LSL language looks like. An LSL document is stored as a file coded in UTF-8 with extension `.lsl`. Before processing an LSL file, the linker preprocesses it using a standard C preprocessor. Following this, the linker interprets the LSL file using a scanner and parser. Finally, the linker uses the information found in the LSL file to guide the locating process.

17.2.1. Preprocessing

When the linker loads an LSL file, the linker first processes it with a C-style preprocessor. As such, it strips C and C++ comments. Lines starting with the `#` character are taken as commands for the preprocessor. You can use the standard ISO C99 preprocessor directives, including:

```
#include "file"
#include <file>
```

Preprocess and include file *file* at this point in the LSL file.

For example:

```
#include "arch.lsl"
```

Preprocess and include the file `arch.lsl` at this point in the LSL file.

```
#if condition
#else
#endif
```

If the *condition* evaluates to a non-zero value, copy the following lines, up to an `#else` or `#endif` command, skip lines between `#else` and `#endif`, if present. If the condition evaluates to zero, skip the lines up to the `#else` command, or `#endif` if no `#else` is present, and copy the lines between the `#else` and `#endif` commands.

```
#ifdef identifier
#else
#endif
```

Same as `#if`, but with `defined(identifier)` as condition.

```
#error text
```

Causes a fatal error the given message (optional).

17.2.2. Lexical Syntax

The following lexicon is used to describe the syntax of the Linker Script Language:

$A ::= B$	=	A is defined as B
$A ::= B C$	=	A is defined as B and C ; B is followed by C
$A ::= B \mid C$	=	A is defined as B or C
$\langle B \rangle^{0 1}$	=	zero or one occurrence of B
$\langle B \rangle^{>=0}$	=	zero or more occurrences of B
$\langle B \rangle^{>=1}$	=	one or more occurrences of B
<i>IDENTIFIER</i>	=	a character sequence starting with 'a'-'z', 'A'-'Z' or '_'. Following characters may also be digits and dots '.'
<i>STRING</i>	=	sequence of characters not starting with <code>\n</code> , <code>\r</code> or <code>\t</code>
<i>DQSTRING</i>	=	" <i>STRING</i> " (double quoted string)
<i>OCT_NUM</i>	=	octal number, starting with a zero (06, 045)
<i>DEC_NUM</i>	=	decimal number, not starting with a zero (14, 1024)

HEX_NUM = hexadecimal number, starting with '0x' (0x0023, 0xFF00)

OCT_NUM, *DEC_NUM* and *HEX_NUM* can be followed by a **k** (kilo), **M** (mega), or **G** (giga).

Characters in **bold** are characters that occur literally. Words in *italics* are higher order terms that are defined in the same or in one of the other sections.

To write comments in LSL file, you can use the C style `/* */` or C++ style `/**/`.

17.2.3. Identifiers and Tags

```

arch_name      ::= IDENTIFIER
bus_name      ::= IDENTIFIER
core_name     ::= IDENTIFIER
derivative_name ::= IDENTIFIER
file_name     ::= DQSTRING
group_name    ::= IDENTIFIER
heap_name     ::= section_name
map_name      ::= IDENTIFIER
mem_name      ::= IDENTIFIER
proc_name     ::= IDENTIFIER
section_name  ::= DQSTRING
space_name    ::= IDENTIFIER
stack_name    ::= section_name
symbol_name   ::= DQSTRING
tag_attr      ::= (tag<,tag>=>0)
tag           ::= tag = DQSTRING

```

A tag is an arbitrary text that can be added to a statement.

17.2.4. Expressions

The expressions and operators in this section work the same as in ISO C.

```

number        ::= OCT_NUM
                  | DEC_NUM
                  | HEX_NUM

expr          ::= number
                  | symbol_name
                  | unary_op expr
                  | expr binary_op expr
                  | expr ? expr : expr
                  | ( expr )
                  | function_call

unary_op      ::= !      // logical NOT
                  | ~      // bitwise complement
                  | -      // negative value

```

```

binary_op      ::= ^      // exclusive OR
                | *      // multiplication
                | /      // division
                | %      // modulus
                | +      // addition
                | -      // subtraction
                | >>     // right shift
                | <<     // left shift
                | ==     // equal to
                | !=     // not equal to
                | >      // greater than
                | <      // less than
                | >=     // greater than or equal to
                | <=     // less than or equal to
                | &      // bitwise AND
                | |      // bitwise OR
                | &&     // logical AND
                | ||     // logical OR

```

17.2.5. Built-in Functions

```

function_call  ::= absolute ( expr )
                | addressof ( addr_id )
                | checksum ( checksum_algo , expr , expr )
                | exists ( section_name )
                | max ( expr , expr )
                | min ( expr , expr )
                | sizeof ( size_id )

addr_id        ::= sect : section_name
                | group : group_name
                | mem : mem_name

checksum_algo  ::= crc32w

size_id        ::= sect : section_name
                | group : group_name
                | mem : mem_name

```

- Every space, bus, memory, section or group you refer to, must be defined in the LSL file.
- The addressof() and sizeof() functions with the **group** or **sect** argument can only be used in the right hand side of an assignment. The sizeof() function with the **mem** argument can be used anywhere in section layouts.
- The checksum() function can only be used in a **struct** statement.

You can use the following built-in functions in expressions. All functions return a numerical value. This value is a 64-bit signed integer.

absolute()

```
int absolute( expr )
```

Converts the value of *expr* to a positive integer.

```
absolute( "labelA" - "labelB" )
```

addressof()

```
int addressof( addr_id )
```

Returns the offset of *addr_id*, which is a named section, group, or memory in the address space of the section layout. If the referenced object is a group or memory, it must be defined in the LSL file. To get the offset of the section with the name *asect*:

```
addressof( sect: "asect" )
```

This function only works in assignments and **struct** statements.

checksum()

```
int checksum( checksum_algo, expr, expr )
```

Returns the computed checksum over a contiguous address range. The first argument specifies how the checksum must be computed (see below), the second argument is an expression that represents the start address of the range, while the third argument represents the end address (exclusive). The value of the end address expression must be strictly larger than the value of the start address (i.e. the size of the checksum address range must be at least one MAU). Each address in the range must point to a valid memory location. Memory locations in the address range that are not occupied by a section are filled with zeros.

The only checksum algorithm (*checksum_algo*) currently supported is **crc32w**. This algorithm computes the checksum using a Cyclic Redundancy Check with the "CRC-32" polynomial 0xEDB88320. The input range is processed per 4-byte word. Those 4 bytes are passed to the checksum algorithm in reverse order if the target architecture is little-endian. For big-endian targets, this checksum algorithm is equal to a regular byte-wise CRC-32 implementation. Both the start address and end address values must be aligned on 4 MAUs. The behavior of this checksum algorithm is undefined when used in an address space that has a MAU size not equal to 8.

```
checksum( crc32w,  
    __BMHD_ALIGN( addressof( mem:foo ), -4 ),  
    __BMHD_ALIGN( addressof( mem:foo ) + sizeof( mem:foo ), 4 ) )
```

This function only works in **struct** statements.

exists()

```
int exists( section_name )
```

The function returns 1 if the section *section_name* exists in one or more object file, 0 otherwise. If the section is not present in input object files, but generated from LSL, the result of this function is undefined.

To check whether the section *mysection* exists in one of the object files that is specified to the linker:

```
exists( "mysection" )
```

max()

```
int max( expr, expr )
```

Returns the value of the expression that has the largest value. To get the highest value of two symbols:

```
max( "sym1" , "sym2" )
```

min()

```
int min( expr, expr )
```

Returns the value of the expression that has the smallest value. To get the lowest value of two symbols:

```
min( "sym1" , "sym2" )
```

sizeof()

```
int sizeof( size_id )
```

Returns the size of the object (group, section or memory) the identifier refers to. To get the size of the section "asection":

```
sizeof( sect: "asection" )
```

The **group** and **sect** arguments only work in assignments and **struct** statements. The **mem** argument can be used anywhere in section layouts. If the referenced object is a group or memory, it must be defined in the LSL file.

17.2.6. LSL Definitions in the Linker Script File

```
description      ::= <definition>*>=1
definition       ::= architecture_definition
                       | derivative_definition
                       | board_spec
                       | section_definition
                       | section_setup
```

- At least one *architecture_definition* must be present in the LSL file.

17.2.7. Memory and Bus Definitions

mem_def ::= **memory** *mem_name* <*tag_attr*>^{0|1} { <*mem_descr* ;>⁼⁰ }

- A *mem_def* defines a memory with the *mem_name* as a unique name.

mem_descr ::= **type** = <**reserved**>^{0|1} *mem_type*
 | **mau** = *expr*
 | **size** = *expr*
 | **speed** = *number*
 | **priority** = *number*
 | **exec_priority** = *number*
 | **fill** <= *fill_values*>^{0|1}
 | **write_unit** = *expr*
 | *mapping*

- A *mem_def* contains exactly one **type** statement.
- A *mem_def* contains exactly one **mau** statement (non-zero size).
- A *mem_def* contains exactly one **size** statement.
- A *mem_def* contains zero or one **priority** (or **speed**) statement (if absent, the default value is 1).
- A *mem_def* contains zero or one **exec_priority** statement.
- A *mem_def* contains zero or one **fill** statement.
- A *mem_def* contains zero or one **write_unit** statement.
- A *mem_def* contains at least one *mapping*

mem_type ::= **rom** // attrs = rx
 | **ram** // attrs = rw
 | **nvram** // attrs = rwx
 | **blockram**

fill_values ::= *expr*
 | [*expr* <, *expr*>⁼⁰]

bus_def ::= **bus** *bus_name* { <*bus_descr* ;>⁼⁰ }

- A *bus_def* statement defines a bus with the given *bus_name* as a unique name within a core architecture.

bus_descr ::= **mau** = *expr*
 | **width** = *expr* // bus width, nr
 | // of data bits
 | *mapping* // legal destination
 // 'bus' only

- The **mau** and **width** statements appear exactly once in a *bus_descr*. The default value for **width** is the **mau** size.
- The bus width must be an integer times the bus MAU size.
- The MAU size must be non-zero.
- A bus can only have a *mapping* on a destination bus (through **dest = bus:**).

mapping ::= **map** <map_name>^{0|1} (*map_descr* <, *map_descr*>^{>=0})

map_descr ::= **dest** = *destination*
 | **dest_dbits** = *range*
 | **dest_offset** = *expr*
 | **size** = *expr*
 | **src_dbits** = *range*
 | **src_offset** = *expr*
 | **reserved**
 | **cached**
 | **priority** = *number*
 | **exec_priority** = *number*
 | *tag*

- A *map_descr* requires at least the **size** and **dest** statements.
- A *map_descr* contains zero or one **cached** statement.
- A *map_descr* contains zero or one **priority** statement (if absent, the default value is 0).
- A *map_descr* contains zero or one **exec_priority** statement.
- Each *map_descr* can occur only once.
- You can define multiple mappings from a single source.
- Overlap between source ranges or destination ranges is not allowed.
- If the **src_dbits** or **dest_dbits** statement is not present, its value defaults to the **width** value if the source/destination is a bus, and to the **mau** size otherwise.
- The **reserved** statement is allowed only in mappings defined for a memory.

destination ::= **space** : *space_name*
 | **bus** : <*proc_name* |
 core_name :>^{0|1} *bus_name*

- A *space_name* refers to a defined address space.
- A *proc_name* refers to a defined processor.
- A *core_name* refers to a defined core.
- A *bus_name* refers to a defined bus.

- The following mappings are allowed (source to destination)
 - space => space
 - space => bus
 - bus => bus
 - memory => bus

range ::= *expr* .. *expr*

- With address ranges, the end address is not part of the range.

17.2.8. Architecture Definition

```
architecture_definition
    ::= architecture arch_name
       <( parameter_list )>0|1
       <extends arch_name
         <( argument_list )>0|1 >0|1
       { <arch_spec>>=0 }
```

- An *architecture_definition* defines a core architecture with the given *arch_name* as a unique name.
- At least one *space_def* and at least one *bus_def* have to be present in an *architecture_definition*.
- An *architecture_definition* that uses the **extends** construct defines an architecture that inherits all elements of the architecture defined by the second *arch_name*. The parent architecture must be defined in the LSL file as well.

parameter_list ::= *parameter* <, *parameter*>^{>=0}

parameter ::= IDENTIFIER <= *expr*>^{0|1}

argument_list ::= *expr* <, *expr*>^{>=0}

```
arch_spec
    ::= bus_def
       | space_def
       | endianness_def
       | mpu_layout_def
```

space_def ::= **space** *space_name* <*tag_attr*>^{0|1} { <*space_descr*>^{>=0} }

- A *space_def* defines an address space with the given *space_name* as a unique name within an architecture.

```
space_descr
    ::= space_property ;
       | section_definition //no space ref
       | reserved_range
```

```
space_property      ::= id = number // as used in object
                    | mau = expr
                    | align = expr
                    | page_size = expr <[ range ] <| [ range ]>=>0|1
                    | page
                    | direction = direction
                    | stack_def
                    | heap_def
                    | copy_table_def
                    | start_address
                    | mapping
```

- A *space_def* contains exactly one **id** and one **mau** statement.
- A *space_def* contains at most one **align** statement.
- A *space_def* contains at most one **page_size** statement.
- A *space_def* contains at least one *mapping*.

```
stack_def           ::= stack stack_name ( stack_descr
                                     <, stack_descr >=>0 )
```

- A *stack_def* defines a stack with the *stack_name* as a unique name.

```
stack_descr         ::= min_size = expr
                    | grows = direction
                    | align = expr
                    | fixed
                    | threads = expr
                    | entry_points = entry_point_list
                    | attributes
                    | tag
```

```
entry_point_list    ::= symbol_name
                    | [ symbol_name <, symbol_name >=>0 ]
```

- The **min_size** statement must be present.
- The **min_size** value must be 1 or greater.
- You can specify at most one **align** statement, one **grows** statement and one **threads** statement.
- Each stack definition can have one or more **entry_points** statements for stack estimation. The *symbol_name* corresponds to the caller name in the **.CALLS** directive as generated by the compiler.

```
heap_def            ::= heap heap_name ( heap_descr
                                     <, heap_descr >=>0 )
```

- A *heap_def* defines a heap with the *heap_name* as a unique name.

```

heap_descr      ::= min_size = expr
                  | grows = direction
                  | align = expr
                  | fixed
                  | attributes
                  | tag

```

- The **min_size** statement must be present.
- The **min_size** value must be 1 or greater.
- You can specify at most one **align** statement and one **grows** statement.

```

direction      ::= low_to_high
                  | high_to_low

```

- If you do not specify the **grows** statement, the stack and heap grow **low-to-high**.

```

copy_table_def ::= copytable <( copy_table_descr
                               <, copy_table_descr >>=0 )>0|1

```

- A *space_def* contains at most one **copytable** statement.
- Exactly one copy table must be defined in one of the spaces.

```

copy_table_descr ::= align = expr
                  | copy_unit = expr
                  | dest <space_name>0|1 = space_name
                  | page
                  | table { <subtable_descr; >>=0 }
                  | tag

```

```

subtable_descr  ::= symbol = symbol_name
                  | space = subtable_space_ref

```

```

subtable_space_ref ::= <processor_name>0|1 : <core_name>0|1 : space_name

```

- The **copy_unit** is defined by the size in MAUs in which the startup code moves data.
- The **dest** statement is only required when the startup code initializes memory used by another processor that has no access to ROM.
- A *space_name* refers to a defined address space.

```

start_addr      ::= start_address ( start_addr_descr
                                   <, start_addr_descr>>=0 )

```

```

start_addr_descr ::= run_addr = expr
                  | symbol = symbol_name

```

- A *symbol_name* refers to the section that contains the startup code.

```
reserved_range ::= reserved <tag_attr>0|1 expr .. expr ;
```

- The end address is not part of the range.

```
endianness_def ::= endianness { <endianness_type;>=1 }
```

```
endianness_type ::= big
                  | little
```

```
mpu_layout_def ::= mpu_layout { <mpu_layout_descr;>=1 }
```

```
mpu_layout_descr ::= register_sets = number
                  | data_ranges = number
                  | code_ranges = number
```

17.2.9. Derivative Definition

```
derivative_definition
    ::= derivative derivative_name
       <( parameter_list )>0|1
       <extends derivative_name
         <( argument_list )>0|1 >0|1
       { <derivative_spec>=0 }
```

- A *derivative_definition* defines a derivative with the given *derivative_name* as a unique name.

```
derivative_spec ::= core_def
                 | bus_def
                 | mem_def
                 | section_definition // no processor name
                 | section_setup
```

```
core_def ::= core core_name { <core_descr ;>=0 }
```

- A *core_def* defines a core with the given *core_name* as a unique name.
- At least one *core_def* must be present in a *derivative_definition*.

```
core_descr ::= architecture = arch_name
            <( argument_list )>0|1
            | copytable_space <core_name :>0|1 space_name
            | endianness = ( endianness_type
                           <, endianness_type>=0 )
            | import core_name
            | space_id_offset = number
```

- An *arch_name* refers to a defined core architecture.
- Exactly one **architecture** statement must be present in a *core_def*.

- Exactly one **copytable_space** statement must be present in a *core_def*, or in exactly one space in that core, a **copytable** statement must be present.

17.2.10. Processor Definition and Board Specification

```
board_spec      ::= proc_def
                  | bus_def
                  | mem_def

proc_def        ::= processor proc_name
                  { proc_descr ; }

proc_descr      ::= derivative = derivative_name
                  <( argument_list )>0|1
```

- A *proc_def* defines a processor with the *proc_name* as a unique name.
- If you do not explicitly define a processor for a derivative in an LSL file, the linker defines a processor with the same name as that derivative.
- A *derivative_name* refers to a defined derivative.
- A *proc_def* contains exactly one **derivative** statement.

17.2.11. Section Setup

```
section_setup   ::= section_setup space_ref <tag_attr>0|1
                  { <section_setup_item>>=0 }

section_setup_item
                ::= reserved_range
                  | stack_def ;
                  | heap_def ;
                  | copy_table_def ;
                  | mpu_data_table;
                  | start_address ;
                  | reference_space_restriction ;
                  | modify linktime_modification
                  | section_reference_restriction
```

- At most one **mpu_data_table** statement can be present in a **section_setup** for a link task.

```
reference_space_restriction
                ::= prohibit_references_to subtable_space_ref
                  <,subtable_space_ref>>=0

linktime_modification
                ::= input ( input_modifier <,input_modifier>>=0 )
                  { <select_section_statement ; >>=0 }
```

```
input_modifier      ::= space = subtable_space_ref
                       | attributes = < <+|-> attribute>*>0
                       | copy
```

- An *input_modifier* contains at most one **space** statement.
- An *input_modifier* contains at most one **attributes** statement.

```
section_reference_restriction
    ::= section_reference_restriction
       { <section_reference_restriction_item ; >*>0 }
```

```
section_reference_restriction_item
    ::= safety_class = number
       | target_safety_class = number
       | attributes = < <attribute>*>0 | - >
```

- Each *section_reference_restriction_item* can occur at most once in a **section_reference_restriction** statement.

17.2.12. Section Layout Definition

```
section_definition ::= section_layout <space_ref>*>0|1
                    <( space_layout_properties )>*>0|1
                    { <section_statement>*>0 }
```

- A section definition inside a space definition does not have a *space_ref*.
- All global section definitions have a *space_ref*.

```
space_ref           ::= <proc_name>*>0|1 : <core_name>*>0|1
                       : space_name <| space_name>*>0
```

- If more than one processor is present, the *proc_name* must be given for a global section layout.
- If the section layout refers to a processor that has more than one core, the *core_name* must be given in the *space_ref*.
- A *proc_name* refers to a defined processor.
- A *core_name* refers to a defined core.
- A *space_name* refers to a defined address space.

```
space_layout_properties
    ::= space_layout_property <, space_layout_property >*>0
```

```
space_layout_property
    ::= locate_direction
       | tag
```

```
locate_direction    ::= direction = direction
```

```
direction      ::= low_to_high
                | high_to_low
```

- A section layout contains at most one **direction** statement.
- If you do not specify the **direction** statement, the locate direction of the section layout is **low-to-high**.

```
section_statement
    ::= simple_section_statement ;
       | aggregate_section_statement
```

```
simple_section_statement
    ::= assignment
       | select_section_statement
       | special_section_statement
       | memcpy_statement
```

```
assignment     ::= symbol_name assign_op expr
```

```
assign_op      ::= =
                | :=
```

```
select_section_statement
    ::= select <ref_tree>0|1 <section_name>0|1
        <section_selections>0|1
```

- Either a *section_name* or at least one *section_selection* must be defined.

```
section_selections
    ::= ( section_selection
        <, section_selection>>=0 )
```

```
section_selection
    ::= attributes = < <+|-> attribute>>0
       | tag
```

- **+attribute** means: select all sections that have this attribute.
- **-attribute** means: select all sections that do not have this attribute.

```
special_section_statement
    ::= heap heap_name <stack_heap_mods>0|1
       | stack stack_name <stack_heap_mods>0|1
       | copytable
       | reserved section_name <reserved_specs>0|1
```

- Special sections cannot be selected in load-time groups.

```
stack_heap_mods ::= ( stack_heap_mod <, stack_heap_mod>>=0 )
```

```
stack_heap_mod  ::= size = expr
                | tag
```

```
reserved_specs ::= ( reserved_spec <, reserved_spec>>=0 )
```

```
reserved_spec ::= attributes
                  | fill_spec
                  | size = expr
                  | alloc_allowed = absolute | ranged
```

- If a **reserved** section has attributes **r**, **rw**, **x**, **rx** or **rwX**, and no fill pattern is defined, the section is filled with zeros. If no attributes are set, the section is created as a scratch section (attributes **ws**, no image).

```
memcpy_statement ::= memcpy section_name
                      ( memcpy_spec <, memcpy_spec>0|1 )
```

```
memcpy_spec ::= memory = memory_reference
                | fill_spec
```

- A **memcpy** statement must contain exactly one **memory** statement.
- A **memcpy** statement can contain at most one *fill_spec*.

```
fill_spec ::= fill = fill_values
```

```
fill_values ::= expr
                | [ expr <, expr>>=0 ]
```

```
aggregate_section_statement ::= { <section_statement>>=0 }
                                | group_descr
                                | if_statement
                                | section_creation_statement
                                | struct_statement
```

```
group_descr ::= group <group_name>0|1 <( group_specs )>0|1
                section_statement
```

- For every group with a name, the linker defines a label.
- No two groups for address spaces of a core can have the same *group_name*.

```
group_specs ::= group_spec <, group_spec >>=0
```

```
group_spec ::= group_alignment
                | attributes
                | copy
                | nocopy
                | group_load_address
                | fill <= fill_values>0|1
                | group_page
                | group_run_address
                | group_type
```

```
| allow_cross_references
| priority = number
| group_safety_class
| tag
```

- The **allow-cross-references** property is only allowed for *overlay* groups.
- The **copy** and **nocopy** properties cannot be applied both to the same group.
- Sub groups inherit all properties from a parent group.

```
group_alignment ::= align = expr
```

```
attributes ::= attributes = <attribute>*=1
```

```
attribute ::= r // readable sections
| w // writable sections
| x // executable code sections
| i // initialized sections
| s // scratch sections
| b // blanked (cleared) sections
| p // protected sections
| u // uncached sections
```

```
group_load_address ::= load_addr <= load_or_run_addr>0|1
```

```
group_page ::= page <= expr>0|1
| page_size = expr <[ range ] <| [ range ]>*=0>0|1
```

```
group_run_address ::= run_addr <= load_or_run_addr>0|1
```

```
group_type ::= clustered
| contiguous
| ordered
| overlay
```

- For *non-contiguous* groups, you can only specify *group_alignment* and *attributes*.
- The **overlay** keyword also sets the **contiguous** property.
- The **clustered** property cannot be set together with **contiguous** or **ordered** on a single group.

```
group_safety_class ::= safety_class = number
```

```
load_or_run_addr ::= addr_absolute
| addr_range <| addr_range>*=0
```

```
addr_absolute ::= expr
| memory_reference [ expr ]
```

- An absolute address can only be set on *ordered* groups.

```

addr_range      ::= [ expr .. expr ]
                  | memory_reference
                  | memory_reference [ expr .. expr ]

```

- The parent of a group with an *addr_range* or **page** restriction cannot be **ordered**, **contiguous** or **clustered**.
- The end address is not part of the range.

```

memory_reference ::= mem : <proc_name >0|1 mem_name </ map_name>0|1

```

- A *proc_name* refers to a defined processor.
- A *mem_name* refers to a defined memory.
- A *map_name* refers to a defined memory mapping.

```

if_statement    ::= if ( expr ) section_statement
                  <else section_statement>0|1

```

```

section_creation_statement
                  ::= section section_name ( section_specs )
                  { <section_statement2>>=0 }

```

```

section_specs   ::= section_spec <, section_spec >>=0

```

```

section_spec    ::= attributes
                  | fill_spec
                  | size = expr
                  | blocksize = expr
                  | overflow = section_name
                  | tag

```

```

section_statement2
                  ::= select_section_statement ;
                  | group_descr2
                  | { <section_statement2>>=0 }

```

```

group_descr2    ::= group <group_name>0|1
                  ( group_specs2 )
                  section_statement2

```

```

group_specs2    ::= group_spec2 <, group_spec2 >>=0

```

```

group_spec2     ::= group_alignment
                  | attributes
                  | load_addr
                  | nocopy
                  | tag

```

```

struct_statement
                  ::= struct { <struct_item>>=0 }

```

```
struct_item      ::= expr : number ;
```

17.2.13. Veneer Layout Definition

```
veneer_layout    ::= veneer_layout <space_ref>0|1
                    { <section_statement3>>=0 }

section_statement3
    ::= select_section_statement ;
       | group_descr3
       | { <section_statement3>>=0 }

group_descr3     ::= group <group_name>0|1
                    ( group_specs3 )
                    section_statement3

group_specs3     ::= group_spec3 <, group_spec3 >>=0

group_spec3      ::= group_run_address
                    | tag
```

17.3. Expression Evaluation

Only *constant* expressions are allowed, including sizes, but not addresses, of sections in object files.

All expressions are evaluated with 64-bit precision integer arithmetic. The result of an expression can be absolute or relocatable. A symbol you assign is created as an absolute symbol. Symbol references are only allowed in symbol assignments and **struct** statements.

17.4. Semantics of the Architecture Definition

Keywords in the architecture definition

```

architecture
  extends
endianness          big  little
bus
  mau
  width
  map
space
  id
  mau
  align
  page_size
  page
  direction          low_to_high  high_to_low
  stack
    min_size
    grows            low_to_high  high_to_low
    align
    fixed
    threads
    entry_points
    attributes       b
  heap
    min_size
    grows            low_to_high  high_to_low
    align
    fixed
    attributes       b
  copytable
    align
    copy_unit
    dest
    page
    table            space  symbol
  reserved
  start_address
    run_addr
    symbol
  map
  map
    dest             bus  space
    dest_dbits
    dest_offset
    size
    src_dbits

```



```

    src_offset
    cached
    priority
    exec_priority

mpu_layout
    register_sets
    data_ranges
    code_ranges

```

17.4.1. Defining an Architecture

With the keyword **architecture** you define an architecture and assign a unique name to it. The name is used to refer to it at other places in the LSL file:

```

architecture name
{
    definitions
}

```

If you are defining multiple core architectures that show great resemblance, you can define the common features in a parent core architecture and extend this with a child core architecture that contains specific features. The child inherits all features of the parent. With the keyword **extends** you create a child core architecture:

```

architecture name_child_arch extends name_parent_arch
{
    definitions
}

```

A core architecture can have any number of parameters. These are identifiers which get values assigned on instantiation or extension of the architecture. You can use them in any expression within the core architecture. Parameters can have default values, which are used when the core architecture is instantiated with less arguments than there are parameters defined for it. When you extend a core architecture you can pass arguments to the parent architecture. Arguments are expressions that set the value of the parameters of the sub-architecture.

```

architecture name_child_arch (parm1, parm2=1)
    extends name_parent_arch (arguments)
{
    definitions
}

```

17.4.2. Defining Internal Buses

With the **bus** keyword you define a bus (the combination of data and corresponding address bus). The bus name is used to identify a bus and does not conflict with other identifiers. Bus descriptions in an architecture definition or derivative definition define *internal* buses. Some internal buses are used to communicate with the components outside the core or processor. Such buses on a processor have physical pins reserved for the number of bits specified with the **width** statements.

- The **mau** field specifies the MAU size (Minimum Addressable Unit) of the data bus. This field is required and must be non-zero.
- The **width** field specifies the width (number of address lines) of the data bus. The default value is the MAU size.
- The **map** keyword specifies how this bus maps onto another bus (if so). Mappings are described in [Section 17.4.4, Mappings](#).

```
bus bus_name
{
    mau = 8;
    width = 8;
    map ( map_description );
}
```

17.4.3. Defining Address Spaces

With the **space** keyword you define a logical address space. The space name is used to identify the address space and does not conflict with other identifiers.

- The **id** field defines how the addressing space is identified in object files. In general, each address space has a unique ID. The linker locates sections with a certain ID in the address space with the same ID. This field is required.
- The **mau** field specifies the MAU size (Minimum Addressable Unit) of the space. This field is required and must be non-zero.
- The **align** value must be a power of two. The linker uses this value to compute the start addresses when sections are concatenated. An align value of *n* means that objects in the address space have to be aligned on *n* MAUs.
- The **page_size** field sets the page alignment and page size in MAUs for the address space. It must be a power of 2. The default value is 1. If one or more page ranges are supplied the supplied value only sets the page alignment. The ranges specify the available space in each page, as offsets to the page start, which is aligned at the page alignment.

See also the **page** keyword in subsection [Locating a group](#) in [Section 17.8.2, Creating and Locating Groups of Sections](#).

- With the optional **direction** field you can specify how all sections in this space should be located. This can be either from **low_to_high** addresses (this is the default) or from **high_to_low** addresses.
- The **map** keyword specifies how this address space maps onto an internal bus or onto another address space. Mappings are described in [Section 17.4.4, Mappings](#).

Stacks and heaps

- The **stack** keyword defines a stack in the address space and assigns a name to it. The architecture definition must contain at least one stack definition. Each stack of a core architecture must have a unique name. See also the **stack** keyword in [Section 17.8.3, Creating or Modifying Special Sections](#).

The stack is described in terms of a minimum size (**min_size**) and the direction in which the stack grows (**grows**). This can be either from **low_to_high** addresses (stack grows upwards, this is the default) or from **high_to_low** addresses (stack grows downwards). The **min_size** is required.

By default, the linker tries to maximize the size of the stacks and heaps. After locating all sections, the largest remaining gap in the space is used completely for the stacks and heaps. If you specify the keyword **fixed**, you can disable this so-called 'balloon behavior'. The size is also fixed if you used a stack or heap in the software layout definition in a restricted way. For example when you override a stack with another size or select a stack in an ordered group with other sections.

A stack may have an **attributes** property with value **b**. Such a stack must be cleared at program startup. No other attributes are allowed.

Optionally you can specify an alignment for the stack with the argument **align**. This alignment must be equal or larger than the alignment that you specify for the address space itself.

For each stack, a stack size estimation may be computed (and listed in a map file) from a call graph. Each root node of the call graph is treated as a separate thread that can run independently from the other threads. Root nodes can be specified using the **entry_points** keyword. The estimated stack usage for a root node is the highest sum of stack usage values along a path to a leaf node. The total estimated stack usage of a link task is the sum of the calculated stack usage of such independent call graphs. If only a limited number of these threads can make use of a specific stack at any time, you can specify this by assigning a number to the **threads** keyword on that stack's definition. When **threads** is set to *n*, only the *n* highest stack usage numbers of root nodes are summed. A **threads** argument equal to zero or negative is ignored.

A stack definition may have one or more **entry_points** statements that specify the code that uses that stack - all functions that are reachable (by calls) from the entry points are considered to be using the stack at run-time. Each symbol name specified as an entry point must match a node in the call graph, which must not have more than one caller. As a result of the entry point specification, the specific call (edge) is removed from the call graph (so the symbol becomes a root). A symbol may be declared as entry point for multiple stacks.

In the LSL file the value of the **threads** argument can be determined by the macro **__MAX_CONCURRENT_HANDLERS + 1**. You can define **__MAX_CONCURRENT_HANDLERS** to the number of interrupt and trap handlers that should contribute to stack size estimation. The handlers that use the most stack space are selected. Allowed values are 0 or higher values. By default the macro is not set. Defining this macro adds a **threads** keyword to **ustack**, with the macro value increased by one.

In the LSL file you can use a macro to set the value of an **entry_points** argument. Pre-AURIX TriCore derivative LSL files use **__USTACK_ENTRY_POINTS** and **__ISTACK_ENTRY_POINTS**, whereas AURIX and AURIX 2G derivative LSL files use **__USTACK_n_ENTRY_POINTS** or **__ISTACK_n_ENTRY_POINTS** for core *n*. Multiple entry points are separated by commas and enclosed in square brackets [].

```
#define __USTACK0_ENTRY_POINTS ["_START", "my_func"]

stack "ustack_tc0"
(
    min_size = 16k,
    fixed,
    align = 8,
```

```
#ifdef __MAX_CONCURRENT_HANDLERS
    threads = (__MAX_CONCURRENT_HANDLERS)+1,
#endif
    entry_points = __USTACK0_ENTRY_POINTS
);
```

- The **heap** keyword defines a heap in the address space and assigns a name to it. The definition of a heap is similar to the definition of a stack. See also the **heap keyword** in [Section 17.8.3, *Creating or Modifying Special Sections*](#).

Stacks and heaps are only generated by the linker if the corresponding linker labels are referenced in the object files.

See [Section 17.8, *Semantics of the Section Layout Definition*](#), for information on creating and placing stack sections.

Copy tables

- The **copytable** keyword defines a copy table in the address space. The content of the copy table is created by the linker and contains the start address and size of all sections that should be initialized by the startup code. You must define exactly one copy table in one of the address spaces (for a core).

Optionally you can specify an alignment for the copy table with the argument **align**. This alignment must be equal or larger than the alignment that you specify for the address space itself. If smaller, the alignment for the address space is used.

The **copy_unit** argument specifies the size in MAUs of information chunks that are copied. If you do not specify the copy unit, the MAU size of the address space itself is used.

The **dest** argument specifies the destination address space that the code uses for the copy table. The linker uses this information to generate the correct addresses in the copy table. The memory into where the sections must be copied at run-time, must be accessible from this destination space.

Sections generated for the copy table may get a page restriction with the address space's page size, by adding the **page** argument.

One or more **table** arguments split off one or more sub-tables from a copy table. Each sub-table can be handled separately from the others and from the main table. This can be useful in a multi-core system, for example. All initialization entries generated from a section in an address space can be redirected to a sub-table by putting the address space name in a comma-separated list following **space** =. The initialization code that handles a sub-table needs a reference to it. This can be accomplished by specifying a symbol for the sub-table using **symbol = symbol_name**; . Each sub-table including the main table (which is filled with all entries not redirected to a sub-table) ends with a terminator entry and they are all placed in the regular copy table section.

```
copytable
(
    align = 4,
    dest = linear,
    table
    {
```

```

        symbol = "_lc_ub_table_tc1";
        space = :tc1:linear, :tc1:abs24, :tc1:abs18, :tc1:csa;
    },
    table
    {
        symbol = "_lc_ub_table_tc2";
        space = :tc2:linear, :tc2:abs24, :tc2:abs18, :tc2:csa;
    }
);

```

Reserved address ranges

- The **reserved** keyword specifies to reserve a part of an address space even if not all of the range is covered by memory. See also the **reserved** keyword in [Section 17.8.3, Creating or Modifying Special Sections](#).

Start address

- The **start_address** keyword specifies the start address for the position where the C startup code is located. When a processor is reset, it initializes its program counter to a certain start address, sometimes called the reset vector. In the architecture definition, you must specify this start address in the correct address space in combination with the name of the label in the application code which must be located here.

The **run_addr** argument specifies the start address (reset vector). If the core starts executing using an entry from a vector table, and directly jumps to the start label, you should omit this argument.

The **symbol** argument specifies the name of the label in the application code that should be located at the specified start address. The **symbol** argument is required. The linker will resolve the start symbol and use its value after locating for the start address field in IEEE-695 files and Intel Hex files. If you also specified the **run_addr** argument, the start symbol (label) must point to a section. The linker locates this section such that the start symbol ends up on the start address.

```

space space_name
{
    id = 1;
    mau = 8;
    align = 8;
    page_size = 1;
    stack name (min_size = 1k, grows = low_to_high);
    reserved start_address .. end_address;
    start_address ( run_addr = 0x0000,
                   symbol = "start_label" )
    map ( map_description );
}

```

17.4.4. Mappings

You can use a mapping when you define a space, bus or memory. With the **map** field you specify how addresses from the source (space, bus or memory) are translated to addresses of a destination (space, bus). The following mappings are possible:

- space => space
- space => bus
- bus => bus
- memory => bus

With a mapping you specify a range of source addresses you want to map (specified by a source offset and a size), the destination to which you want to map them (a bus or another address space), and the offset address in the destination.

- The **dest** argument specifies the destination. This can be a **bus** or another address **space** (only for a space to space mapping). This argument is required.
- The **src_offset** argument specifies the offset of the source addresses. In combination with size, this specifies the range of address that are mapped. By default the source offset is 0x0000.
- The **size** argument specifies the number of addresses that are mapped. This argument is required.
- The **dest_offset** argument specifies the position in the destination to which the specified range of addresses is mapped. By default the destination offset is 0x0000.

If you are mapping a bus to another bus, the number of data lines of each bus may differ. In this case you have to specify a range of source data lines you want to map (**src_dbits = begin..end**) and the range of destination data lines you want to map them to (**dest_dbits = first..last**).

- The **src_dbits** argument specifies a range of data lines of the source bus. By default all data lines are mapped.
- The **dest_dbits** argument specifies a range of data lines of the destination bus. By default, all data lines from the source bus are mapped on the data lines of the destination bus (starting with line 0).

A mapping can optionally have a name which can be referenced in an address assignment.

If you define a memory and the memory mapping must not be used by default when locating sections in address spaces, you can specify the **reserved** argument. This marks all address space areas that the mapping points to as reserved. If a section has an absolute or address range restriction, the reservation is lifted and the section may be located at these locations. This feature is only useful when more than one mapping is available for a range of memory addresses, otherwise the **memory** keyword with the same name would be used.

For example:

```
memory xrom
{
```

```

    mau = 8;
    size = 1M;
    type = rom;
    map    cached (dest=bus:spe:fpi_bus, dest_offset=0x80000000,
                  size=1M, cached);
    map not_cached (dest=bus:spe:fpi_bus, dest_offset=0xa0000000,
                  size=1M, reserved);
}

```

Mappings that do not guarantee cross-core memory (cache) consistency

The optional mapping keyword **cached** means sections that require cross-core memory access consistency must not be located at addresses that use this mapping.

Mapping priority

If you define a memory you can set a locate priority on a mapping with the keywords **priority** and **exec_priority**. The values of these priorities are relative which means they add to the priority of memories. Whereas a priority set on the memory applies to all address space areas reachable through any mapping of the memory, a priority set on a mapping only applies to address space areas reachable through the mapping. The memory mapping with the highest priority is considered first when locating. To set only a priority for non-executable (data) sections, add a **priority** keyword with the desired value and an **exec_priority** set to zero. To set only a priority for executable (code) sections, simply set an **exec_priority** keyword to the desired value.

The default for a mapping **priority** is zero, while the default for **exec_priority** is the same as the specified **priority**. If you specify a value for **priority** in LSL it must be greater than zero. A value for **exec_priority** must be greater or equal to zero.

For more information about priority values see the description of the [memory priority](#) keyword.

```

memory dspram
{
    mau = 8;
    size = 112k;
    type = ram;
    map (dest=bus:tc0:fpi_bus, dest_offset=0xd0000000,
        size=112k, priority=8, exec_priority=0);
    map (dest=bus:sri, dest_offset=0x70000000,
        size=112k);
}

```

From space to space

If you map an address space to another address space (nesting), you can do this by mapping the subspace to the containing larger space. In this example a small space of 64 kB is mapped on a large space of 16 MB.

```

space small
{
    id = 2;
}

```

```
    mau = 4;
    map (src_offset = 0, dest_offset = 0,
        dest = space : large, size = 64k);
}
```

From space to bus

All spaces that are not mapped to another space must map to a bus in the architecture:

```
space large
{
    id = 1;
    mau = 4;
    map (src_offset = 0, dest_offset = 0,
        dest = bus:bus_name, size = 16M );
}
```

From bus to bus

The next example maps an external bus called `e_bus` to an internal bus called `i_bus`. This internal bus resides on a core called `mycore`. The source bus has 16 data lines whereas the destination bus has only 8 data lines. Therefore, the keywords **src_dbits** and **dest_dbits** specify which source data lines are mapped on which destination data lines.

```
architecture mycore
{
    bus i_bus
    {
        mau = 4;
    }

    space i_space
    {
        map (dest=bus:i_bus, size=256);
    }
}

bus e_bus
{
    mau = 16;
    width = 16;
    map (dest = bus:mycore:i_bus, src_dbits = 0..7, dest_dbits = 0..7 )
}
```

It is not possible to map an internal bus to an external bus.

17.4.5. Defining the MPU Layout

For each AURIX architecture definition that supports it, the layout of the Memory Protection Unit (MPU) is defined with the keyword **mpu_layout**.

- The **register_sets** keyword specifies the number of memory protection register sets. The sets specify memory protection ranges and permissions for code and data.
- The **data_ranges** keyword specifies the total number of data protection ranges that is available in a register set.
- The **code_ranges** keyword specifies the total number of code protection ranges that is available in a register set.

For example:

```
mpu_layout
{
    register_sets = 4;
    data_ranges = 16;
    code_ranges = 8;
}
```

17.5. Semantics of the Derivative Definition

Keywords in the derivative definition

```
derivative
    extends
core
    architecture
    import
    space_id_offset
    copytable_space
bus
    mau
    width
    map
memory
    type          reserved rom  ram  nvram  blockram
    mau
    size
    speed
    priority
    exec_priority
    fill
    write_unit
    map
```

```
section_layout
section_setup
```

```
map
    dest          bus  space
    dest_dbits
    dest_offset
    size
    src_dbits
    src_offset
    cached
    priority
    exec_priority
    reserved
```

17.5.1. Defining a Derivative

With the keyword **derivative** you define a derivative and assign a unique name to it. The name is used to refer to it at other places in the LSL file:

```
derivative name
{
    definitions
}
```

If you are defining multiple derivatives that show great resemblance, you can define the common features in a parent derivative and extend this with a child derivative that contains specific features. The child inherits all features of the parent (cores and memories). With the keyword **extends** you create a child derivative:

```
derivative name_child_deriv extends name_parent_deriv
{
    definitions
}
```

As with a core architecture, a derivative can have any number of parameters. These are identifiers which get values assigned on instantiation or extension of the derivative. You can use them in any expression within the derivative definition.

```
derivative name_child_deriv (parm1,parm2=1)
    extends name_parent_deriv (arguments)
{
    definitions
}
```

17.5.2. Instantiating Core Architectures

With the keyword **core** you instantiate a core architecture in a derivative.

- With the keyword **architecture** you tell the linker that the given core has a certain architecture. The architecture name refers to an existing architecture definition in the same LSL file.

For example, if you have two cores (called `mycore_1` and `mycore_2`) that have the same architecture (called `mycorearch`), you must instantiate both cores as follows:

```
core mycore_1
{
    architecture = mycorearch;
}

core mycore_2
{
    architecture = mycorearch;
}
```

If the architecture definition has parameters you must specify the arguments that correspond with the parameters. For example `mycorearch1` expects two parameters which are used in the architecture definition:

```
core mycore
{
    architecture = mycorearch1 (1,2);
}
```

- With the keyword **import** you can combine multiple cores with the same architecture into a single link task. The imported cores share a single symbol namespace.
- The address spaces in each imported core must have a unique ID in the link task. With the keyword **space_id_offset** you specify for each imported core that the space IDs of the imported core start at a specific offset.
- With the keyword **copytable_space** you can specify that writable sections for a core must be initialized by using the copy table of a different core.

```
core mycore_1
{
    architecture = mycorearch;
    space_id_offset = 100; // add 100 to all space IDs in
                          // the architecture definition
    copytable_space = mycore:myspace; // use copytable from core mycore
}
core mycore_2
{
    architecture = mycorearch;
    space_id_offset = 200; // add 200 to all space IDs in
                          // the architecture definition
    copytable_space = mycore:myspace; // use copytable from core mycore
}

core mycore
{
    architecture = mycorearch;
    import mycore_1; // add all address spaces of mycore_1 for linking
}
```

```
    import mycore_2; // add all address spaces of mycore_2 for linking
}
```

17.5.3. Defining Internal Memory and Buses

With the keyword **memory** you define physical memory that is present on the target board. The memory name is used to identify the memory and does not conflict with other identifiers. It is common to define internal memory (on-chip) in the derivative definition. External memory (off-chip memory) is usually defined in the board specification (See [Section 17.6.3, Defining External Memory and Buses](#)).

- The **type** field specifies a memory type:
 - **rom**: read-only memory - it can only be written at load-time
 - **ram**: random access volatile writable memory - writing at run-time is possible while writing at load-time has no use since the data is not retained after a power-down
 - **nvr****ram**: non volatile ram - writing is possible both at load-time and run-time
 - **blockram**: writing is possible both at load-time and run-time. Changes are applied in RAM, so after a full device reset the data in a blockram reverts to the original state.

The optional **reserved** qualifier before the memory type, tells the linker not to locate any section in the memory by default. You can locate sections in such memories using an absolute address or range restriction (see subsection [Locating a group](#) in [Section 17.8.2, Creating and Locating Groups of Sections](#)).

- The **mau** field specifies the MAU size (Minimum Addressable Unit) of the memory. This field is required and must be non-zero.
- The **size** field specifies the size in MAU of the memory. This field is required.
- The **priority** field specifies a locate priority for a memory. The **speed** field has the same meaning but is considered deprecated. By default, a memory has its priority set to 1. The memories with the highest priority are considered first when trying to locate a rule. Subsequently, the next highest priority memories are added if the rule was not located successfully, and so on until the lowest priority that is available is reached or the rule is located. The lowest priority value is zero. Sections with an **ordered** and/or **contiguous** restriction are not affected by the locate priority. If such sections also have a **page** restriction, the locate priority is still used to select a page.
- If an **exec_priority** is specified for a memory, the regular priority (either specified or its default value) does not apply to locate rules with only executable sections. Instead, the supplied value applies for such rules. Additionally, the **exec_priority** value is used for any executable unrestricted sections, even if they appear in an unrestricted rule together with non-executable sections.
- The **map** field specifies how this memory maps onto an (internal) bus. The mapping can have a name. Mappings are described in [Section 17.4.4, Mappings](#).
- The optional **write_unit** field specifies the minimum write unit (MWU). This is the minimum number of MAUs required in a write action. This is useful to initialize memories that can only be written in units of two or more MAUs. If **write_unit** is not defined the minimum write unit is 0.

- The optional **fill** field contains a bit pattern that the linker writes to all memory addresses that remain unoccupied during the locate process. The result of the expression, or list of expressions, is used as values to write to memory, each in MAU.

```
memory mem_name
{
    type = rom;
    mau = 8;
    write_unit = 4;
    fill = 0xaa;
    size = 64k;
    priority = 2;
    map map_name ( map_description );
}
```

With the **bus** keyword you define a bus in a derivative definition. Buses are described in [Section 17.4.2, Defining Internal Buses](#).

17.6. Semantics of the Board Specification

Keywords in the board specification

```
processor
    derivative
bus
    mau
    width
    map
memory
    type          reserved rom ram nvram blockram
    mau
    size
    speed
    priority
    exec_priority
    fill
    write_unit
    map
    map
        dest      bus space
        dest_dbits
        dest_offset
        size
        src_dbits
        src_offset
        cached
        priority
```

```
exec_priority
reserved
```

17.6.1. Defining a Processor

If you have a target board with multiple processors that have the same derivative, you need to instantiate each individual processor in a processor definition. This information tells the linker which processor has which derivative and enables the linker to distinguish between the present processors.

If you use processors that all have a unique derivative, you may omit the processor definitions. In this case the linker assumes that for each derivative definition in the LSL file there is one processor. The linker uses the derivative name also for the processor.

With the keyword **processor** you define a processor. You can freely choose the processor name. The name is used to refer to it at other places in the LSL file:

```
processor proc_name
{
    processor definition
}
```

17.6.2. Instantiating Derivatives

With the keyword **derivative** you tell the linker that the given processor has a certain derivative. The derivative name refers to an existing derivative definition in the same LSL file.

For example, if you have two processors on your target board (called `myproc_1` and `myproc_2`) that have the same derivative (called `myderiv`), you must instantiate both processors as follows:

```
processor myproc_1
{
    derivative = myderiv;
}

processor myproc_2
{
    derivative = myderiv;
}
```

If the derivative definition has parameters you must specify the arguments that correspond with the parameters. For example `myderiv1` expects two parameters which are used in the derivative definition:

```
processor myproc
{
    derivative = myderiv1 (2,4);
}
```

17.6.3. Defining External Memory and Buses

It is common to define external memory (off-chip) and external buses at the global scope (outside any enclosing definition). Internal memory (on-chip memory) is usually defined in the scope of a derivative definition.

With the keyword **memory** you define physical memory that is present on the target board. The memory name is used to identify the memory and does not conflict with other identifiers. If you define memory parts in the LSL file, only the memory defined in these parts is used for placing sections.

If no external memory is defined in the LSL file and if the linker option to allocate memory on demand is set then the linker will assume that all virtual addresses are mapped on physical memory. You can override this behavior by specifying one or more memory definitions.

```
memory mem_name
{
    type = rom;
    mau = 8;
    write_unit = 4;
    fill = 0xaa;
    size = 64k;
    priority = 2;
    map map_name ( map_description );
}
```

For a description of the keywords, see [Section 17.5.3, Defining Internal Memory and Buses](#).

With the keyword **bus** you define a bus (the combination of data and corresponding address bus). The bus name is used to identify a bus and does not conflict with other identifiers. Bus descriptions at the global scope (outside any definition) define external buses. These are buses that are present on the target board.

```
bus bus_name
{
    mau = 8;
    width = 8;
    map ( map_description );
}
```

For a description of the keywords, see [Section 17.4.2, Defining Internal Buses](#).

You can connect off-chip memory to any derivative: you need to map the off-chip memory to a bus and map that bus on the internal bus of the derivative you want to connect it to.

17.7. Semantics of the Section Setup Definition

Keywords in the section setup definition

```
section_setup
    stack
```

```

    min_size
    grows          low_to_high  high_to_low
    align
    fixed
    id
heap
    min_size
    grows          low_to_high  high_to_low
    align
    fixed
    id
copytable
    align
    copy_unit
    dest
    page
    table          space  symbol
reserved
start_address
    run_addr
    symbol
prohibit_references_to
modify input
    space
    attributes
    copy
section_reference_restriction
    safety_class
    target_safety_class
    attributes
mpu_data_table

```

17.7.1. Setting up a Section

With the keyword **section_setup** you can define stacks, heaps, copy tables, start address and/or reserved address ranges outside their address space definition. In addition you can configure space reference restrictions, input section modifications, section reference restrictions and MPU data table.

```

section_setup ::my_space
{
    reserved address range
    stack definition
    heap definition
    copy table definition
    start address
    space reference restrictions
    input section modifications
    section reference restrictions
    MPU data table
}

```


See the subsections [Stacks and heaps](#), [Copy tables](#), [Start address](#) and [Reserved address ranges](#) in [Section 17.4.3, Defining Address Spaces](#) for details on the keywords **stack**, **heap**, **copytable** and **reserved**.

Space reference restrictions

With a space reference restriction, references from the section setup's address space to sections in specific address spaces can be deleted and blocked. If sections, for example code, in space A are not allowed or not able to access sections (functions or variables) in space B, you can configure this in LSL as follows:

```
section_setup ::A
{
    prohibit_references_to ::B;
}
```

The linker emits an error when such a reference is found in a relocation.

For example, for a multi-core AURIX, the following fragment disallows references from private0 sections to private1 and private2 sections, from private1 sections to private0 and private2 sections, and from private2 sections to private0 and private1 sections.

```
section_setup :tc0:linear
{
    prohibit_references_to :tc1:linear, :tc1:abs18, :tc2:linear, :tc2:abs18;
}

section_setup :tc1:linear
{
    prohibit_references_to :tc0:linear, :tc0:abs18, :tc2:linear, :tc2:abs18;
}

section_setup :tc2:linear
{
    prohibit_references_to :tc0:linear, :tc0:abs18, :tc1:linear, :tc1:abs18;
}
```

Input section modifications

Before sections are located and before selections defined in `section_layout` are performed, you can still modify a few section properties. These are:

- change the address space of a section
- add (+w) or remove (-w) the writable attribute
- add (+p) or remove (-p) the protected attribute
- add (+u) or remove (-u) the uncached attribute

You cannot set the protected attribute on linker created sections like reserved sections and output sections.

Sections are selected the same way as in groups in a `section_layout`. Instead of `attributes=+w` you can use the **copy** keyword.

```
section_setup ::A
{
    modify input (space>::B, attributes=+w)
    {
        select "mysection";
    }
}
```

Note that the new address space must be used to select a modified section in a `section_layout`. To locate the section `mysection` in the example somewhere, it must be selected in a `section_layout` for space `::B`. If the link result is output to a file, for example by only linking or incremental linking, the modified properties are exported. So, when the resulting file is used in another invocation of the linker, the section can appear in a different address space.

Section reference restrictions

With a **section_reference_restriction** statement you can specify what access operations are allowed between specific combinations of sections. Instead of selecting sections directly, the concept of a *safety class* is introduced. A safety class is associated with a collection of sections and is represented by a positive integer number. Safety classes are disjunct subsets of the set of sections that is emitted in absolute output object files. Initially, all sections are in safety class 0. By default, all access is prohibited unless the two sections are part of the same safety class. In that case, all access is allowed.

For example,

```
#define SC_ASIL_A 3
section_setup ::linear
{
    section_reference_restriction
    {
        safety_class = SC_ASIL_A;
        target_safety_class = 7;
        attributes = r;
        // safety class '3' only has read access in safety class '7'
    }
}
```

MPU data table

With a **mpu_data_table** statement you can specify in which address space the linker should generate MPU configuration data when you specify linker option **--mpu-configuration-data**. The contents of the MPU data table contains the required information to configure the Memory Protection System for devices that support it.

For example,

```
section_setup ::linear
{
```

```

    mpu_data_table;
}

```

17.8. Semantics of the Section Layout Definition

Keywords in the section layout definition

```

section_layout
    direction      low_to_high  high_to_low
group
    align
    attributes     + -  r w x b i s p u
    copy
    nocopy
    fill
    ordered
    contiguous
    clustered
    overlay
    allow_cross_references
    load_addr
        mem
    run_addr
        mem
    page
    page_size
    priority
    safety_class
    select
stack
    size
heap
    size
reserved
    size
    attributes     r w x
    fill
    alloc_allowed absolute ranged
copytable
memcpy
    memory
    fill
section
    size
    blocksize
    attributes     r w x
    fill
    overflow

```

```
struct
    checksum

if
else
```

17.8.1. Defining a Section Layout

With the keyword **section_layout** you define a section layout for exactly one address space. In the section layout you can specify how input sections are placed in the address space, relative to each other, and what the absolute run and load addresses of each section will be.

You can define one or more section definitions. Each section definition arranges the sections in one address space. You can precede the address space name with a processor name and/or core name, separated by colons. You can omit the processor name and/or the core name if only one processor is defined and/or only one core is present in the processor. A reference to a space in the only core of the only processor in the system would look like ": :my_space". A reference to a space of the only core on a specific processor in the system could be "my_chip: :my_space". The next example shows a section definition for sections in the my_space address space of the processor called my_chip:

```
section_layout my_chip::my_space ( locate_direction )
{
    section statements
}
```

Clone sections

Clone sections are placed in special address spaces that are characterized by a set of regular address spaces. Those spaces each have different "local" memory across some common address range. The clone space is created in the "main" core of the task, not in a core that is imported. For a section layout, the clone space is specified by replacing the simple space name by a list of full names for the "real" spaces. These names are separated by vertical bars (|), and the colons in the full names are replaced by underscores. The processor name (including the following underscore) can be omitted from the space names in the list if it is equal to the clone space's processor, and the order of spaces in the list is not important. The next example shows a definition for a multi-core TriCore with clone sections in address space abs18:

```
section_layout mpe:vtc:tc1_abs18|tc2_abs18|mpe_tc0_abs18
{
    section statements
}
```

You can see in the example that the mpe_ (the processor name) can be omitted.

Locate direction

With the optional keyword **direction** you specify whether the linker starts locating sections from **low_to_high** (default) or from **high_to_low**. In the second case the linker starts locating sections at the highest addresses in the address space but preserves the order of sections when necessary (one processor and core in this example).

```
section_layout ::my_space ( direction = high_to_low )
{
    section statements
}
```

If you do not explicitly tell the linker how to locate a section, the linker decides on the basis of the section attributes in the object file and the information in the architecture definition and memory parts where to locate the section.

17.8.2. Creating and Locating Groups of Sections

Sections are located per group. A group can contain one or more (sets of) input sections as well as other groups. Per group you can assign a mutual order to the sets of sections and locate them into a specific memory part.

```
group ( group_specifications )
{
    section_statements
}
```

With the *section_statements* you generally select sets of sections to form the group. This is described in subsection [Selecting sections for a group](#).

Instead of selecting sections, you can also modify special sections like stack and heap or create a reserved section. This is described in [Section 17.8.3, Creating or Modifying Special Sections](#).

With the *group_specifications* you actually locate the sections in the group. This is described in subsection [Locating a group](#).

Selecting sections for a group

With the keyword **select** you can select one or more sections for the group. You can select a section by name or by attributes. If you select a section by name, you can use a wildcard pattern:

- * matches with all section names
- ? matches with a single character in the section name
- \ takes the next character literally
- [abc] matches with a single 'a', 'b' or 'c' character
- [a-z] matches with any single character in the range 'a' to 'z'

```
group ( ... )
{
    select "mysection";
    select "*";
}
```

The first **select** statement selects the section with the name "mysection". The second **select** statement selects all sections that were not selected yet.

A section is selected by the first select statement that matches, in the union of all section layouts for the address space. Global section layouts are processed in the order in which they appear in the LSL file. Internal core architecture section layouts always take precedence over global section layouts.

When you use wildcards, the linker skips sections with an absolute address from the selection process, for example, a start section already having an absolute start address.

Note that when you select sections with an exact name (no wildcards), all sections with that name are automatically protected against unreferenced section removal. With a selection using wildcards, matching sections are selected, but matching sections that are unreferenced may be removed.

Restriction: Keep in mind that all section selections are restricted to the address space of the section layout in which this group definition occurs. So, for example for a multi-core TriCore where you have three cores, tc0, tc1 and tc2, that are imported into a virtual core vtc, when you have a section called .text.private0.lmu.p0_in_lmu in core tc0, you can select this section in the section layout for core tc0 only, so not in the root core vtc.

```
section_layout mpe:tc0:code      // must be core tc0; vtc is not allowed here
{
    group LMU_code0 ( run_addr = mem:mpe:lmuram )
    {
        select ".text.*.lmu.*"; // select section from tc0
    }
}
```

- The **attributes** field selects all sections that carry (or do not carry) the given attribute. With **+attribute** you select sections that have the specified attribute set. With **-attribute** you select sections that do not have the specified attribute set. You can specify one or more of the following attributes:

- **r** readable sections
- **w** writable sections
- **x** executable sections
- **i** initialized sections
- **b** sections that should be cleared at program startup
- **s** scratch sections (not cleared and not initialized)
- **p** protected sections
- **u** uncached sections (sections requiring memory coherence)

To select all read-only sections:

```
group ( ... )
{
    select (attributes = +r-w);
}
```

Keep in mind that all section selections are restricted to the address space of the section layout in which this group definition occurs.

- With the **ref_tree** field you can select a group of related sections. The relation between sections is often expressed by means of references. By selecting just the 'root' of tree, the complete tree is selected. This is for example useful to locate a group of related sections in special memory (e.g. fast memory). The (referenced) sections must meet the following conditions in order to be selected:
 1. The sections are within the section layout's address space
 2. The sections match the specified attributes
 3. The sections have no absolute restriction (as is the case for all wildcard selections)

For example, to select the code sections referenced from `foo1`:

```
group refgrp (contiguous, run_addr=mem:ext_c)
{
    select ref_tree "foo1" (attributes=+x);
}
```

If section `foo1` references `foo2` and `foo2` references `foo3`, then all these sections are selected by the selection shown above.

Locating a group

```
group group_name ( group_specifications )
{
    section_statements
}
```

With the *group_specifications* you actually define how the linker must locate the group. You can roughly define three things: 1) assign properties to the sections in a group like alignment and read/write attributes, 2) define the mutual order in the address space for sections in the group and 3) restrict the possible addresses for the sections in a group.

The linker creates labels that allow you to refer to the begin and end address of a group from within the application software. Labels `_lc_gb_group_name` and `_lc_ge_group_name` mark the begin and end of the group respectively, where the begin is the lowest address used within this group and the end is the highest address used. Notice that a group not necessarily occupies all memory between begin and end address. The given label refers to where the section is located at run-time (versus load-time).

1. Assign properties to the sections in a group like alignment and read/write attributes.

These properties are assigned to all sections in the group (and subgroups) and override the attributes of the input sections.

- The **align** field tells the linker to align all sections in the group according to the align value. The alignment of a section is first determined by its own initial alignment and the defined alignment for the address space. Alignments are never decreased, if multiple alignments apply to a section, the largest one is used.

- The **attributes** field tells the linker to assign one or more attributes to all sections in the group. This overrides the default attributes. By default the linker uses the attributes of the input sections. You can set the **r**, **w**, or **rw** attributes and you can switch between the **b** and **s** attributes.
- The **copy** field tells the linker to locate a read-only section in RAM and generate a ROM copy and a copy action in the copy table. This property makes the sections in the group writable which causes the linker to generate ROM copies for the sections.
- The effect of the **nocopy** field is the opposite of the **copy** field. It prevents the linker from generating ROM copies of the selected sections. You cannot apply both **copy** and **nocopy** to the same statement.

2. Define the mutual order of sections in an LSL group.

By default, a group is *unrestricted* which means that the linker has total freedom to place the sections of the group in the address space.

Note that when you use the linker optimization option **--optimize=+copytable-compression**, unrestricted sections affected by the copy table are located as if they were in a clustered LSL group. This option is enabled by default.

- The **ordered** keyword tells the linker to locate the sections in the same order in the address space as they appear in the group (but not necessarily adjacent).

Suppose you have an ordered group that contains the sections 'A', 'B' and 'C'. By default the linker places the sections in the address space like 'A' - 'B' - 'C', where section 'A' gets the lowest possible address. With **direction=high_to_low** in the **section_layout** space properties, the linker places the sections in the address space like 'C' - 'B' - 'A', where section 'A' gets the highest possible address.

- The **contiguous** keyword tells the linker to locate the sections in the group in a single address range. Within a contiguous group the input sections are located in arbitrary order, however the group occupies one contiguous range of memory. Due to alignment of sections there can be 'alignment gaps' between the sections.

When you define a group that is both **ordered** and **contiguous**, this is called a *sequential* group. In a sequential group the linker places sections in the same order in the address space as they appear in the group and it occupies a contiguous range of memory.

- The **clustered** keyword tells the linker to locate the sections in the group in a number of *contiguous* blocks. It tries to keep the number of these blocks to a minimum. If enough memory is available, the group will be located as if it was specified as **contiguous**. Otherwise, it gets split into two or more blocks.

If a contiguous or clustered group contains *alignment gaps*, the linker can locate sections that are not part of the group in these gaps. To prevent this, you can use the **fill** keyword. If the group is located in RAM, the gaps are treated as reserved (scratch) space. If the group is located in ROM, the alignment gaps are filled with zeros by default. You can however change the fill pattern by specifying a bit pattern. The result of the expression, or list of expressions, is used as values to write to memory, each in MAU.

- The **overlay** keyword tells the linker to overlay the sections in the group. The linker places all sections in the address space using a contiguous range of addresses. (Thus an overlay group is automatically also a contiguous group.) To overlay the sections, all sections in the overlay group share the same run-time address.

For each input section within the overlay the linker automatically defines two symbols. The symbol `_lc_cb_section_name` is defined as the load-time start address of the section. The symbol `_lc_ce_section_name` is defined as the load-time end address of the section. C (or assembly) code may be used to copy the overlaid sections.

If sections in the overlay group contain references between groups, the linker reports an error. The keyword **allow_cross_references** tells the linker to accept cross-references. Normally, it does not make sense to have references between sections that are overlaid.

```
group ovl (overlay)
{
    group a
    {
        select "my_ovl_p1";
        select "my_ovl_p2";
    }
    group b
    {
        select "my_ovl_q1";
    }
}
```

It may be possible that one of the sections in the overlay group already has been defined in another group where it received a load-time address. In this case the linker does not overrule this load-time address and excludes the section from the overlay group.

3. Restrict the possible addresses for the sections in a group.

The load-time address specifies where the group's elements are loaded in memory at download time. The run-time address specifies where sections are located at run-time, that is when the program is executing. If you do not explicitly restrict the address in the LSL file, the linker assigns addresses to the sections based on the restrictions relative to other sections in the LSL file and section alignments. The program is responsible for copying overlay sections at appropriate moment from its load-time location to its run-time location (this is typically done by the startup code).

- The **run_addr** keyword defines the run-time address. If the run-time location of a group is set explicitly, the given order between groups specify whether the run-time address propagates to the parent group or not. The location of the sections in a group can be restricted either to a single absolute address, or to a number of address ranges (not including the end address). With an expression you can specify that the group should be located at the absolute address specified by the expression:

```
group (ordered, run_addr = 0xa00f0000)
```

A group with an absolute address must be ordered, the first section in the group is located at the specified absolute address.

You can use the '[offset]' variant to locate the group at the given absolute offset in memory:

```
group (ordered, run_addr = mem:A[0x1000])
```

A group with an absolute address must be ordered, the first section in the group is located at the specified absolute offset in memory.

A range can be an absolute space address range, written as `[expr .. expr]`, a complete memory device, written as `mem:mem_name`, or a memory address range, `mem:mem_name[expr .. expr]`

```
group (run_addr = mem:my_dram)
```

You can use the '|' to specify an address range of more than one physical memory device:

```
group (run_addr = mem:A | mem:B)
```

When used in top-level section layouts, a memory name refers to a board-level memory. You can select on-chip memory with `mem:proc_name:mem_name`. If the memory has multiple parallel mappings towards the current address space, you can select a specific named mapping in the memory by appending `/map_name` to the memory specifier. The linker then maps memory offsets only through that mapping, so the address(es) where the sections in the group are located are determined by that memory mapping.

```
group (run_addr = mem:CPU1:A/cached)
```

- The **load_addr** keyword changes the meaning of the section selection in the group: the linker selects the load-time ROM copy of the named section(s) instead of the regular sections. Just like **run_addr** you can specify an absolute address or an address range.

```
group (contiguous, load_addr)
{
    select "mydata"; // select ROM copy of mydata:
                    // "[mydata]"
}
```

The load-time and run-time addresses of a group cannot be set at the same time. If the load-time property is set for a group, the group (only) restricts the positioning at load-time of the group's sections. It is not possible to set the address of a group that has a not-unrestricted parent group.

The properties of the load-time and run-time start address are:

- At run-time, before using an element in an overlay group, the application copies the sections from their load location to their run-time location, but only if these two addresses are different. For non-overlay sections this happens at program start-up.
- The start addresses cannot be set to absolute values for unrestricted groups.
- For non-overlay groups that do not have an overlay parent, the load-time start address equals the run-time start address.
- For any group, if the run-time start address is not set, the linker selects an appropriate address.

- If an ordered group or sequential group has an absolute address and contains sections that have separate page restrictions (not defined in LSL), all those sections are located in a single page. In other cases, for example when an unrestricted group has an address range assigned to it, the paged sections may be located in different pages.

For overlays, the linker reserves memory at the run-time start address as large as the largest element in the overlay group.

- The **page** keyword tells the linker to place the group in one page. Instead of specifying a run-time address, you can specify a page and optional a page number. Page numbers start from zero. If you omit the page number, the linker chooses a page.

The **page** keyword refers to pages in the address space as defined in the architecture definition.

- With the **page_size** keyword you can override the page alignment and size set on the address space. When you set the page size to zero, the linker removes simple (auto generated) page restrictions from the selected sections. See also the **page_size** keyword in [Section 17.4.3, Defining Address Spaces](#).
- With the **priority** keyword you can change the order in which sections are located. This is useful when some sections are considered important for good performance of the application and a small amount of fast memory is available. The value is a number for which the default is 1, so higher priorities start at 2. Sections with a higher priority are located before sections with a lower priority, unless their relative locate priority is already determined by other restrictions like **run_addr** and **page**.

```
group (priority=2)
{
    select "importantcode1";
    select "importantcode2";
}
```

- With the **safety_class** keyword you can assign a safety class to the sections in the group. The safety class must be a positive integer value. The default value is zero. The meaning of the safety class value is user defined.

```
group ASIL_A (safety_class=3)
{
    select ".*.ASIL_A.*" (attributes=-w);
}
```

17.8.3. Creating or Modifying Special Sections

Instead of selecting sections, you can also create a reserved section or an output section or modify special sections like a stack or a heap. Because you cannot define these sections in the input files, you must use the linker to create them.

Stack

- The keyword **stack** tells the linker to reserve memory for the stack. The name for the stack section refers to the stack as defined in the architecture definition. If no name was specified in the architecture definition, the default name is `stack`.

With the keyword **size** you can specify the size for the stack. If the size is not specified, the linker uses the size given by the **min_size** argument as defined for the stack in the architecture definition. Normally the linker automatically tries to maximize the size, unless you specified the keyword **fixed**.

```
group ( ... )
{
    stack "mystack" ( size = 2k );
}
```

The linker creates two labels to mark the begin and end of the stack, `_lc_ub_stack_name` for the begin of the stack and `_lc_ue_stack_name` for the end of the stack. The linker allocates space for the stack when there is a reference to either of the labels.

See also the **stack** keyword in [Section 17.4.3, Defining Address Spaces](#).

Heap

- The keyword **heap** tells the linker to reserve a dynamic memory range for the `malloc()` function. Each heap section has a name. With the keyword **size** you can change the size for the heap. If the **size** is not specified, the linker uses the size given by the **min_size** argument as defined for the heap in the architecture definition. Normally the linker automatically tries to maximize the size, unless you specified the keyword **fixed**.

```
group ( ... )
{
    heap "myheap" ( size = 2k );
}
```

The linker creates two labels to mark the begin and end of the heap, `_lc_ub_heap_name` for the begin of the heap and `_lc_ue_heap_name` for the end of the heap. The linker allocates space for the heap when a reference to either of the section labels exists in one of the input object files.

Reserved section

- The keyword **reserved** tells the linker to create an area or section of a given size. The linker will not locate any other sections in the memory occupied by a reserved section, with some exceptions. Each reserved section has a name. With the keyword **size** you can specify a size for a given reserved area or section.

```
group ( ... )
{
    reserved "myreserved" ( size = 2k );
}
```

The optional **fill** field contains a bit pattern that the linker writes to all memory addresses that remain unoccupied during the locate process. The result of the expression, or list of expressions, is used as values to write to memory, each in MAU. The first MAU of the fill pattern is always the first MAU in the section.

By default, no sections can overlap with a reserved section. With **alloc_allowed=absolute** sections that are located at an absolute address due to an absolute group restriction can overlap a reserved section. The same applies for reserved sections with **alloc_allowed=ranged** set. Sections restricted to a fixed address range can also overlap a reserved section.

With the **attributes** field you can set the access type of the reserved section. The linker locates the reserved section in its space with the restrictions that follow from the used attributes, **r**, **w** or **x** or a valid combination of them. The allowed attributes are shown in the following table. A value between < and > in the table means this value is set automatically by the linker.

Properties set in LSL		Resulting section properties		
attributes	filled	access	memory	content
x	yes		<rom>	executable
r	yes	r	<rom>	data
r	no	r	<rom>	scratch
rx	yes	r	<rom>	executable
rw	yes	rw	<ram>	data
rw	no	rw	<ram>	scratch
rwX	yes	rw	<ram>	executable

```
group ( ... )
{
    reserved "myreserved" ( size = 2k,
                           attributes = rw, fill = 0xaa );
}
```

If you do not specify any attributes, the linker will reserve the given number of maus, no matter what type of memory lies beneath. If you do not specify a fill pattern, no section is generated.

The linker creates two labels to mark the begin and end of the section, **_lc_ub_name** for the begin of the section and **_lc_ue_name** for the end of the reserved section.

Output sections

- The keyword **section** tells the linker to accumulate sections obtained from object files ("input sections") into an output section of a fixed size in the locate phase. You can select the input sections with **select** statements. You can use groups inside output sections, but you can only set the **align**, **attributes**, **nocopy** and **load_addr** properties and the **load_addr** property cannot have an address specified.

The **fill** field contains a bit pattern that the linker writes to all unused space in the output section. When all input sections have an image (code/data) you must specify a fill pattern. If you do not specify a fill pattern, all input sections must be scratch sections. The fill pattern is aligned at the start of the output section.

As with a reserved section you can use the **attributes** field to set the access type of the output section.

```
group ( ... )
{
    section "myoutput" ( size = 4k, attributes = rw,
                        fill = 0xaa )
    {
        select "myinput1";
        select "myinput2";
    }
}
```

The available room for input sections is determined by the **size**, **blocksize** and **overflow** fields. With the keyword **size** you specify the fixed size of the output section. Input sections are placed from output section start towards higher addresses (offsets). When the end of the output section is reached and one or more input sections are not yet placed, an error is emitted. If however, the **overflow** field is set to another output section, remaining sections are located as if they were selected for the overflow output section.

```
group ( ... )
{
    section "tsk1_data" (size=4k, attributes=rw, fill=0,
                        overflow = "overflow_data")
    {
        select ".data.tsk1.*"
    }
    section "tsk2_data" (size=4k, attributes=rw, fill=0,
                        overflow = "overflow_data")
    {
        select ".data.tsk2.*"
    }
    section "overflow_data" (size=4k, attributes=rx,
                            fill=0)
    {
    }
}
```

With the keyword **blocksize**, the size of the output section will adapt to the size of its content. For example:

```
group flash_area (run_addr = 0x10000)
{
    section "flash_code" (blocksize=4k, attributes=rx,
                          fill=0)
    {
        select ".*.flash";
    }
}
```

If the content of the section is 1 MB, the size will be 4 kB, if the content is 11 kB, the section will be 12 kB, etc. If you use **size** in combination with **blocksize**, the **size** value is used as default (minimal) size for this section. If it is omitted, the default size will be of **blocksize**. It is not allowed to omit both **size** and **blocksize** from the section definition.

The linker creates two labels to mark the begin and end of the section, **_lc_ub_name** for the begin of the section and **_lc_ue_name** for the end of the output section.

When the **copy** property is set on an enclosing group, a ROM copy is created for the output section and the output section itself is made writable causing it to be located in RAM by default. For this to work, the output section and its input sections must be read-only and the output section must have a **fill** property.

A copy table can also be inserted into an output section, but only if two additional conditions are met:

- The copy table is the last section added to the output section.
- There must be sufficient room in the output section to accommodate the additional size of the copy table.

A copy table will likely increase in size after being added to the output section, so if you would add sections after the copy table selection, this would overwrite part of the copy table. The linker will emit an error message if either of the conditions is not met.

```
group ( ... )
{
    section "myoutput_tbl" ( size = 4k, attributes = r, fill = 0)
    {
        select "myinput";
        select "table"; // select the copy table
    }
}
```

Copy table

- The keyword **copytable** tells the linker to select a section that is used as *copy table*. The content of the copy table is created by the linker. It contains the start address and length of all sections that should be initialized by the startup code.

The linker creates two labels to mark the begin and end of the section, **_lc_ub_table** for the begin of the section and **_lc_ue_table** for the end of the copy table. The linker generates a copy table when a reference to either of the section labels exists in one of the input object files.

Memory copy sections

- If a memory (usually RAM) needs to be initialized by a different core than the one(s) that will use it, a copy of the contents of the memory can be placed in a section using a **memcpy** statement in a **section_layout**. All data (including code) present in the specified memory is then placed in a new section with the provided name and appropriate attributes. Unused areas in the memory are filled in the section using the supplied fill pattern or with zeros if no fill pattern is specified. If the memory contains a memory copy section the result is undefined. The actual initialization of the memory at run-time needs

to be done separately, this LSL feature only directs the linker to make the data located in the memory available for initialization. Note that a memory of type **ram** cannot hold initialized data, use type **blockram** instead.

Structures

- A **struct** statement in a **section_layout** creates a section and fills it with numbers that each occupy one or more MAUs. The new section must be named by providing a double-quoted string after the **struct** keyword. Each element has the form *expr* : *number* ;, where the expression provides the value to insert in the section and the number determines the number of MAUs occupied by the expression value. Elements are placed in the section in the order in which they appear in the **struct** body without any gaps between them. Multi-MAU elements are split into MAUs according to the endianness of the target. A **struct** section is read-only and it cannot be copied to RAM at startup (using the **copy** group attribute). No default alignment is set.

For example,

```
struct "mystruct"
{
    0x1234                                     : 2;
    __BMHD_ALIGN( addressof( mem:foo ), -4 )   : 4;
    __BMHD_ALIGN( addressof( mem:foo ) + sizeof( mem:foo ), 4 ) : 4;
    checksum( crc32w,
        __BMHD_ALIGN( addressof( mem:foo ), -4 ),
        __BMHD_ALIGN( addressof( mem:foo ) + sizeof( mem:foo ), 4 ) ) : 4}
}
```

17.8.4. Creating Symbols

You can tell the linker to create symbols before locating by putting assignments in the section layout definition. Symbol names are represented by double-quoted strings. Any string is allowed, but object files may not support all characters for symbol names. You can use two different assignment operators. With the simple assignment operator '=', the symbol is created unconditionally. With the ':=' operator, the symbol is only created if it already exists as an undefined reference in an object file.

The expression that represents the value to assign to the symbol may contain references to other symbols. If such a referred symbol is a special section symbol, creation of the symbol in the left hand side of the assignment will cause creation of the special section.

```
section_layout
{
    "_lc_cp" := "_lc_ub_table";
    // when the symbol _lc_cp occurs as an undefined reference
    // in an object file, the linker generates a copy table
}
```

17.8.5. Conditional Group Statements

Within a group, you can conditionally select sections or create special sections.

- With the **if** keyword you can specify a condition. The succeeding section statement is executed if the condition evaluates to TRUE (1).
- The optional **else** keyword is followed by a section statement which is executed in case the if-condition evaluates to FALSE (0).

```
group ( ... )
{
    if ( exists( "mysection" ) )
        select "mysection";
    else
        reserved "myreserved" ( size=2k );
}
```

17.9. Semantics of the Veneer Layout Definition

Keywords in the veneer layout definition

```
veneer_layout
group
    run_addr
    mem
    select
```

17.9.1. Veneer Placement

If the linker creates additional sections while executing section placement (after section placement directions have been fully evaluated), such sections cannot be selected in a **section_layout**. You can use a **veneer_layout** instead for veneer sections that do not have an automatic explicit restriction.

The specification of the address space to which a **veneer_layout** definition applies, works in the same way as the specification of the address space for a **section_layout** definition.

```
veneer_layout my_chip::my_space
{
    section statements
}
```

The following differences apply with respect to the **section_layout**:

- Only **group** and **select** statements are allowed in a **veneer_layout**.
- On groups in a **veneer_layout** only a **run_addr** statements or tags are allowed.
- **select** statements only select veneers (sections created after locating has started) that do not have a restriction; this is true for veneers created with linker option **--long-branch-veneers=a**.
- **select** statements only select one veneer at a time.

- For absolute address restrictions, the group is considered 'ordered' (so the requirement that groups with an absolute address restriction must be ordered is met).

Example:

```
veneer_layout :vtc:abs24
{
    group mygroup (run_addr=mem:mpe:plash0/not_cached)
    {
        select  "";
    }
}
```

Chapter 19. CPU Problem Bypasses and Checks

Infineon Technologies regularly publishes microcontroller errata sheets for reporting both functional problems and deviations from the electrical and timing specifications.

For some of these functional problems in the microcontroller itself, the TASKING VX-toolset for TriCore provides workarounds. In fact these are software workarounds for hardware problems.

Support to deal with CPU functional problem is provided in three areas:

- Whenever possible and relevant, compiler bypasses will modify the code in order to avoid the identified erroneous code sequences;
- The assembler gives warnings for suspicious or erroneous code sequences;
- Ready-built, 'protected' standard C libraries with bypasses for all identified TriCore CPU functional problems are included in the toolset.

This chapter lists a summary of functional problems which can be bypassed by the TASKING VX-toolset for TriCore. Please refer to the Infineon errata sheets for the CPU step you are using, to verify if you need to use one of these bypasses.

To set a CPU bypass or check

1. From the **Project** menu, select **Properties for**

The Properties dialog appears.

2. In the left pane, expand **C/C++ Build** and select **Processor**.

In the right pane the Processor page appears.

3. From the **Processor selection** list, select a processor.

*The **CPU problem bypasses and checks** box shows the available workarounds/checks available for the selected processor.*

4. (Optional) Select **Show all CPU problem bypasses and checks**.
5. Click **Select All** or select one or more individual options.

Overview of the CPU problem bypasses and checks

The following table contains an overview of the silicon bugs you can provide to the [C compiler option `--silicon-bug`](#) and the [assembler option `--silicon-bug`](#). WA means a workaround by the compiler, assembler and/or linker, CK means a check by the compiler or assembler.

CPU Problem	Description	Compiler	Assembler	Linker	CPU
CPU TC.013	Unreliable context load/store operation following an address register load instruction	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.048	CPU fetches program from unexpected address	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.060	LD.[A,DA] followed by a dependent LD.[DA,D,W] can produce unreliable results	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.065	Error when unconditional loop targets unconditional jump	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.068	Potential PSW corruption by cancelled DVINIT instructions	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.069	Potential incorrect operation of RSLCX instruction	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.070	Error when conditional jump precedes loop instruction	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.071	Error when Conditional Loop targets Unconditional Loop		CK		TC1130, TC1766, TC1792, TC1796
CPU TC.072	Error when Loop Counter modified prior to Loop instruction	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.074	Interleaved LOOP/LOOPU instructions may cause GRWP Trap		WA		TC1130, TC1766, TC1792, TC1796
CPU TC.081	Error during Load A[10], Call / Exception Sequence		CK		TC1130, TC1766, TC1792, TC1796
CPU TC.082	Data corruption possible when Memory Load follows Context Store		CK		TC1130, TC1766, TC1792, TC1796
CPU TC.083	Interrupt may be taken following DISABLE instruction	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.094	Potential Performance Loss when CSA Instruction follows IP Jump	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.095	Incorrect Forwarding in SAT, Mixed Register Instruction Sequence	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.096	Error when Conditional Loop targets Single Issue Group Loop	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.103	Spurious parity errors can be generated	WA		WA	TC1130, TC1766

CPU Problem	Description	Compiler	Assembler	Linker	CPU
CPU TC.104	Double-word Load instructions using Circular Addressing mode can produce unreliable results	WA	CK		TC1130, TC1766, TC1792, TC1796
CPU TC.105	User / Supervisor mode not staged correctly for Store Instructions	CK	CK		TC1766, TC1767, TC1792, TC1796, TC1797
CPU TC.106	Incorrect PSW update for certain IP instructions dual-issued with MTCR PSW	CK	CK		TC1767, TC1797
CPU TC.108	Incorrect Data Size for Circular Addressing mode instructions with wrap-around	WA	CK		TC1736, TC1766, TC1767, TC1792, TC1796, TC1797
CPU TC.109	Circular Addressing Load can overtake conflicting Store in Store Buffer	WA	CK		TC1736, TC1766, TC1767, TC1792, TC1796, TC1797
CPU TC.116	Parity bit corruption may occur when loop counter is read at start of loop body	WA	CK		TC1130, TC1792, TC1796

TC1164, TC1166, TC1762, TC1764 have the same silicon bugs as the TC1766.

CPU_TC.013

Command line option

--silicon-bug=cpu-tc013

Description

To bypass this CPU functional problem, the C compiler generates a NOP16 instruction if a 16-bit load/store address register instruction (instructions: LD16.A and ST16.A) is followed by a lower context load/store instruction (instructions: LDLCX and STLCX).

The assembler issues a warning if a 16-bit load/store address register instruction (instructions: LD16.A and ST16.A) is followed by a lower context load/store instruction (instructions: LDLCX and STLCX).

CPU_TC.048

Command line option

--silicon-bug=cpu-tc048

Description

To bypass this CPU functional problem, the C compiler generates a NOP instruction before a JI or CALLI instruction when this instruction is not directly preceded by either a NOP instruction or an integer instruction or a MAC instruction. The compiler also generates a NOP instruction before a RET and RET16 instruction if there is no or just one instruction before RET, starting from the function entry point.

The assembler issues a warning when a load address register instruction LD.A/LD.DA is being followed immediately by an indirect jump JI, JLI or indirect call CALLI instruction with the same address register as parameter, because the CPU might fetch program from an unexpected address. In case there is an IP instruction between the J and LD, then the problem arises too and the assembler issues a warning. The assembler also issues a warning when there is no or just one instruction (not a NOP instruction) between label and RET or RET16.

CPU_TC.060

Command line option

--silicon-bug=cpu-tc060

Description

To bypass this CPU functional problem, the C compiler generates a NOP instruction between an LD.A / LD.DA instruction and a following LD.W / LD.D instruction, even if an integer instruction occurs in between.

The assembler issues a warning when an LD.A / LD.DA instruction is directly followed by an LD.W / LD.D instruction, or when only an integer instruction is in between.

CPU_TC.065

Command line option

--silicon-bug=cpu-tc065

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction before a jump, when a label is directly followed by an unconditional jump.

The assembler issues a warning when a label is directly followed by an unconditional jump, only when debug information is turned off.

CPU_TC.068

Command line option

--silicon-bug=cpu-tc068

Description

To bypass this CPU functional problem, the C compiler inserts a DISABLE and two NOP instructions before each DVINIT instruction (and if necessary an ENABLE after DVINIT).

The assembler issues a warning when a DVINIT instruction is not preceded by a DISABLE and two NOP instructions.

CPU_TC.069

Command line option

--silicon-bug=cpu-tc069

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction after each RSLCX instruction.

The assembler issues a warning when an RSLCX instruction is not followed by a NOP instruction.

CPU_TC.070

Command line option

--silicon-bug=cpu-tc070

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction before a loop instruction, when a conditional jump, based on the value in an address register, is directly followed by a loop instruction.

The compiler inserts two NOP instructions before a loop instruction, when a conditional jump, based on the value in a data register, is directly followed by a loop instruction.

The assembler issues a warning when a conditional jump, based on the value in an address register, is directly followed by a loop instruction.

The assembler issues a warning when a conditional jump, based on the value in a data register, is directly followed by a loop instruction or when only a single NOP instruction is in between.

CPU_TC.071

Command line option

--silicon-bug=cpu-tc071

Description

The assembler issues a warning when a label is directly followed by an unconditional loop instruction, only when debug information is turned off.

The L00PU instruction is never generated by the compiler.

CPU_TC.072

Command line option

--silicon-bug=cpu-tc072

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction before a loop instruction, when an instruction that updates an address register is followed by a conditional loop instruction which uses this address register.

The assembler issues a warning when an instruction that updates an address register is followed by a conditional loop instruction which uses this address register.

CPU_TC.074

Command line option

--silicon-bug=cpu-tc074

Description

The C compiler has no workaround for this problem.

To bypass this CPU functional problem, the assembler encodes the LOOPU instruction in such a way that bits 12-15 get the value 1.

CPU_TC.081

Command line option

--silicon-bug=cpu-tc081

Description

The C compiler has no workaround for this problem.

The assembler issues a warning when an address register load instruction, LD . A or LD . DA, targeting the A[10] register, is immediately followed by an operation causing a context switch.

CPU_TC.082

Command line option

--silicon-bug=cpu-tc082

Description

The assembler issues a warning when a context store operation, STUCX or STLCX, is immediately followed by a memory load operation which reads from the last double-word address written by the context store.

The STUCX and STLCX instructions are never generated by the compiler.

CPU_TC.083

Command line option

--silicon-bug=cpu-tc083

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction after each DISABLE instruction.

The assembler issues a warning when the DISABLE instruction is not followed by a NOP instruction.

CPU_TC.094

Command line option

--silicon-bug=cpu-tc094

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction between an IP jump and CSA list instruction.

The assembler issues a warning when an IP jump is followed by a CSA list instruction.

CPU_TC.095

Command line option

--silicon-bug=cpu-tc095

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction between any SAT.B/SAT.H instruction and a following load-store instruction with a DGPR source operand (addsc.a, addsc.at, mov.a, mtc r).

The assembler issues a warning when a SAT.B/SAT.H instruction is immediately followed by a load-store instruction with a DGPR source operand (addsc.a, addsc.at, mov.a, mtc r).

CPU_TC.096

Command line option

--silicon-bug=cpu-tc096

Description

To bypass this CPU functional problem, the C compiler inserts NOP instructions for a single group loop. How many depends on the pattern of the assembler instructions.

Pattern 1:

<pre>label: loop Ax, label</pre>	<pre>label: nop ; Inserted by compiler nop ; Inserted by compiler loop Ax, label</pre>
--------------------------------------	--

Pattern 2:

<pre>label: <any IP instruction> loop Ax, label</pre>	<pre>label: <any IP instruction> nop ; Inserted by compiler nop ; Inserted by compiler loop Ax, label</pre>
---	---

Pattern 3:

<pre>label: <any non IP instruction> loop Ax, label</pre>	<pre>label: <any non IP instruction> nop ; Inserted by compiler loop Ax, label</pre>
---	--

Pattern 4:

<pre>label: <any IP instruction> <any LS instruction> loop Ax, label</pre>	<pre>label: <any IP instruction> <any LS instruction> nop ; Inserted by compiler loop Ax, label</pre>
--	---

The assembler issues a warning if it finds one of these patterns without NOPs.

Be careful with handwritten assembly and relative addressing against the location counter. e.g.:

```
label:
    <any IP instruction>
    <any LS instruction>
    loop Ax, *-6
```

In this case it is impossible to test if one of the patterns occur. Therefore, the assembler always issues a warning when a loop instruction is used with relative addressing.

CPU_TC.103

Command line option

--silicon-bug=cpu-tc103

Description

To bypass this CPU functional problem, the C compiler directs certain program flow instructions, such as RET, RFE, CALL and JI, running in spram (scratch pad ram) via a stub located in safe memory. In order to be able to tell the C compiler that certain code is predetermined for spram, the pragma spram and option **--spram** are introduced.

To bypass this CPU functional problem, the preprocessor define `__CPU_TC103__` is used in the `tc*.ls1` linker script files. The linker will collect the stubs as generated by the C compiler and locate them in safe non-spram memory. Furthermore it is tested if (the start of) the interrupt and trap table are located at safe addresses.

Safe non-SPRAM addresses are defined as any address except:

```
bit [15:14] = 11b (TC1130 PMEM)
bit [14:13] = 11b (TC1762, TC1764, TC1766 PMEM)
```

CPU_TC.104

Command line option

--silicon-bug=cpu-tc104

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction before a double-word load instruction using circular addressing mode (LD.D instruction).

The assembler issues a warning when a double-word load instruction using circular addressing mode (LD.D instruction) is not preceded by a NOP instruction.

CPU_TC.105

Command line option

--silicon-bug=cpu-tc105

Description

The C compiler has no workaround for this problem. The compiler issues a warning when an MTCR instruction is generated, which is not preceded by a DSYNC instruction.

Under some conditions the use of MTCR leads to errors, a DSYNC before the MTCR prevents these problems, but is in most situations not necessary.

The assembler issues a warning when an MTCR instruction is not preceded by a DSYNC instruction.

CPU_TC.106

Command line option

--silicon-bug=cpu-tc106

Description

The C compiler has no workaround for this problem. The compiler issues a warning when an MTCR instruction is generated, which is preceded by a MUL, MADD, MSUB or RSTV instruction.

Under some conditions the use of MTCR directly after a MUL/MADD/MSUB/RSTV instruction leads to errors, a NOP before the MTCR prevents these problems, but is in most situations not necessary.

The assembler issues a warning when an MTCR instruction is preceded by a MUL, MADD, MSUB or RSTV instruction.

CPU_TC.108

Command line option

--silicon-bug=cpu-tc108

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction before a load or store instruction using a circular addressing mode.

The assembler issues a warning when a load or store instruction using a circular addressing mode is not preceded by a NOP instruction.

CPU_TC.109

Command line option

--silicon-bug=cpu-tc109

Description

To bypass this CPU functional problem, the C compiler inserts a NOP instruction before the load word instruction when a load word instruction, using circular addressing mode, is preceded by a store byte instruction, or when only a single IP instruction is in between such a load word instruction and a store byte instruction.

The assembler issues a warning when a load word instruction, using circular addressing mode is preceded by a store byte instruction, or when only a single IP instruction is in between such a load word instruction and a store byte instruction.

CPU_TC.116

Command line option

--silicon-bug=cpu-tc116

Description

To bypass this CPU functional problem, the C compiler inserts two NOP instructions to avoid that the loop counter is read in the first two Load-Store instructions of the loop body.

The assembler issues a warning when the first two Load-Store instructions of the loop body are not preceded by two NOP instructions.

Details

The problem affects the following generic code sequence:

```
...
loop_target_:
    < Optional IPinst >
    LSinst1
    < Optional IPinst >
    LSinst2
    ...
    ...
    loop Ax, loop_target_
    ...
```

IPinst is any Integer Pipeline (IP) instruction, LSinst1 and LSinst2 are Load-Store (LS) pipeline instructions. See *TriCore1 Architecture Manual, Volume 2* (Instruction Set for V1.3/1.3.1 Architecture), Chapter 4 *Summary Tables of LS and IP Instructions* for a complete list of IP and LS instructions.

The problem may occur when either of the first two LS instructions of a loop body (LSinst1, LSinst2) read the loop variable Ax as a source operand. The problem occurs due to a missing forwarding path between the loop counter writeback to the register file and a following instruction reading this register. In this case the data seen by the LS instruction reading Ax is that currently being written through the transparent phase of the register file, potentially leading to setup timing violations where Ax is used. Such setup timing violations may lead to data corruption or spurious parity errors from memories if Ax is used as part of a memory address calculation.

Note: loop instructions themselves are not affected by this issue, so where LSinst1 or LSinst2 is a loop instruction then that instruction is not affected.

